09.10.2026

Co to jest JavaScript?

Ten wszechobecny dziś język skryptowy został stworzony w 1995 roku, na mocy porozumienia z firmą Sun Microsystems, w zamian za włączenie do przeglądarki Netscape obsługi technologii Java, język otrzymał oficjalną nazwę JavaScript. W grudniu tego samego roku został zaimplementowany w najnowszej wówczas wersji Navigatora i rozpoczął triumfalny pochód przez Sieć.
JavaScript jest przede wszystkim **językiem skryptowym – to znaczy interpretowanym**. Nie musi zostać skompilowany do kodu maszynowego, aby można było zobaczyć efekty jego działania. Wystarczy nam do tego przeglądarka internetowa, która ten język obsługuje.
Ze względów bezpieczeństwa JavaScript ma znacznie ograniczone uprawnienia dostępu do zasobów komputera, przy użyciu którego przeglądana jest dana strona, a wszelkie odwołania do funkcji i obiektów wykonywane są w trakcie wykonywania programu.
Javascript jest językiem **programowania obiektowego** ponieważ wykorzystuje definicje obiektów, metod, zdarzeń, które mogą być zastosowane do opisu właściwości obiektów. Autorzy JS zdefiniowali bardzo dużo obiektów, metod, zdarzeń. W JS obiekty posiadają pewną strukturę. Struktura to określana jest przez DOM (Document Object Model) Możesz korzystać z zdefiniowanych obiektów oraz metod. Możesz też tworzyć swoje jak i modyfikować istniejące.

```
<!DOCTYPE html>
<html lang="pl">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Tytuł strony</title>
</head>
<body>
  <h1>Witaj ZSE</h1>
</body>
</html>
```

Kluczowe elementy:

<!DOCTYPE html>: Definiuje dokument jako HTML5.
<html lang="pl">: Określa główny język treści jako polski (ważne dla czytników ekranu i SEO).
<meta charset="UTF-8">: Kluczowy tag zapewniający poprawne wyświetlanie polskich znaków.
<meta name="viewport" ...>: Zapewnia poprawne wyświetlanie strony na urządzeniach mobilnych.
<title>: Tytuł strony wyświetlany na karcie przeglądarki.
<body>: Zawiera widoczną treść strony.

Zalety JavaScriptu

1)Największą zaletą JavaScriptu jest niewątpliwie jego prostota. Język ten uważa się więc za relatywnie łatwy do opanowania i rzeczywiście – nic nie stoi na przeszkodzie, aby zacząć używać go już po kilku minutach od rozpoczęcia nauki.
2)Ponieważ jest to obecnie najbardziej rozpowszechniony język skryptowy, w Internecie można znaleźć miliony praktycznych przykładów – witryn wykorzystujących różnorodne funkcje JavaScriptu.

3)O olbrzymiej popularności języka zadecydowały z pewnością jego uniwersalność oraz fakt, iż pozwala on na odciążenie serwerów i ograniczenie ilości zbędnych danych przesyłanych pomiędzy nimi a przeglądarką klienta.
4) JavaScript pomaga użytkownikom tworzyć nowoczesne aplikacje internetowe do bezpośredniej interakcji bez ponownego ładowania strony za każdym razem.
5)Z pomocą **frameworków** (rodzaj programu do tworzenia innych programów w JS) zostały stworzone znane na całym świecie aplikacje, np. Google używa Angular, a Facebook — React.
6)W praktyce developerskiej stosowana jest JavaScript w tworzeniu **dynamicznej zawartości witryn** internetowych, aktualizowaniu treści w czasie rzeczywistym, interaktywnych formularzach, różnych animacjach i zaawansowanych funkcjach, które zwiększają wydajność aplikacji internetowej i angażują użytkownika.
7)Funkcje i zastosowania JavaScript sprawiają, że jest to potężne narzędzie do tworzenia **aplikacji mobilnych.** Bardzo popularnym frameworkiem do tworzenia aplikacji jest React Native. Z jego pomocą możemy budować aplikacje mobilne na różne systemy operacyjne (cross-platform apps). Inne popularne front-endowe frameworki to React, Angular i Vue.
Budowanie serwerów WWW i tworzenie aplikacji serwerowych
8)Oprócz witryn i aplikacji, programiści mogą również używać JavaScriptu do **tworzenia prostych serwerów internetowych** i rozwijania infrastruktury zaplecza przy użyciu biblioteki Node.js, który zapewnia środowisko zawierające niezbędne narzędzia wymagane do uruchomienia JS na serwerach.
9)JavaScript służy również do **tworzenia gier przeglądarkowych**. To świetny sposób dla początkujących programistów na ćwiczenie swoich umiejętności. Posiada różne biblioteki i frameworki do tworzenia gier, takie jak PhysicsJS, Pixi.js. Możemy również użyć WebGL (biblioteki grafiki internetowej), która jest API JS do renderowania obrazów 2D i 3D w przeglądarkach.
10)**Responsywność** JavaScript udostępnia metody, których można użyć w aplikacji, aby uzyskać bieżący rozmiar ekranu i wykorzystać wynik do wykonania działań w oparciu o ten rozmiar. Używając CSS i HTML można zmienić rozmiar, ukrywać, zmniejszać, powiększać lub przenosić treści, aby wyglądały dobrze na dowolnym ekranie.
11) **AJAX (Asynchronous JavaScript and XML)**AJAX umożliwia nam dynamiczne ściąganie i wysyłanie danych bez potrzeby przeładowania całej strony. W dzisiejszych czasach jest to praktycznie normą. Gdy wchodzisz na Facebooka, przeglądasz sobie profil wczytując kolejne posty, możesz równocześnie pisać komentarz.

Dział 1 Wiadomości wstępne.

Dwa sposoby dołączenia/osadzania pliku JavaScript do kodu strony WWW.

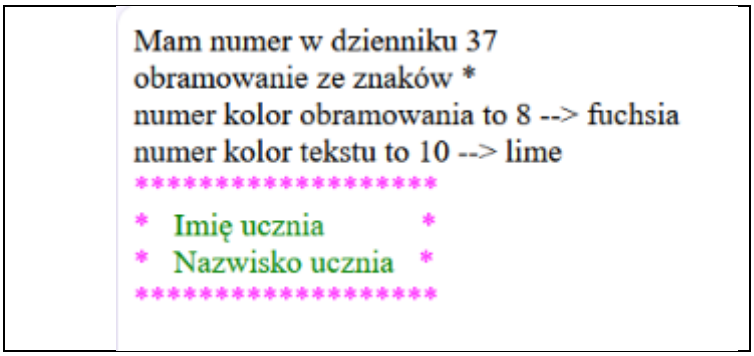
Sposób1:

Zadanie 1.
Plik *.html o nazwie **zadanie1_cztery_litery_nazwiska_ucznia.html**

Umieszczenie kodu źródłowego skryptu wprost pomiędzy znacznikami **script:**

```
<!DOCTYPE html>
<html lang="pl-PL">
  <HEAD>
    <meta charset="utf-8">
    <TITLE>Osadzanie pliku JS-nazwisko_ucznia(wpisz Twoje)</TITLE>
  </HEAD>
<BODY>
 <SCRIPT>
document.write("******************".fontcolor("#FF00FF")+"<br>");
document.write("*".fontcolor("#FF00FF")+"   Imię ucznia
```

```
".fontcolor("green")+"           
*".fontcolor("#FF00FF")+"<br>");
document.write("*".fontcolor("#FF00FF")+"   Nazwisko ucznia
".fontcolor("green")+"  *".fontcolor("#FF00FF")+"<br>");
document.write("**********".fontcolor("#FF00FF")+"<br>");
 </SCRIPT>
</BODY>
</HTML>
```



Wykonaj:

1)Na podstawie kodu powyżej (możesz kopiować) wykonaj program robiący wizytówkę zawierający obramowanie, imię, nazwisko ucznia(wpisz swoje). W tabelach poniżej masz kolory oraz znaki obramowań.

2)Gdy kopiujesz kod programu musisz instrukcję document.write(); **umieścić w jednej**, tak aby linia kodu kończyła się na znaku ; [średnik]

3)łączenie tekstów przy użyciu +, teksty są w cudzysłowie " tekst "

4)" " to dwie spacje

5) "**********".fontcolor("#FF00FF") gdy chcesz nadać tekstowi pewne właściwości to stosujemy konstrukcję z kropką. Jest to tzw wywołanie metody w programowaniu obiektowym.

6) nazwa pliku: z1_kowa.html z1_cztery_pierwsze_litery_nazwiska.html

7)Na zakładce strony w przeglądarce ma się pokazać Twoje nazwisko

Numer w dzienniku	Numer kolorów (patrz tabela poniżej) dla obramowania	Numer kolorów (patrz tabela poniżej) dla imienia i nazwiska	Znak obramowania
1, 15	16	9	+
2, 16	15	8	=
3, 17	14	7	!
4, 18	13	6	#
5, 18, 36	12	5	$
6, 19,35	11	3	%
7, 20, 34	10	2	&
8, 21, 33	9	15	+
9, 22, 32	8	14	=
10, 23, 31	7	13	!
11, 24, 30	6	12	#
12, 25, 29	5	11	$

13, 26, 28	3	10	%
14, 27	2	11	&

Tabela kolorów

numer koloru	Nazwa	HEX	Kolor
1	Black	#000000	
2	silver	#C0C0C0	
3	Gray	#808080	
4	white	#FFFFFF	
5	maroon	#800000	
6	red	#FF0000	
7	purple	#800080	
8	fuchsia	#FF00FF	
9	Green	#008000	
10	lime	#00FF00	
11	olive	#808000	
12	yellow	#FFFF00	
13	navy	#000080	
14	blue	#0000FF	
15	teal	#008080	
16	Adua	#00FFFF	

Sposób2:

Zadanie 2.

Temat: Zastosowanie pliku zewnętrznego javascript *.js do określenia ostatniej daty modyfikacji.

Przykłady określania właściwości tekstu w JS

```
var txt = "Hello World!";
document.write("ZSE najlepszą szkoła w kodowaniu" + txt);
document.write("<p>Big: " + txt.big() + "</p>");
document.write("<p>Small: " + txt.small() + "</p>");
document.write("<p>Bold: " + txt.bold() + "</p>");
document.write("<p>Italic: " + txt.italics() + "</p>");
document.write("<p>Fixed: " + txt.fixed() + "</p>");
document.write("<p>Strike: " + txt.strike() + "</p>");
document.write("<p>Fontcolor: " + txt.fontcolor("green") + "</p>");
document.write("<p>Fontsize: " + txt.fontsize(6) + "</p>");
document.write("<p>Subscript: " + txt.sub() + "</p>");
document.write("<p>Superscript: " + txt.sup() + "</p>");
document.write("<p>Link: " + txt.link("http://www.w3ii.com") + "</p>");
document.write("<p>Blink: " + txt.blink() + " (works only in Opera)</p>");
```

1)**Plik *.html** o nazwie **zadanie2_cztery_litery_nazwiska_ucznia.html**

```
<!DOCTYPE html>
<html lang="pl-PL">
  <HEAD>
      <meta charset="utf-8">
       <TITLE>SKRYPT zewnętrzny-nazwisko_ucznia(wpisz twoje)</TITLE>
<  /HEAD>
<BODY>
     <SCRIPT type="text/javascript" src="nazwisko_ucznia_zewnetrzny.js"></SCRIPT>
</BODY>
</HTML>
```

2)Plik *.js o nazwie **nazwisko_ucznia_zewnetrzny.js (**bez polskich liter)

Poniżej masz kodu pliku zewnętrznego wpisz go do pliku nazwisko_ucznia_zewnetrzny.js

```
document.write("ostatnia modyfikacja strony".fontcolor("red").bold().fontsize(7)+"<br>");
document.write(document.lastModified);
```

Wyglą ekranu to: (zwróć uwagę na liczby I napisy znajdują się pod sobą [w kolumne]), odpowiednie efekty uzyskasz poprzez odpowiednią ilość spacji nełąmliwych.

Mój numer w dzienniku to 37
Pierwszy wiersz to: 14 Tak Tak 5
Drugi wiersz to: 2 Nie Tak 6

ostatnia modyfikacja strony

02/03/2014 09:17:42

Wykonaj:

1)W celu wykonania zadania musisz skopiować dwa pliki (są powyżej) i nadać im odpowiednie nazwy.

2)Zmień właściwości tekstu wg tabeli→dla **pierwszego wiersza**

Numer w dzienniku	Numer kolorów (patrz tabela) dla tekstu „ostatnia modyfikacja strony” Pierwszy wiersz	Test pogrubiony	Tekst przekreślony	Wielkość liter: fontsize(()
1, 15	16	Tak	Nie	6
2, 16	15	Nie	Tak	7
3, 17	14	Tak	Tak	5
4, 18	13	Tak	Nie	6
5, 18, 36	12	Nie	Tak	7
6, 19,35	11	Tak	Tak	5
7, 20, 34	10	Tak	Nie	5
8, 21, 33	9	Nie	Tak	6
9, 22, 32	8	Tak	Tak	7
10, 23, 31	7	Tak	Nie	5
11, 24, 30	6	Nie	Tak	6
12, 25, 29	5	Tak	Tak	7
13, 26, 28	3	Tak	Nie	5
14, 27	2	Nie	Tak	6

3)Zmień właściwości tekstu wg tabeli→dla **drugiego wiersza**

Numer w dzienniku	**Numer kolorów (patrz tabela) dla tekstu „ostatnia modyfikacja strony” Pierwszy wiersz**	**Test pogrubiony**	**Tekst przekreślony**	**Wielkość liter: fontsize(()**
14, 27	16	Tak	Nie	6
1, 15	13	Tak	Nie	6
2, 16	15	Nie	Tak	7
3, 17	14	Tak	Tak	5
4, 18	2	Nie	Tak	6
5, 18, 36	12	Nie	Tak	7
7, 20, 34	11	Tak	Tak	5
6, 19,35	10	Tak	Nie	5
8, 21, 33	9	Nie	Tak	6
9, 22, 32	8	Tak	Tak	7
10, 23, 31	7	Tak	Nie	5
12, 25, 29	6	Nie	Tak	6
11, 24, 30	5	Tak	Tak	7
13, 26, 28	3	Tak	Nie	5

Dział 2 Praca z oknami

Zadanie 3.

Temat: Demonstracja pracy z oknami.

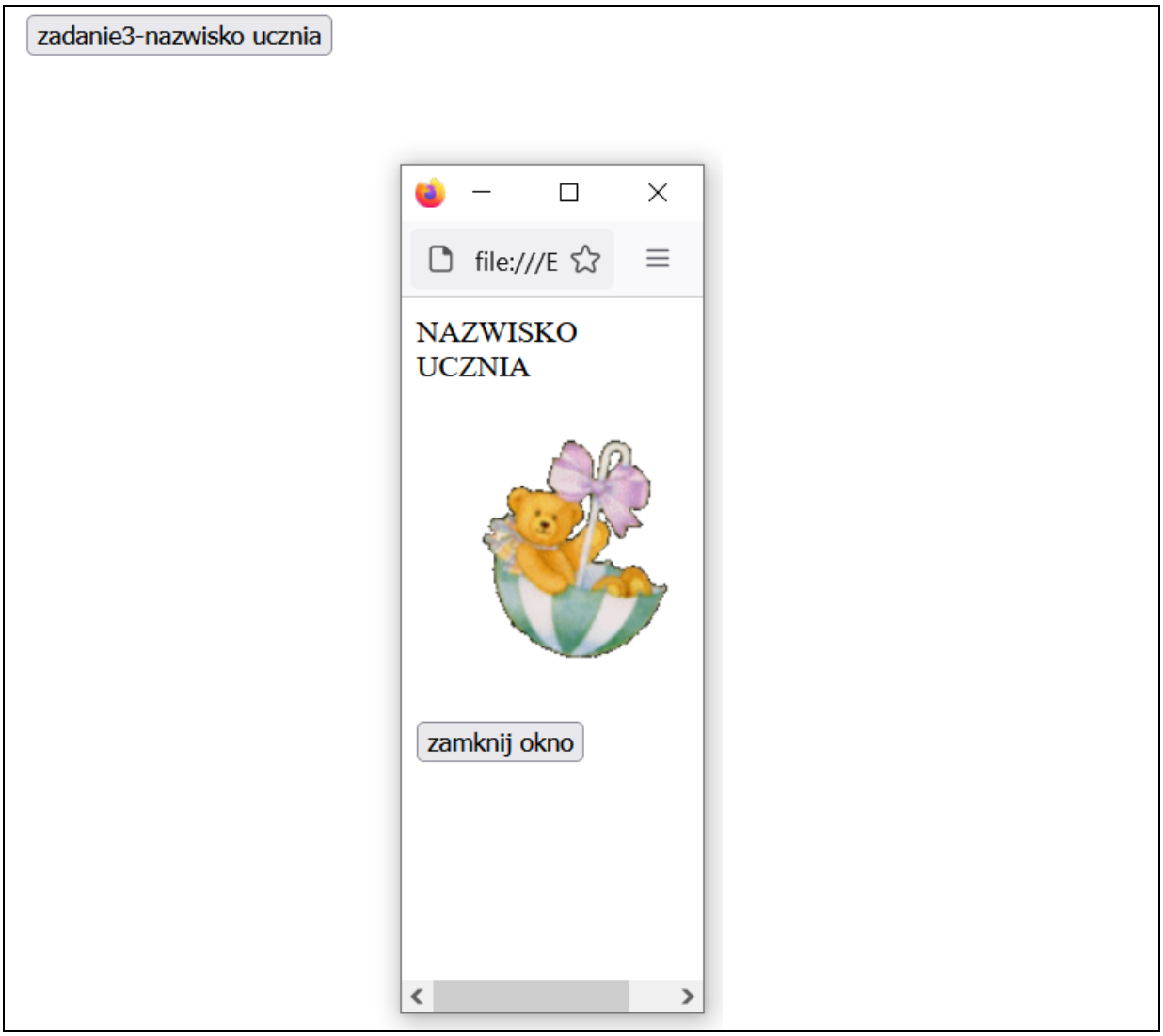
Plik o nazwie zadanie3_trzy_litery_nazwiska.html
Np. zadanie3_kow.html (bez polskich liter)

```
<html>
   <head>
        <title> Tutaj wpisz Twoje nazwisko i klasę</title>
       <script language="JavaScript">
         function WinOpen()
          {
window.open("obraz.html","okienko","toolbar=no,directories=no,menubar=no,height=380,width=160,top=200,left=200");
          }
       </script>
    </head>
<body>
  <form>
 <input type="button" name="zadanie3" value="zadanie3-nazwisko ucznia" onclick="WinOpen(' ')">
  </form>
</body>
</html>
```

Plik o nazwie obraz.html

```
<html>
   <head>
       <script type="text/javascript">
           function okno_zamknij()
               {
                   window.close()
               }
        </script>
   </head>
<body>
     NAZWISKO UCZNIA
    <img src="obraz1.gif" />
    <input type="button" value="zamknij okno" onclick="okno_zamknij()"/>
</body>
</html>
```

3)dograj do **folderu zadanie3** darmowego animowanego gifa i zmień mu nazwę na **obraz1.gif**
Zwróć uwagę na wymiar gifa, muszą być one takie aby cały gif mieścił się w oknie. (konieczne będzie **przeskalowanie grafiki** np. w programie Paint)
4)
Zmień nazwy funkcji w pliku zadanie3_kow.html tak, aby było tam nazwisko:
function WinOpen() →na WinOpen_kowalski()
function okno_zamknij() →okno_zamknij_kowalski()



Zadanie 4.

Temat: Wykonanie strony zaliczeniowej do zadań z JavaScript. Wyświetlanie teorii w oknie.

Wykonaj:

1)Przeczytaj tekst poniżej.

Poniżej masz kod strony zaliczeniowej. (wywołanie dwóch stron: jedna to rozwiązanie zadanie 3 a druga to strona rozwiązanie zadania 4 zawierająca teorię Javscript. Zwróć uwagę, że wszystkie mają przycisk zamykający okno. Możesz dokonać kopiowania kodu lecz musisz pamiętać o zamianie nazw funkcji oraz plików na z literami Twojego nazwiska. Dla okna wywoływanego w instrukcji

window.open("z4_kow.html","okienko_z4","toolbar=no,directories=no,menubar=no,height=380,width=160,top=200,left=400");

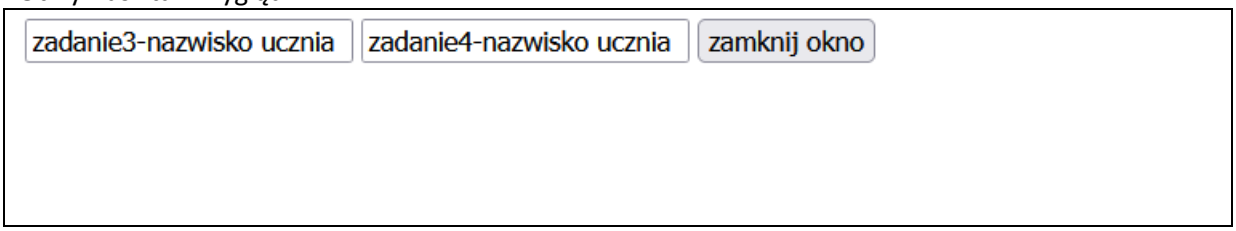
musisz zmienić wymiary okna na takie aby zmieściła się cała teoria.

2)Dokonaj kopiowania

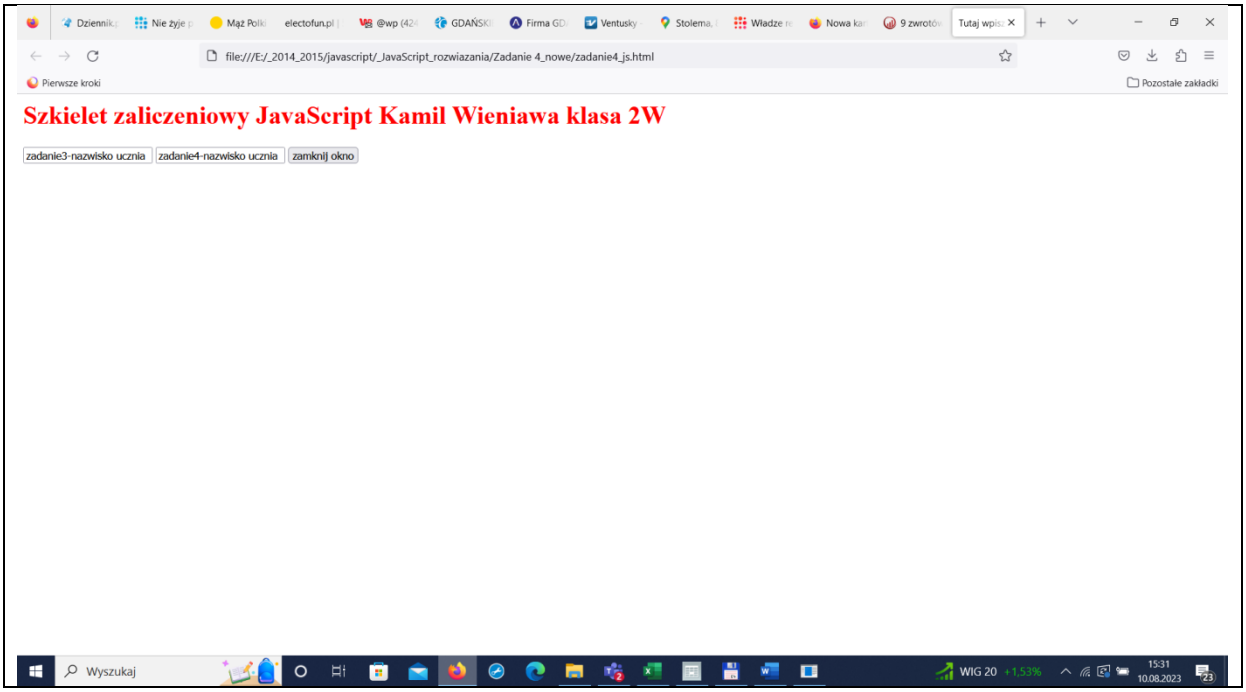
```
<html>
  <head>
    <title> Tutaj wpisz Twoje nazwisko i klasę</title>
```

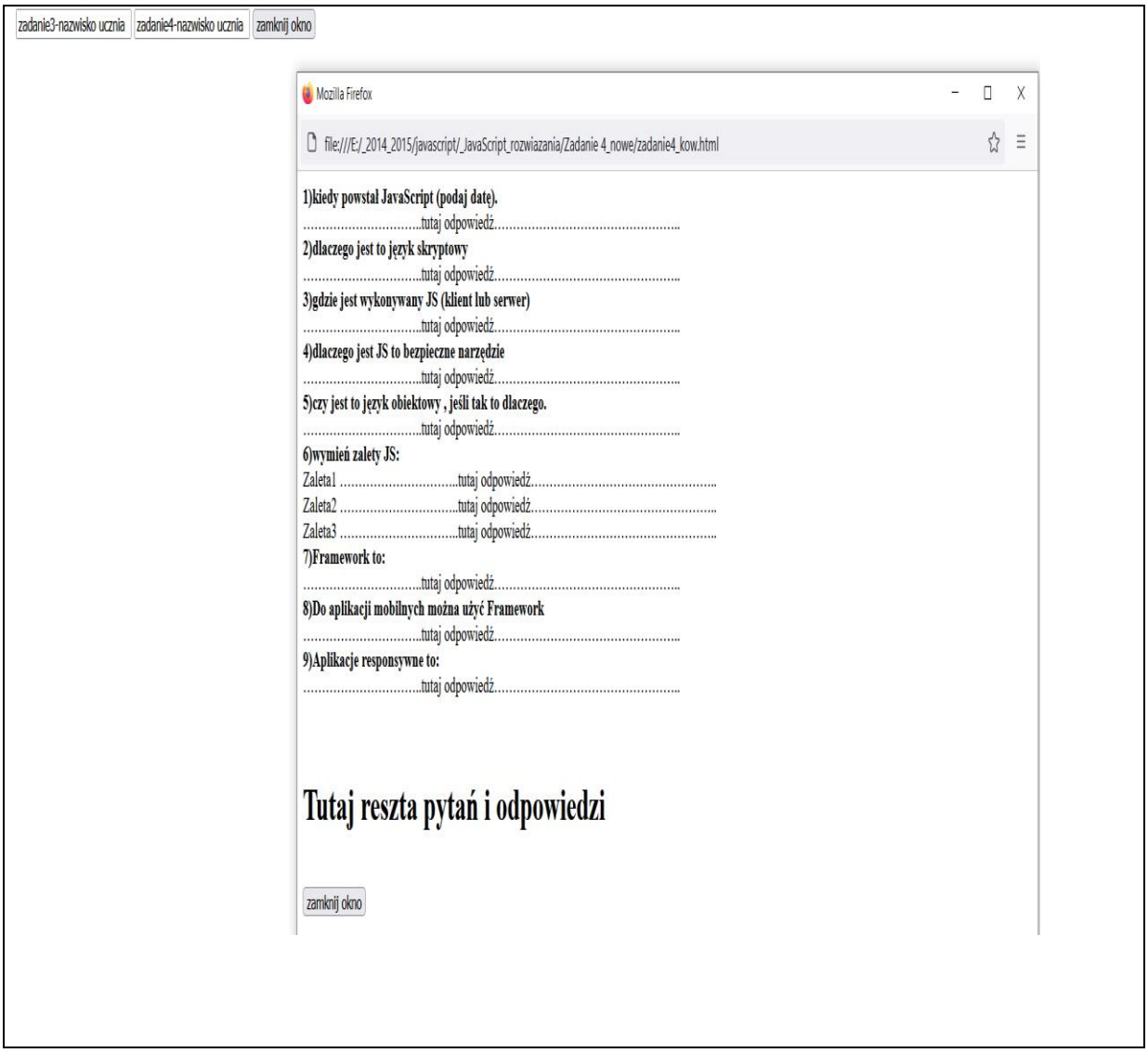
```
        <script language="JavaScript">
          function WinOpen_z3()
           {
window.open("z3_kow.html","okienko_z3","toolbar=no,directories=no,menubar=no,height=380,width=160,top=200,left=200");
           }
function WinOpen_z4()
           {
window.open("z4_kow.html","okienko_z4","toolbar=no,directories=no,menubar=no,height=380,width=160,top=200,left=400");
           }
function okno_zamknij()
                {
                    window.close()
                }
       </script>
    </head>
<body>
  <form>
      <input type="button_z3" name="zadanie3" value="zadanie3-nazwisko ucznia" onclick="WinOpen_z3(' ')">
      <input type="button_z4" name="zadanie4" value="zadanie4-nazwisko ucznia" onclick="WinOpen_z4(' ')">
      <input type="button" value="zamknij okno" onclick="okno_zamknij()"/>
   </form>
</body>
</html>
```

Otrzymasz taki wygląd



3) Dopisz linię identyfikującą autora „szkieletu zaliczeniowego”, tak jak poniże (identyczna)j. Wielkość liter 2 cm.





4)wykonaj odpowiedzi na pytania teoretyczne do pliku **zadanie4_kow.html**
Najpierw pytania (czcionką pogrubioną) a potem w nowym wierszu odpowiedź.

Teorii proszę szukać w tej instrukcji (kliknij).

1)kiedy powstał JavaScript (podaj datę).
……..tutaj odpowiedź……..

2)dlaczego jest to język skryptowy
……..tutaj odpowiedź……..

3)gdzie jest wykonywany JS (klient lub serwer)
……..tutaj odpowiedź……..

4)dlaczego jest JS to bezpieczne narzędzie

……..tutaj odpowiedź……..
5)czy jest to język obiektowy , jeśli tak to dlaczego.
……..tutaj odpowiedź……..

6)wymień zalety JS:

Zaleta1 ……..tutaj odpowiedź……..

Zaleta2 ……..tutaj odpowiedź……..

Zaleta3 ……..tutaj odpowiedź……..

7)Framework to:
……..tutaj odpowiedź……..

8)Do aplikacji mobilnych można użyć Framework
……..tutaj odpowiedź……..

9)Aplikacje responsywne to:
……..tutaj odpowiedź……..

10)Przepisz linie kody pod nimi wytłumaczenie jak działa ten fragment kodu
<input type="button" name="przycisk" value="Nowa Strona" onclick="WinOpen(' ')">
……*wytłumaczenie jak działa linia kodu*…….
Uwaga:
gdy chcesz wyświetlić instrukcję, która nie powinna się wykonywać a tylko wyświetlać masz do dyspozycji wyświetlanie znaków, które nie będą interpretowane jako instrukcja czyli > lub <
znak < możesz wyświetlić jako ‹
znak > możesz wyświetlić jako ›
11)Przepisz linie kody pod nimi wytłumaczenie jak działa ta ten fragment kodu
window.open("obraz.html","okienko","toolbar=no,directories=no,menubar=no,height=280,width=160,top=200,left=200");
……*wytłumaczenie jak działa linia kodu*…….
kliknij tutaj aby uzyskać opis poniższych opcji
lub **https://kursjs.pl/kurs/rozne/okna**

toolbar=no →*wytłumaczenie* np. ukrywa przyciski katalogów
directories=no, →*wytłumaczenie*
menubar=no, →*wytłumaczenie*
height=280, →*wytłumaczenie*
width=160, →*wytłumaczenie*
top=200, →*wytłumaczenie*
left=200 →*wytłumaczenie*

12)Opisz następującą instrukcję window.close()
……*wytłumaczenie*…….

==========

Gdy chcesz wyświetlić instrukcję, która nie powinna się wykonywać a tylko wyświetlać masz: dwie możliwości:

pierwsza możliwość

znak < możesz wyświetlić jako ‹
znak > możesz wyświetlić jako ›

więcej na :

https://unicode-table.com/pl/html-entities/

tekst możesz umieścić w

druga możliwość

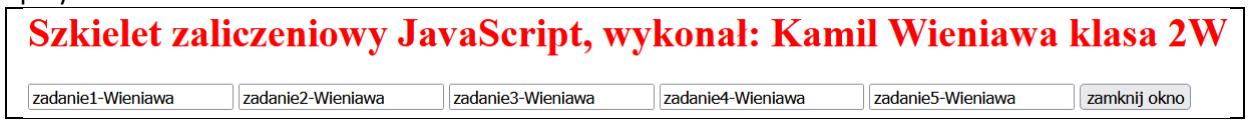
……wytłumaczenie……

```
<!DOCTYPE html>
<html lang="pl">
 <head>
  <meta http-equiv="Content-Type" content="text/html; charset=utf-8">
  <meta http-equiv="Content-Script-Type" content="text/javascript">
  <title>Moja strona WWW</title>
  <script type="text/javascript">
  function onclickHandler()
  {
   alert('Akapit został kliknięty!');
  }
  </script>
 </head>
 <body>
  <p onclick="onclickHandler();">Kliknij ten akapit,
   a pojawi się okno dialogowe.</p>
 </body>
</html>
```

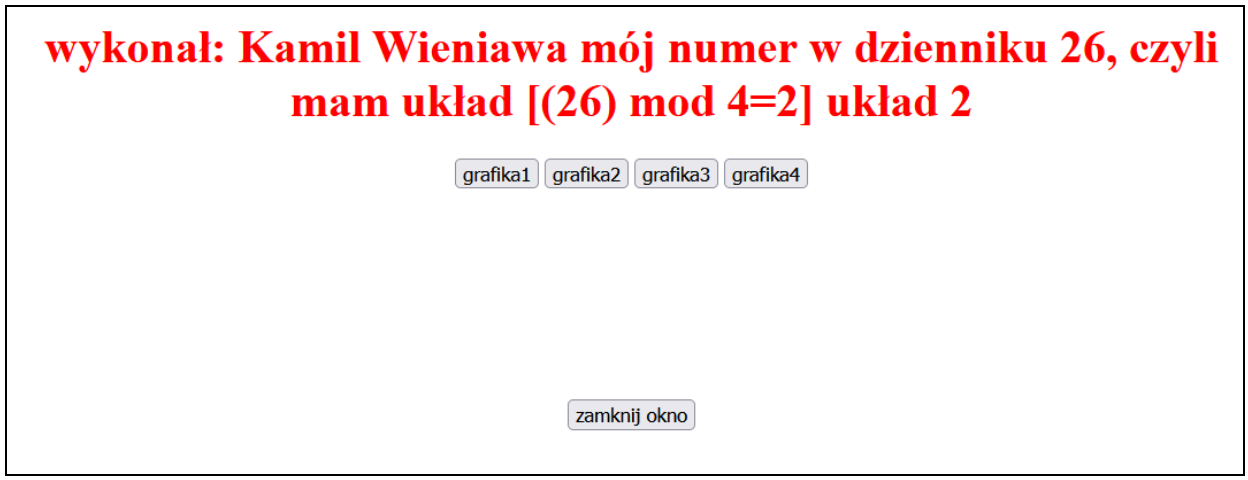
==========

Zadanie 5.
Temat: Praca z oknami.

1)Dodaj trzecią opcję jako wywołanie zadania 5 (tutaj przycisk zadanie5-Wieniawa) oraz dopisz przyciski do zadania 1 i zadania 2



2)Po wyborze przycisku zadanie5-Wieniawa, uruchomi się następujące okienko (napis konieczny, obliczamy układ) oraz 4 przyciski.



3)Przyciski grafika1, grafika2, grafika3, grafika4 będą realizowały:,

- wyświetlały się, cztery pliki graficzne z opisami odpowiadającymi grafice. Grafiki otwierają się w nowych oknach, opis umieść pod grafiką w tym samym oknie (każdy opis **innym kolorem**). Całość ma tworzyć śmieszną przyzwoita historyjkę. Grafiki pobierz z Internetu. Wielkość grafik przeskaluj tak aby mieściły się w całości w oknie (na przykładowym ekranie nie skalowania). Opisy wymyśl sam.
- na stronie głównej wykonaj przyciski do każdej grafiki z odpowiednim napisem na przycisku.
- okna z grafiki powinny być tak rozmieszczone aby po otwarciu wszystkich nie nachodziły na siebie.
- Funkcje muszą mieć nazwy z Twoim nazwiskiem

WinOpen_kowalski_1()
okno_zamknij_kowalski_1()

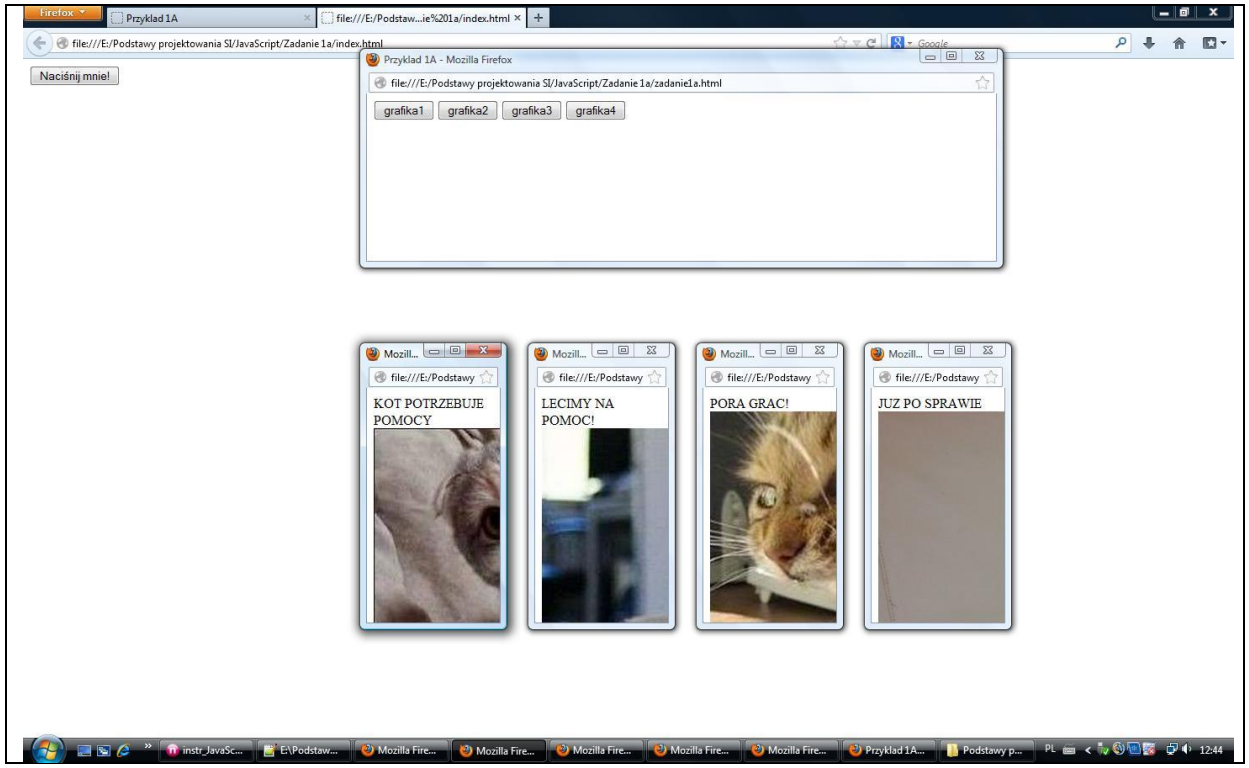
- pamiętaj o tym, że okna musza mieć różne nazwy

==========

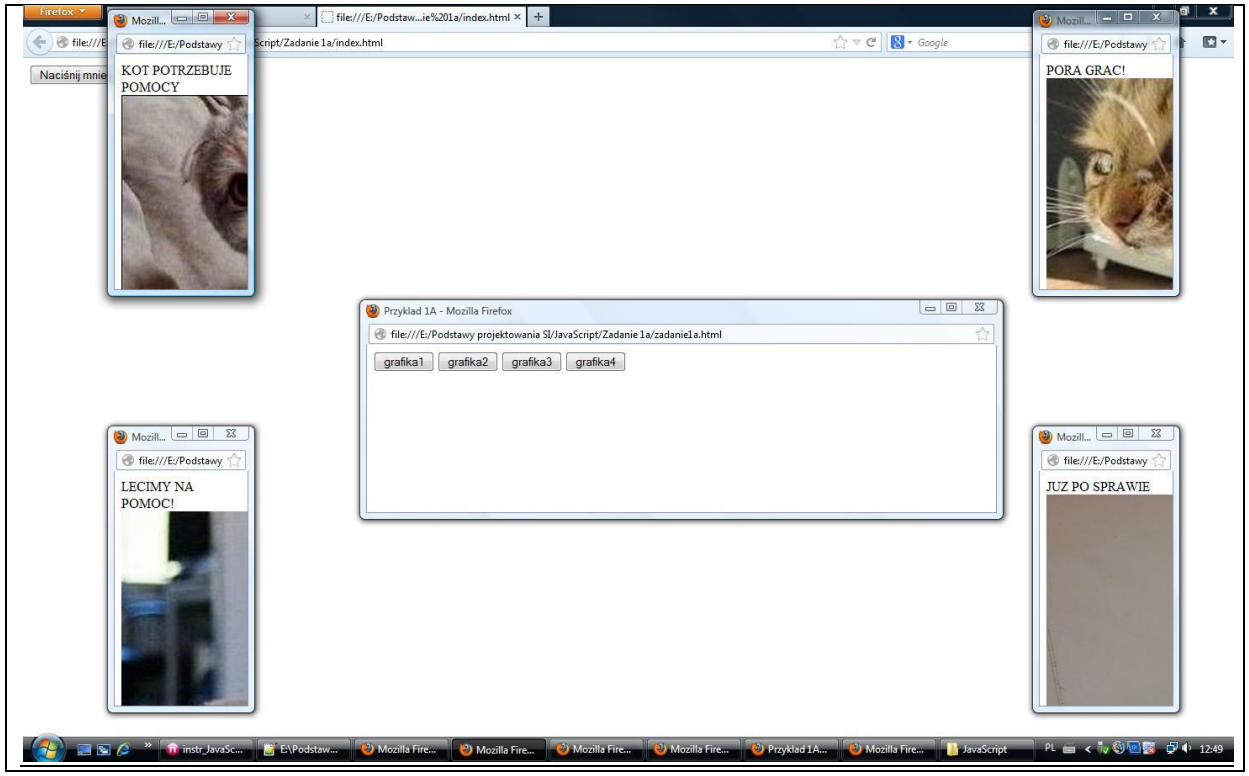
Wykonujesz układ jak układach zademonstrowanych poniżej.

Obliczenie	**Który układ**
(Numer_w_dzienniku) mod 4=1	Układ1
(Numer_w_dzienniku) mod 4=2	Układ2
(Numer_w_dzienniku) mod 4=0	Układ4
(Numer_w_dzienniku) mod 4=3	Układ3

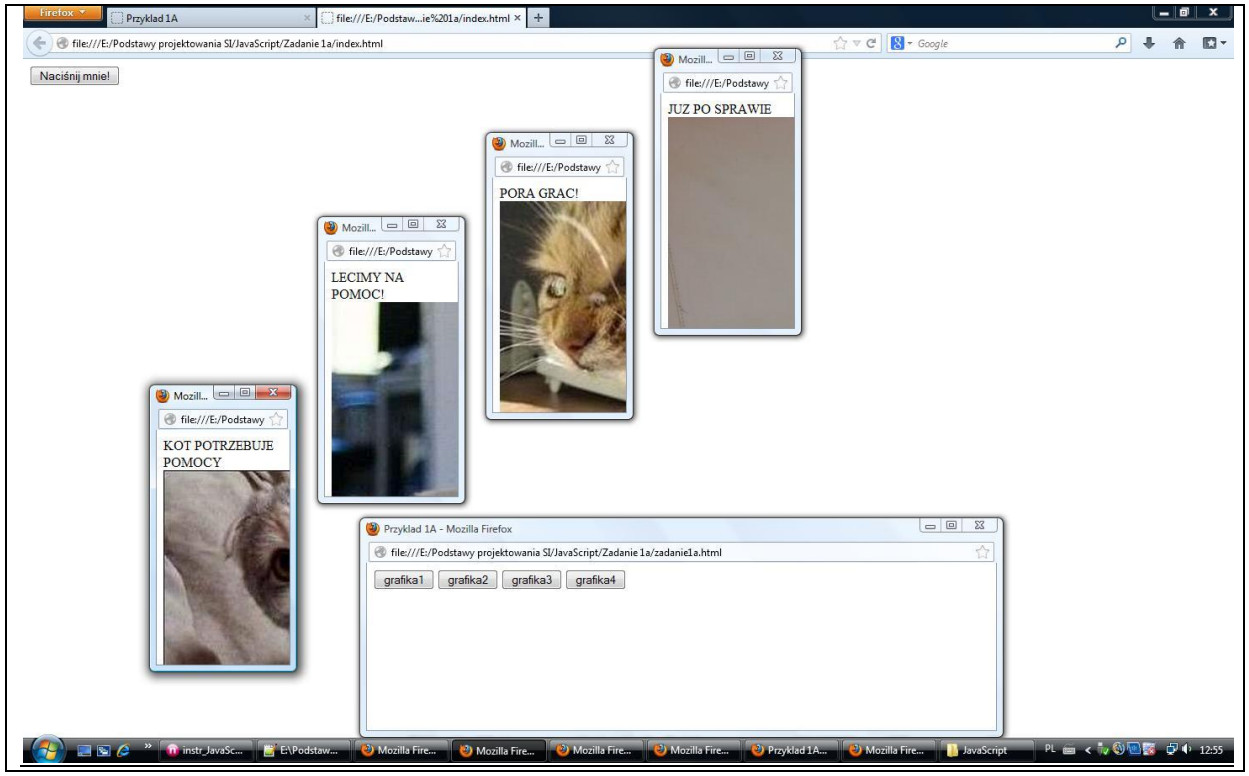
Układ1



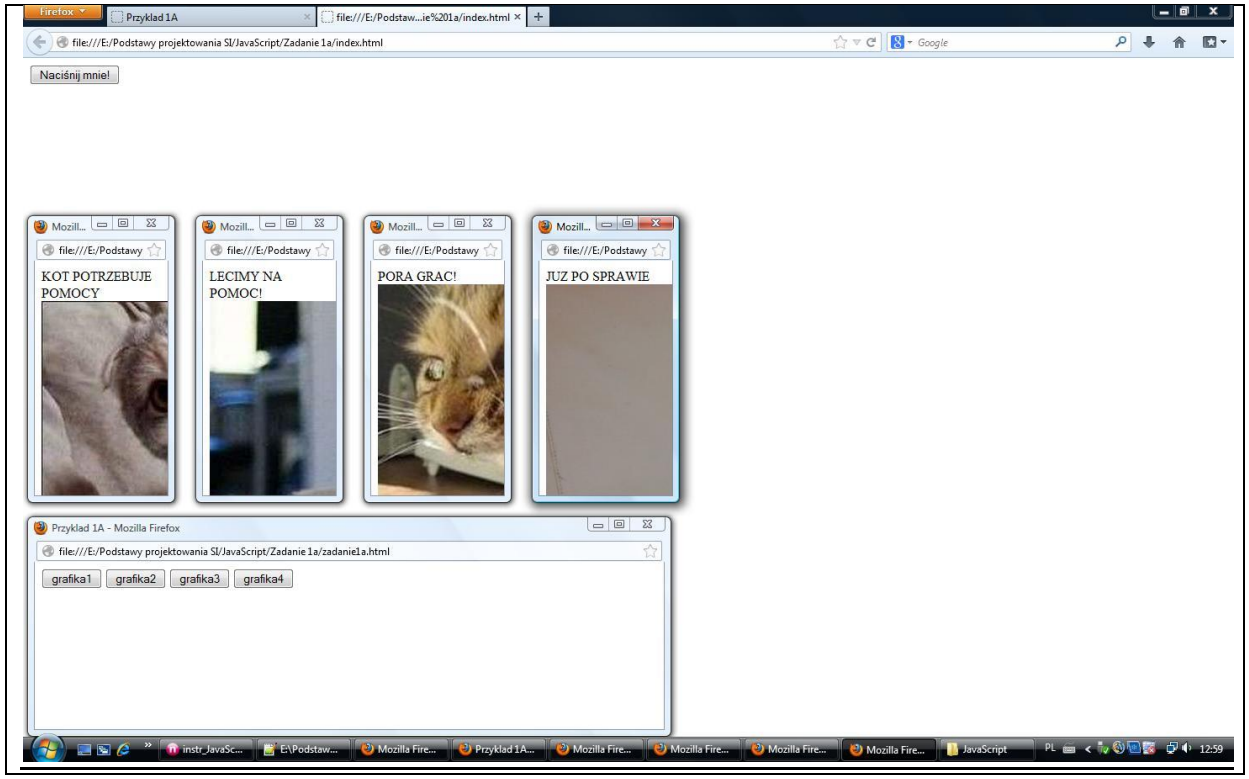
Układ2



Układ3



Układ4



==========koniec działu 2==========

Narzędzia programisty w przeglądarce FireFox

Narzędzia-klawisze skrótu	
Polecenie	**Skrót**
Pobieranie	Ctrl + J
Dodatki	Ctrl + Shift + A
Otwieranie narzędzi twórców witryn	F12 Ctrl + Shift + I
Konsola WWW	Ctrl + Shift + K
Inspektor	Ctrl + Shift + C
Debugger	Ctrl + Shift + S
Edytor stylów	Shift + F7
Profiler	Shift + F5
Sieć	Ctrl + Shift + Q
Pasek programisty	Shift + F2
Widok responsywny	Ctrl + Shift + M
Brudnopis	Shift + F4
Źródło strony	Ctrl + U
Konsola przeglądarki	Ctrl + Shift + J

Otwieranie dodatkowego, głównego narzędzia.

Postępowanie:

1) wybierz F12 –> **Otwieranie narzędzi twórców witryn**

2) Uzyskasz dostęp do:

Inspektor	Konsola	Debugger	Edytor stylów	Wydajność	Sieć

==========

Dział 3

Okienka dialogowe. Sterowanie wydrukami. Obliczenia część 1.

Zadanie 6.

Temat: Zadanie teoretyczne.

Wykonaj:
***przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie6)

*omów następujące zagadnienia teoretyczne(teoria poniżej):
Najpierw pytania (czcionką pogrubioną) a potem w nowym wierszu odpowiedź.
1) Rodzaje okienek,
2) Konwersji liczb.
w oparciu o opis teoretyczne znajdujący się poniżej (**wytłuczenia oraz podkreślenia tekstu konieczne**).
3) Przy nadawaniu nazw zmiennym należy przestrzegać pewnych zasad:
(trzy zasady)
4)Omów typy zmiennych w Javascript.

==========Początek opisu teorii==========

Rodzaje okienek:

- Okienko **alert** – wyświetlająca okienko dialogowe z informacją i przyciskiem OK,
- Okienko **prompt** – generująca okno, w którym użytkownik może wpisać wartość, zwracaną następnie do programu.
- Okienko **confirm** służy do tego, aby użytkownik coś potwierdził (stąd nazwa confirm).

Przy nadawaniu nazw zmiennym należy przestrzegać pewnych zasad:
1. Nazwa zmiennej musi się zaczynać od litery, znaku "$" lub "_", może zawierać tylko litery (bez polskich znaków), cyfry oraz "$" i "_".
Poprawne nazwy zmiennych: zmienna1, $wynik, _rozmiar, wynik1, Rekord, $_gracz
Błędne nazwy zmiennych: 1zmienna, wynik gry, wysokosc&szerokosc, wynik.rekord, &gracz, wysokość, imie+nazwisko.
2. Wielkość liter ma znaczenie.
Rekord i rekord to dwie różne zmienne! Jest to bardzo częsta przyczyna błędów w kodzie początkujących programistów.
3. Należy nie używać słów kluczowych JavaScript jako nazw zmiennych.

Typy zmiennych w Javascript
W JavaScript nie ma możliwości definiowania różnych typów zmiennych tak jak w C++, typy zmiennych są definiowane w trakcie ich przypisywania.
Podstawowe zmienne liczbowe w JavaScript to:
Number → Typ liczbowy

- zmienna całkowite
- zmiennoprzecinkowe

String → Typ tekstowy
Boolea → Typ logiczny
Null → Pusta zawartość
Undefined → Typ niezidentyfikowany

Object → Obiekt

Konwersji zmiennych.
Wczytanie zmiennej z użyciem **prompt** wykonywane jest jako ciąg znaków dlatego konieczna jest konwersji z ciągu znaków na wartości liczbowe aby wykonywac obliczenia.

parseFloat(zmienna) - konwersja zmiennej do typu float
parseInt(zmienna) - konwersja zmiennej do typu int (liczbowego)
toString(zmienna) - konwersja zmiennej do typu łańcuchowego
========== **Koniec opisu teorii** ==========

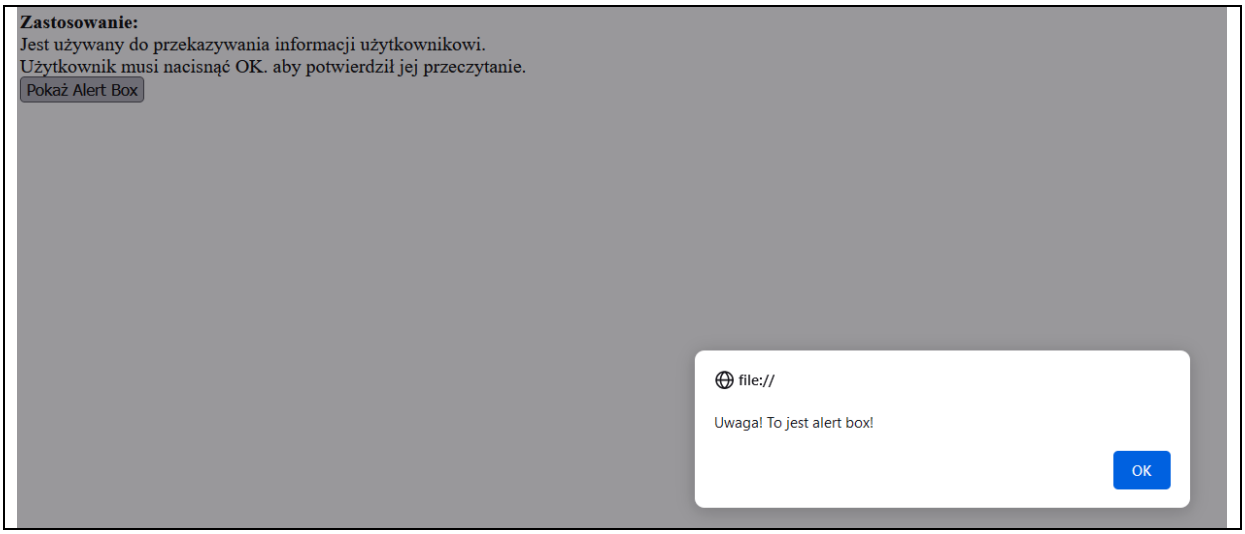
Zadanie 7.

Temat: **Alert Box**

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie7) uruchamiający Alert Box
2)Dokonaj kopiowania kodu do pliku o nazwie zadanie7_kow.html
3)Dopisz teorię
4)Podłącz plik zadanie7_kow.html do przycisku Nazwisko_ucznia-Zadanie7
5)Dokonaj testowania tak aby otrzymać efekt przedstawiony poniżej.

Zastosowanie:
Jest używany do przekazywania informacji użytkownikowi. Użytkownik musi nacisnąć ok. aby potwierdził jej przeczytanie.

```
<!DOCTYPE html>
  <head>
    <script type="text/javascript"> function show_alert() { alert("Uwaga!"); } </script>
  </head>
 <body>
      <input type="button" onclick="show_alert()" value="Pokaż Alert Box" />
 </body>
</html>
```



Zadanie 8.

Temat: **Confirm Box**

Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie8) uruchamiający Confirm Box

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie8) uruchamiający Confirm Box
2)Dokonaj kopiowania kodu do pliku o nazwie zadanie8_kow.html
3)Dopisz teorię
4)Podłącz plik zadanie8_kow.html do przycisku Nazwisko_ucznia-Zadanie8
5)Dokonaj testowania tak aby otrzymać efekt przedstawiony poniżej.

Zastosowanie:
Jest używany, jeśli chcemy zapytać użytkownika o weryfikacje, lub akceptację jakichś danych. Użytkownik musi nacisnąć „OK.” w celu akceptacji bądź „Cancel” w celu odrzucenia. Jeżeli zostanie wybrane „OK.” metoda zwraca true w przeciwnym wypadku false.

```
<!DOCTYPE html>
<head>
<script type="text/javascript">
  function show_confirm()
    {
      var r=confirm("Naciśnij klawisz");
       if (r==true)
          {  document.write("Nacisnąłeś OK!");  }
       else
        {  document.write("Nacisnąłeś Cancel!");  }
    }
</script>
</head>
 <body>
   <input type="button" onclick="show_confirm()" value="Pokaż confirm box" />
 </body>
</html>
```

Zadanie 9.

Temat: **Prompt Box**

Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie6) uruchamiający Prompt Box
Nie musisz wykonywać przycisku do kodu zadania6.

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie9)
2)Dokonaj kopiowania kodu do pliku o nazwie zadanie9_kow.html
3)Dopisz teorię
4)Podłącz plik zadanie9_kow.html do przycisku Nazwisko_ucznia-Zadanie9
5)Dokonaj testowania

Zastosowanie:
Jest używany kiedy użytkownik ma wpisać coś np. zanim wejdzie w dany moduł strony. Np. hasło. Użytkownik po wpisaniu wartość może nacisnął „OK.” albo „Cancel” Jeżeli użytkownik kliknie „OK.” metoda zwraca wpisaną wartość, a w przeciwnym wypadku zwraca null.

```
<!DOCTYPE html>
 <head>
   <script type="text/javascript">
    function show_prompt()
      {
        var name=prompt("Napisz imię","Imię Nazwisko");
          if (name!=null && name!="")
            {
              document.write("Witaj " + name + "! jak się masz?");
            }
      }
    </script>
</head>
 <body>
  <input type="button" onclick="show_prompt()" value="Pokaż prompt box" />
 </body>
</html>
```

Operator	Działanie operatora
+	dodawanie liczb oraz klejenie łańcuchów (konkatenacja)
-	odejmowanie liczb
*	mnożenie liczb
/	dzielenie liczb
++	inkrementacja (zwiększenie liczby o dokładnie 1)
--	dekrementacja (zmniejszenie liczby o dokładnie 1)
%	reszta z dzielenia (tzw. modulo), na przykład: 27 % 6 jest równe 3, gdyż cztery szóstki mieszczą się w 27 (co daje 24 i reszty zostaje 3)
=	przypisanie wartości do zmiennej
==	porównanie wartości
<	mniejsze od
<=	mniejsze lub równe od
>	większe od
>=	większe lub równe od
!	zaprzeczenie (negacja)
&&	logiczne "i" (and)
||	logiczne "lub" (or)
+=	skrócony zapis, np. a += b odpowiada: a = a + b
-=	skrócony zapis, np. a -= b odpowiada: a = a - b
*=	skrócony zapis, np. a *= b odpowiada: a = a * b
/=	skrócony zapis, np. a /= b odpowiada: a = a / b
%=	skrócony zapis, np. a %= b odpowiada: a = a % b

Zadanie 10.

Temat: Wczytanie dwóch liczb oraz obliczenie sumy tych liczb.

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie10)
2)Dokonaj kopiowania kodu do pliku o nazwie zadanie10_kow.html
3)Podłącz plik zadanie10_kow.html do przycisku Nazwisko_ucznia-Zadanie10
4)Dokonaj testowania

```
<!DOCTYPE html>
  <head>
      <title> proste obliczenia</title>
      <meta charset="utf-8">
   </head>
 <body>
         <div style="color:red;font-size:50px;">
           <script language="JavaScript">
                  var liczba1=prompt("liczba1=","");
                  var liczba2=prompt("liczba2=","");
                  var suma;
                   var a=parseFloat(liczba1);
                   var b=parseFloat(liczba2);
                    suma=a+b;
                    document.write("suma="+a+" + "+b+" = "+suma+"<br>");
            </script>
         </div>
 </body>
</html>
```

Zadanie 11.

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie11)
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie11-KOD)
3)Przeczytaj treść zadania, rozwiąż to zadanie. (pamiętaj o odpowiednich nazwach)
4)Dokonaj testowania zadania
5)Podłącz plik zadanie11_kow.html do przycisku Nazwisko_ucznia-Zadanie11
6)Wykonaj stronę wyświetlająca kod programu, możesz przy użyciu TEXTAREA o nazwie zadanie11_kow_KOD.html (patrz opis poniżej oraz przykład)
7)Podłącz plik zadanie11_kow_KOD.html do przycisku Nazwisko_ucznia-Zadanie11-KOD
8)Zmień wygląd szkieletu (patrz informacje poniżej)

Temat: Ile sekund.

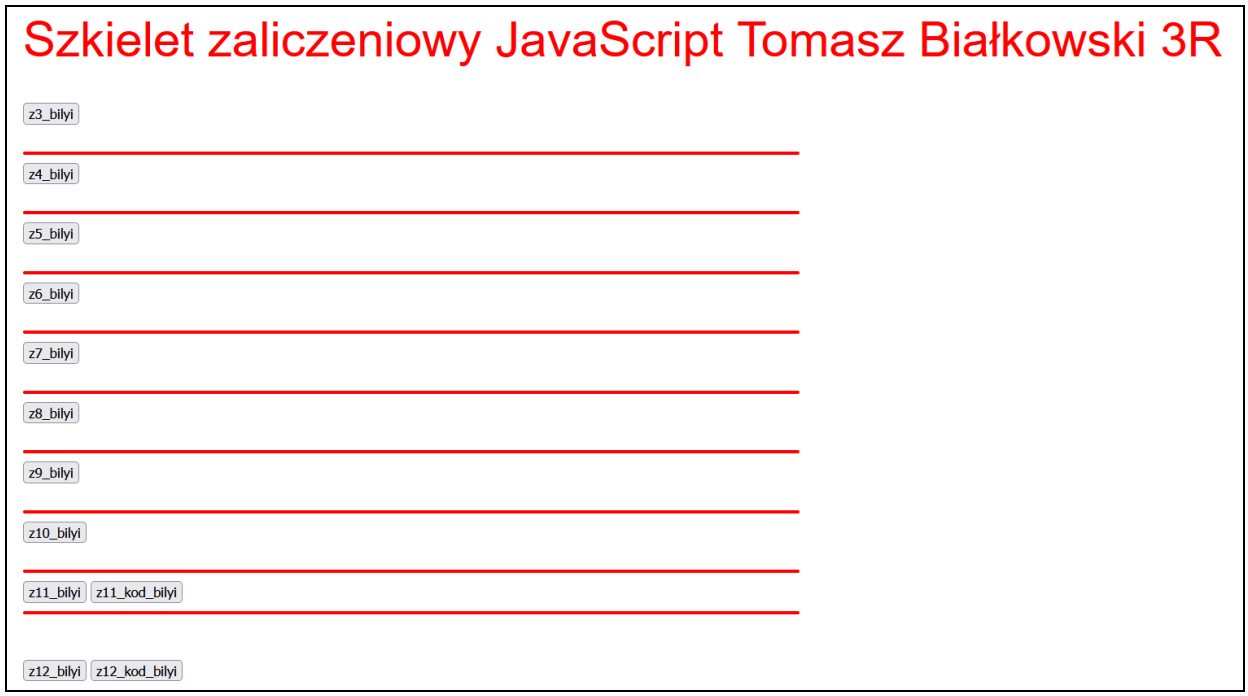
Wczytać dwa numer dni z miesiąca stycznia D1_kow, D2_kow (trzy litery nazwiska ucznia) i obliczyć ile sekund minęło między tymi dniami dla północy obu dni. Program nie sprawdza poprawności wczytania danych, czyli D1>=D2 1<=D1,D2<=31 D1,D2 należy do naturalnych. Kolor wyświetlania zielony wielkość znaków 30+numer_dnia urodzenia.

Dane do sprawdzenia:
D1=30
D2=28
ilość sekund= 172800 sekund

Wypisywanie KODu do zadań

Zmień wygląd szkieletu.

1. Tak aby od zadania11 był przycisk do wykonania i obok przycisk kodu
2. Wstaw linie rozdzielające. (patrz poniżej)



Przykład jak wykonać przyciski do zadania i kodu.

```
<html>
 <head>
 <title>Tomasz Białkowski</title>
<p align="left"> <font color="red" size="7" face="Arial"> Szkielet zaliczeniowy JavaScript Tomasz
```

```
Białkowski  3R </font> </p> <br>
 <script language="JavaScript">
function WinOpen_z3()
 {
window.open("z3_bil.html","okienko_z3","toolbar=no,directories=no,menubar=no,height=380,width
=160,top=200,left=200");
 }
function WinOpen_z4()
 {
window.open("z4_bil.html","okienko_z4","toolbar=no,directories=no,menubar=no,height=380,width
=160,top=200,left=400");
 }
function WinOpen_z5()
 {
window.open("z5_bil.html","okienko_z5","toolbar=no,directories=no,menubar=no,height=380,width
=160,top=200,left=400");
 }
function WinOpen_z6()
 {
window.open("z6_bil.html","okienko_z6","toolbar=no,directories=no,menubar=no,height=380,width
=160,top=200,left=400");
 }
 function WinOpen_z7()
 {
window.open("z7_bil.html","okienko_z7","toolbar=no,directories=no,menubar=no,height=380,width
=160,top=200,left=200");
 }
function WinOpen_z8()
 {
window.open("z8_bil.html","okienko_z8","toolbar=no,directories=no,menubar=no,height=380,width
=160,top=200,left=400");
 }
function WinOpen_z9()
 {
window.open("z9_bil.html","okienko_z9","toolbar=no,directories=no,menubar=no,height=380,width
=160,top=200,left=400");
 }
function WinOpen_z10()
 {
window.open("z10_bil.html","okienko_z10","toolbar=no,directories=no,menubar=no,height=380,wi
dth=160,top=200,left=400");
 }
function WinOpen_z11()
 {
window.open("z11_bil.html","okienko_z11","toolbar=no,directories=no,menubar=no,height=380,wi
dth=160,top=200,left=400");
 }
function WinOpen_z11_kod()
 {
window.open("z11_kod.html","okienko_z11_kod","toolbar=no,directories=no,menubar=no,height=3
80,width=160,top=200,left=400");
 }
```

```
function okno_zamknij()
 {
 window.close()
 }
 </script>
 </head>
<body>
 <form>
 <input type="button" name="z3" value="z3_bilyi" onclick="WinOpen_z3(' ')"> <br><br>
<HR SIZE=4 WIDTH=50% ALIGN=LEFT COLOR=red>
 <input type="button" name="z4" value="z4_bilyi" onclick="WinOpen_z4(' ')"> <br><br>
<HR SIZE=4 WIDTH=50% ALIGN=LEFT COLOR=red>
 <input type="button" name="z5" value="z5_bilyi" onclick="WinOpen_z5(' ')"> <br><br>
<HR SIZE=4 WIDTH=50% ALIGN=LEFT COLOR=red>
 <input type="button" name="z6" value="z6_bilyi" onclick="WinOpen_z6(' ')"> <br><br>
<HR SIZE=4 WIDTH=50% ALIGN=LEFT COLOR=red>
 <input type="button" name="z7" value="z7_bilyi" onclick="WinOpen_z7(' ')"> <br><br>
<HR SIZE=4 WIDTH=50% ALIGN=LEFT COLOR=red>
 <input type="button" name="z8" value="z8_bilyi" onclick="WinOpen_z8(' ')"> <br><br>
<HR SIZE=4 WIDTH=50% ALIGN=LEFT COLOR=red>
 <input type="button" name="z9" value="z9_bilyi" onclick="WinOpen_z9(' ')"> <br><br>
<HR SIZE=4 WIDTH=50% ALIGN=LEFT COLOR=red>
 <input type="button" name="z10" value="z10_bilyi" onclick="WinOpen_z10(' ')"> <br><br>
<HR SIZE=4 WIDTH=50% ALIGN=LEFT COLOR=red>
 <input type="button" name="z11" value="z11_bilyi" onclick="WinOpen_z11(' ')">
 <input type="button" name="z11" value="z11_kod_bilyi" onclick="WinOpen_z11_kod(' ')">
<HR SIZE=4 WIDTH=50% ALIGN=LEFT COLOR=red>
<br><br>
<input type="button" value="zamknij okno" onclick="okno_zamknij()"/>
 </form>
</body>
</html>
```

<u>Uwagi dotyczące wyświetlania kodu zadania w szkielecie</u>

1. Nazwa pliku zad11_kod_bialkowski.html
2. Okienko wyświetlania powinno być tak dobrane aby było dopasowane do długości i szerokości, tutaj to <textarea cols="90" rows="24"> 90 kolumn i 24 wierszy (patrz rzut ekranu poniżej).
3. W pliku wstaw w komentarzu Imię Nazwisko klasa oraz data i w nowym wierszu numer zadania np.

// Tomasz Białkowski 3R 14.09.2029
// Zadanie 11
Używamy instrukcji textarea z odpowiednimi parametrami. Poniżej <textarea cols="90" rows="24"> wpisujemy kod, koniec to </textarea>

początek i koniec instrukcji textarea
kod który ma być wyświetlony

```
<html>
  <body>
   <textarea cols="90" rows="24">
     <html>
      <body>
       <div style="color:green;font-size:50px;">
        <script language="JavaScript">
// Tomasz Białkowski  3R   14.09.2029
// Zadanie 11
          var D1_bil=prompt("Data1=","");
          var D2_bil=prompt("Data2=","");
          var sekund;
          var a=parseFloat(D1_bil);
          var b=parseFloat(D2_bil);
           sekund=(a-b)*24*60*60;
          document.write("Ilość sekund między dniami "+a+" i "+b+" = "+sekund+"<br>");
           function okno_zamknij_bilyi()
           {
             window.close()
           }
          </script>
        </div>
       <input type="button" value="zamknij okno" onclick="okno_zamknij_bilyi()"/>
      </body>
     </html>
   </textarea>
  </body>
</html>
```

Rzut pokazujący jak wyświetlić kod zadania

```
        <html>
         <body>
          <div style="color:green;font-size:50px;">
            <script language="JavaScript">
// Tomasz Białkowski  3R   14.09.2029
// Zadanie 11
              var D1_bil=prompt("Data1=","");
              var D2_bil=prompt("Data2=","");
              var sekund;
              var a=parseFloat(D1_bil);
              var b=parseFloat(D2_bil);
                sekund=(a-b)*24*60*60;
              document.write("Ilość sekund między dniami "+a+" i "+b+" = "+sekund+"<br>");
                function okno_zamknij_bilyi()
                 {
                   window.close()
                 }
              </script>
            </div>
          <input type="button" value="zamknij okno" onclick="okno_zamknij_bilyi()"/>
         </body>
        </html>
```

Zadanie 12.

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie12)
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie12-KOD)
3)Przeczytaj treść zadania, rozwiąż to zadanie. (pamiętaj o odpowiednich nazwach)
4)Dokonaj testowania zadania
5)Podłącz plik zadanie12_kow.html do przycisku Nazwisko_ucznia-Zadanie12
6)Podłącz plik zadanie12_kow_KOD.html do przycisku Nazwisko_ucznia-Zadanie12-KOD

Temat: Ile sekund→modyfikacja zadania 11.

Wczytać dwa numer dni z miesiąca stycznia D1_kow, D2_kow (trzy litery nazwiska ucznia) oraz dwie godziny (G1_kow, G2_kow) obliczyć ile sekund minęło między tymi dniami i godzinami. Program nie sprawdza poprawności wczytania danych czyli D1>=D2 1<=D1,D2<=31 1<=G1,G2<=24 D1,D2,G1,G2 należy do naturalnych. Kolor wyświetlania zielony wielkość znaków 30+numer_dnia urodzenia.

Dane do sprawdzenia:
D1=30
G1=4
D2=28
G2=22
ilość sekund= 108000 sekund

Zadanie 13.

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie13)
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie13-KOD)
3)Przeczytaj treść zadania, rozwiąż to zadanie. (pamiętaj o odpowiednich nazwach)
4)Dokonaj testowania zadania
5)Podłącz plik zadanie13_kow.html do przycisku Nazwisko_ucznia-Zadanie13
6)Podłącz plik zadanie13_kow_KOD.html do przycisku Nazwisko_ucznia-Zadanie13-KOD

Temat: Rozkład ułamka egipskiego na dwa ułamki egipskie.

Użyj instrukcji document.write i prompt do wczytania wartości zmiennej **m_kow**.

Opis teoretyczny.
Ułamki egipskie są to inaczej ułamki proste- ułamki postaci

$$\frac{1}{m}$$

gdzie m jest liczbą naturalną większą od 0.
Ułamek ten jest rozkładany na ułamki, które mają w liczniku też jeden. Stosujemy wzór:

$$\frac{1}{m} = \frac{1}{(m+1)} + \frac{1}{m*(m+1)}$$

Dane do sprawdzenia:
m=4

1/4=1/5+1/20
Dane przykłady ułamków egipskich
Teraz przedstawimy liczbę 1 za pomocą sumy różnych ułamków.

Otrzymujemy:

$$1=\frac{1}{2}+\frac{1}{3}+\frac{1}{6}$$

$$1=\frac{1}{2}+\frac{1}{3}+\frac{1}{7}+\frac{1}{42}$$

$$1=\frac{1}{2}+\frac{1}{3}+\frac{1}{7}+\frac{1}{43}+\frac{1}{1806}\ \text{itd.}$$

Zadanie 14.

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie14)
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie14-KOD)
3)Przeczytaj treść zadania, rozwiąż to zadanie. (pamiętaj o odpowiednich nazwach)
4)Dokonaj testowania zadania
5)Podłącz plik zadanie14_kow.html do przycisku Nazwisko_ucznia-Zadanie14
6)Podłącz plik zadanie14_kow_KOD.html do przycisku Nazwisko_ucznia-Zadanie14-KOD

Temat: Rozkład ułamka egipskiego na trzy ułamki egipskie.

Wskazówka:
Rozłóż najpierw na dwa ułamki i następnie drugi lub pierwszy rozłóż na dwa a otrzymasz rozkład na trzy.

Dane do sprawdzenia:
m=4

1/4=1/6 + 1/30 + 1/20

Zadanie 15.

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie15)
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie15-KOD)
3)Przeczytaj treść zadania, rozwiąż to zadanie. (pamiętaj o odpowiednich nazwach)
4)Dokonaj testowania zadania
5)Podłącz plik zadanie15_kow.html do przycisku Nazwisko_ucznia-Zadanie15
6)Podłącz plik zadanie15_kow_KOD.html do przycisku Nazwisko_ucznia-Zadanie15-KOD

Temat: Rozkład ułamka egipskiego na dwa takie same ułamki egipskie.

Dane do sprawdzenia:
m=3

1/3=1/6 + 1/6

m=7

1/7=1/14+1/14

Zadanie 16.

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie16)
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie16-KOD)
3)Przeczytaj treść zadania, rozwiąż to zadanie. (pamiętaj o odpowiednich nazwach)
4)Dokonaj testowania zadania

5)Podłącz plik zadanie16_kow.html do przycisku Nazwisko_ucznia-Zadanie16
6)Podłącz plik zadanie16_kow_KOD.html do przycisku Nazwisko_ucznia-Zadanie16-KOD

Temat: Sterowanie wydrukami.

Użyj instrukcji document.write i prompt. Napisz program obliczając, który po podaniu dwóch liczb naturalnych wykona obliczenia według wzoru: $(A+B)^3 = A^3 + 3A^2B + 3AB^2 + B^3$
Kolor liter granatowy a ich wielkość tak aby druga linia wydruków skryptu zajmowała całą szerokość ekranu.

Wygląd ekranu dla okienek prompt
Podaj pierwszą liczbę A=
Podaj drugą liczbę B=

Wygląd ekranu
wczytane liczby: A=1 B=2

wygląd ekranu dla A=1 B=2 np.
(1 + 2)^3= 1^3 + 3*((1)^2)*(2) + 3*(1)*((2)^2) + 2^3 = 1 + 6 + 12 + 8 = 27
uwaga: przy innych danych A i B liczby w napisie będą się zmieniać

Dział 4

Sterowanie wydrukami. Obliczenia część druga. Obiekty math

Zadanie 17.

Temat: Zadanie teoretyczne.

Wykonaj:

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie17)
2)Przeczytaj treść zadania, rozwiąż to zadanie. (pamiętaj, że jest to zadanie teoretyczne)
4)Dokonaj testowania zadania
5)Podłącz plik zadanie17_kow.html do przycisku Nazwisko_ucznia-Zadanie13
6)Podłącz plik zadanie17_kow_KOD.html do przycisku Nazwisko_ucznia-Zad

Treść zadania:

Udziel odpowiedzi na następujące pytania teoretyczne: (zastosuj pogrubienia oraz podkreślenia podczas udzielania odpowiedzi, najpierw pytanie potem odpowiedź).

Pytania:

1. Wymień cztery sposoby wyprowadzania danych w JS,
2. Czym jest funkcja w JS, omów przy użyciu 4 podpunktów?
3. Omów metody **getElementsByName, getElementById, getElementByTagName** (może być w postaci tabeli),
4. Omów właściwości **innerHTML innerText className.** (każdą osobno)

==========Początek opisu teorii==========

Cztery sposoby **wyprowadzania** danych na stronę:

- z użyciem dokument.write()
- z użyciem okienka alert()
- do okienek formularza
- do tabeli

Co to jest funkcja?

- Jest to zbiór zgrupowanych instrukcji, które możemy uruchamiać poprzez podanie ich nazwy,
- Każda funkcja po wywołaniu wykonuje swój wewnętrzny kod,
- Funkcja może mieć argumenty, które wchodzą do wnętrza funkcji i są tam przetwarzane np. suma_kry(a, b),
- Może(nie zawsze) zwrócić obliczoną wartość zapisaną po słowie kluczowym **return**.

Wybrane metody obiektu document

Metoda	opis
getElementById("id ")	dostęp do wszystkich elementów o konkretnej wartości atrybutu id
getElementByTagName("znacznik ")	dostęp do wszystkich elementów o typie znacznika, np. P lub DIV
getElementByName("nazwa")	dostęp do wszystkich elementów o konkretnej wartości atrybutu

Właściwości tak pobranego elementu to:

- **innerHTML** – jest to właściwość, która umożliwia odczyt, zmodyfikowanie, wstawienie, albo usunięcie kodu HTML z danego elementu HTML np. w tabeli,
- **innerText** – umożliwia określenie ciągu znaków który zostanie umieszczony w elemencie,
- **className** – umożliwia określenie klasy CSS użytej do prezentacji elementu.

Przykład

```
<!DOCTYPE html>
  <head>
      <title> Przykłady dostępu do elementów </title>
      <meta charset="utf-8">
         <style type="text/css">
             .wyroznienieCzerwone { color: Red; font-weight: 600; }
        </style>
   </head>
  <body>
     <div id="nazwa"></div>
  <script>
      document.getElementById("nazwa").innerHTML = "Witaj";
      document.getElementById("nazwa").className = "wyroznienieCzerwone";
 </script>
</body>
```

========== **Koniec opisu teorii** ==========

Zadanie 18.
Temat: Definiowanie oraz użycie funkcji w JS

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie18)
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie18-KOD)
3)Przeczytaj treść zadania, rozwiąż to zadanie. (pamiętaj o odpowiednich nazwach)
4)Dokonaj testowania zadania
*uruchom przykład i zmień nazwę z:
suma_kry na **suma_trzy_pierwsze_liter_nazwisko_małe_litery_bez_polskich_liter**
*wykonaj funkcje:
odejmowanie_xxx(?,?)
mnozenie_xxx(?,?)
od_kwadratu_xxx(?,?) → do kwadratu pierwszy parametr oraz do kwadratu drugi parametr
np.
od_kwadratu_xxx(3,4)
3^2 = 9
4^2= 16
reszta_z_dzielenia_a_przez_b_xxx(?,?)
xxx- **>trzy_pierwsze_liter_nazwisko_małe_litery_bez_polskich_liter**
*wyprowadź wyniki dla czterech nowych funkcji.
5)Podłącz plik zadanie18_kow.html do przycisku Nazwisko_ucznia-Zadanie18
6)Podłącz plik zadanie18_kow_KOD.html do przycisku Nazwisko_ucznia-Zadanie18-KOD

```
<!DOCTYPE html>
  <head>
      <title> Użycie fukcji </title>
      <meta charset="utf-8">
   </head>
<body>
      <div style="color:navy;font-size:50px;">
        <script language="JavaScript">
            var a=parseFloat(prompt("a=",""));
            var b=parseFloat(prompt("b=",""));

           function suma_kry(a, b) {
                return a + b;
            }

          document.write("suma="+suma_kry(a,b)+"<br>");

</script>
</div>
</body>
</html>
```

Zadanie 19.

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie19)
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie19-KOD)
3)Przeczytaj treść zadania.
4)Dokonaj kopiowania zadania.
5)Zmień
Nazwy funkcji:
Pole_kowalski()
obwod_kowalski()
nazwy zmiennych
a_kow
b_kow
6)Dokonaj testowania zadania
7)Podłącz plik zadanie19_kow.html do przycisku Nazwisko_ucznia-Zadanie19
8)Podłącz plik zadanie19_kow_KOD.html do przycisku Nazwisko_ucznia-Zadanie19-KOD

Temat: Wczytywanie danych w okienku. Wykonywanie obliczeń.

Treść zadania.
Obliczanie pola i obwodu kwadratu z użyciem funkcji, przycisków.
Wygląd ekranu.

Wszystko o KWADRACIE

Podaj długość boku: 5.67 cm

Oblicz pole

Oblicz obwod

Pole kwadratu= 32.15 cm^2

Wytłumaczenie kodu.

var a= parseFloat(document.getElementById('dlugosc').value);
Pobierz do zmiennej **a** z elementu o nazwie Id ‘dlugosc’ jego wartość (.value)

document.getElementById("nazwa").innerHTML ="Pole kwadratu= "+b.toFixed(2)+" cm^2";
Wypisz w elemencie o Id "nazwa" z użyciem metody **.innerHTML** ciąg znaków "Pole kwadratu= "+b.toFixed(2)+" cm^2"; Uwaga: toFixed(2) oznacza metodę dwa miejsca popprzecinku.

Kod programu

```
<html>
 <head>
  <meta charset="UTF-8">
  <link rel="stylesheet" href="styl.css">
 </head>
<body>
 <script type="text/javascript">
  function pole()
  {
            var a= parseFloat(document.getElementById('dlugosc').value);
            var b= a*a;
      document.getElementById("nazwa").innerHTML ="Pole kwadratu= "+b.toFixed(2)+" cm^2";
  }
  function obwod()
  {
            var a= parseFloat(document.getElementById('dlugosc').value);
            var b= 4*a;
      document.getElementById("nazwa").innerHTML ="Obwód kwadratu= "+b.toFixed(2)+" cm";
  }
 </script>
  <h1>Wszystko o KWADRACIE</h1><br><br>
  Podaj długość boku:        <input type="text" id="dlugosc" value="podaj długość"> cm<br><br><br>
  <input type="button" name="pole" value="Oblicz pole" Onclick="pole()"><br><br>
  <input type="button" name="obwod" value="Oblicz obwod" Onclick="obwod()"><br><br>
  <div  style="color:red;font-size:50px;"id="nazwa"></div>
</body>
</html>
```

Zadanie 20.

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie20)
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie20-KOD)
3)Przeczytaj treść zadania.
4)Rozwiąż zadanie.
a)Nazwy funkcji:
function AB_kry()
function BC_kry()
function AC_kry()
b)Nazwy zmiennych
A(xa_kow,ya_kow) B(xb_kow,yb_kow) C(xc_kow,yc_kow)
c)Wczytaj punkty w okienkach (sześć okienek).
d)Wypisz obliczone długości boków.
5)Dokonaj testowania zadania.
7)Podłącz plik zadanie20_kow.html do przycisku Nazwisko_ucznia-Zadanie20
8)Podłącz plik zadanie20_kow_KOD.html do przycisku Nazwisko_ucznia-Zadanie20-KOD

Treść zadania

Wczytaj trzy punkty nie współliniowe. Punkty tworzą trójkąt. A(xa_kow,ya_kow) B(xb_kow,yb_kow) C(xc_kow,yc_kow) z użyciem sześciu okienek formularza (w poprzednim zadaniu było jedno okienko do wczytaniu boku kwadratu). Oblicz z użyciem funkcji długości boków AB BC AC.
Wskazówka: odległość między punktami F(x1,y1) oraz G(x2,y2)
$|FG| = \sqrt{(x1 - x2)^2 + (y1 - y2)^2}$
Do obliczeń użyj modułu Math. Przejście do obiektów Math.
var a = Math.sqrt(4); pierwiastek z 2
var a = Math.pow(4,2); 4 do potęgi 2 czyli do kwadratu
Wypisz obliczone długości boków.

Zadanie 21.

Temat: Użycie formularza do obliczeń pola powierzchni oraz objętości walca.

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie21)
2)Przeczytaj temat zadania i przeanalizuj kod zadania.
3)Dokonaj testowania zadania
4)Podłącz plik zadanie21_kow.html do przycisku Nazwisko_ucznia-Zadanie21

```
<html>
<head>
<title>
 Przykład obliczenia V  i  Pc walca
</title>
 <script language="JavaScript">
function V(r,h)
{
   r_liczba=parseFloat(r)
   h_liczba=parseFloat(h)
   V_wynik=Math.PI*r_liczba*r_liczba*h_liczba
   document.getElementsByName('obj')[0].value=V_wynik.toFixed(2);   /* na dokument  */
   alert("V="+V_wynik.toFixed(2)+" cm^3");                          /*do okna Alert */
   pokaz1.innerHTML="V="+V_wynik.toFixed(2)+" cm^3"+"<br>";       /* do tabeli */
```

```
 }
function Pc(r,h)
{
   r_liczba=parseFloat(r)
   h_liczba=parseFloat(h)
  Pc_wynik=2*Math.PI*r_liczba*r_liczba+2*Math.PI*r_liczba*h_liczba
  document.getElementsByName('pole')[0].value=Pc_wynik.toFixed(2);
  alert("Pc="+Pc_wynik.toFixed(2)+" cm^2");
  pokaz2.innerHTML="Pc="+Pc_wynik.toFixed(2)+"cm^2";
}
     </script>
</head>
<body>
 Pobliczanie V   i    Pc walca
<br> <br>
<form name="formularz" action="..." onsubmit="V(this.promien.value,this.wysokosc.value);
Pc(this.promien.value,this.wysokosc.value);return false">
    Podaj r=
      <input  size="6"  name="promien">
    Podaj h=
      <input  size="6"  name="wysokosc">
                <br>  <br>
       <input  TYPE="submit"  Value="oblicz" >
                <br>  <br>
     V=
        <input  size="6"  name="obj">
     Pc=
       <input  size="6"  name="pole">
</form>
<table>
          <tr id="pokaz1"></tr>
          <tr id="pokaz2"></tr>
</table>
</body>
</html>
```

Zadanie 22.

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie22)
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie22-KOD)
3)Przeczytaj temat oraz treść zadania.
4)Rozwiąż zadanie.
Uwaga: proszę zwrócić uwagę na nazwy zmiennych (powinny mieć trzy litery nazwiska ucznia)
Nazwy funkcji:
Pole_kowalski(a_kow,b_kow)
obwod_kowalski(a_kow,b_kow)
nazwy pól do wczytywania danych:
a_dana_kow
b_dana_kow
nazwy pól tabeli
wyprowadz_pole_kow
wyprowadz_obwod_kow
nazwy pól Input do wyprowadzania pola i obwodu

pole
obw

5)Dokonaj testowania zadania
6)Podłącz plik zadanie20_kow.html do przycisku Nazwisko_ucznia-Zadanie22
7)Podłącz plik zadanie20_kow_KOD.html do przycisku Nazwisko_ucznia-Zadanie22-KOD

Temat: Obliczenie pola oraz obwodu prostokąta.

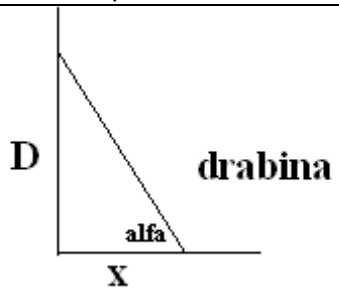
Napisz skrypty, który obliczy z użyciem funkcji
a) pole,
b) obwód,
Prostokąta

Zadanie 23.

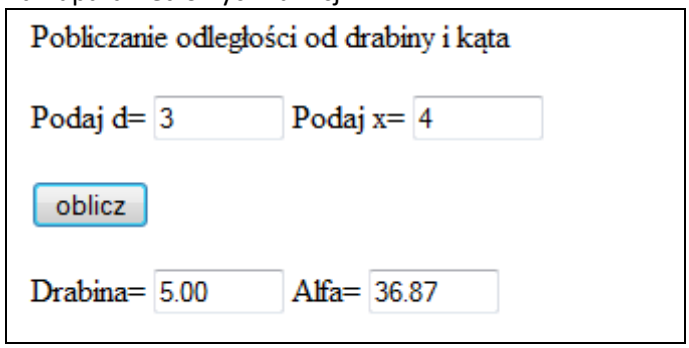
Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie23) uruchamiający zadanie23
Proszę wykonać przycisk **do kodu** zadania23.

Temat: Używając formularzy do wpisywania danych oraz do wypisywania rozwiąż zadanie jak poniżej.

Drabina stoi oparta o mur

	Wczytaj: D-wysokość oparcia drabiny X- odległość oparcia drabiny Oblicz: długość drabiny [m] kąt nachylenia drabiny w [stopniach]

Do obliczenia długości drabiny użyj twierdzenia Pitagorasa. Do obliczenia kata użyj funkcji arcustangens. Pamiętaj o zamianie radianów na stopnie. Do obliczeń użyj definiowanych własnych dwuparametrowych funkcji.



Zadanie 24.

Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie24) uruchamiający zadanie24
Proszę wykonać przycisk **do kodu** zadania24.

Temat: Obliczenia z użyciem Math

Wykonaj:
1)Wczytaj dwie liczby x1_kow i x2_kow z użyciem prompt
2)Wypisz wczytane dane:

Wygląd ekranu
wczytane liczby: x1=1 (pięć spacji) x2=2 (na ekranie ma się pojawić wczytane wartości x1 i x2)
większą ilość spacji uzyskujemy poprzez:

3)Oblicz i wypisz

$$y1 = 3 * x1^{\frac{liczba\ \ liter\ \ imienia}{liczba\ \ liter\ \ imienia\ +2}}$$

$$y1 = 3 * x1^{\frac{liczba\ liter\ imienia}{liczba\ liter\ imienia+2}}$$

Uwaga: x1 jest podnoszone do potęgi.
Wyświetl y1, y2 i y3 z dwoma miejscami po kropce (w JS części dziesiętne pisane są po kropce).
Obliczenia→ dwa miejsca po przecinku metoda .toFixed(2)
Podajemy nazwę zmiennej a następnie .toFixed(2) {kropka konieczna}
Np. +x.toFixed(2);

4)Oblicz i wypisz

$$y2 = 2 * \cos(x1) + 3 * \sin(x1)$$

$$y2 = 2 * \cos(x1) + 3 * \sin(x1)$$

Pamiętaj, że x1 będzie wczytane w stopniach i konieczne jest zamienienie ze stopni na radiany

5)Wyświetl dodatkowo (ponownie) x1[w stopniach] i x1[radianach]
Wygląd ekranu
wczytane liczby:
w stopniach x1=30 [stopniach] x1=0.52 [radianach]

rad – wartość w radianach
st - wartość w stopniach

```
rad=(st*Math.PI)/180;
```

5) Oblicz i wypisz

$$\boldsymbol{y3 = \sqrt[3]{(x1)^5 + 2 * (x2)^4 * |x1 - x2|}}$$

$$y3 = \sqrt[3]{(x1)^5 + 2 * (x2)^4 * |x1 - x2|}$$

6)Wygląd ekranu po wykonaniu zadania

```
x1 = 30     x2 = 2
x1 = 30     x1 = 0.52
y1 = 38.46
y2 = 3.23
y3 = 8110.11
```

Zadanie 25. (na celującą)

Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie25) uruchamiający zadanie25
Proszę wykonać przycisk **do kodu** zadania25.

Temat: Napisz program obliczający ilość atomów pierwiastka promieniotwórczego N [sztuk]
po czasie t [rok] oraz czasu połowicznegoT1/2 rozpadu danego pierwiastka.

Przejście do obiektów Math.

Dane wejściowe:
No, t, T1/2
Dane wyjściowe
λ, N

Najpierw oblicz λ i wyprowadź na ekran ze wzoru:

Następnie oblicz N

$$\lambda = \frac{\ln(2)}{T_{1/2}}$$

$$N = N_o * e^{-\lambda * t}$$

gdzie:
e - liczba Eulera
ln(x) - logarytm naturalny
T1/2 – okres połowicznego zaniku, (czas po jakim pozostanie połowa atomów)

T1/2- dla izotopu węgla C14 wynosi 5700 lat
λ [1/ rok], stała rozpadu promieniotwórczego (określa prawdopodobieństwo zajścia rozpadu jednego jądra w jednostce czasu),

N_o [sztuk]-ilość początkowa atomów pierwiastka rozpadu.

1)Wczytaj dane→prompt.
2)Wypisz wczytane dane oraz wyniki z jednostkami, dobierz kolor oraz wielkość(nie stosuj standardowych).
3)Użyj formatowania poprzez toFixed(2)(patrz przykład następny) danych dwa miejsca po przecinku dla lambdy siedem miejsc.
4)Użyj komentarzy→podaj autora.
5)do testowania użyj T1/2dla węgla, wielkość masz w treści zadania oraz dla radu (odszukaj w necie)
6) Użyj obiekty **Obiekty Math** (patrz opis w dalszej części instrukcji) np. e to Math.E podnoszenie do potęgi x^y to Math.pow(x,y)



Zadanie 26. (na celującą)

Wykonaj przycisk uruchamiający następujące zadanie.

Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie26) uruchamiający zadanie26
Proszę wykonać przycisk **do kodu** zadania26.

Temat: Obliczenie pola oraz obwodu trapez równoramiennego po podaniu górnej i dolnej podstawy oraz wysokości.

Uwagi:
1)Użyj dwóch funkcji z odpowiednią ilością parametrów,
2)Nazwy funkcji to **Pow_kowalski(parametry)** oraz **Obw_kowalski(parametry),**
3)W celu uniknięcia błędu, która podstawa jest dłuższa użyj wartości bezwzględnej,
4)ramie oblicz z twierdzenia pitagorasa,
5)wprowadzaj dane z użyciem formularza,
5)Wyprowadzanie danych z użyciem document.write oraz formularza.

Zadanie 27. (Na celującą)

Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie27) uruchamiający zadanie27
Proszę wykonać przycisk **do kodu** zadania27.

Temat: Używając formularzy do wpisywania danych oraz do wypisywania rozwiąż zadanie jak poniżej.

Artur i Dawid stoją w odległości odl_kow (taka zmienna do, której wczytujemy daną). Artur jest na lotnisku z którego startuje samolot pod pewnym katem (kat_kow). Samolot gdy znajduje się pionowo nad Dawidem przebył drogę s_kow (taka zmienna do, której wczytujemy daną). Oblicz na jakiej wysokości h_kow oraz jaki był kąt startu samolotu. Do obliczeń użyj definiowanych własnych dwuparametrowych funkcji.

Dział 5 Programowanie obiektowe

Zadanie 28.
Wykonaj:
Udziel odpowiedzi na następujące pytania teoretyczne:
(najpierw pytania potem odpowiedzi, czcionka powiększona znacznik <Hn> takie n aby było największe czcionka).

1. Rodzaje „filozofii" programowania.
2. Opisz paradygmat programowania strukturalnego. Założenia programowania strukturalnego.
3. Programowanie obiektowe: definicja (angielski skrót, obiekty, stan, zachowanie, końcowa konkluzja czym jest programowanie obiektowe).
4. Dlaczego stosujemy programowanie obiektowe?
5. Omów następujące pojęcia (w krótkiej formie),
    - Klasa
    - Obiekt
    - Właściwość
    - Metoda
    - Konstruktor
    - Dziedziczenie
    - Hermetyzacja
    - Polimorfizm

Koniec pytań teoretycznych
==========

Rodzaje „filozofii-paradygmatów" programowania:

1)Programowanie strukturalne
2)Programowanie obiektowe

==========

Programowanie strukturalne – paradygmat programowania opierający się na podziale kodu źródłowego programu na procedury i hierarchicznie ułożone bloki z wykorzystaniem struktur kontrolnych w postaci instrukcji wyboru i pętli.
Programowanie strukturalne zwiększa czytelność i ułatwia analizę programów, co stanowi znaczącą poprawę w stosunku do trudnego w utrzymaniu ***„spaghetti code"*** często wynikającego z użycia ***instrukcji goto***.

Założenia programowania strukturalnego

1. Podzielone bloki kodu mają jeden punkt wejścia (mogą mieć wiele punktów wyjścia),
2. Wykonywanie wyrażeń w określonej kolejności,
3. Używanie instrukcji warunkowych (if, if else),
4. Używanie pętli (for, while, do while),
5. Unikanie instrukcji skoku (goto),
6. Unikanie instrukcji break, continua,
7. W programowaniu proceduralnym (strukturalne) funkcje i zmienne opisujące dany przedmiot nie są ze sobą powiązane.

==========

Opis programowania obiektowego

Krótki opis.

Programowanie obiektowe jest to podejście bardziej naturalne dla ludzi, bardziej zgodne z rzeczywistością. W pamięci komputera tworzona jest wirtualna rzeczywistość. Definiowane (powoływane są do wirtualnego życia w pamięci komputera) **obiekty** na podstawie wcześniej zdefiniowanych **klas**. Obiekty mają zdefiniowane **metody** oraz **pola**.

Programowanie obiektowe definicja:

Programowanie obiektowe (ang. object-oriented programming) — sposób programowania, w którym programy definiuje się za pomocą:

- **obiektów** (obiekty powoływane są do życia wirtualnego przez programistę, mogą być kasowane przez programistę) np. okienka w Windows→okienko jest obiektem,
- obiekt ma jakiś **stan** (czyli dane, w programowaniu obiektowym nazywane najczęściej **polami**) np. wielość okienka w Windows,
- obiekt wykazuje pewne **zachowanie** (czyli posiada pewne procedury w programowaniu obiektowym nazywane metody). Np. jest zdefiniowana metoda do zmiany wielkości okienka.

Czyli **obiektowy program komputerowy** wyrażony jest jako zbiór takich obiektów, komunikujących się pomiędzy sobą w celu wykonywania zadań.

Dlaczego stosujemy programowanie obiektowe:

- Jest to bardziej naturalne podejście przez programistę podczas tworzenia programów ponieważ otaczający nas świat to różnego rodzaju przedmioty. Tworząc program komputerowy dokonujemy pewnego ich odwzorowania rzeczywistości np. definiujemy obiekt samochód. Definiujemy pola czyli jego stan np. rok produkcji, marka, kolor. Definiujemy również metody czyli co umie np. hamowanie, przyśpieszanie. Definicja obiektu jego metod i pól znajduje się w klasie. Następnie na podstawie zdefiniowanego obiektu (może mieć pola oraz metody) powołujemy obiekt w pamięci wirtualnej
- Programowanie obiektowe pozwala na lepsze gospodarowanie pamięciom komputera, ponieważ gdy obiekt staje się zbędny możemy go usunąć co zwolni pamięć zajmowaną przez ten obiekt. W programowaniu strukturalnym aby zwalniać pamięć konieczne było stosowanie dynamicznych struktur danych np. stos czy kolejka.
- W programowaniu obiektowym dobrze napisana i przetestowana klasa pozwala na stosowaniu przez innych programistów. Zaoszczędza to czas i środki pieniężne.

W przypadku programowania zorientowanego na obiekt (programowania obiektowego) dokonuje się powiązania metod (funkcji programu) z danymi (zmiennymi) definiującymi przedmiot. Wszystko to grupuje się w tzw. klasie zawierającej zarówno dane (pola) definiowanego przedmiotu, jak i funkcje (metody). W ten sposób definicje przedmiotu i jego właściwości znajdują się w jednym miejscu programu. W języku JavaScript, opis klasy jest dokonywany za pomocą pól (zmienne) i metod(funkcje).

Terminologia (podsumowanie)

Klasa (ang. "class")

Klasa rodzaj foremki, która opisuje nam jak będą wyglądać tworzone na jej podstawie nowe obiekty. Klasa jest to złożony typ będący opisem (definicją) obiektu/obiektów wraz z definicją pól i metod obiektów.

Obiekt (ang. "object")

Instancja (byt, twór) który powstał na podstawie klasy. Czyli Obiekty są konstrukcjami programistycznymi mającymi tak zwane właściwości (inaczej pola obiektu), którymi mogą być zmienne lub inne obiekty. Z obiektami powiązane są funkcje wykonujące operacje na ich właściwościach, nazywane metodami.

Właściwość (ang. "property")

Własność obiektu, np. kolor.

Metoda (ang. "method")

Zdolność (czynność) którą umie wykonywać obiekt, np. chodzenie (idź). Realizowane w postaci konstrukcji podobnej do funkcji.

Konstruktor (ang. "constructor")

Metoda wywoływana w momencie inicjalizacji obiektu. np. tworzymy obiekt a wraz z tworzeniem obiektu uruchomi się metoda, która nada np. Imię i Nazwisko.

Destruktor (ang. "descructor "jest wywoływany automatycznie podczas niszczenia obiektów. Nie przyjmuje żadnych argumentów. W przeciwieństwie do konstruktorów, w klasie może być tylko jeden destruktor.

Dziedziczenie (ang. "inheritance")

Klasa może dziedziczyć własności od innej klasy. Tworzymy klasę na podstawie innej klasy pobierając od „rodzica" właściwości i metody, może dopisać nowe pola i nowe metody.

Hermetyzacja (lub enkapsulacja - ang. "encapsulation")

Wewnątrz ciała klasy znajdują się pola i metody. Część pól i metod można odpowiednio ukryć przed "zewnętrznym światem" klasy tak jak to ma miejsce z przedmiotami ze świata rzeczywistego.

Polimorfizm (ang. "polymorphism")

Polimorfizm czyli wielopastaciowość. Oznacza to, że dany obiekt może zmieniać swoją postać w zależności od potrzeb. Może to być realizowane w formie metody wywoływanej z różną ilością parametrów. Tak więc utorzony obiekt może być traktowany polimorficznie bo zachowuje się różnie w zależności od sposobu uruchamiania jego metody.

Zadanie 29.

Wykonaj:

Udziel odpowiedzi na następujące pytania teoretyczne:

(najpierw pytania potem odpowiedzi, czcionka powiększona znacznik <Hn> takie n aby było największe czcionka).

1. W jaki sposób można określić właściwość danego obiektu (dwa sposoby wraz z przykładami)? (odpowiedź poniżej)
2. W jaki sposób można odwołać się do pól (właściwości) i metod? (odpowiedź poniżej)
3. Wymień cechy charakterystyczne konstruktora. (odpowiedź poniżej)
4. Konstruowanie obiektu na podstawie przykładu→obiekt ma trzy pola oraz jedną metodę.
5. Znaczenie słowa **this**. Jak oddzielamy pola oraz metody? (odpowiedź poniżej)
6. Konstruktor w JS na podstawie przykładu wraz z opisem.
7. Znaczenie słowa kluczowego **New** wraz z przykładem.
8. Znaczenie słowa kluczowego **prototype** wraz z przykładem i opisem **(szczególnie ważny tekst pogrubiony)**.

Koniec pytań teoretycznych
==========

Właściwość danego obiektu można określić (wpisać daną do pola odpowiedzialnego za właściwość)

Sposób 1

Używając zapisu postaci:

a)

nazwa_obiektu.nazwa_właściwości="tekst"; → dla właściwości typu tekstowego

np.

```
auto.marka="Opel Omega";
```

b)

nazwa_obiektu.nazwa_właściwości=liczba; → dla właściwości typu liczbowego

np.

```
auto.rok=1996;
auto.cena=25000;
```

Sposób 2

używając zapisu postaci:

nazwa_obiektu[nazwa_właściwości];

np. auto[cena]=25000;

Odwołanie do pól (właściwości) i metod.

Do właściwości i metod można się odwołać podobnie jak do zwykłych zmiennych i funkcji, trzeba tylko przed ich nazwą umieścić nazwę obiektu, którego są elementami (np. zmienną, która przechowuje dany obiekt), i kropkę.

np. console.log(samochod1.kolor); → wyświetla kolor obiektu samochod

Cechy charakterystyczne konstruktora:

- jest wywoływany (uruchamiany) automatycznie w chwili tworzenia obiektu danej klasy - na rzecz tego obiektu,
- nazywa się tak samo jak klasa,
- nie zwraca żadnej wartości ,
- może przyjmować argumenty - tak jak zwykła funkcja czy metoda, aby następnie powołać do życia obiekt za pomocą operatora **new**.

Przykład

Temat: Konstruowanie obiektów→obiekt ma trzy pola oraz jedną metodę.

```
var osoba_1 =
{
nazwisko        : 'Kowalski',
imie            :'Jan',
zawod           :'piekarz',
wyświetl        :function  ()
        {
                Document.write(this.nazwisko + ' ' + this.imie)
```

```
        }
}
```

Uwaga1:
Słowo kluczowe **this** → pozwala odwołać się do pola lub metody obiektu z wnętrza tego obiektu.
Uwaga2:
Metody i pola muszą być odzielne przecinkami w obrębie jednego obiektu.

Definiwonie klasy w JS

Zamiast klas, JavaScript stosuje funkcje. Zdefiniowanie klasy ogranicza się do prostej czynności, jaką jest zdefiniowanie funkcji, która pełni funkcję klasy. Tak działa JS jest ponieważ JavaScript jest językiem opartym na **prototypie**, w którym nie występuje pojęcie klasy, w przeciwieństwie do języków takich, jak C++ czy Java. Fakt ten bywa dezorientujący dla programistów przyzwyczajonych do języków z pojęciem klasy

Przykład: Tworzenie konstruktora.

Zostanie utworzony konstruktor o nazwie **klient** z właściwościami **nazwisko, imie, zawod** oraz metodą **wypisz().** Właściwościom obiektu zostały przypisane wartości parametrów. Użyte słowo kluczowe **this** odnosi się do aktualnego obiektu i pozwala na przypisanie wartości parametru do odpowiedniego pola tego obiektu.

```
function klient(nazwisko_k,imie_k,zawod_k)
{
 this.nazwisko=nazwisko_k;
 this.imie=imie_k;
 this.zawod=zawod_k;

wypisz = function ()
  {
    alert(this.nazwisko + ' ' + this.imie)
  }
}
```

Słowo kluczowe new

Do utworzenia **nowego obiektu** na podstawie konstruktora stosowane jest **słowo kluczowe new**.

Przykład

```
var osoba1 = new klient('Kowalski', 'Jan', 'kierowca');
var osoba2 = new klient('Nowak', 'Anna', 'sekretarka');
```

Powstały dwa nowe obiekty osoba1 i osoba2 należące do klasy klient.

Właściwość prototype

Do definowania metod i właściwości dla obiektu jest wykorzystana właściwości **prototype**.

Przykład

```
function klient()
{
 this.nazwisko='Bielski';
 this.imie='Paweł';
 }
klient.prototype.pisz_dane = function ()
  {
    document.write(this.nazwisko + ' ' + this.imie)
   }
klient.prototype.zawod = 'kierowca';
var osoba1 = new klient();
osoba1.pisz_dane();
```

Opis do przykładu powyżej:

W definicji konstruktora nie zostały zadeklarowane żadne metody i właściwości. Dopiero po użyciu właściwości prototype została dodana metoda pisz_dane oraz właściwość zawod. Od tej pory każdy nowo tworzony obiekt na podstawie konstruktora klient będzie posiadał tę dodatkową właściwość i metodę.

Właściwość prototype może być również wykorzystana do dodawania dodatkowych metod lub właściwości do istniejących obiektów.

==========

Przykład

Temat: Nadawanie nowych właściwości i metod.

```
var osoba_1 =
{
nazwisko        : 'Kowalski',
imie            :'Jan',
zawod           :'piekarz',
wyświetl        :function  ()
        {
                Document.write(this.nazwisko + '  '  +  this.imie)
        }
}
czlowiek.wiek = 25;
czlowiek wypisz_wiek = function() {alert('wiek: ' + this.wiek + 'lat')}
czlowiek.wypisz();
```

Przykład:

Temat: Tworzenie pojedynczego obiektu

Obiekt posiada dwie ustalone właściwości

- name i height
- jedną metodę, która wypisuje jego imię.

```
var myObject = {
   name: "Marcin",
   height: 184,
   print : function() {
      console.log(this.name)
   }
}
    myObject.print(); //wypisze w konsoli "Marcin"
    console.log(myObject.height); //wypisze 184
```

Przykład
Temat: Nadanie nowej własności number i obliczenie kwadratu.

Aby odwołać się do danego obiektu z jego wnętrza stosujemy instrukcję this. Dzięki temu możemy w łatwy sposób wywoływać z wnętrza inne metody danego obiektu lub korzystać z jego właściwości:

```
var myObject = {
   number: 100,
   square : function() {
      return this.number * this.number
   }
}

obiekt2.number = 200;
console.log(obiekt2.square())
```

Przykład
Temat: Definiowanie obiektów z użyciem konstruktora oraz tworzenie nowego obiektu.
W poniższym przykładzie wygenerujemy więc dwa nowe obiekty – „auto” i „osoba”, wykorzystamy je do zgromadzenia odpowiednich informacji, po czym te ostatnie wyświetlimy na ekranie komputera:

```
function auto(marka,rok,cena,wlasciciel)
{
        this.marka=marka;
        this.rok=rok;
        this.cena=cena;
        this.wlasciciel=wlasciciel;
}

function osoba(imie,nazwisko)
{
        this.nazwisko=nazwisko;
        this.imie=imie;
}
posiadacz=new osoba("Jan","Kowalski");
bryka=new auto("Ferrari",2003,200000,posiadacz);
document.write("Marka: "+ bryka.marka +" rocznik: "+ bryka.rok +" cena: "+ bryka.cena);
document.write("<br>Wlasciciel: "+ bryka.wlasciciel.imie +" "+ bryka.wlasciciel.nazwisko);
```

Uwaga1
Konstruktor obiektu przypomina zwykłą funkcję.
Uwaga2
Słowo kluczowe **new** służy do tworzenia obiektu na podstawie konstruktora.

Przykład
Temat: Operowanie metodami →definiowanie metod wyświetlających właściwości obiektów.

```
function pokaz_auto()
{
        dane="Marka: "+ this.marka +" Rocznik: "+ this.rok +" Cena: "+ this.cena +"<br>";
        document.write(dane);
        this.wlasciciel.pokaz()                         // metoda pokaż obiektu osoba
}

function pokaz_osoba()
{
        dane="imie: "+ this.imie +" nazwisko: "+ this.nazwisko +"<br>";
        document.write(dane);
}

function auto(marka,rok,cena,wlasciciel)
{
        this.marka=marka;
        this.rok=rok;
        this.cena=cena;
        this.wlasciciel=wlasciciel;
        this.pokaz=pokaz_auto           // dodajemy metodę pokazującą dane naszego auta
}

function osoba(imie,nazwisko)
{
        this.nazwisko=nazwisko;
        this.imie=imie;
        this.pokaz=pokaz_osoba                  // dodajemy metodę pokazującą naszą osobę
}

posiadacz=new osoba("Jan","Kowalski");
bryka=new auto("Ferrari",2003,200000,posiadacz);
bryka.pokaz()                           // pokazuje nam wszystkie właściwości naszego obiektu
```

Przykład
Temat: Właściwości i metody możemy deklarować dla nowych obiektów:

```
var myObject = {
   name: "Marcin",
   height: 184,
   print : function() {
      alert(this.name)
   }
}

myObject.weight = 73; //dodaliśmy nową właściwość
myObject.printDetail = function() {
   return {
      height : this.height,
```

```
      name :
   }
}
myObject.printDetail()
```

Przykład
Temat: Usuwanie właściwości
Aby usunąć właściwość obiektu, skorzystamy z operatora delete:

```
var myObject = {
   color: 'zielony',
   size: 'duzy',
   price: 'wielka'
}

console.log(price);

delete myObject.price; //Usuwamy właściwość cena

console.log(price); //wypisze błąd
```

Przykład
Temat:Tworzenie obiektu za pomocą konstruktora.
Chcemy utworzyć ich kilka. Każdy obiekt powinien posiadać jakieś właściwości i metody np. width, height i wypisz().
Aby utworzyć kilka podobnych obiektów posłużymy się klasą obiektu.
W JS w przeciwieństwie do innych języków nie mamy mechanizmu konstruktora, ale samą klasę możemy stworzyć za pomocą zwykłej funkcji:

```
function SuperObjectClass(_width,_height) {
   this.width = _width;
   this.height = _height;
   this.print = function() {
      console.log(this.width + 'x' + this.height)
   }
}

var myObject1 = new SuperObjectClass(200, 100);
var myObject2 = new SuperObjectClass(300, 200);

myObject1.print(); //wypisze 200x100

myObject2.width = 600;
myObject2.print() //wypisze 600x200
```

Zadanie 30.

Wykonaj:
1)w szkielecie wykonaj przyciski do zadania i KODu
2)Na podstawie Przykładu1 (poniżej), wykonaj program z użyciem programowania obiektowego. Dokonaj kopiowania **przykładu 1** i zmień aby zrealizował twój obiekt z **tabeli** poniżej.
Zdefiniuj obiekt [patrz tabela poniżej], trzy właściwości z wpisanymi danymi, jedną metodę wyświetlającą wartości pól właściwości (nazwa metody to nazwa obiekt_wyswietl_nazwisko_ucznia

np. samochod_wyswietl_kowalski). Twój program powinien wyświetlać nazwę obiektu, nazwy pól, oraz nazwę metody. (zmień drugiego w przykładzie 1)

Nr w dzienniku	**Nazwa obiektu**	**Pola (Właściwości)**	**Metoda** Końcowa nazwa metody ma zawierać Twoje nazwisko
1,16	Samochod	Np. **kolor**, dopisz jeszcze dwie właściwości pasujące do obiektu	samochod_wyswietl_nazwisko
2,17	Ciezarowka	Np. **ladownosc**, dopisz jeszcze dwie właściwości pasujące do obiektu	Ciezarowka_wyswietl_nazwisko
3,18	Dom	Np. **wysokosc**, dopisz jeszcze dwie właściwości pasujące do obiektu	Dom_wyswietl_nazwisko
4,19	Pudełko	Np. **kolor**, dopisz jeszcze dwie właściwości pasujące do obiektu	Pudelko_wyswietl_nazwisko
5,20	Wazon	Np. **kolor**, dopisz jeszcze dwie właściwości pasujące do obiektu	Wazon_wyswietl_nazwisko
6,21	Pociag	Np. **ile_wagonow**, dopisz jeszcze dwie właściwości pasujące do obiektu	Pociag_wyswietl_nazwisko
7,22	Mebel	Np. **rodzaj**, dopisz jeszcze dwie właściwości pasujące do obiektu	Mebel_wyswietl_nazwisko
8,23	Autobus	Np. **ilu_pasazerow**, dopisz jeszcze dwie właściwości pasujące do obiektu	Autobus_wyswietl_nazwisko
9,24	Komputer	Np. **marka**, dopisz jeszcze dwie właściwości pasujące do obiektu	Komputer_wyswietl_nazwisko
10,25	Monitor	Np. **marka**, dopisz jeszcze dwie właściwości pasujące do obiektu	Monitor_wyswietl_nazwisko
11,26	Router	Np. **rodzaj**, dopisz jeszcze dwie właściwości pasujące do obiektu	Router_wyswietl_nazwisko
12,27	Telefon	Np. **siec**, dopisz jeszcze dwie właściwości pasujące do obiektu	Telefon_wyswietl_nazwisko
13,28	Spodnie	Np. **rodzaj**, dopisz jeszcze dwie właściwości pasujące do obiektu	Spodnie_wyswietl_nazwisko
14,29	Owoc	Np. **jaki_owoc**, dopisz jeszcze dwie właściwości pasujące do obiektu	Owoc_wyswietl_nazwisko
15, 30, 31,32,33,34	Wieżowiec	Np. **ile_pieter** dopisz jeszcze dwie właściwości pasujące do obiektu	Wiezowiec_wyswietl_nazwisko

Przykład 1→dotyczący programowania obiektowego.

Temat: Utwórz obiekt o nazwie **człowiek**, polach **nazwisko, imie, zawod** i metodzie **pokaz, która** wyświetla nazwisko i imię.

Uwagi do przykładu poniżej:

1)Przy definiowaniu obiektu deklarowane właściwości i funkcje muszą być oddzielone przecinkami.

2)Słowo kluczowe **this** pozwala odwołać się do właściwości lub metod danego obiektu z jego wnętrza tego obiektu.

tło	Kolor liter pierwszego diva	Kolor liter drugiego diva	wielkość liter pierwszego diva	wielkość liter drugiego diva
żółte	zielony	niebieski	35+numer miesiąca urodzenia px	20+numer dzień urodzenia px

```
<!DOCTYPE html>
<html lang="pl">
 <head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Zadanie 30 – Nazwisko ucznia np. Kowalski</title>
 </head>
<body bgcolor="silver">
    <div style="color:navy;font-size:50px;">
        <script type="text/javascript">
            var czlowiek = {
                                nazwisko: 'Nowacki',
                                imie: 'Marek',
                                zawod: 'informatyk',
                                pokaz: function ()
                                 {
                    document.write("Dane obiektu nazwisko oraz imię"+"<br>")
                                    document.write("Moje dane ==>"+this.nazwisko + " " + this.imie)
                                }
                     }
                     czlowiek.pokaz();
        </script>
    </div>
 <br>
<div style="color:red;font-size:30px;">
 <br>
Mój numer w dzienniku jest 38 i moje dane to:<br>
 Obiekt  to  człowiek<br>
 pola:<br>
               Nazwisko<br>
               imie<br>
               zawod<br>
   metoda: <br>
           pokaz<br>
<div>
</body>
</html>
```

Zadanie 31.

1)w szkielecie wykonaj przyciski do zadania i KODu
2)Na podstawie Przykładu2 (poniżej), uzupełnij program z zadania przedniego (każdy uczeń miał inny obiekt) z użyciem programowania obiektowego.
Dopisz jedną właściwość (poza ciałem obiektu) z wpisaną daną, jedną metodę wyświetlającą wartości pola właściwości (dopisaną w zadaniu poprzednim),
nazwa metody to:
nazwa obiekt_pokaz_nazwisko_ucznia
np. samochod_pokaz_kowalski (metoda dopisana poza ciałem obiektu).

3)Program wyświetli nazwę twojego obiektu, twoje nazwy pól, twoją nazwę metody, nazwę nowego pola poza obiektu poza ciałem obiektu , nazwę nowej metody poza ciałem obiektu
np. samochod_wyswietl_kowalski). Twój program powinien wyświetlać nazwę obiektu, nazwy pól, oraz nazwę metody.

tło	Kolor liter pierwszego diva	Kolor liter drugiego diva	wielkość liter pierwszego diva	wielkość liter drugiego diva
Jasno niebieski	czerwony	niebieski	40+numer miesiąca urodzenia px	25+numer dzień urodzenia px

Przykład 2

Temat: Dla istniejących już obiektów można deklarować nowe właściwości i metody (poza klasa/funkcją).

```
<!DOCTYPE html>
<html lang="pl">
 <head>
  <meta charset="UTF-8">
  <meta name="viewport" content="wi""dth=device-width, initial-scale=1.0">
  <title>Zadanie 31 – Nazwisko ucznia np. Kowalski</title>
 </head>
<body bgcolor="silver">
    <div style="color:navy;font-size:50px;">
        <script type="text/javascript">
            var czlowiek = {
                            nazwisko: "Nowacki",
                            imie: "Marek",
                            zawod: "informatyk",
                            pokaz: function ()
                             {
                 document.write("Dane obiektu nazwisko oraz imię"+"<br>")
                                document.write("Moje dane ==>"+this.nazwisko + " " + this.imie)
                            }
                    }
                    czlowiek.pokaz();
           czlowiek.wiek=19;
                    czlowiek.wypisz=function() {alert("Wiek: " + this.wiek + " lat")}
                    czlowiek.wypisz();
           wypisz();
        </script>
    </div>
 <br>
<div style="color:red;font-size:30px;">
 <br>
Mój numer w dzienniku jest 38 i moje dane to:<br>
 Obiekt  to  człowiek<br>
 pola:<br>
               Nazwisko<br>
               imie<br>
               zawod<br>
   metoda: <br>
            pokaz();<br>
==========================================================================<br>
nowe pole to wiek dla obiektu człowiek warość pola to 19<br>
   nowa  defincja  metoda to wypisz(); z użyciem alert();<br>
użycie metody wypisz();

<div>
</body>
</html>
```

Przykład 4

Temat: **Tworzenie obiektu za pomocą konstruktora**

Chcemy utworzyć ich kilka. Każdy obiekt powinien posiadać jakieś właściwości i metody np. width, height i wypisz(). Aby utworzyć kilka podobnych obiektów posłużymy się klasą obiekt. W JS w przeciwieństwie do innych języków nie mamy mechanizmu konstruktora, ale samą klasę możemy stworzyć za pomocą zwykłej funkcji:

```
function SuperObjectClass(_width,_height) {
   this.width = _width;
   this.height = _height;
   this.print = function() {
      console.log(this.width + 'x' + this.height)
   }
}

var myObject1 = new SuperObjectClass(200, 100);
var myObject2 = new SuperObjectClass(300, 200);

myObject1.print(); //wypisze 200x100

myObject2.width = 600;
myObject2.print() //wypisze 600x200
```

Konstruktor jest wywoływany w momencie instancjalizacji (moment, w którym instancja obiektu zostaje utworzona). Konstruktor jest metodą klasy. W JavaScript, funkcja służy za konstruktor obiektu. Nie ma jednak wyraźnej potrzeby definiowania konstruktora. Każda akcja zadeklarowana w konstruktorze zostanie wykonana w momencie utworzenia obiektu.

Konstruktor jest używany do ustawienia właściwości obiektu lub do wywołania metod przygotowujących obiekt do użytku.

W języku JavaScript istnieje możliwość tworzenia wielu obiektów posiadających podobne właściwości. W tym celu można posłużyć się konstruktorem obiektu. Konstruktor przypomina zwykłą funkcję.

Do właściwości obiektu można się do nich odwoływać na dwa sposoby:

```
nazwa_obiektu.nazwa_właściwości
np.
klient.nazwisko,

nazwa_obiektu["nazwa_właściwości"]
np.
klient["nazwisko"].
```

Przykład

Temat: instrukcja Prototype

Przypuśćmy, że mamy już stworzone jakieś obiekty. Po jakimś czasie chcielibyśmy dodać do nich nową metodę. Dodajemy więc nową metodę do naszej foremki (klasy), ale okazuje się, że dostaną ją dopiero nowe obiekty stworzone na podstawie tej formy. Aby zmienić prototyp (a co za tym idzie i wszystkie obiekty stworzone na jego podstawie) posłużymy się instrukcją prototype.

```
function SuperObjectClass() {
      this.name = 'Marcin';
      this.height = 183
}

//dodaję właściwość i metodę do prototypu
SuperObjectClass.prototype.weight = 73;
SuperObjectClass.prototype.showInfo = function () {
   alert(this.name + ' ma ' + this.height + 'cm wzrostu.')
}
var myObject = new SuperObjectClass();
myObject.showInfo();
console.log(myObject.weight); //wypisze sie 73
```

Zadanie 32.

1)w szkielecie wykonaj przyciski do zadania i KODu
2)Na podstawie Przykładu3 (poniżej), uzupełnij program z dwóch zadań poprzednich z użyciem programowania obiektowego.
Program powinien zawierać:

- Definiowanie konstruktora (klasy). Należy zauważyć, że w JS w wersji przed 2015 niema możliwości definiowania klasy (tak jak w innych językach obiektowych). Dlatego definiowanie klasy odbywa się z użyciem konstruktora (słowo kluczowe function).
- dodawanie metody do konstruktora z użyciem prototypu, (z czterema polami),
- powoływanie obiektów, (powołanie 3 obiektów),
- dostęp do właściwości (pól) obiektów, (wyświetlenie danych o wszystkich obiektach),
- użycie zdefiniowanej metody. (wyświetlenie danych o wszystkich obiektach).

Dopisz jedną właściwość (poza ciałem obiektu) z wpisaną daną, jedną metodę wyświetlającą wartości pola właściwości (dopisaną w zadaniu 2??), nazwa metody to nazwa obiekt_pokaz_nazwisko_ucznia np. samochod_pokaz_kowalski (metoda dopisana poza ciałem obiektu).

tło	Kolor liter pierwszego diva	Kolor liter drugiego diva	wielkość liter pierwszego diva	wielkość liter drugiego diva
żółte	zielony	niebieski	35+numer miesiąca urodzenia px	20+numer dzień urodzenia px

Przykład 3

Temat: Definiowanie konstruktora, dodawanie metody do konstruktora z użyciem prototypu, powoływanie obiektów, dostęp do właściwości (pól) obiektów, użycie zdefiniowanej metody.

```
<!DOCTYPE html>
<html lang="pl">
 <head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title size="2xl">Zadanie 32 – Nazwisko ucznia np. Kowalski</title>
 </head>
<body bgcolor="silver">
 <div style="color:navy;font-size:50px;">
  <script type="text/javascript">
        function klient(nazwisko_k,imie_k,zawod_k)   //początek konstruktora
                {
                        this.nazwisko=nazwisko_k;
                        this.imie=imie_k;
                        this.zawod=zawod_k;
                }                               //koniec konstruktora
        klient.prototype.wypisz = function () //definicja metody
                                {
                                        alert(this.nazwisko + " " + this.imie+ " " + this.zawod);
                                };
                var osoba1 = new klient("Kowalski", "Jan", "kierowca"); // nowy obiekt osoba1
                var osoba2 = new klient("Nowak", "Anna", "sekretarka"); // nowy obiekt osoba2

          osoba1.wypisz(); // uruchomienie metody wypisz() dotyczącej osoby1 z użyciem alert();
          osoba2.wypisz(); // uruchomienie metody wypisz() dotyczącej osoby2 z użyciem alert();
          document.write("Moje nazwisko to KOWALSKI  mam numer 37 a obiekt to klient"+"<br>");
          document.write("dostęp do pól osoba1 i  osoba2"+"<br>");
          document.write("czyli wypisanie danych"+"<br>");

// poniżej dostęp/wypisanie danych  do pól osoby1 i  osoba2
 document.write("klient pierwszy =>"+"   "+osoba1.nazwisko+"   "+osoba1.imie+"   "+osoba1.zawod+"<br>");
 document.write("klient pierwszy =>"+"   "+osoba2.nazwisko+"   "+osoba2.imie+"   "+osoba2.zawod+"<br>");
  </script>
 </div>
 <div style="color:red;font-size:30px;"><br>
programowanie obiektowe:
Słowo kluczowe prototype stosowane jest do:<br>
.....tutaj opis..... <br>
Słowo kluczowe new stosowane jest do:<br>
.....tutaj opis..... <br>
</div>
</body>
</html>
```

Przykład
Temat: Użycie Instrukcji **prototype** do utworzenia dodatkowej funkcjonalność, dla obiektu np String czy Array. Tutaj:

- String.prototype.firstCapital→pierwsza litera duża
- String.prototype.mixLetterSize→litery nieparzyste małe, parzyste duże

```
String.prototype.firstCapital = function() {
   return this.charAt(0).toUpperCase() + this.substr(1);
}

function mixLetterSize() {
   var tekst = '';
   for (var x=0; x<this.length; x++) {
      tekst += (x%2==0)?this.charAt(x).toUpperCase() : this.charAt(x).toLowerCase();
   }
   return tekst;
}
String.prototype.mixLetterSize = mixLetterSize;

var text1 = "marcinek";
console.log(text1.firstCapital()) //wypisze Marcinek

var text2 = "marcinek";
console.log(text2.mixLetterSize()) //wypisze mArCiNeK
```

Przykład
Temat: Obiekty nadrzędne, kontekst obiektu.

Przypuśćmy, że mamy obiekt w obiekcie:

```
var SuperObjectClass = function() {
   this.name = 'Marcin';
   this.height = 183
   this.button = null;

   this.init = function() {
      this.button = document.createElement('input');
      this.button.addEventListener('click', function() {
         this.value = this.height ???????
      });
      document.querySelector('body').appendChild(this.button);
   }
   this.init();
}

var someObject = new SuperObjectClass();
```

Po wywołaniu metody init tworzymy nowy guzik. Po jego kliknięciu powinien on ustawić swoje value na height SuperObjectClass. Jak jednak to zrobić, skoro instrukcja this wewnątrz obiektu guzika wskazuje na niego samego?
Są na to dwa sposoby: posłużenie się dodatkową zmienną that:

```
var SuperObjectClass = function() {
```

```
    this.name = 'Marcin';
    this.height = 183
    this.button = null;

    this.init = function() {
        var that = this;

        this.button = document.createElement('input');
        this.button.value = 'Kliknij';
        this.button.type = 'button';
        this.button.addEventListener('click', function() {
            alert('To jest ' + this.nodeName); //przycisk
            alert('Wzrost Marcina: ' + that.height); //obiekt SuperObiectClass
        });
        document.querySelector('body').appendChild(this.button);
    }
    this.init();
}
var someObject = new SuperObjectClass();
```

lub skorzystanie z instrukcji bind(), za pomocą której możemy przekazać kontekst do danej funkcji:

```
var SuperObjectClass = function() {
    this.name = 'Marcin';
    this.height = 183
    this.button = null;

    this.init = function() {
        this.button = document.createElement('input');
        this.button.value = 'Kliknij';
        this.button.type = 'button';
        this.button.addEventListener('click', function(e) {
            //tutaj już this wskazuje na SuperObjectClass
            //dlatego dany przycisk musimy pobrac za pomocą e.target
            alert(e.target.value = this.height);
        }.bind(this));
        document.querySelector('body').appendChild(this.button);
    }
    this.init();
}
var someObject = new SuperObjectClass();
```

Bardzo dużo osób neguje stosowanie dodatkowej zmiennej do przekazywania kontekstu obiektu. Pamiętać należy jednak, że stosując instrukcję bind() zatracamy dostęp do this danego pod obiektu.

Przykład
Temat: Metoda Watch oraz Unwatch

Javascript udostępnia nam metodę:
watch("nazwaWlasciwosci", funkcja(id, staraWartosc, nowaWartosc),
która służy do podglądu właściwości obiektów. Jej działanie jest takie samo jak w innych językach. Gdy podglądana wartość się zmieni, wówczas watch wywoła funkcję podaną w drugim atrybucie. Przekazujemy jej 2 parametry - pierwszy określa nazwę obserwowanej właściwości, drugi to funkcja z

3 atrybutami: id, oldValue, newValue. Parametry te są wypełniane automatycznie. Gdy obserwowana wartość się zmieni, funkcja podana w 2 atrybucie zostanie wywołana:

```
var obiekt = {p:1}

//śledzenie właściwości "p" obiektu "obiekt"
obiekt.watch("p", function (id, oldValue, newValue) {
   console.log("o." + id + " zmieniło sie z " + oldValue + " na " + newValue)
   return nowaWart;
});

//teraz przy każdej zmianie wartości obiektu wywołany zostanie callback
//który śledzi zmiany w obiekcie
obiekt.p = 2
obiekt.p = 3
```

Aby zakończyć obserwowanie, korzystamy z metody unwatch("nazwa_obserwowanej_wlasciwości").

```
obiekt.unwatch('p')'
```

Zadanie 33.

1)Stwórz nowy konstruktor "**User**", którego argumentami będą:

- adres e-mail,
- hasło
- adres url awatara

2)Stwórz obiekt **userList**, który będzie posiadał metodę "**addUser**", przyjmującą jako argument instancję User i zapamiętującą ją w obiekcie, oraz metodę "login" z argumentami "email" oraz "password", zwracającą true w przypadku podania prawidłowych danych dla któregoś z użytkowników, oraz "false" w przypadku podania nieprawidłowych danych.
3)Stwórz prosty formularz, zawierający pola "login" oraz "hasło", który wyświetli komunikat z informacją, czy udało się zalogować

Zadanie →do zredagowania

1)Stwórz nowy konstruktor "UserView", którego argumentem będzie instancja User
W konstruktorze UserView wygeneruj element li, zawierający obrazek z awatarem użytkownika, oraz jego adres e-mail umieszczony jako link z protokołem "mailto:" (nadaj tym elementom jakieś ładne style :)). Przypisz element dom oraz obiekt User jako własności tworzonego obiektu.
W prototypie UserView utwórz metodę refresh odświeżającą dane użytkownika w elemencie dom.

Dział 6 Instrukcje warunkowe.

Zadanie 34.

Temat: Zadanie teoretyczne dotyczące instrukcji warunkowej.

Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie34)
Podłącz plik kowa_z34.html do przycisku Nazwisko_ucznia-Zadanie34

Nazwa pliku HTML
cztery_pierwsze_litery_nazwiska_z34.html np. kowa_z34.html

wstaw do pliku
<title>Kowalski Z34_html</title>

wstaw do pliku na sam koniec pliku komentarz
<!-- Kowalski Z34_JS -->

Najpierw pytanie (czcionką pogrubioną) (od 1 do 6) pytanie i odpowiedź na zmianę.
Uwaga: Schematy blokowe możesz wycinać jak zrzuty ekrany z tej instrukcji.

Pytania

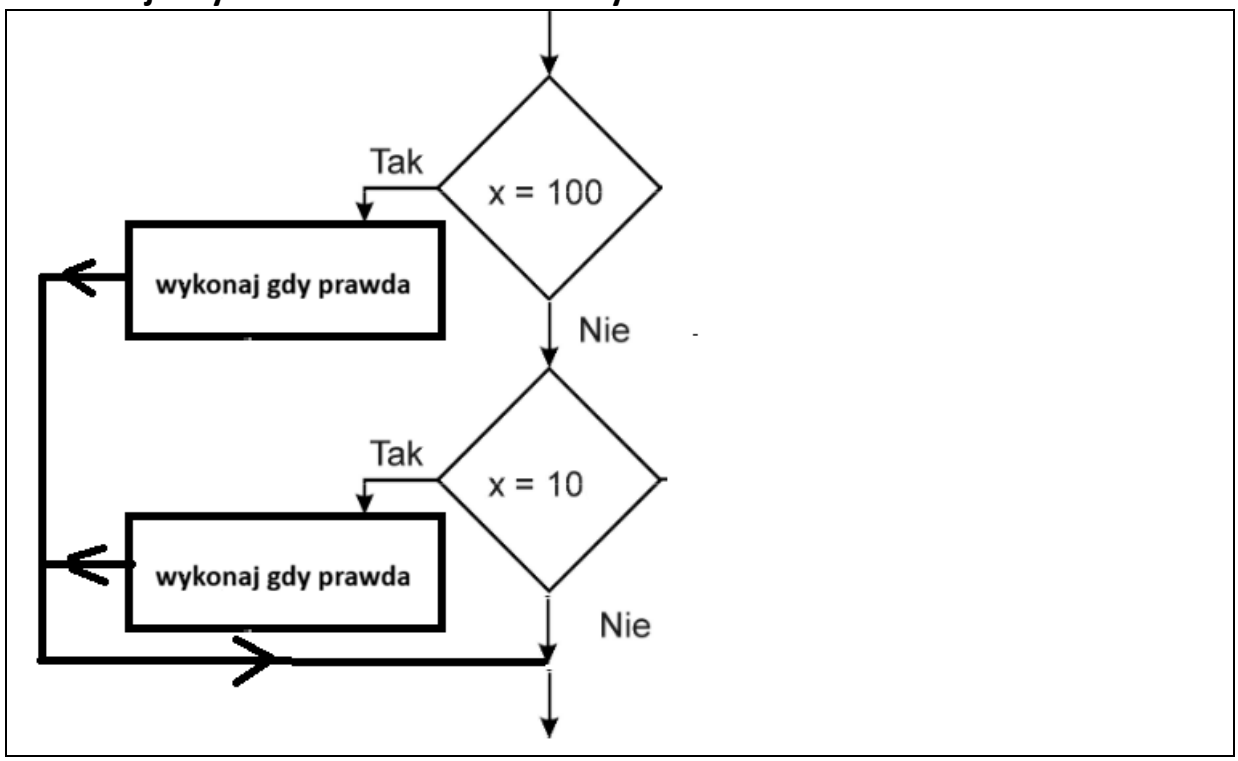
1) Instrukcja wyboru → składania teoretyczna.
2) Instrukcja wyboru → schemat blokowy.
3)Instrukcja alternatywy → składania teoretyczna.
4)Instrukcja alternatywy → schemat blokowy.
5)Operatory porównania.
6)Uwaga do operatorów porównania.
7)Operatory logiczne.

==========Opis teorii==========

Instrukcja wyboru → składania teoretyczna

```
if (warunek_logiczny1)
 {
   // instrukcje jeśli warunek_logiczny1 prawdziwy
 }
if (warunek_logiczny2)
 {
   // instrukcje jeśli warunek_logiczny2 prawdziwy
 }
```

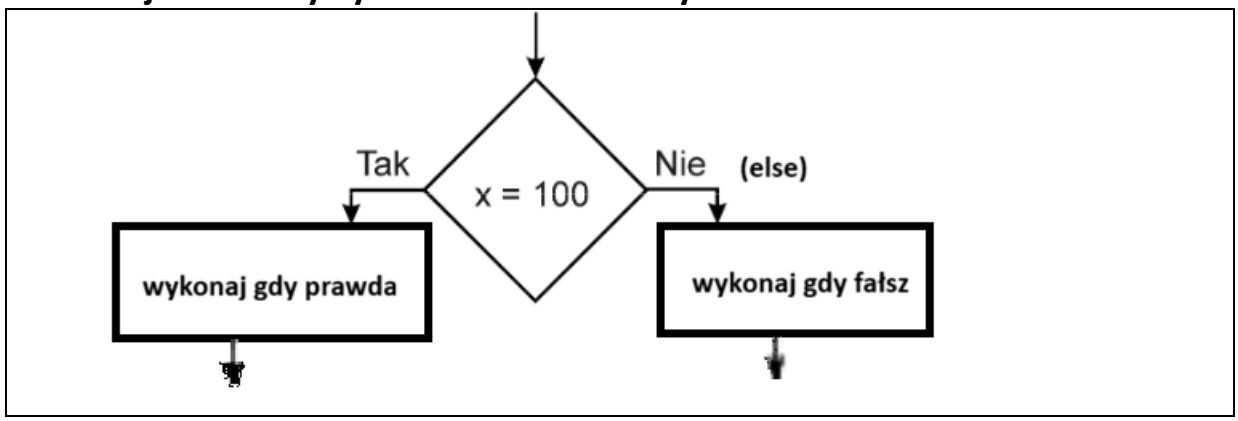
Instrukcja wyboru → schemat blokowy



Instrukcja alternatywy → składania teoretyczna

```
if (warunek_logiczny)
 {
   // instrukcje jeśli warunek_logiczny prawdziwy
 }
 else
 {
   // instrukcje jeśli warunek_logiczny fałszywy
 }
```

Instrukcja alternatywy → schemat blokowy



==========

Operatory porównania

Operator	Opis	Przykład	
==	jest równe	2==3	wynik→fałsz
!=	nie jest równe	2!=3	wynik→prawda
>	jest większe	25>3	wynik→prawda
<	jest mniejsze	2<3	wynik→prawda
>=	większe lub równe	25>=3	wynik→prawda
<=	mniejsze lub równe	2<=3	wynik→prawda

Uwaga do operatorów porównania:
Należy odróżnić == (dwa razy) od = (jeden raz)
== (dwa razy) → instrukcja PORÓWNANIA
= (jeden raz) → instrukcja PRZYPISANIA np. a=5;

Operatory logiczne

Operator	Opis	Przykład		
&&	i	x=3 y=4	(x < 9 && y > 2)	wynik→prawda
||	lub	x=3 y=4	(x==8 || y==6)	wynik→fałsz
!	zaprzeczenie	x=3 y=4	!(x==y)	wynik→prawda

==========

Zadanie 35.
Temat: Użycie instrukcji if dla określenia znaku liczby wczytanej z klawiatury.

Nazwa pliku HTML/JS
cztery_pierwsze_litery_nazwiska_z35.kltml np. kowa_z35.html

wstaw do pliku
<title>Kowalski Z35_JS</title>

wstaw do pliku na sam koniec pliku komentarz
<!-- Kowalski Z35_JS -->

Nazwa pliku KOD
cztery_pierwsze_litery_nazwiska_z35_kod.kltml np. kowa_z35_kod.html

wstaw do pliku
<title>Kowalski Z35_KOD</title>

wstaw do pliku na sam koniec pliku komentarz
<!-- Kowalski Z35_KOD -->

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie35)
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie35-KOD)
3)Przeczytaj temat. Pod pojęciem **znaku liczby** rozumiemy, że może być dodatnia lub zero lub dodatnia.
4)Dokonaj kopiowania zadania i uzupełnij aby rozwiązać zadanie.
5)Zmień nazwę zmiennej, powinna mieć trzy litery nazwiska ucznia

a_dana_kow

6)Dokonaj testowania zadania
7)Podłącz plik kowa_z35.html do przycisku Nazwisko_ucznia-Zadanie35
8)Podłącz plik kowa_z35_KOD.html do przycisku Nazwisko_ucznia-Zadanie35-DOD

```
<!DOCTYPE html>
<html lang="pl-PL">
                    <!-- polskie litery mają działać(uzupełnij w kodzie strony meta) -->
                    <!-- title -> Z35 oraz nazwisko ucznia -->
   <body>
        <div style="color:red;font-size:50px;">
          <script language="JavaScript">
                  var x=prompt("podaj liczbę x=","");
                        var a=parseFloat(x);
//zmien zmienną „a” na a_dana_kow
                    document.write("czytana liczba x="+a+"<br>");
          if(a>0)
              {document.write("czytana liczba jest większa od 0"+"<br>");}
// Dokończ dwa pozostałe przypadki a==0   i  a<0
            </script>
          </div>
 </body>
</html>
<!--  Kowalski Z35_JS  -->
```

==========

Zadanie 36.

Temat: Użycie instrukcji if dla określenia czy liczby wczytanej z klawiatury parzysta/nieparzysta.

Wczytać liczbę i stwierdzić czy jest parzysta czy nieparzysta:
Gdy jest parzysta to: czerwony napis *„liczba jest parzysta”* wielkością 3 (size)
Gdy jest nieparzysta to: srebrny napis *„liczba jest nieparzysta”* wielkością 5 (size)
Użyj instrukcji HTML z użyciem instrukcji dokument.write(…….); (kliknij)

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie36)
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie36-KOD)

Nazwa pliku HTML/JS
cztery_pierwsze_litery_nazwiska_z36.kltml np. kowa_z36.html

wstaw do pliku
<title>Kowalski Z36_JS</title>

wstaw do pliku na sam koniec pliku komentarz
<!-- Kowalski Z36_JS -->

Nazwa pliku KOD
cztery_pierwsze_litery_nazwiska_z36_kod.kltml np. kowa_z36_kod.html

wstaw do pliku
<title>Kowalski Z36_KOD</title>

wstaw do pliku na sam koniec pliku komentarz
<!-- Kowalski Z36_KOD -->

Po wykonaniu zadania:
7)Podłącz plik kowa_z36.html do przycisku Nazwisko_ucznia-Zadanie36
8)Podłącz plik kowa_z36_KOD.html do przycisku Nazwisko_ucznia-Zadanie36-DOD

Użycie instrukcji document.write do wykonania instrukcji HTML na przykładzie formatowania tekstu

```
document.write("<font color=silver size=4>wpisałeś kolor czerwony</font>"+"<br>");
```

Użycie instrukcji document.write do wykonania instrukcji CSS na przykładzie formatowania tekstu

Uwaga:
Konieczne jest użycie \" ponieważ zastosowanie samego znaku " bez \ spowoduje, ze interpreter Javascript będzie chciał zakończyć instrukcję document.write

```
document.write ("<p style=\"font-size: 40.px; color: black;\"><br>Piękna pogoda<br></p>");
```

Zadanie 37.
Temat: Użyj instrukcji if…. else ….. do sprawdzenia czy użytkownik wie od, którego roku Javascript jest w użyciu.

Wczytać rok do zmiennej **rok_kow** i są dwie własności możliwości:
Brawo znasz rok od kiedy działa Javasript (niebieski, wielkość 4)
Źle musisz jeszcze poczytać (czerwony, wielkość 7)
Wyświetli się również napis „wczytany rok=…..”

Nazwa pliku HTML/JS
cztery_pierwsze_litery_nazwiska_z37.kltml np. kowa_z37.html

wstaw do pliku
<title>Kowalski Z37_JS</title>

wstaw do pliku na sam koniec pliku komentarz
<!-- Kowalski Z37_JS -->

Nazwa pliku KOD
cztery_pierwsze_litery_nazwiska_z37_kod.kltml np. kowa_z37_kod.html

wstaw do pliku
<title>Kowalski Z37_KOD</title>

wstaw do pliku na sam koniec pliku komentarz
<!-- Kowalski Z37_KOD -->

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie37)
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie37-KOD)

Po wykonaniu zadania:
7)Podłącz plik kowa_z37.html do przycisku Nazwisko_ucznia-Zadanie37
8)Podłącz plik kowa_z37_KOD.html do przycisku Nazwisko_ucznia-Zadanie37-DOD

.

Operatory logiczne

Operator	Opis	Przykład		
&&	i	x=3 y=4	(x < 9 && y > 2)	wynik→prawda
||	lub	x=3 y=4	(x==8 || y==6)	wynik→fałsz
!	zaprzeczenie	x=3 y=4	!(x==y)	wynik→prawda

Zadanie 38.

Temat: Rozpoznawanie: rozdzielczość ekranu/monitora oraz szerokość/wysokość ekranu przeglądarki w pikselach.

Wykonaj PRZYCISKI

Nazwisko_ucznia-Zadanie38_E1	Nazwisko_ucznia-Zadanie38_E1_kod
Nazwisko_ucznia-Zadanie38_E2	Nazwisko_ucznia-Zadanie38_E2_kod
Nazwisko_ucznia-Zadanie38_E3	Nazwisko_ucznia-Zadanie38_E3_kod
Nazwisko_ucznia-Zadanie38_E4	Nazwisko_ucznia-Zadanie38_E4_kod

Do przycisków(patrz powyżej) dodaj pliki o nazwach jak poniżej.

Nazwa pliku HTML/JS
cztery_pierwsze_litery_nazwiska_z38_e1.kltml np. kowa_z38_e1.html

wstaw do pliku
<title>Kowalski Z38_JS_e1</title>

wstaw do pliku na sam koniec pliku komentarz
<!-- Kowalski Z38_JS_e1 -->

Nazwa pliku KOD
cztery_pierwsze_litery_nazwiska_z38_kod.ktml np. kowa_z38_kod_e1.html

wstaw do pliku
<title>Kowalski Z38_KOD_e1</title>

wstaw do pliku na sam koniec pliku komentarz
<!-- Kowalski Z38_KOD_e1 -->

Nazwa pliku do etapu4
zadanie38_e4_kryniewski_start

Wykonaj:
1)Etap1
Dokonaj kopiowania poniższego pliku:
a)zmień na Twoje nazwisko,

b) Pojęcie rozdzielczość ekranu/monitora oznacza……wpisz sam……→ odszukaj definicję w Necie i wpisz
c) Pojęcie szerokość/wysokość ekranu przeglądarki w pikselach oznacza……wpisz sam…… odszukaj definicję w Necie i wpisz

Uwaga
Istnieją dwa różne pojęcia:

- rozdzielczość ekranu/monitora,
- szerokość/wysokość ekranu przeglądarki w pikselach.

Czyli pamiętaj:
Pamiętaj, że rozdzielczość ekranu to nie to samo co szerokość/wysokość ekranu przeglądarki w pikselach:-)

```
<!DOCTYPE html>
<html lang="pl">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Kowalski Z38</title>
</head
 <html>
  <body>
     <div>Nazwisko ucznia</div><br>
      <div> Pojęcie rozdzielczość ekranu/monitora oznacza……wpisz sam…… </div><br>
      <div> Pojęcie szerokość/wysokość ekranu przeglądarki w pikselach oznacza……wpisz sam…… </div><br>

  </body>
 </html>
<!--  Kowalski Z38_JS  -->
```

Nazwisko ucznia (1cm)

Pojęcie rozdzielczość ekranu/monitora oznacza……wpisz sam……(12mm)

Pojęcie szerokość/wysokość ekranu przeglądarki w pikselach oznacza……wpisz sam…… (50px)

2)Etap2
Jak wykryć aktualny rozmiar ekranu przeglądarki (strony).
Użyj **window.innerWidthi , window.innerHeightaby** aby uzyskać aktualny rozmiar ekranu strony. (NIE uwzględniając pasków narzędzi/pasków przewijania):
Przykład
Wyświetla wysokość i szerokość okna przeglądarki

```
var w = window.innerWidth;
var h = window.innerHeight;
```

a)Dopisz przed </body>

```
        <p id="demo_kryniewski"></p>
```

```
<script>
   var w = window.innerWidth;
   var h = window.innerHeight;
   var x = document.getElementById("demo_kryniewski");
        x.innerHTML = "szerokość przeglądarki: " + w + ", wysokość: " + h + ".";
</script>
```

b)Dopisz w sekcji <head> ……</head> kod poniższy i zmień kryniewski na Twoje nazwisko (bez polskich liter)

```
<style>
    #demo_kryniewski {
    color: red;
    font-size: 40px;
    }
</style>
```

c)a otrzymasz

Nazwisko ucznia (1cm)

Pojęcie rozdzielczość ekranu/monitora oznacza……wpisz sam……(12mm)

Pojęcie szerokość/wysokość ekranu przeglądarki w pikselach oznacza……wpisz sam…… (50px)

szerokość przeglądarki: 1485, wysokość: 827.

d)wstaw ten rzut ekranu na Twoją stronę.(ten powyżej)

e)zmień wymiar przeglądarki i uruchom Twoją stronę a otrzymasz.

Nazwisko ucznia (1cm)

Pojęcie rozdzielczość ekranu/monitora oznacza……wpisz sam……(12mm)

Pojęcie szerokość/wysokość ekranu przeglądarki w pikselach oznacza……wpisz sam…… (50px)

szerokość przeglądarki: 958, wysokość: 628.

f)wstaw ten rzut ekranu na Twoją stronę.(ten powyżej)

3)Etap3

Jak wykryć rozdzielczość ekranu/monitora?

Chcesz sprawdzić jaką **rozdzielczość ekranu** ma ustawioną użytkownik, aby w razie potrzeby poinformować go, że powinien ja zwiększyć, żeby strona WWW była wyświetlana poprawnie ponieważ stronę zaprojektowane na 1024x768 pikseli .

Rozdzielczość ekranu każdy może mieć inną. np.

1920x1200 pikseli
1024x768 pikseli
800x600 pikseli
640x480 pikseli

ale wciąż garść osób korzysta z rozdzielczości 640x480. Można im więc zasugerować, że strony lepiej wyglądają przy rozdzielczości np. 1024x768 pikseli .

screen.width→szerokość ekranu,
screen.height→wysokość ekranu.

a)**Wstaw przykład 2 (jest poniżej). Zmień size na największy z możliwych**

b) **a otrzymasz**

Nazwisko ucznia (1cm)

Pojęcie rozdzielczość ekranu/monitora oznacza……wpisz sam……(12mm)

Pojęcie szerokość/wysokość ekranu przeglądarki w pikselach oznacza……wpisz sam…… (50px)

szerokość przeglądarki: 1485, wysokość: 827.

Twoja rozdzielczość ekranu to:1536 na 960

Przykład1

```
<script>
document.write("Twoja rozdzielczość ekranu to: "+screen.width+"na"+screen.height);
</script>
```

Przykład2

```
<script>
     document.write("<font color=silver size=6>Twoja rozdzielczość ekranu to:</font>");
    document.write("<font color=silver size=6>"+screen.width+"</font>");
    document.write("<font color=silver size=6> na </font>");
    document.write("<font color=silver size=6>"+screen.height+"</font>");
</script>
```

Przykład3

```
<script>
if (screen.width<800 || screen.height<600)
  document.write("Rozdzielczość ekranu powinna być większa niż 800x600");
```

```
</script>
```

Przykład4
Możesz też przekierować użytkownika na stronę przeznaczoną dla wybranej rozdzielczości:

```
<script>
if (screen.width==800 && screen.height==600) {
  window.location="index800.html";
} else if (screen.width==1024 && screen.height==768) {
  window.location="index1024.html";
} else {
  window.location="inna.html";
}
</script>
```

4)Etap4

Temat: Uruchamianie strony w zależności od rozdzielczości ekranu.

a)Dokonaj kopiowania pliku.
Nadaj mu nazwę→ zadanie38_e4_kryniewski_start.html
Zmień na Twoje nazwisko w pięciu miejscach pliku.

```
<!DOCTYPE html>
<html lang="pl">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Kowalski Z38</title>
<body>
<script>
 if (screen.width==800 && screen.height==600) {
  window.location="zadanie38_e4_kryniewski_800.html";
} else if (screen.width==1024 && screen.height==768) {
  window.location="zadanie38_e4_kryniewski_1024.html";
} else {
  window.location="zadanie38_e4_kryniewski_inna.html";
}
</script>
  </body>
 </html>
<!--  Kowalski Z38_e4_JS  -->
```

b)Wykonaj trzy strony o nazwach: (zmień na Twoje nazwisko-bez polskich liter, patrz niżej), Wykonanie trzech nowych stron uzyskasz poprzez skopiowanie trzy razy strony z etapu:

index800_kryniewski.html
index1024_kryniewski.html
inna_kryniewski.html

c)Uruchom stronę

d)Odczytaj rozdzielszość i zmień linię kodu:
else if (screen.width==1024 && screen.height==768)
na **twoje wymiary** i uruchom program

Zadanie 39.
Temat: Wczytać kolor wyświetlania tekstu w postaci numeru lub tekstu.

Nazwa pliku HTML/JS
cztery_pierwsze_litery_nazwiska_z39.ktml np. kowa_z39.html

wstaw do pliku
<title>Kowalski Z39_JS</title>

wstaw do pliku na sam koniec pliku komentarz
<!-- Kowalski Z39_JS -->

Nazwa pliku KOD
cztery_pierwsze_litery_nazwiska_z39_kod.ktml np. kowa_z39_kod.html

wstaw do pliku
<title>Kowalski Z39_KOD</title>

wstaw do pliku na sam koniec pliku komentarz
<!-- Kowalski Z39_KOD -->

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie39)
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie39-KOD)
3)Przeczytaj temat oraz treść zadania.
4)Dokonaj kopiowania kodu zadania (patrz poniżej) i uzupełnij aby rozwiązać zadanie.
5)Zmienne powinna mieć trzy litery nazwiska ucznia np.:
opcja_dana_kow
5)Dokonaj testowania zadania
6)Podłącz plik cztery_pierwsze_litery_nazwiska_z39.ktml do przycisku Nazwisko_ucznia-Zadanie39
7)Podłącz plik cztery_pierwsze_litery_nazwiska_z39_kod.ktml do przycisku Nazwisko_ucznia-Zad39

Treść zadania.

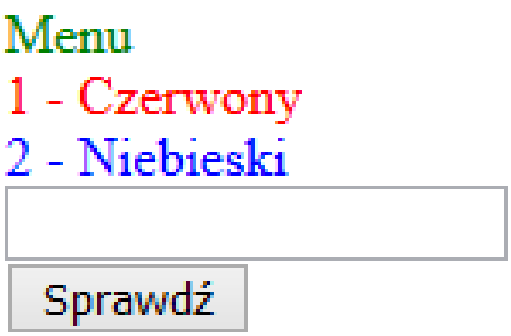
Jeżeli:
wczytamy liczbę (1) **lub** (słowo „czerwony”) to [tekst uzyskamy tekst *“wczytałeś kolor czerwony”* wyświetli się na czerwono czcionką wielkość o numerze 7 (size)]
wczytamy liczbę (2) **lub** (słowo „niebieski”) to [tekst uzyskamy tekst *“wczytałeś kolor niebieski”* wyświetli się na niebiesko wielkość o numerze 1 (size)]
Wskazówka: Wielkość liter (duże czy małe litery) wczytywanych danych nie będzie miało znaczenia. Użyj metody x.toLowerCase() [dla zmiennej x]
Wskazówka:

Wykonaj menu: (wygląd patrz poniżej)

MENU: (napis kolorem zielonym)

1-CZERWONY (napis kolorem czerwonym)

2-NIEBIESKI (napis kolorem niebieskim)

Wygląd ekranu dwa przykłady (jeden dwóch, uczeń wybiera sam)	
Przykład 1	**Przykład 2**
	
Po kliknięciu przycisku **Oblicz** wyświetli się okienko do wczytywania danych, zostanie wyświetlone wynik zgodnie z treścią zadania	Po wpisaniu danych oraz kliknięciu przycisku **Sprawdź**, zostanie wyświetlone wynik zgodnie z treścią zadania

Przykładowe rozwiązanie: (zmienne z trzema literami, funkcje z trzema literami→ zmień)

```
<input type="button" name="nazwisko_ucznia" value="Oblicz" onclick="fun_kow()" />
 <script>
  function fun_kow() {
   var x_kow = prompt("Podaj kolor", "");
    if(x_kow == 1 || x_???.toLowerCase() == "czerwony") {
                document.write('<h1 style="color:???;">wczytałeś kolor ?????</h1><br />'); }
    else if(x_kow == 2 || x_???.toLowerCase() == "????????????") {
                document.write('<h6 style="????????????;">wczytałeś kolor ??????</h6><br />');
                }
                }
        </script>
```

Zadanie 40.

Temat: Program obliczania wartości funkcji „f(x)” dla wczytanego argumentu „x”. Funkcja podana jest w postaci „klamerkowej”

Nazwa pliku HTML/JS

cztery_pierwsze_litery_nazwiska_z40.kml np. kowa_z40.html

wstaw do pliku

<title>Kowalski Z40_JS</title>

wstaw do pliku na sam koniec pliku komentarz
<!-- Kowalski Z40_JS -->

Nazwa pliku KOD
cztery_pierwsze_litery_nazwiska_z40_kod.ktml np. kowa_z40_kod.html

wstaw do pliku
<title>Kowalski Z40_KOD</title>

wstaw do pliku na sam koniec pliku komentarz
<!-- Kowalski Z40_KOD -->

Równanie w postaci klamerkowej.

$$f(x) = \begin{cases} 1 \; dla \; x \in [1; \infty) \\ x^2 dla \; x \in (-2; 1) \\ 4 \; dla \; x \in (-\infty; 1] \end{cases}$$

Założenie do programu:
1)Funkcja w postaci klamerkowej w programie będzie zdefiniowana jako funkcja z jednym parametrem **x_kow** o nazwie function **f_kow**(x_kow)
2)program będzie się pytał o wartość x (argument) i dla tej wartości obliczy wartość funkcji.
3)Dwa miejsca po przecinku.
4)Dokonaj kopiowania programu.
5)Zmień komentarze (html oraz Java) na linie kodu.
6)Dopisz do kodu : odpowiednie wypisanie wartości funkcji oraz dwa miejsca po przecinku .
Patrz rzut ekranu

wykonał Jan Kowalski
czytany argument x=0.50
f(0.50)=0.25

7)Przetestuj dla x=0.3 x=5 x=-3
8)Zmień program aby realizował nową funkcję klamerkową:

$$f(x) = \begin{cases} 9 \; dla \; x \in [2; \infty) \\ (x+1)^2 dla \; x \in (-2; 0) \\ x^3 + 1 \; dla \; x \in [0; 2) \\ 1 \; dla \; x \in (-\infty; -2] \end{cases}$$

9)Wykonaj **przycisk** na stronie zaliczeniowej i podłącz cztery_pierwsze_litery_nazwiska_z40.ktml
10) Wykonaj **przycisk** na stronie zaliczeniowej i podłącz
cztery_pierwsze_litery_nazwiska_z40_kod.ktml

```
<!DOCTYPE html>
<html lang="pl-PL">
<!-- polskie litery mają działać(uzupełnij w kodzie strony meta) -->
<!-- title -> Z40 oraz nazwisko ucznia -->
<body>
          <div style="color:red;font-size:50px;">
            <script language="JavaScript">
                    var x=prompt("podaj argument x=","");
                            var x_kow=parseFloat(x);
//zmienna x_kow (zmień _kow na Twoje nazwisko)
//document.write(napisz tekst--> wykonał Jan Kowalski);
                     document.write("czytany argument x="+x_kow+"<br>");

             function f_kow(x_kow) {
                    if(x_kow>=1) {return 1;}
                    if((x_kow=>-2)&&(x_kow<1)) {return x_kow*x_kow;}
                    if(x_kow<-2) {return 4;}
              }

            document.write("f="+f_kow(x_kow)+"<br>");

        </script>
          </div>
 </body>
</html>
<!--  Kowalski Z40_JS  -->
```

Zadanie 41.

Temat: Zadanie z egzaminu zawodowego.

Treść zadania.

1)Po wciśnięciu dowolnego z przycisków, skrypt sprawdzi czy:

Warunek 1

jedno lub oba pola są puste(nie wypełnione przez użytkownika), jeśli są niewypełnione to wyświetli się tekst: *Proszę uzupełnić obie liczby.*

Warunek 2

po wciśnięciu przycisku DZIELENIE, druga liczba jest równa zero. Jeśli tak, wyświetli się tekst *Nie wolno dzielić przez zero.*

2)W innym przypadku skrypt wykona działania dodawania, odejmowania, mnożenia lub dzielenia liczb, w zależności od wybranego przycisku.

3)Po wykonaniu działania zostanie wyświetlony wynik poprzedzony tekstem:

Wynik działania wynosi: ????? (wyniki działań dwa miejsca po przecinku)

4) Komunikaty oraz wyniki powinny być wyświetlone w treści strony pod przyciskami

Nazwa pliku HTML/JS

cztery_pierwsze_litery_nazwiska_Z41.ktml np. kowa_Z41.html

wstaw do pliku

<title>Kowalski Z41_JS</title>

wstaw do pliku na sam koniec pliku komentarz
<!-- Kowalski Z41_JS -->

Nazwa pliku KOD
cztery_pierwsze_litery_nazwiska_Z41_kod.ktml np. kowa_Z41_kod.html

wstaw do pliku
<title>Kowalski Z41_KOD</title>

wstaw do pliku na sam koniec pliku komentarz
<!-- Kowalski Z41_KOD -->

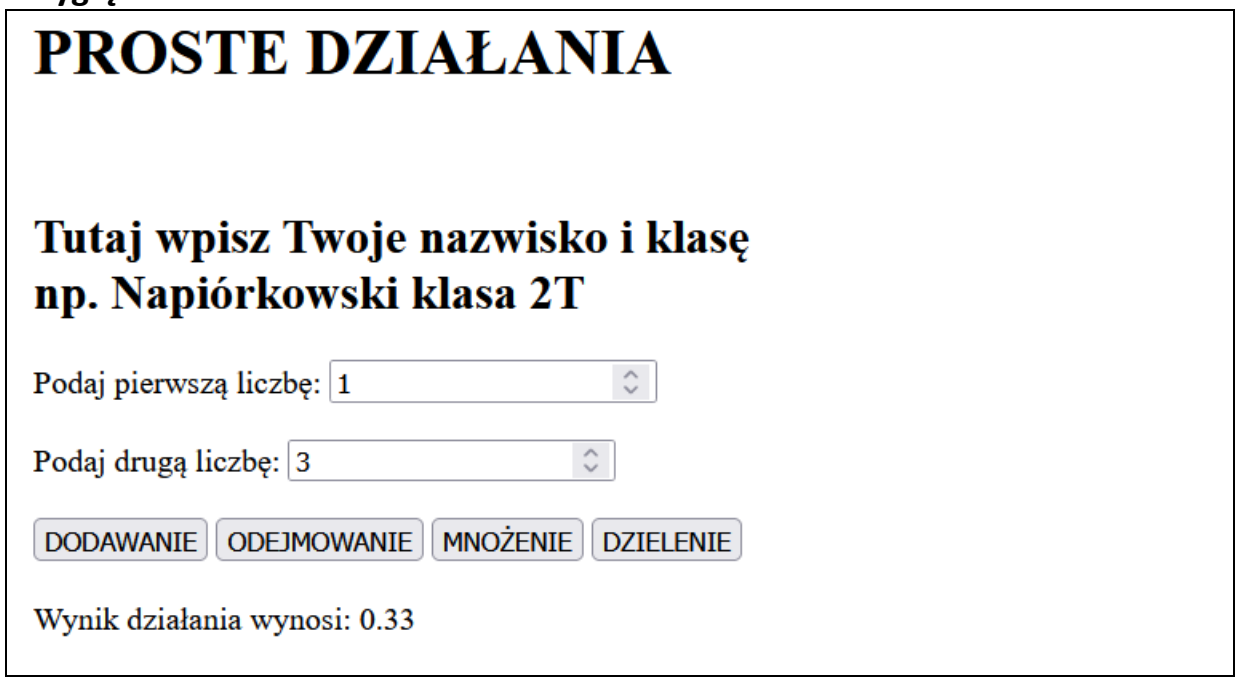
Wykonaj:

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie40)
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie40-KOD)
3)Przeczytaj temat oraz treść zadania.
4)Dokonaj kopiowania kodu zadania(jest na końcu zadania) i **uzupełnij** aby rozwiązać zadanie.
5)Zmień nazwy funkcji: (trzy pierwsze litery nazwisk ucznia)

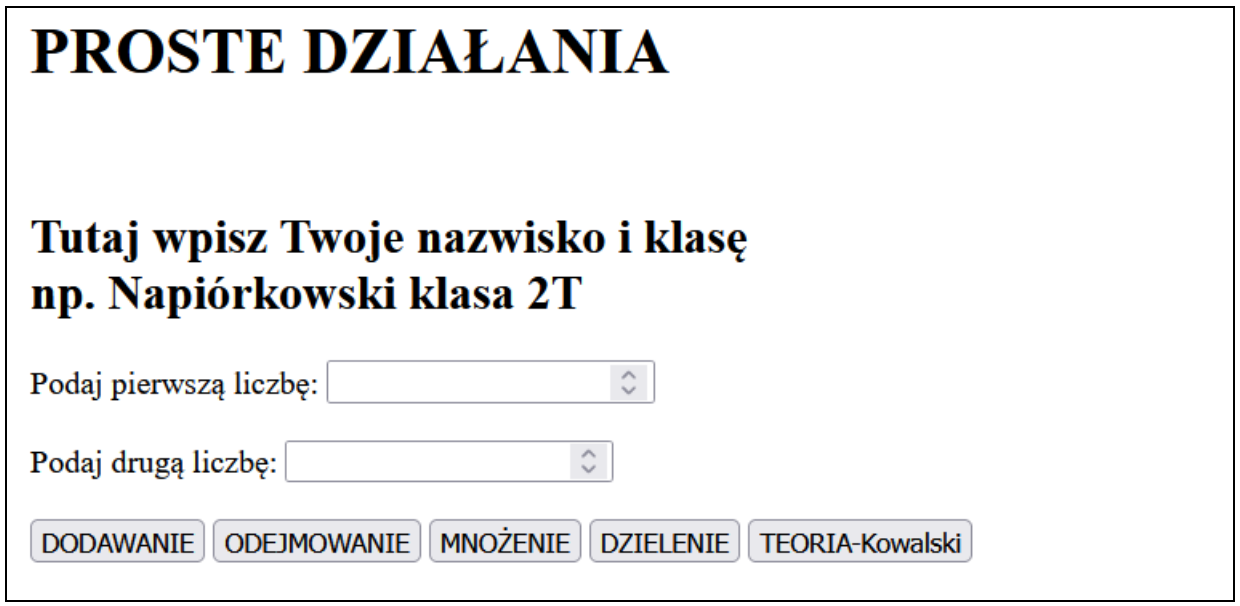
Jest	Ma być
suma()	suma_kow()
roznica()	roznica_kow()
iloczyn()	iloczyn_kow()
iloraz()	iloraz_kow()

6)Zmień kod aby program wypisywał Twoje nazwisko i klasę (zamiast Napiórkowskiego). Dokonaj testowania zadania.

Wygląd ekranu.



7)Dopisz nową opcję TEORIA. Wykonaj przycisk TEORIA-Kowalski (tutaj Twoje nazwisko).



8)Dokonaj oprogramowania przycisku TEORIA-Kowalski poprzez utworzenie nowej funkcji o nazwie:
function teoria_kow()
Działanie tego przycisku będzie wyświetlało linijki kodu wraz z wytłumaczeniem. Lnie kodów wraz z wytłumaczeniami masz poniżej na następnej stronie).

1. Wyświetlą się Pierwsza linia kodu+opis
2. Wyświetlą się Druga linia kodu+opis
3. Wyświetlą się Trzecia linia kodu+opis
4. Wyświetlą się Czwarta linia kodu+opis
5. Wyświetlą się Piąta linia kodu+opis

Poniżej masz efekt działanie kodu ale tylko Pierwsza linia kodu+opis

```
pierwsza linia teorii
<label for="a">Podaj pierwszą liczbę:</label><input type="number" id="a">
wytłumaczenia
wykonanie numerycznego pola formularza z opisem 'Podaj pierwszą liczbę:' a nazwa pola to 'a'
```

Efekt taki uzyskasz poprzez:

```
function teoria()
{
document.write("<font color=green size=4>pierwsza linia teorii</font>"+"<br>");
document.write("<font color=red size=4>&lt;</font>"+"<font color=red size=4>label
for=</font>"+"<font color=red size=4>&quot;</font>");
document.write("<font color=red size=4>a</font>"+"<font color=red size=4>&quot;</font>"+"<font
color=red size=4>&gt;</font>");
document.write("<font color=red size=4>Podaj pierwszą liczbę:</font>");
document.write("<font color=red size=4>&lt;</font>"+"<font color=red
size=4>/label</font>"+"<font color=red size=4>&gt;</font>");
document.write("<font color=red size=4>&lt;</font>"+"<font color=red size=4>input
type=</font>"+"<font color=red size=4>&quot;</font>");
document.write("<font color=red size=4>number</font>"+"<font color=red
size=4>&quot;</font>");
document.write("<font color=red size=4> id=</font>"+"<font color=red size=4>&quot;</font>");
document.write("<font color=red size=4>a</font>"+"<font color=red size=4>&quot;</font>"+"<font
color=red size=4>&gt;</font>"+"<br>");
document.write("<font color=green size=4>wytłumaczenia</font>"+"<br>");
document.write("<font color=red size=4>wykonanie numerycznego pola formularza z opisem 'Podaj
pierwszą liczbę:' a nazwa pola to 'a' </font>"+"<br>");
}
```

Uwaga**: Po kopiowaniu kodu złącz linijki kodu**, tak aby document.write tworzyły jedną linię.

Dopisz pozostałe linie kody wraz z wytłumaczeniem. (ma być 5 linii kodu+wytłumaczeniem)

Kod	Znak
>	>
"	"
<	<

9) cztery_pierwsze_litery_nazwiska_z40.ktml do przycisku Nazwisko_ucznia-Zadanie40
10) cztery_pierwsze_litery_nazwiska_z40_kod.kml do przycisku Nazwisko_ucznia-Zad40

Pierwsza linia kodu → Wytłumaczenie linijki kodu.

```
<label for="a">Podaj pierwszą liczbę:</label> <input type="number" id="a">
```

Ta linijka kodu HTML tworzy interfejs użytkownika składający się z **etykiety (label)**[opis wprowadzania danych, czyli tekst przed polem input] i **pola do wprowadzania danych (input)** w języku JavaScript (oraz HTML). **label** i **input** współpracują między sobą.

Oto szczegółowy opis działania każdego elementu:

- `<label for="a">Podaj pierwszą liczbę:</label>`: Jest to element etykiety

(<label>), który jest używany do identyfikacji pola formularza. Atrybut for w etykiecie odnosi się do identyfikatora (id) elementu formularza, do którego etykieta jest przypisana. W tym przypadku, for="a" oznacza, że etykieta jest przypisana do pola formularza z identyfikatorem id="a". Tekst Podaj pierwszą liczbę: jest wyświetlany jako opis pola formularza, pomagając użytkownikowi zrozumieć, jakie informacje powinien wprowadzić.

- <input type="number" id="a">: Jest to element formularza (<input>), który pozwala użytkownikowi na wprowadzenie danych. Atrybut type="number" określa, że pole przyjmie tylko wartości liczbowe, co oznacza, że użytkownik może wprowadzić tylko liczby (całkowite oraz zmiennoprzecinkowe, w zależności od przeglądarki i ustawień regionalnych). Atrybut id="a" nadaje temu polu formularza unikalny identyfikator, który może być używany w JavaScript i CSS do manipulacji lub stylizacji tego elementu.

W praktyce, połączenie etykiety z polem formularza poprzez for i id ułatwia użytkownikom korzystanie z formularzy, szczególnie w przypadku osób korzystających z czytników ekranu, ponieważ po kliknięciu na etykietę, fokus zostaje automatycznie przesunięty do przypisanego pola formularza, co ułatwia wprowadzanie danych.

Druga linia kodu → Wytłumaczenie linijki kodu.

<input type="button" value="DODAWANIE" onclick="suma()">

Wykonanie przycisku o nazwie DODAWANIE i z takim samym napisem, kliknięcie myszką uruchamia funkcję suma().

Trzecia linia kodu → Wytłumaczenie linijki kodu.

var a = document.getElementById("a").value;

wpisanie do zmiennej "a" wartości z elementu o Id "a" (zbieżność nazw)

Czwarta linia kodu → Wytłumaczenie linijki kodu.

if (a == "" || b == "")

sprawdzenie czy pola "a" lub "b" są puste.

Piąta linia kodu → Wytłumaczenie linijki kodu.

document.getElementById("wynik").innerHTML = "Wynik działania wynosi: " + suma;

pobranie wartości Id o nazwie **"wynik"** i wyprowadzenie w oknie przeglądarki tekstu "Wynik działania wynosi: " oraz wartości zmiennej **suma**;

Kod programu.

```
<!DOCTYPE html>
<html lang="pl">
<head>
	<title>Zadanie JavaScript</title>
	<meta charset="utf-8">
	<style>
		h1 { text-transform: uppercase; }

	</style>
</head>
<body>

	<h1>Proste działania</h1>
	<label for="a">Podaj pierwszą liczbę:</label> <input type="number" id="a">
	<br><br>
	<label for="b">Podaj drugą liczbę:</label> <input type="number" id="b">
	<br><br>
	<input type="button" value="DODAWANIE" onclick="suma()">
	<input type="button" value="ODEJMOWANIE" onclick="roznica()">
	<input type="button" value="MNOŻENIE" onclick="iloczyn()">
	<input type="button" value="DZIELENIE" onclick="iloraz()">
	<div id="wynik" style="margin-top:20px;"></div>
	<script>
		function suma()
		{
			var a = document.getElementById("a").value;
			var b = document.getElementById("b").value;
			if (a == "" || b == "")
			{
			document.getElementById("wynik").innerHTML = "Proszę uzupełnić obie liczby.";
			} else
			{
				a = parseFloat(a);
				b = parseFloat(b);
				var suma = a + b;
		document.getElementById("wynik").innerHTML = "Wynik działania wynosi: " + suma;
			}
			//alert(a);
			//console.log(a);
		}
		function roznica()
		{
			var a = document.getElementById("a").value;
			var b = document.getElementById("b").value;
			if (a == "" || b == "")
			{
		document.getElementById("wynik").innerHTML = "Proszę uzupełnić obie liczby.";
			} else
			{
				a = parseFloat(a);
				b = parseFloat(b);
				var roznica = a - b;
		document.getElementById("wynik").innerHTML = "Wynik działania wynosi: " + roznica;
			}
		}
```

```
		function iloczyn()
		{
			var a = document.getElementById("a").value;
			var b = document.getElementById("b").value;
			if (a == "" || b == "")
			{
		document.getElementById("wynik").innerHTML = "Proszę uzupełnić obie liczby.";
			} else
			{
				a = parseFloat(a);
				b = parseFloat(b);
				var iloczyn = a * b;
		document.getElementById("wynik").innerHTML = "Wynik działania wynosi: " + iloczyn;
			}
		}

		function iloraz()
		{
			var a = document.getElementById("a").value;
			var b = document.getElementById("b").value;
			if (a == "" || b == "")
			{
		document.getElementById("wynik").innerHTML = "Proszę uzupełnić obie liczby.";
			} else
			{
				a = parseFloat(a);
				b = parseFloat(b);
				if (b == 0)
				{
		document.getElementById("wynik").innerHTML = "Nie wolno dzielić przez zero.";
				}
				else
				{
					var iloraz = a / b;
		document.getElementById("wynik").innerHTML = "Wynik działania wynosi: " + iloraz;
				}
			}
		}
	</script>
</body>
</html>
```

Zadanie 42. (na ocenę celującą)

Temat: Zapis i użycie przedziałów liczbowych.

Nazwa pliku HTML/JS

cztery_pierwsze_litery_nazwiska_z42.kltml np. kowa_z42.html

wstaw do pliku

<title>Kowalski Z42_JS</title>

wstaw do pliku na sam koniec pliku komentarz

<!-- Kowalski Z42_JS -->

Nazwa pliku KOD
cztery_pierwsze_litery_nazwiska_z42_kod.ktml np. kowa_z42_kod.html

wstaw do pliku
<title>Kowalski Z42_KOD</title>

wstaw do pliku na sam koniec pliku komentarz
<!-- Kowalski Z42_KOD -->

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie38)
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie38-KOD)
Po wykonaniu zadania:
3)Podłącz plik zadanie39_kow.html do przycisku Nazwisko_ucznia-Zadanie38
4)Podłącz plik zadanie39_kow_KOD.html do przycisku Nazwisko_ucznia-Zad38

Treść zadania:

Program będzie miał trzy opcje menu. Wybór opcji (wejście do opcji) będzie zależny od:
Liczba liter imienia-wyświetlanie teorii o operatorach logicznych
Liczba liter nazwiska+20- sprawdzanie czy liczba należy do przedziału (od ; do]
Dzień urodzenia+50- sprawdzanie czy liczba należy do przedziału (-oo liczba1) lub [liczba2 +oo)

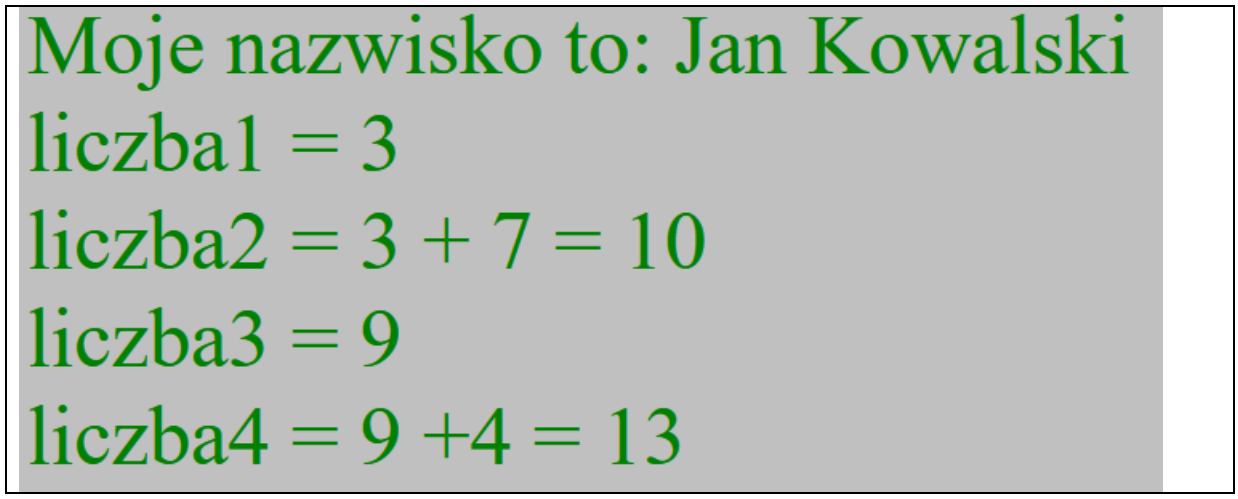
Uwaga:
nawiasy "["]" oznaczają przedziały zamknięte.
nawiasy "(")"oznaczają przedziały zamknięte.

1)Dokonaj obliczenia liczba1 liczba2 liczba3 liczba4 wg wzorów poniżej:

Obliczone liczby, będziesz te liczby wstawał/używał do wyświetlonego/(warunki if) w menu:
liczba1 = liczba liter imienia
liczba2 = liczba liter imienia + 7
liczba3 = liczba liter nazwiska
liczba4 = liczba liter nazwiska +4

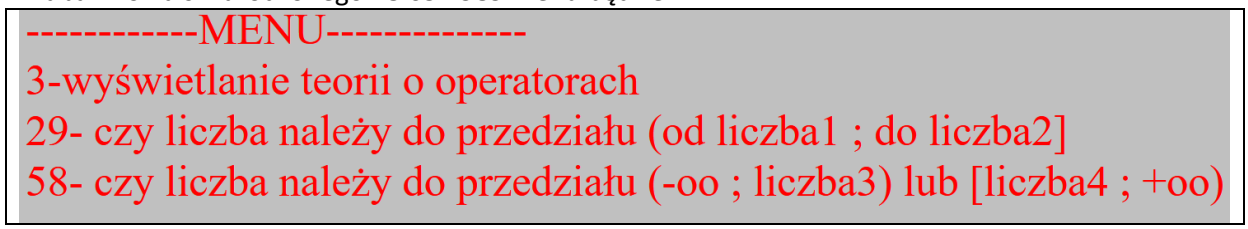
np. dla imienia i nazwiska Jan Kowalski
liczba1 = 3
liczba2 = 3 + 7 = 10
liczba3 = 9
liczba4 = 9 +4 = 13

2)Wypisz na ekranie Twoje nazwsko i imię oraz obliczone liczby.
Kolor znaków zielony, wielkość 40+[(numer w dzienniku) reszta_z_dzielenia_przez 7] px. Tło srebrne.



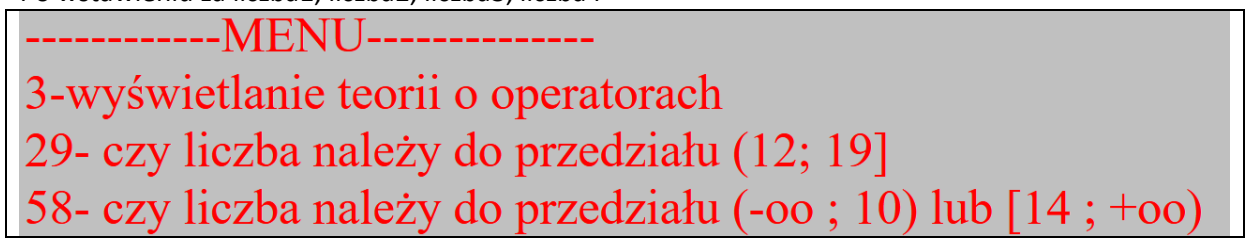
3)Wykonaj menu z wstawionymi liczba1, liczba2, liczba3, liczba4

Dla Jan Kowalski urodzonego 18.05.1989 menu będzie:



Kolor znaków czerwony, wielkość 40+[(numer w dzienniku) reszta_z_dzielenia_przez 7] px. Tło srebrne.

Po wstawieniu za liczba1, liczba2, liczba3, liczba4



---

4)Dokonaj wczytania opcji
Uwaga:
Wszystkie zmienne z trzema literami nazwiska np. opcja_kow

Program w okienku promt będzie się pytał o numer opcji.



5)Dokonaj oprogramowania wyboru opcji wg. **wskazówek poniżej**

Do wyboru opcji 3 oraz 29 oraz 58 użyj konstrukcji:

```
if (opcja==3)
{document.write("<font color=green size=7>Jesteś w opcja 3 --> teoria</font>"+"<br>");
……
……
}
```

uwaga:
Gdy wczytasz nie istniejącą opcję (tutaj opcja=3 lub opcja=29 lub opcja=58). To uzyskasz następujący efekt:

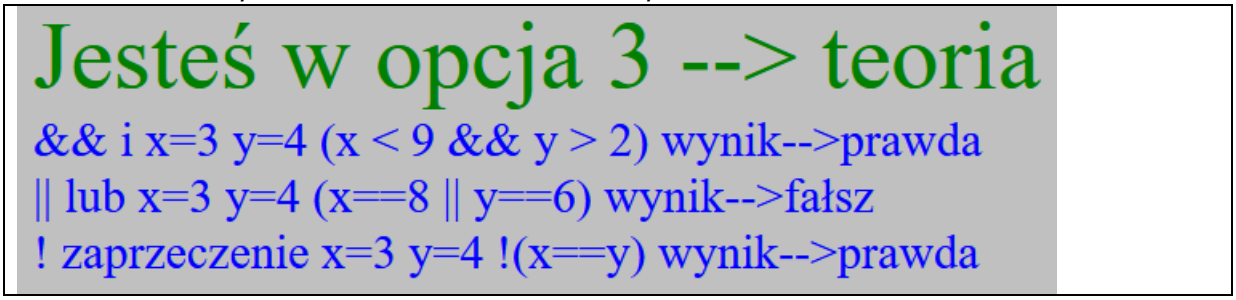


możesz to uzyskać poprzez:

```
if((opcja!=3)&&(opcja!=29)&&(opcja!=58))
 {
   document.write("<font color=red size=7> nie ma takiej opcji</font>"+"<br>");
}
```

6)Dokonaj oprogramowania opcji 3 (u ciebie będzie inna zależna od ilości liter imienia).

Po wyborze opcji 3 (Ty będziesz miał inny numer opcji w twoim programie) uzyskasz:
Zmień kolor zielony na niebieski a niebieski na zielony.



7)Dokonaj oprogramowania opcji 29 (u ciebie będzie inna zależna od liczba liter nazwiska+20).

- sprawdzanie czy liczba należy do przedziału (od ; do] użyj instrukcję **if**

Po wyborze **opcji 29** (Ty będziesz miał inny numer opcji w twoim programie) nastąpi pytanie o liczbę i sprawdzenie czy należy do przedziału (podanego w menu), Ty będziesz miał swój indywidualny przedział. Do zdefiniowania przedziałów użyj informacji w tabeli poniżej.

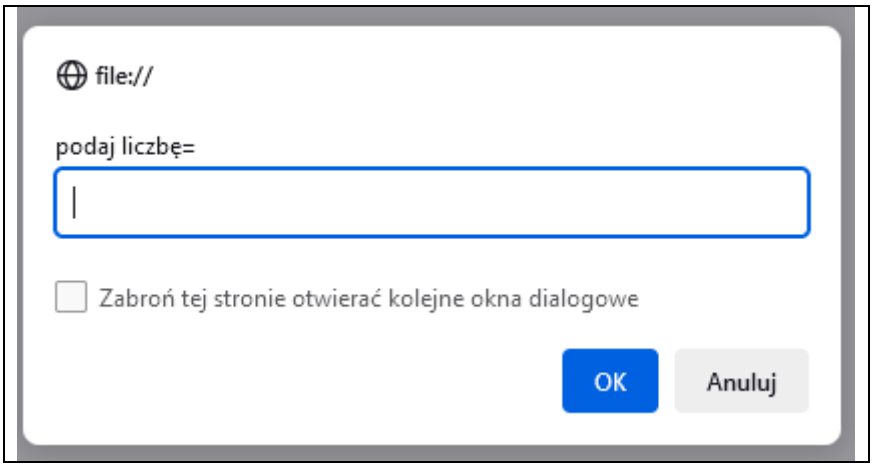
Przedział liczbowy matematyka	Przedział liczbowy informatyka
$x \in < -2,3)$	(x>=-2)&&(x<3)
$x \in (-\infty, -4 > \cup (5,+\infty)$	(x<=-4)||((x>5)

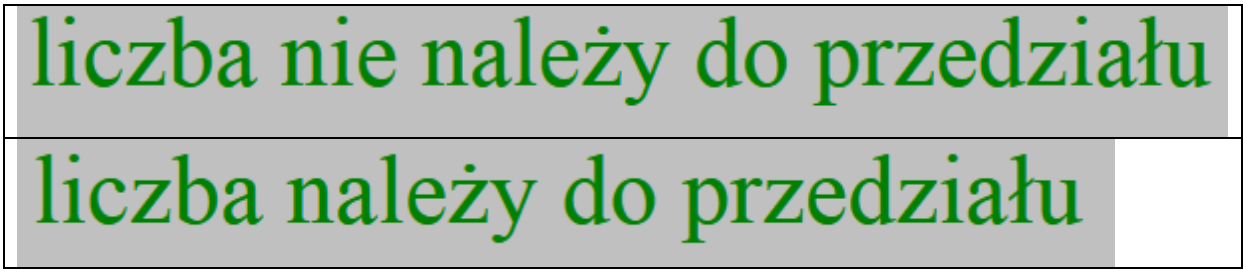
Uwaga:
nawiasy "[" "]" oznaczają przedziały zamknięte.
nawiasy "(" ")"oznaczają przedziały zamknięte.



8)Dokonaj wczytania liczby
Uwaga:
Wszystkie zmienne z trzema literami nazwiska np. liczba_kow

Teraz pytanie o liczbę





==========

9)Dokonaj oprogramowania opcji 58 (u ciebie będzie inna zależna od liczba liter dzień urodzenia+50).

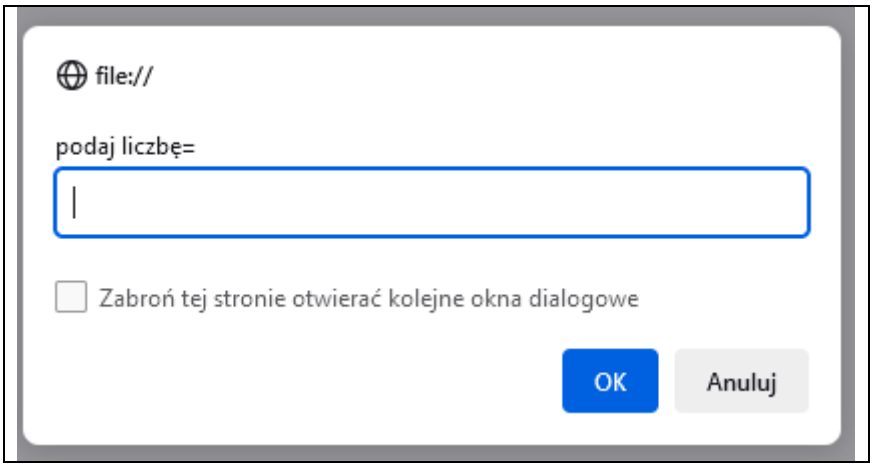
- sprawdzanie czy liczba należy do przedziału (-oo liczba1) lub [liczba2 +oo) użuj instrukcj **if**

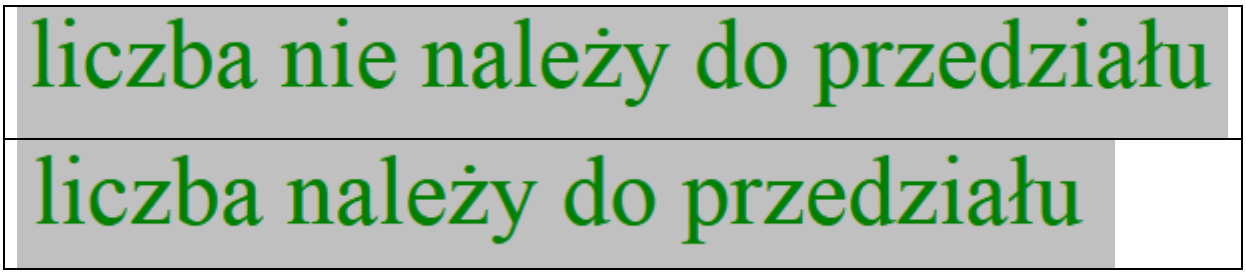
Po wyborze **opcji 58** (Ty będziesz miał inny numer opcji w twoim programie) nastąpi pytanie o liczbę i sprawdzenie czy należy do przedziału (podanego w menu), Ty będziesz miał swój indywidualny przedział. Do zdefiniowania przedziałów użyj informacji w tabeli powyżej.

10)Dokonaj wczytania liczby

Uwaga:

Wszystkie zmienne z trzema literami nazwiska np. liczba2_kow





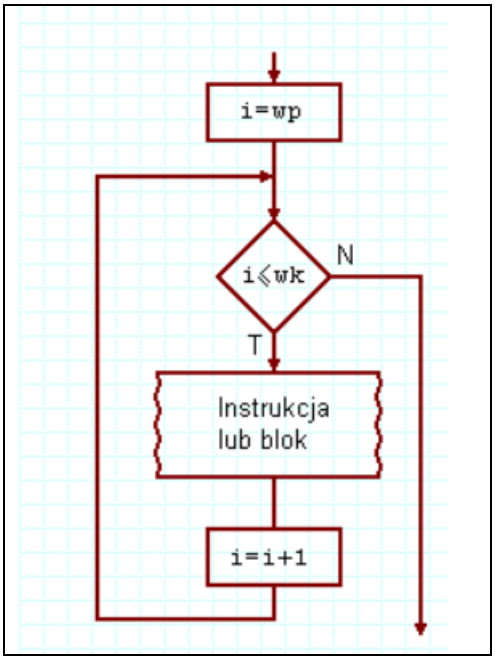
==========koniec zadania 42==========

Dział 6 Pętle

Rodzaje pętli.

1)Pętla for

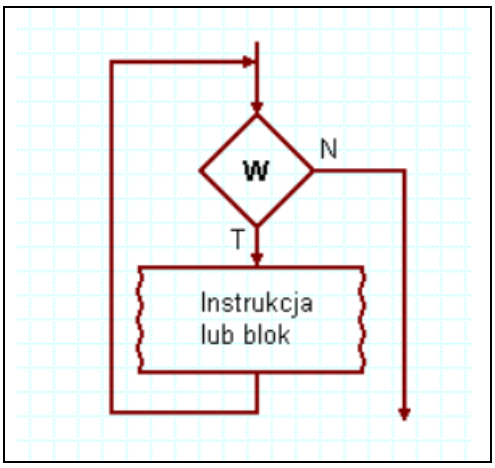
```
for (zainicjowanie_zmiennej;   warunek_kończący_wykonywanie_pętli;
zmiana_zmiennej) {
    kod który zostanie wykonany pewną ilość razy
}
```



2)Pętla while→z kontrolowanym wejściem

```
while (wyrażenie_sprawdzające_zakończenie_pętli) {
    ...fragment kodu który będzie powtarzany...
}
```

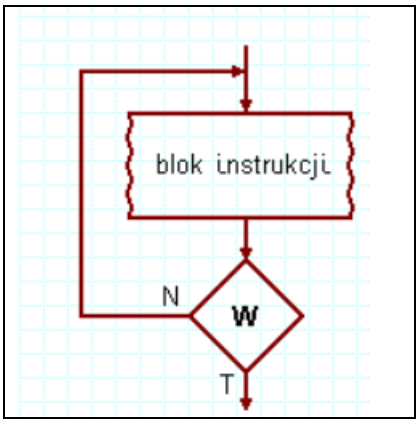
pętla while najpierw sprawdza warunek, potem coś wykonuje, pętla może się nie wykonać.



3)Pętla do while→z kontrolowanym wyjściem

```
Do
{
    ...fragment kodu który będzie powtarzany...
} while (false)
```

pętla do...while zawsze wykona jedną iterację, zanim sprawdzi warunek. Zawsze więc wykona jakieś zadanie



Zadanie 43.
Temat: Teoria dotycząca pętli

Wykonaj:

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie41).
2)Wykonaj stronę, która wyświetli informacje o trzech rodzajach pętli:

- Pętla for
- Pętla while→z kontrolowanym wejściem
- Pętla do while→z kontrolowanym wyjściem

Wyświetlając informacje uwzględnij:

- nazwa pętli
- konstrukcja
- opis
- schemat blokowy

Zadanie 44.
Temat: Użycie pętli **for** do wypisywania Twoje nazwiska 10 razy.

Wykonaj:

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie42),
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie42-KOD),
3)Przeczytaj temat oraz kod zadania,
4)Dokonaj kopiowania kodu zadania(jest na końcu zadania) i **uzupełnij** wpisaniem Twojego nazwiska,
5)Zmień nazwę zmiennej „i” na i_pierwsza_litera_nazwiska np. i_k: (trzy pierwsze litery nazwiska ucznia),
6) Zapewnij wyświetlanie polskich liter w standardzie Utf8.

```
<html>
 <body>
         <div style="color:red;font-size:50px;">
           <script language="JavaScript">
         for(i=1;i<=10;i++)
          {
             document.write("wyświetlam "+i+" raz"+"  tutaj wpisz Twoje nazwisko"+"<br>");
          }
       </script>
         </div>
 </body>
</html>
```

Zadanie 45.

Temat: Wypisz wszystkie liczby parzyste od 2 do liczby wczytanej w okienku.

Wykonaj:

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie43),
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie43-KOD),
3)Wczytanie końca pętli z użyciem instrukcji-okienka **prompt**
4)Nazwa zmiennej „i" na i_pierwsza_litera_nazwiska np. i_k: (trzy pierwsze litery nazwiska ucznia), inne zmienne z trzema literami nazwiska ucznia np. **koniec_petli_kow**
5) Zapewnij wyświetlanie polskich liter w standardzie Utf8.

Zadanie 46.

Temat: Użycie jednej pętli **for** do wypisywania:

Treść zadania.
Wyświetlaj Twoje **nazwisko** lub **imię** na przemian.
Gdy wyświetlasz nieparzysty raz to wyświetli się Twoje **nazwisko**
Gdy wyświetlasz parzysty raz to wyświetli się Twoje **imię**

Ilość wyświetleń (imienia lub nazwiska) to:
Ile_razy_wyświetlić=[(liczba_liter_nazwiska) mod 4]+10 (oblicz w pamięci i wstaw do pętli)
Dla określenia czy liczba jest parzysta czy nie użyj if…. else……. oraz reszty z dzielenia (%)
Wielkość liter:
Wielkość_liter=[(liczba_liter_nazwiska) mod 3]+40px (oblicz w pamięci i wstaw do programu)

Wykonaj:

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie44),
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie44-KOD),
3)Przeczytaj temat oraz treść zadania,
4)Nazwa zmiennej „i" na i_pierwsza_litera_nazwiska np. i_k: (trzy pierwsze litery nazwiska ucznia), inne zmienne z trzema literami nazwiska ucznia np. **koniec_petli_kow**
5) Zapewnij wyświetlanie polskich liter w standardzie Utf8.
6)Przed wyświetlania Twojego nazwiska i imienia [w pętli], wyświetl obliczoną ilość wykonania pętli oraz wielkość liter w px.
Np.
ilość wykonania pętli = 16
wielkość liter = 44px

Zadanie 47.

Temat: Użycie jednej pętli **for** do wypisywania twojego nazwiska lub imienia.

Treść zadania.

Wczytujemy jedną zmienną w okienku prompt.
I gdy:
→wczytana dana jest parzysta. Wypisujemy Twoje **nazwisko** (ile razy patrz poniżej) dla danej wczytanej w okienku. Kolor wyświetlania to niebieski.
→wczytana dana jest nieparzysta. Twoje **imienię** (ile razy patrz poniżej) dla danej wczytanej w okienku Kolor wyświetlania to zielony.

Ilość wyświetleń (imienia lub nazwiska) to:
Ile_razy_wyświetlić=[(liczba_liter_nazwiska) mod 4]+10 (oblicz w pamięci i wstaw do pętli)
Dla określenia czy liczba jest parzysta czy nie użyj if…. else……. oraz reszty z dzielenia (%)
Wielkość liter:
Wielkość_liter=[(liczba_liter_nazwiska) mod 3]+40px (oblicz w pamięci i wstaw do programu)
Zapewnij wyświetlanie polskich liter w standardzie Utf8.

Wykonaj:

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie45),
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie45-KOD),
3)Przeczytaj temat oraz treść zadania,
4)Nazwa zmiennej „i" na i_pierwsza_litera_nazwiska np. i_k: (trzy pierwsze litery nazwiska ucznia), inne zmienne z trzema literami nazwiska ucznia np. **koniec_petli_kow**
5) Zapewnij wyświetlanie polskich liter w standardzie Utf8.
6)Przed wyświetlania Twojego nazwiska i imienia [w pętli], wyświetl obliczoną ilość wykonania pętli oraz wielkość liter w px.
Np.
ilość wykonania pętli = 16
wielkość liter = 44px

Zadanie 48.

Temat: Użycie **do…… while…….** do sprawdzania wczytywanych danych.

Wykonaj:
1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie46)
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie46-KOD)
3)Przeczytaj temat oraz treść zadania.
4)Dokonaj kopiowania zadania i uzupełnij aby rozwiązać zadanie.
5)Zmień nazwy zmiennych a i h na a_kow h_kow (trzy pierwsze litery nazwisk ucznia)
6)Dokonaj testowania zadania
7)Podłącz plik zadanie46_kow.html do przycisku Nazwisko_ucznia-Zadanie46
8)Podłącz plik zadanie46_kow_KOD.html do przycisku Nazwisko_ucznia-Zad46

Treść zadania:
Program sprawdza czy wczytane zmienne: wysokość oraz podstawa trójkąta są większe od zera.
Wczytywanie następuje z użyciem **do…… while…….** tak długo aż wczytane dane będą większe od zera.

Zwróć uwagę, że wyjście z pierwszej pętli **do…… while……..** następuje z użyciem warunku while (h<=0);
a z drugiej pętli z użycie operatora zaprzeczenia „!” (wykrzyknik) while(!(a>0));.

```
<!DOCTYPE html>
  <head>
      <title>Sprawdzenie danych Twoje nazwisko </title>
      <meta charset="utf-8">
   </head>
 <body>
         <div style="color:red;font-size:50px;">
           <script language="JavaScript">
      do {
                  var liczba1=prompt("wysokość=","");
          var h=parseFloat(liczba1);
        }
      while ( h<=0);
      do {
                  var liczba1=prompt("podstawę=","");
          var a=parseFloat(liczba1);
        }
      while(!( a>0));
                  var Pole=a*h/2;
                   document.write("Pole="+Pole.toFixed(2)+" cm^2"+"<br>");
           </script>
         </div>
 </body>
</html>
```

Zadanie 49.

Temat: Użycie **do…… while……..** do sprawdzania wczytywanych danych.

Wykonaj:
1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie47)
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie47-KOD)
3)Przeczytaj temat oraz treść zadania.
4)Rozwiązać zadanie.
5)Nazwy zmiennych a b c (boki trójkąta) a_kow b_kow c_kow (trzy pierwsze litery nazwisk ucznia).
6)Dokonaj testowania zadania.
7)Podłącz plik zadanie47_kow.html do przycisku Nazwisko_ucznia-Zadanie47
8)Podłącz plik zadanie47_kow_KOD.html do przycisku Nazwisko_ucznia-Zad47

Treść zadania:
Wczytujemy trzy boki trójkąta. Program sprawdza czy wczytane trzy boki trójkąta tworzą trójkąt. Jeśli nie to komputer ponownie pyta się o boki trójkąta, aż do skutku. Warunki to: każdy bok musi być większy od zera oraz boki muszą spełniać nierówność trójkąta w trzech kombinacjach
czyli (a>0) i (b>0) i (c>0) i (a+b>c) i (a+c>b) i (c+b>a)

Zadanie 50.

Temat: Napisać program obliczający sumę szeregu harmonicznego od wyrazu pierwszego do milionowego.

Wykonaj:

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie48)
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie48-KOD)
3)Przeczytaj temat oraz treść zadania.
4)Rozwiązać ćwiczenia wstępne. Przeczytaj algorytm w postaci liczby kroków.
5)Nazwy zmiennych **suma** to **suma_kow** (trzy pierwsze litery nazwisk ucznia).
6)Rozwiąż zadanie i dokonaj testowania zadania.
7)Podłącz plik zadanie48_kow.html do przycisku Nazwisko_ucznia-Zadanie48
8)Podłącz plik zadanie48_kow_KOD.html do przycisku Nazwisko_ucznia-Zad48

Ćwiczenia wstępne:
oblicz:

$$suma = \sum_{2}^{4} i^2$$

$$suma = \sum_{1}^{3} \frac{i}{i+2}$$

zapisz w postaci sumy:

$$suma = 1 + 8 + 27 + 64$$

Wyrazu szeregu harmonicznego powstają następująco:

$$a_n = \frac{1}{n}$$

sumę wyrazów od pierwszego do milionowego można zapisać z użyciem znaku sigmy

$$suma = \sum_{i=1}^{1000000} \frac{1}{i}$$

Lista kroków obliczania sumy szeregu harmonicznego od wyrazy 1 do 1000000

Krok1 Zainicjuj zmienną sumy na zero czyli suma_kow=0;

Krok2 Ustaw zmienną sterującą i_kow na jeden czyli i_kow=1;

Krok3 Dodaj do sumy_kow odwrotność aktualnej wartości zmiennej i_kow. Czyli

suma_kow=suma_kow+1/i_kow;

Krok4 Zwiększ zmienną sterującą i_kow o jeden. Czyli i_kow++

Krok5 Sprawdź czy i_kow>1000000 jeśli nie przejdź do Kroku3 jeśli nie do Kroku6

Krok6 Wypisz wartość zmiennej suma_kow

Uwaga: zmienne „i” oraz „suma” z trzema literami nazwiska.

Zadanie 51.

Temat: Obliczanie silni w sposób iteracyjny.

Opis teoretyczny:

silnie oznaczamy n! (jest użyty wykrzyknik)

Jest to iloczyn liczb **od 1 do n** ale 0!=1

np. 5!=1*2*3*4*5 i wynik 5!=120

Wykonaj:

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie49)
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie49-KOD)
3)Przeczytaj temat oraz treść zadania.
4)Rozwiązać zadanie (według algorytmu poniżej)i dokonać testowania.
5)Nazwy zmiennych silnia_kow (trzy pierwsze litery nazwisk ucznia).
6)Dokonaj testowania zadania.
7)Podłącz plik zadanie49_kow.html do przycisku Nazwisko_ucznia-Zadanie49
8)Podłącz plik zadanie49_kow_KOD.html do przycisku Nazwisko_ucznia-Zad49

Rozwiązanie

Użyj zmiennych z trzema literami nazwiska np. n_kow oraz silnia_kow .
Program pyta się o liczbę n= (tylko jedną liczbę) np. n_kow
i wynikiem programu będzie taki efekt na ekranie
np.
n=6
6!=720

Wskazówka:

Wczytaj n, gdy n=0 to **nie wchodzimy** do pętli tylko robimy przypisanie silnia_kow=1 i wyprowadzamy na ekran wynik.
gdy n>0 to wtedy pętla w odpowiednich granicach[od 1 do n] . Pamiętaj o wartości początkowej zmiennej silnia_kow=1 [ustawiamy na jeden].

Zadanie 52.

Temat: Wyprowadzanie liczb od 1 do 10 przesuniętych o jedną spację w prawo. Demonstarcja pętli zagnieżdżonych (czyli pętla w pętli).

Wykonaj:

1)Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie50)
2) Wykonaj **przycisk** na stronie zaliczeniowej (Nazwisko_ucznia-Zadanie50-KOD)
3)Przeczytaj temat oraz treść zadania.
4)Dokonaj kopiowania zadania (pamiętaj o odpowiedniej nazwie pliku).
5)Dopisz na początku zadania(pierwsze wyświetlenie przez zadanie):

- wyprowadzanie nazwisko ucznia→ 2cm, kolor czerwony
- wyprowadzanie tematu → 5 mm, kolor niebieski

5)Dokonaj testowania zadania.
6)Podłącz plik zadanie50_kow.html do przycisku Nazwisko_ucznia-Zadanie50
7)Podłącz plik zadanie50_kow_KOD.html do przycisku Nazwisko_ucznia-Zad50

```
<!DOCTYPE html>
<html lang="pl">
<head>
   <meta charset="utf-8">
    <title>  Przykład użycia pętli </title>
  <script language="JavaScript">
   for ( var i=1; i<=10; i++)
    {
       for ( var k=1; k<=i; k++)
        {
           document.write(" ");
         }
         document.write(i+"<br>");
     }
  </script>
</head>
<body>
</body>
</html>
```

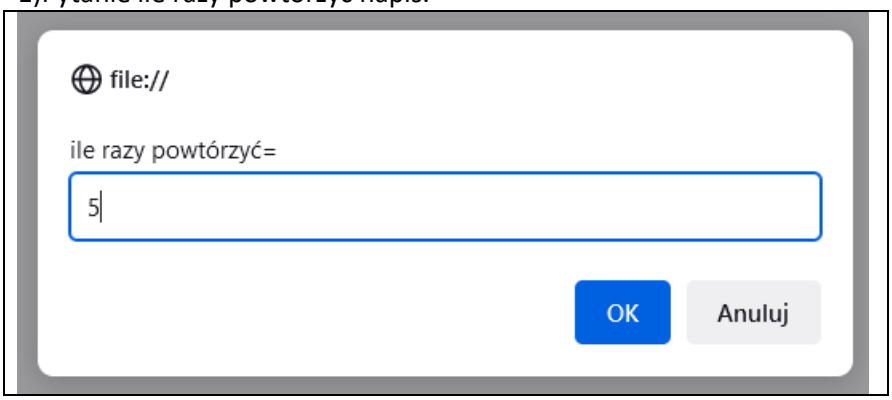
Efekt działania programu

```
 1
 2
  3
  4
   5
    6
     7
      8
       9
        10
```

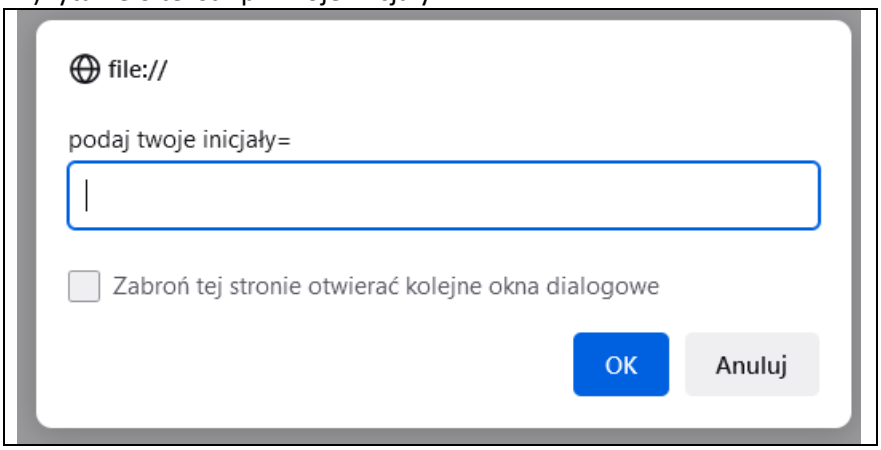
Zadanie 53.

Dokonaj modyfikacji zadania poprzedniego przez:

1)Pytanie ile razy powtórzyć napis.



2)Pytanie o tekst np. Twoje inicjały



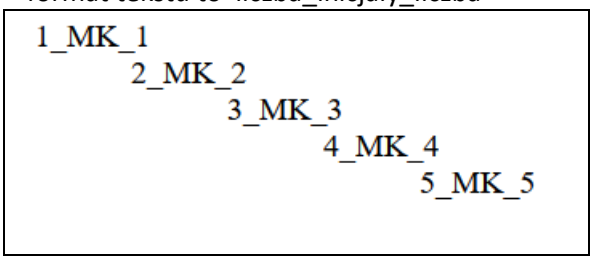
3)Zmień treść kod zadania aby otrzymać następujący efekt.

-będziesz musiał zmienić koniec pętli zewnętrznej tak, aby powtarzała się określoną ilość razy (wczytaną w okienku),

- w pętli wewnętrznej będziesz musiał zmienić ilość spacji [`  ]`

-liczba zapisze się pod liczbą wczytanych dla inicjałów)

-format tekstu to liczba_inicjały_liczba



Zadanie 54.

Temat: Użycie pętli **for** do rysowania w trybie tekstowym.

Wykonaj:

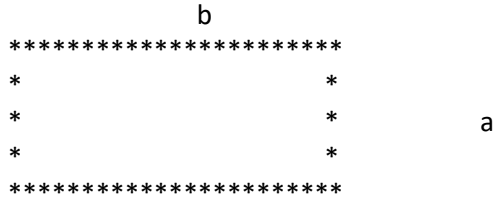
Zmienne a i b mają nazwy a_kow i b_kow (trzy pierwsze litery nazwiska [bez polskich liter małymi literami].

Z użyciem 3 trzech pętli narysuj prostokąt z gwiazdek o zadanych (obliczonych) wymiarach a i b.

Czyli program nie pyta się o **a** i **b**. Należy obliczyć **a** i **b** przed wykonaniem zadania.

Oblicz a i b ze wzorów zapisanych poniżej:

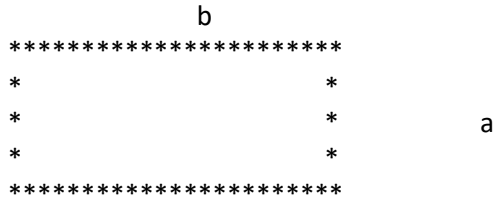
a=[(liczba_liter_nazwiska) mod 4]+5
b=[(liczba_liter_imienia) mod 6]+20
Wygląd ekranu:
Zauważ, że **a** w przykładzie poniżej a=5. Musisz uwzględnić ten fakt przy rozwiązywaniu tego zadania. b=23.



Pierwsza pętla to górne poziome gwiazdki, środkowe gwiazdki czyli pionowe gwiazdki to druga pętla, trzecia pętla to dolne poziome gwiazdki.
Uwaga: Pierwsza osoba (oraz zadania indywidualne), które rozwiążą zadania z użyciem będą punktowane +1 punkt.

Zadanie 55.

Temat: Użycie pętli **for** do rysowania w trybie tekstowym
Zmienne a i b mają nazwy a_kow i b_kow (trzy pierwsze litery nazwiska [bez polskich liter małymi literami]).
Z użyciem 4 czterech pętli narysuj prostokąt z gwiazdek o wczytanych wymiarach a i b.



Pierwsza pętla to górne poziome gwiazdki,
Druga pętla to przygotowanie wiersza środkowego, czyli pionowe gwiazdki i spacje,
Zauważ, że wiersze skradają się z „*_odpowiednia ilość spacji_*”
Uwagi:
1)Spacje wykonujemy ,
2)ilość spacji jest dwukrotnie większa od ilości gwiazdek, ze względu, że matryca spacji jest dwa razy mniejsza od matrycy „*”,
3)pętla jest od dwa mniejsza od szerokości, ze względu na gwiazdę na końcu i początku wiersza,
4)zmienna wiersz ma odpowiednią nazwę i wiersz_kow (trzy pierwsze litery nazwiska [bez polskich liter małymi literami]).

```
var wiersz_kow="*";
for ( var i=1; i<=(b-2); i++)
  {
     Wiersz_kow=wiersz_kow+"  ";
  }
 wiersz=wiersz+"*";
```

Trzecia pętla to wyświetlenie przygotowanego wiersza w pętli drugiej
Czwarta pętla to dolne poziome gwiazdki.
Program będzie pytał się o **a** i **b** i następnie rysował.
Uwaga:
Przejście do nowego wiersza w JS to:

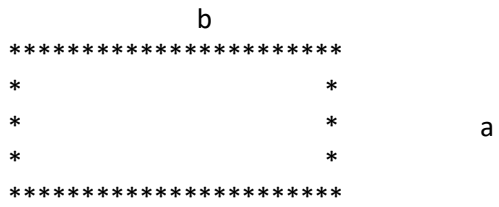
```
document.write("<br>");
```

Uwaga: Pierwsza osoba (oraz zadania indywidualne), które rozwiążą zadania z użyciem będą punktowane +1 punkt.

Zadanie 56.

Temat: Użycie pętli **for** do rysowania w trybie tekstowym

Z użyciem 3 trzech pętli narysuj prostokąt z gwiazdek o zadanych (obliczonych) wymiarach a i b.



Pierwsza pętla to górne poziome gwiazdki, środkowe gwiazdki czyli pionowe gwiazdki to druga pętla, trzecia pętla to dolne poziome gwiazdki.
Program będzie pytał się o **imię i nazwisko** następnie **obliczał** (z użyciem odpowiednich formuł) **a** i **b** ze wzorów zapisanych poniżej:
a=[(liczba_liter_nazwiska) mod 4]+5
b=[(liczba_liter_imienia) mod 6]+20

Uwaga: Pierwsza osoba (oraz zadania indywidualne), które rozwiążą zadania z użyciem będą punktowane +1 punkt.

Zadanie 57.

Temat: Użycie pętli **for** do zmiany wielkości tekstu

Program pyta się o:
1)**dwie liczby** z przedziały <20,60> wczytane do zmiennych o nazwach **l1_t_kow, l2_t_kow**, zmienne zawierają trzy litery nazwiska ucznia (bez polskich liter), program sprawdza z użyciem pętli do{…..}while(warunek) czy liczby są z przedziału <20,60> (każda zmienna sprawdzana jest osobno), wyświetlany jest komentarz, że liczba nie należy do przedziału (użyj instrukcji if z odpowiedni zdefiniowanym przedziałem <20,60>)
2)**wartość początkową wielkości tekstu** wczytaną do zmiennej o nazwie **pocz_kow**, zmienna zawiera trzy litery nazwiska ucznia (bez polskich liter),
3)**dowolny tekst**, wczytany do zmiennej o nazwie **tekst_kow**, zmienna zawiera trzy litery nazwiska ucznia (bez polskich liter),

Wczytany tekst zostanie wyświetlony określaną ilość razy:

- Dla zwiększania liter przerywamy wyświetlanie gdy tekst jest większy od 100px
- Dla zmniejszania liter przerywamy wyświetlanie gdy tekst jest mniejszy od 0px

Gdy zachodzi zależność liczba1<liczba2 to tekst zwiększa się o 5 pikseli,
Gdy zachodzi zależność liczba2<liczba1 to tekst zmniejsza się o 5 pikseli,
Gdy zachodzi zależność liczba2=liczba1 to tekst wyświetli się raz.

Program sprawdza wczytane dane i informuje o tym np. „liczba l1 nie należy do przedziału <20,60>”.
Wskazówki:

1) Ilość powtórzeń pętli będzie zapewniona poprzez odpowiednią konstrukcję pętli.
2) Stosujemy następującą metodę użycia wartości zmiennych dla np. wyświetlania wielkości tekstu zdefiniowane w zmiennej.

```
document.write("<p style=\"font-size:"+i+"px; color: black;\"><br>"+tekst_kow+"<br></p>");
```

Wielkość w zmiennej → i
Tekst w zmiennej →tekst_kow

Zadanie 58.

Temat: Obliczanie liczb pierwszych podanych wzorem.

Istnieje wzór wielomianu o postaci:

$$w(i) = i^2 + i + 41$$

którego wartości dla i = 0, 1, 2, ..., 39 są liczbami pierwszymi Napisz program znajdujący czterdzieści liczb pierwszych z użyciem pętli, korzystając ze wzoru. Zatrzymanie ekranu po pięciu liniach.
Przykładowy wydruk zadania

i=0 w(0)=41
i=1 w(1)=43

Zadanie 59.

Temat :Napisać program drukujący liczby, ich pierwiastki drugie oraz pierwiastki czwartego stopnia od numeru w dzienniku do numeru w dzienniku+15. Zatrzymanie ekranu po trzech liniach.
Wskazówki:

- Pierwiastek czwartego stopnia obliczamy jako pierwiastek z pierwiastka drugie stopnia.
- W celu uzyskania dwóch miejsc po przecinku zastosuj metodę .toFixed(2) np. x1. .toFixed(2)
- W celu uzyskania pierwiastka zastosuj metodę Math.sqrt(liczba)

Przykładowy wydruk zadania

I=4 pierw_2(4)=1.41 pierw_4(4)=1.19
I=5 pierw_2(5)=2.34 pierw_4(5)=1.49

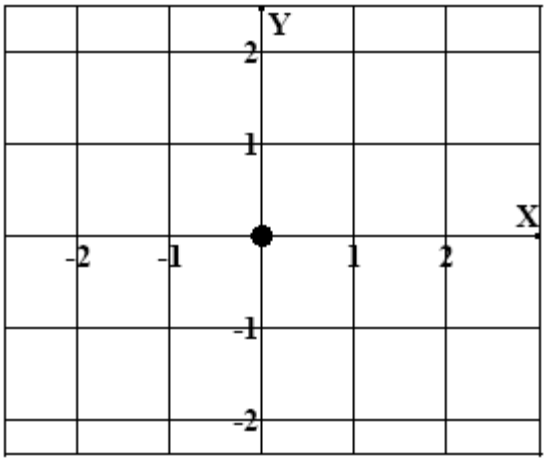
Zadanie 60.

Temat: Wczytanej w osobnych okienkach dwie własności liczby. Wypisz wszystkie liczby nie parzyste zawarte między tymi liczbami. Gdy pierwsza liczba jest mniejsza od drugiej to liczby wypisywane są od najmniejszej do największej kolorom zielonym. Gdy pierwsza liczba jest większa od drugiej to liczby wypisywane są od największej do najmniejszej kolorom niebieskim czcionka dwa razy mniejsza niż czcionka w pierwszy przypadku. Gdy pierwsza liczba wczytana liczba jest równa drugiej to komunikat i brak obliczeń w okienku confirm.

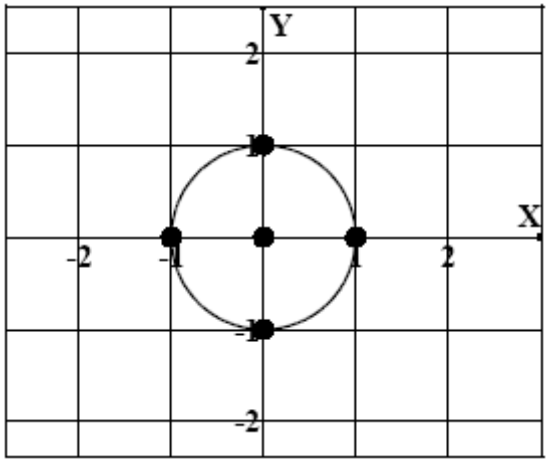
Zadanie 61.

Temat: Obliczanie ilości punktów kratowych z użyciem pętli podwójnej.

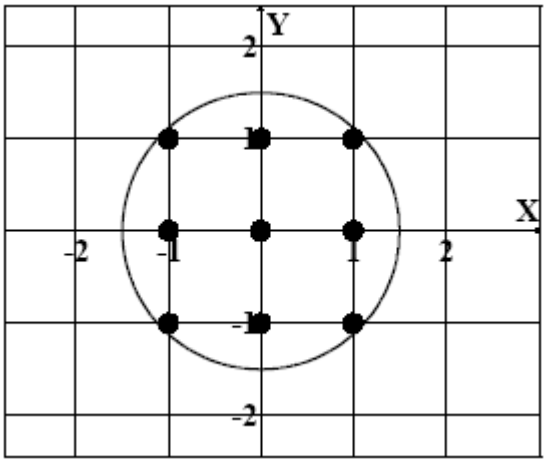
Punkt kratowy to punkt, którego współrzędne w układzie kartezjańskim są liczbami całkowitymi.
Przykłady punktów kratowych:
(−100,101) , (1,1) , (0,0) , (− 1, − 3) .
Rozważamy koła o środku w początku układu współrzędnych. Dla nieujemnej liczby rzeczywistej R przez K (R) oznaczmy koło o promieniu R (brzeg koła należy do koła).
Niech N (R) będzie liczbą punktów kratowych zawartych w kole K(R) .
Przykłady:
Jeżeli R = 0, to N (R) = 1.



Jeżeli R = 1, to w kole K(R) mieści się pięć punktów kratowych, czyli N(R) = 5 .



Jeżeli R =1,5 ,to w kole K(R) mieści się dziewięć punktów kratowych, zatem N(R) = 9.



Do wykonania

a)Zapisz specyfikację w zeszycie.

Specyfikacja:

Dane: R – promień koła o środku znajdującym się w początku układu współrzędnych (0,0); liczba całkowita nieujemna.

Wynik: liczba całkowita N(R) – liczba punktów kratowych zawierających się w kole o środku (0,0) i promieniu R

b)Napisz program obliczający liczbę punktów kratowych zawierających się w kole o promieniu R. Użyj zmiennych sterujących i_trzy_pierwsze_litery_nazwiska_ucznia oraz j_trzy_pierwsze_litery_nazwiska_ucznia np.i_kow oraz j_kow.

c) Uzupełnij i zapisz w zeszycie poniższą tabelę używając napisany program:

Promień koła R	Liczba punktów kratowych N(R)
2,01	
4,50	
100+numer_w_dzieniku	
1000+numer_w_dzienniku	

Rozwiązanie:
Należy przeszukać obszar w postaci kwadratu w układzie XOY $(-1-R) \leq x \leq (1+R)$ $(-1-R) \leq y \leq (1+R)$. W tym celu należy stworzyć dwie własności pętle(jedna w drugiej, czyli pętla podwójna). Pętla podwójna będzie generować współrzędne wszystkich punktów o zmiennych całkowitych wewnątrz tego kwadratu. Należy zwrócić uwagę, że zmienna R jest liczbą rzeczywistą a nie całkowitą dlatego przed użyciem w pętli należy zamienić ją na całkowitą. np. (int)(-R_kow-1). Dla wszystkich wygenerowanych punktów sprawdź czy odległość wygenerowanego punktu od początku układu współrzędnych (punkt (0,0)) jest mniejsza lub równa od R. Jeśli tak to zwiększ licznik (na początku zadania musi być zerowany). Po wykonaniu zadania wyświetl licznik.
Jeśli wygenerowany punkt ma współrzędne (i_kow,j_kow) to warunek ma postać:

$$\sqrt{(i_kow)^2 + (j_kow)^2} \leq R_kow$$

Zadanie → dla osób nie uczestniczących w zajęciach

Temat: Określanie punktów kratowych z użyciem pętli podwójnej.

Wstęp teoretyczny
Punkt kratowy to punkt, którego współrzędne w układzie kartezjańskim są liczbami całkowitymi.
Przykłady punktów kratowych:
(−100,101) , (1,1) , (0,0) , (− 1, − 3)

Wykonanie zadania:
Przeczytaj poprzednie zadanie. Podpowiedzi do jego rozwiązania mogą być pomocne przy rozwiązaniu tego zadania.

1)Wczytaj cztery liczby
x_MAX → maxymalna wartość osi x
x_MIN → minimalna wartość osi x
y_MAX → maxymalna wartość osi y
y_MIN → minimalna wartość osi y

2)Wypisać punkty kratowe
x_MAX = -1
x_MIN = 2
y_MAX = -2
y_MIN =1

(-1,-2) (-1,-1) (-1,-0) (-1,1)
(0,-2) (0,-1) (0,-0) (0,1)
(1,-2) (1,-1) (1,-0) (1,1)
(2,-2) (2,-1) (2,-0) (2,1)

3)Oblicz maxymalną odległość punktu kratowego od początku układu współrzędnych (kilka punktów może mieć spełniać warunki zadania).

Jeśli wygenerowany punkt ma współrzędne (x , y) to wzór ma postać:
$d = \sqrt{(x)^2 + (y)^2}$

Dla danych z punktu 2) wydruk będzie następujący:

Punkt (2,-2) d=2.83

Dwa miejsca po przecinku

4)Istnieją dwie własności możliwe strategie obliczenia d (odległości).

- Maksymalna odległość znajduje się w punktach narożnych prostokąta obszaru punktów kratowych.
- Można obliczać odległość dla wszystkich punktów kratowych, znajdować d_max (przy zapamiętanych aktualnych iteracyjnych „x” i „y”).

Dział 7 Tablice

Definicja

Tablica (Array) jest to struktura złożona z elementów tego samego typu. Możemy przechowywać zbiory wartości, kolejno ułożonych, do których odwołujemy się przez nazwę tablicy i indeks, czyli elementy tablicy są skazywane przez indeks lub zespół indeksów. Określają one miejsce gdzie dany element znajduje się w tablicy.

tablica jednowymiarowa

1	3	4	10
A[0]	A[1]	A[2]	A[3]

w komórce jest umieszczona A[0]=1 A[1]=3 A[2]=4 A[3]=10

tablica dwuwymiarowa

A[0][0]	A[0][1]	A[0][2]	A[0][3]	A[0][4]	A[0][5]
3	**1**	**2**	**10**	**6**	**5**
1	**3**	**4**	**20**	**7**	**9**
A[1][0] Pierwsza kolumna	A[1][1]	A[1][2]	A[1][3]	A[1][4]	A[1][5] szósta kolumna

czyli w komórce jest np.
A[0][0]=3 A[1][4]=7 A[0][2]=2 A[1][5]=9
miejsce w tablicy określone jest przez podanie nazwy tablicy i w nawiasach kwadratowych podajemy wiersz potem po przecinku kolumnę.

Uwagi na temat **TABLIC**
1)Tablice w Javascript są **OBIEKTAMI**
2)Do obiektów możesz zastosować metody.
3) Tablice wymagają przed użycie deklaracji, komputer musi wiedzieć ile miejsca zarezerwować w pamięci i w jaki sposób rozmieścić kolejne elementy tablicy.
4)Tablicę tworzymy, korzystając z operatora **new.**

Pierwszy sposób tworzenia tablicy

Aby utworzyć nową tablicę, należy zadeklarować obiekt Array
w postaci:

```
var NazwaTablicy = new Array()
lub
var NazwaTablicy = []
```

Jeżeli w nawiasach[] zostanie podana liczba n, to zostanie utworzona tablica zawierająca n pustych elementów.

Przykład

```
var Tab1 = new Array(10)
var Tab2 = [15]
```

```
var owoce=new Array("Gruszka","Banan","Ananas");
```

```
var Tab3 = new Array('Anna', 'Adam', 'Piotr', 'Ewa')

var Tab4 = ['Paweł', 'Marcin', 'Ela']
```

```
<script>
var osoba = ["Krzysztof", "Nowak", 32];

alert(osoba[0]);
alert(osoba[1]);
alert(osoba[2]);
</script>
```

```
<script>
var osoba = [ ["Krzysztof", "Nowak"], ["Jan", "Kowalski"] ];
alert("Imię i nazwisko: " + osoba[0][0] + " " + osoba[0][1]);
alert("Imię i nazwisko: " + osoba[1][0] + " " + osoba[1][1]);

var autor = []
autor[0] = ["Stanisław", "Lem"];
autor[1] =["Adam", "Mickiewicz"];

alert("Autor: Imię i nazwisko : " + autor[0][0] + " " + autor[0][1]);
</script
```

Drugi sposób tworzenia tablicy

```
var owoce=new Array(3);
owoce[0]="Gruszka";
owoce[1]="Banan";
owoce[2]="Ananas";
```

Trzeci sposób tworzenia tablicy

```
sok=new Array();
sok.smak="Pomidorowy";
sok.pojemnosc="1 litr";
sok.producent="Pomidorek";
sok.length                                          // zwrócony wynik=3;
```

```
function sumOfDiagonalElements(matrix) {
    // Sprawdź, czy macierz ma odpowiednią liczbę wierszy i kolumn
    if (matrix.length !== 4 || matrix[0].length !== 4) {
```

```
        console.error("Nieprawidłowy rozmiar macierzy. Oczekiwano macierzy
4x4.");
        return null;
    }

    var sum = 0;

    for (var i = 0; i < 4; i++) {
        // Dodaj element z przekątnej do sumy
        sum += matrix[i][i];
    }

    return sum;
}

// Przykładowa tablica dwuwymiarowa 4x4
var matrix = [
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12],
    [13, 14, 15, 16]
];

// Oblicz sumę elementów przekątnej
var diagonalSum = sumOfDiagonalElements(matrix);

if (diagonalSum !== null) {
    console.log("Suma elementów przekątnej:", diagonalSum);
}
```

Bezpośrednie wczytywanie danych z klawiatury w sposób synchroniczny nie jest standardową funkcjonalnością w środowiskach takich jak przeglądarki internetowe, gdzie JavaScript jest najczęściej stosowany. Operacje wejścia/wyjścia (I/O) w JavaScript, szczególnie w przeglądarce, zazwyczaj opierają się na działaniach asynchronicznych i zdarzeniach, takich jak kliknięcia myszą lub wprowadzanie danych w formularzach.

Jednakże, dla celów edukacyjnych lub eksperymentów, można symulować wczytywanie danych z klawiatury do tablicy dwuwymiarowej w przeglądarce, wykorzystując serię okien dialogowych `prompt`, które pozwalają użytkownikowi na wprowadzenie danych. Poniżej znajdziesz przykładowy kod, który pokazuje, jak można to zrobić:

```
javascript
// Funkcja do wczytywania danych do tablicy dwuwymiarowej 4x4 z klawiatury
function readMatrixFromKeyboard(rows, cols) {
    let matrix = [];

    for (let i = 0; i < rows; i++) {
        matrix[i] = []; // Inicjalizacja wiersza
        for (let j = 0; j < cols; j++) {
            let value = prompt(`Wprowadź wartość dla elementu
[${i}][${j}]:`);
            matrix[i][j] = parseFloat(value); // Konwersja na liczbę i
zapisanie w macierzy
        }
    }
```

```
    return matrix;
}

// Funkcja do obliczania sumy elementów przekątnej w tablicy dwuwymiarowej
function sumOfDiagonal(matrix) {
    let sum = 0;
    for (let i = 0; i < matrix.length; i++) {
        sum += matrix[i][i];
    }
    return sum;
}

// Główna część skryptu
let rows = 4, cols = 4; // Wymiary macierzy
let matrix = readMatrixFromKeyboard(rows, cols); // Wczytanie danych do
macierzy
let diagonalSum = sumOfDiagonal(matrix); // Obliczenie sumy przekątnej
console.log("Suma elementów przekątnej:", diagonalSum);
```

Odwołanie bezpośrednie do elementów tablicy

Żeby uzyskać **dostęp do elementów tablicy**, należy podać numer indeksu danego elementu.

Elementy są **indeksowane od zera** czyli do elementów tablicy odwołujemy się za pomocą nazwy tablicy i indeksu.

```
const arr = ['value1', 'value2', 'value3'];
console.log(arr[0]); // value1
console.log(arr[1]); // value2
console.log(arr[2]); // value3
```

Odwołanie do elementów tablicy z użyciem pętli for

```
<script>
var ptaki = ["kos","gil","czyżyk"];
 for (var i=0; i<ptaki.length; i++) {
   alert(ptaki[i]);
 }
</script>
```

Odwołanie do elementów tablicy z użyciem pętli forEach

```
<script>
function wyswietl(nazwa) {
  alert(nazwa);
}
var ptaki = ["kos","gil","czyżyk"];

ptaki.forEach(wyswietl);
ptaki.forEach(function(nazwa){alert(nazwa);});
</script>
```

Właściwości

nazwa	opis
constructor	Zwraca funkcję, która utworzyła obiekt Array

length	Zwraca liczbę elementów w tablicy.
prototype	Odpowiada za dodawanie właściwości i metod do obiektu Array

Właściwość length:

```
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9];
console.log(arr.lenght); //9
```

Metody dla tablic:

at()

Przyjmuje wartość całkowitą i zwraca element pod tym indeksem, dopuszczając dodatnie i ujemne liczby całkowite. Ujemne liczby całkowite liczą wstecz od ostatniego elementu w tablicy.

```
const arr = ['value1', 'value2', 'value3'];
console.log(arr.at(0)); // value1
console.log(arr.at(-1)); // value3
```

push( )

Dodaje jeden lub więcej elementów na końcu tablicy.

```
const arr = ['value1', 'value2'];
arr.push('value3');
console.log(arr); // ['value1', 'value2', 'value3']
```

pop()

Usuwa ostatni element z końca tablicy i zwraca ten element.

```
const arr = ['value1', 'value2'];
const pop = arr.pop();
console.log(pop); // value2
console.log(arr); // ['value1']
```

shift( )

Usuwa pierwszy element z tablicy i zwraca ten element.

```
const arr = ['value1', 'value2'];
const shift = arr.shift();
console.log(shift); // value1
console.log(arr); // ['value2']
```

unshift( )

Dodaje jeden lub więcej elementów na początek tablicy

```
const arr = ['value1', 'value2'];
arr.unshift('value3');
console.log(arr); // ['value3', 'value1', 'value2']
```

reverse( )

Odwraca tablicę. Element o ostatnim indeksie będzie pierwszym, a element o indeksie 0 ostatnim.

```
const arr = [1, 2, 3, 4, 5];
arr.reverse()
console.log(arr); // [5,4,3,2,1]
```

indexOf( )

Zwraca indeks pierwszego wyszukiwanego elementu w tablicy lub -1, jeśli nie został znaleziony.

```
const arr = ['value1', 'value2', 'value3'];
console.log(arr.indexOf('value3')); // 2
console.log(arr.indexOf('value4')); // -1
```

findIndex( )

Zwraca indeks pierwszego elementu w tablicy, który pomyślnie spełnił warunki w funkcji testującej.

```
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
console.log(arr.findIndex(item => item > 5)); // 5
console.log(arr[5]); //6
```

find( )

Zwraca wartość pierwszego elementu w tablicy, który pomyślnie spełnił warunki w funkcji testującej.

```
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
console.log(arr.find(item => item > 5)); // 6
```

reduce( )

Wykonuje funkcję callback na każdym elemencie tablicy, w podanej kolejności, przekazując wartość zwracaną z obliczeń na poprzedzającym elemencie.
Końcowym wynikiem jest pojedyncza wartość.

```
const arr = [10, 10, 10, 10, 10];
console.log(arr.reduce((acc, cur) => acc + cur)); // 50
```

slice( )

Zwraca nową tablicę z określonymi elementami. Pierwszy parametr wskazuje na początkowy indeks, a drugi na indeks końcowego elementu.

```
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
console.log(arr.slice(0, 5)); // [1, 2, 3, 4, 5]
```

splice()

Zmienia zawartość tablicy, usuwając lub zastępując istniejące elementy i/lub dodając nowe elementy.

```
const arr = ['value1', 'value2', 'value3'];
arr.splice(0, 2);
console.log(arr); // ['value3']
```

map( )

Tworzy nową tablicę z wynikami wywołania podanej funkcji dla każdego elementu.

```
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
console.log(arr.map(item => item * 2)); // [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

filter( )

Tworzy nową tablicę zawierającą elementy, które spełniają warunek wewnątrz podanej funkcji.

```
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
console.log(arr.filter(item => item %2 === 0)); // [2, 4, 6, 8, 10]
```

sort( )

Metoda służy do sortowania elementów tablicy w porządku rosnącym lub malejącym.

```
const arr = ['c', 'b', 'e', 'a', 'd'];
console.log(arr.sort((a, b) => (a > b ? 1 : -1))); // ['a', 'b', 'c', 'd', 'e']
console.log(arr.sort((a, b) => (a > b ? -1 : 1))); // ['e', 'd', 'c', 'b', 'a']
```

every( )

Sprawdza każdy element w tablicy, który spełnia warunek, zwracając odpowiednio wartość true lub false.

```
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
const arr2 = [2, 4, 6, 8, 10];
console.log(arr.every(item => item % 2 === 0)); // false
console.log(arr2.every(item => item % 2 === 0)); // true
```

some( )

Sprawdza, czy co najmniej jeden element w tablicy spełnia warunek, zwracając odpowiednio wartość true lub false.

```
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
console.log(arr.some(item => item >= 10)); // true
console.log(arr.some(item => item > 10)); // false
```

includes( )

Sprawdza, czy tablica zawiera element, który spełnia warunek, zwracając odpowiednio wartość true lub false.

```
const arr = ['value1', 'value2', 'value3'];
console.log(arr.includes('value1')); // true
console.log(arr.includes('value4')); // false
```

concat( )

Służy do łączenia dwóch lub więcej tablic i zwraca nową tablicę.

```
const arr = [1, 2, 3, 4, 5];
const arr2 = [6, 7, 8, 9, 10];
console.log(arr.concat(arr2)); // [1,2,3,4,5,6,7,8,9,10]
```

join( )

Zwraca nowy ciąg znaków, łącząc wszystkie elementy tablicy oddzielone określonym separatorem.

```
const arr = ['a', 'b', 'c', 'd', 'e'];
console.log(arr.join(' - '); // a - b - c - d - e
```

fill( )

Wypełnia elementy w tabli.

```
const arr = new Array(5);
console.log(arr); // [ <5 empty items> ]
console.log(arr.fill(10)); // [10, 10, 10, 10, 10]
```

forEach( )

Wykonuje podaną funkcję callback dla każdego elementu w tablicy.

```
const arr = ['value1', 'value2', 'value3'];
arr.forEach(value => {
    console.log(value);
```

```
});
// 'value1'
// 'value2'
// 'value3'
```

flat()

Tworzy nową tablicę ze wszystkich zagnieżdżonych tablic.

```
const arr = ['value1', 'value2', ['value3', ['value4', 'value5']]];
console.log(arr.flat(1));; // ['value1', 'value2', 'value3', ['value4', 'value5']
console.log(arr.flat(2));; // ['value1', 'value2', 'value3', 'value4', 'value5']
```

flatMap()

Zwraca nową tablicę utworzoną przez zastosowanie danej funkcji callback dla każdego elementu tablicy, a następnie spłaszczenie wyniku o jeden poziom.

```
const arr = [
    [1, 2, 3]'
    [4, 5, 6],
    [ 7, 8, 9]
];
console.log(arr.flatMap(item => item)); // [1,2,3,4,5,6,7,8,9]
```

Metody te same oraz inne (inny opis)

nazwa	opis
.concat(obiektArray2)	Łączy dwie własności lub więcej tablic i zwraca nową.
.join("separ")	Zwraca łańcuch znaków elementów tablicy, przedzielonych separatorem.
.pop()	Usuwa i zwraca ostatni element tablicy.
.push("el1"[,"el2"])	Dodaje element lub więcej na koniec tablicy i zwraca nową długość.
.reverse()	Zwraca tablice z elementami w odwrotnej kolejności.
.slice(indexPocz [,indexKońc])	Zwraca tablicę utworzoną z części danej.
.sort([funkcjaPorównawcza])	Zwraca tablicę posortowaną.
.splice(index,ile [,el1, el2])	Dodaje i/lub usuwa elementy tablicy.
.toSource()	Zwraca łańcuch znaków, który reprezentuje źródło tablicy.
.toString()	Zwraca łańcuch znaków, który reprezentuje daną tablicę.
.unshift("el1"[,"el2"])	Dodaje jeden lub kilka elementów na początek tablicy i zwraca nową długość.
.valueOf()	Zwraca wartość tablicy.

Przykład

Dana jest tablica myArray i pięciu elementach. W trakcie wykonywania programu konieczne może się okazać usunięcie niektórych elementów z tablicy – tak jak na poniższym przykładzie. Stosując właściwość length, zobaczymy jednak, że usunięcie elementu z tablicy **nie** powoduje skrócenia jej długości:

```
myArray.length                                  // wynik 5
delete.myArray[2]
myArray.length                                  // wynik 5
myArray[2] // wynik: undefined
```

Przykład

```
document.write(Tab3[2])
```

W wyniku wyświetli się wartość
'Piotr'.

Przykład
Aby **dodać nową wartość** do tablicy, należy przypisać tę wartość do odpowiedniego indeksu tablicy.

```
var Tab5 = new Array('kot', pies', 'koń')
Tab5[3] = 'mysz';
Tab5[4] = 'chomik';
document.write(Tab5[0] + 'i' + Tab5[3])
```

W wyniku wyświetli się tekst 'kot i mysz'.

Dzięki właściwości **length** można określić, z ilu elementów składa się tablica. Jest to bardzo przydatna właściwość, szczególnie gdy chcemy utworzyć pętlę odczytującą wszystkie elementy tablicy.

Zadanie 62.

Temat: Skrypt, który: rezerwuje tablicę, do elementów tablicy wpisuje wartości będące wartością aktualnego indeksu, wypisuje wartości tablicy od pierwszego do ostatniego, oblicza sumę oraz średnią oraz wypisuje te wartości.

Wykonaj:
Zmień zapis skryptu podanego w przykładzie poniżej,
1)nazwy zmiennych T, suma, srednia na trzy litery Twojego nazwiska(bez polskich liter),
2)wykonaj pytanie o koniec pętli i wczytaj liczbę
3)Pamiętaj, że indeksowanie elementów rozpoczyna się od zera.

```
<!DOCTYPE html>
<html lang="pl">
<head>
  <meta charset="UTF-8">
  <title>Tabele zadanie1</title>
</head>
<body>
  <script>
    var suma_kow=0;
    var T_kow = new Array()
    for (i = 0; i < 20; i++) {
      T_kow[i] = i;
      suma_kow=suma_kow+i;
      document.write("T["+i+"]="+T_kow[i]+"<br>");
    }
    var srednia_kow=suma_kow/i;
    document.write("suma="+suma_kow+"<br>");
    document.write("średnia="+srednia_kow.toFixed(2)+"<br>");
  </script>
</body>
</html>
```

Zadanie 63.

Temat Wpisanie do tablicy wielokrotności liczby 3 oraz obliczenie iloczynu .

Wykonaj:

Wykonaj skrypt który:
1)rezerwuje tablicę, nazwy tablicę TT_trzy litery Twojego nazwiska(bez polskich liter),
2)wykonaj pytanie o koniec pętli i wczytaj liczbę,
3)do elementów tablicy wpisz kolejne wielokrotności liczby trzy np. TT_kow[0]=3;
4)wypisz wartości tablicy (wyraz po wyrazie, łącznie z indeksem),
5)oblicz i wypisz iloczyn tych liczb (zmienna ilo_kow, pamiętaj ze zmienna ta powinna mieć wartość początkową jeden[1]).

Zadanie 64.

Temat: Przykład poniżej wyświetla elementy tablicy od pierwszego do ostatniego(siódmego).

Wykonaj:
1)Zmień nazwę tablicy Tablica_N na TaB_na trzy litery Twojego nazwiska(bez polskich liter).
2)Zmień elementy tablicy na (tablica powinna mieć siedem elementów, zmiana podczas pisania kodu) :

Numer w dzienniku	Elementy tablicy
1 ,10, 19, 27,31	owoce
2, 11,20, 28, 5	warzywa
3, 12, 21, 29,15	państwa
4, 14, 22, 30,23	miasta
6, 16, 24, 32	rzeki
7, 17, 25, 33	jeziora
8, 18, 26, 34	napoje
9 ,13, 27, 35	marki samochodów

3)Pozostaw wyświetlanie od pierwszego do ostatniego elementu.
4)Dopisz do skryptu podanego w przykładzie poniżej (elementy wyświetlają się od pierwszego do ostatniego), tak aby wyświetlone zostały elementy tabeli od ostatniego do pierwszego. Pamiętaj, że indeksowanie elementów rozpoczyna się od zera. (czyli będą dwa wyświetlania)

```
<!DOCTYPE html>
<html lang="pl">
<head>
   <meta charset="UTF-8">
   <title>Tabele zadanie2</title>
</head>
<body>
   <script>
    var Tablica_N = new Array('Anna', 'Adam', 'Piotr', 'Ewa', 'Paweł', 'Marcin', 'Ela');
      for (i = 0; i<Tablica_N.length; i++)
          {
            document.write(Tablica_N[i] + "<br>");
          }
   </script>
</body>
```

```
</html>
```

Zadanie 65.

Temat: Dodawanie i usuwanie elementów tablicy do tablicy z poprzedniego zadania oraz odwraca tablicę czyli element o ostatnim indeksie będzie pierwszym, a element o indeksie 0 ostatnim.

Wykonaj:
1)**Dodaj** na ostatnim miejscu tablicy element (zgodny z twoimi elementami), wyświetl tablicę.
Wskazówka: Tablica_N.push('Daniel');
2)**Usuń** pierwszy element tablicy, wyświetl tablicę.
Wskazówka: Tablica_N.shift();
3)Wykonaj **odwracanie** tablicy czyli element o ostatnim indeksie będzie pierwszym, a element o indeksie 0 ostatnim, wyświetl tablicę.
Wskazówka: Tablica_N.reverse();

Otrzymasz taki wygląd (brak po rewersie->Ty wykonaj):

```
Anna
Adam
Piotr
Ewa
Paweł
Marcin
Ela
Po dołączeniu elementu
Anna
Adam
Piotr
Ewa
Paweł
Marcin
Ela
Daniel
Po usunięciu pierwszego elementu
Adam
Piotr
Ewa
Paweł
Marcin
Ela
Daniel
```

Tablice wielowymiarowe

W języku JavaScript można również tworzyć tablice wielowymiarowe. Wtedy element tablicy jest opisywany za pomocą indeksu określającego jego położenie w wierszu i kolumnie.

Przykład

```
var Tablica_Z = [];
Tablica_Z[0] =  ['Anna', 'Nowak'];
Tablica_Z[1] =  ['Adam', 'Kowal'];
Tablica_Z[2] =  ['Piotr', 'Ogórek'];
Tablica_Z[3] =  ['Ewa', 'Lisowska'];
document.write('imię: ' + Tablica_Z[0][0] + 'nazwisko: ' + Tablica_Z[0][1] + "<br>");
document.write('imię: ' + Tablica_Z[1][0] + 'nazwisko: ' + Tablica_Z[1][1] + "<br>");
document.write('imię: ' + Tablica_Z[2][0] + 'nazwisko: ' + Tablica_Z[2][1] + "<br>");
document.write('imię: ' + Tablica_Z[3][0] + 'nazwisko: ' + Tablica_Z[3][1]);
```

Łączenie elementów tablicy

Za pomocą metody **join()** można łączyć elementy tablicy w jeden tekst. W metodzie tej można opcjonalnie podać parametr, który określi znak oddzielający kolejne elementy tablicy. Jeżeli nie zostanie podana wartość tego parametru, domyślnym znakiem będzie przecinek.

Przykład

```
var Tablica = new Array('Anna', 'Adam', 'Piotr');
document.write(Tablica.join()  + "<br>");  /*oddzielający znak , (przecinek) */
document.write(Tablica.join(" - ") + "<br>"); /*oddzielający znak -(minus) */
```

Odwracanie kolejności elementów tablicy

Za pomocą metody reverse() można odwrócić kolejność elementów tablicy.

Przykład

```
var Tablica = new Array('Anna', 'Adam', 'Piotr');
document.write(Tablica.join()  + "<br>";
Tablica.reverse()
document.write(Tablica.join() + "<br>");
```

Sortowanie

Do sortowania elementów tablicy służy metoda **sort()**.

Przykład

```
var Tablica = new Array('Paweł', 'Anna', 'Maria', 'Adam', 'Piotr');
Tablica.sort()
document.write(Tablica.join());
```

Zadanie 66.

łączenie, odwracanie sortowani→napisać zadanie

Domyślnie tablica jest sortowana leksykograficznie. Powoduje to, że liczba 12459 będzie mniejsza od 4567, ponieważ cyfra na pierwszej pozycji jest mniejsza. Aby temu zaradzić, można sortować tablicę według własnych kryteriów. Należy skorzystać z dodatkowego parametru metody sort(). Parametrem będzie własna funkcja sortująca. Tworząc taką funkcję, należy pamiętać o trzech zasadach, które muszą być spełnione:

1)jeżeli funkcja(a,b) zwróci wartość mniejszą od 0, to wartości a zostanie nadany indeks mniejszy od indeksu przyznanego wartości b,

2)jeżeli funkcja(a,b) zwróci wartość równą 0, to wartości indeksów pozostaną bez zmian,

3)jeżeli funkcja(a,b) zwróci wartość większą od 0, to wartości a zostanie nadany indeks większy od indeksu przyznanego wartości b.

Stosując się do tych zasad, można tworzyć własne metody sortowania w tablicy.

Przykład

```
function porownaj(a,b)
{
return a - b
}
var Tablica = new Array(27, 100, 10, 450, 1654, 320);
document.write('Bez sortowania: ' + Tablica.join() );
document.write('Sortowanie domyślne: ');
Tablica.sort()
document.write(Tablica.join());
document.write('Sortowanie poprawne: ');
Tablica.sort(porownaj)
document.write(Tablica.join() );
```

W podanym przykładzie została zdefiniowana funkcja porownaj(a,b), która zwróci wartość mniejszą od zera, równą zero lub większą od zera. W zależności od zwróconej wartości elementy tablicy zostaną uporządkowane od wartości najmniejszej do największej.

Zadanie 67.

Utwórz tablicę, która będzie zawierała elementy zawierające cyfry i litery. Zdefiniuj funkcję, która pozwoli uporządkować elementy tabeli w ten sposób, że najpierw będą umieszczone litery w kolejności alfabetycznej, a następnie cyfry w kolejności od wartości najmniejszej do największej.

→sprawdzić

Zadanie 68.

Utwórz tabelę o wymiarze „trzy na trzy”.

```
var tablica = new Array(1, 2, 3);
document.write(tablica[1] + "<br>");
tablica[1] = 123;
document.write(tablica[1] + "<br>");
```

Umieść w niej obrazki. Zdefiniuj kod, który spowoduje zmianę wyświetlanych na stronie obrazków co 2 sekundy.

→sprawdzić, rozwiązać

Zadanie 69.

Temat: Użycie obiektów-tablic do rozwiązania układu dwóch równań liniowych.

Wykonaj:

I.)Przeczytaj teorię o tablice_obiekty.

II.)Wykonaj praktycznie zadanie

Temat: Rozwiązać układ dwóch równań liniowych w postaci. Do rozwiązania użyj tablic i pętli. Zastosuj wyznaczniki.

Przydatna teoria.

np.

A[0][0] * X + A[0][1] * Y = A[0][2]
A[1][0] * X + A[1][1] * Y = A[1][2]

$$\begin{cases} 2*x+3*y=5 \\ -4*x+8*y=0 \end{cases}$$

i tak A[0][0]=2 A[0][1]=3 A[0][2]=5
A[1][0]=–4 A[1][1]=8 A[1][2]=0

Tak więc w celu rozwiązania układu równań konieczne będzie użycie tablicy o dwóch wierszach i trzech kolumnach czyli konieczna będzie rezerwacja tablicy A[2][3].
W przypadku tablic dwuwymiarowych pierwsza liczba jest ilością wierszy, a druga ilością kolumn.

Wyznaczniki obliczamy stosując wzory:

$$W=\begin{vmatrix} A[0][0] & A[0][1] \\ A[1][0] & A[1][1] \end{vmatrix} = \mathrm{A[0][0]*A[1][1]-A[0][1]*A[1][0]}$$

$$W_X=\begin{vmatrix} A[0][2] & A[0][1] \\ A[1][2] & A[1][1] \end{vmatrix} = \mathrm{A[0][2]*A[1][1]-A[1][2]*A[0][1]}$$

$$W_Y=\begin{vmatrix} A[0][0] & A[0][2] \\ A[1][0] & A[1][2] \end{vmatrix} = \mathrm{A[0][0]*A[1][2]-A[1][0]*A[0][2]}$$

Rozwiązywalność układu równań (możliwe przypadki A) lub B) lub C):
A) gdy (**W<>0)** układ ma jedno rozwiązanie
gdzie: **X=Wx/W** **Y=Wy/W**
W—wyznacznik główny układu równań
Wx—wyznacznik dla zmiennej **X**
Wy—wyznacznik dla zmiennej **Y**
B) gdy ((**W=0**) i ((**Wx<>0)** lub (**Wy<>0**))) układ sprzeczny
C) gdy ((**W=0**) i (**Wx=0**) i (**Wy=0**)) układ ma nieskończenie wiele rozwiązań sposób obliczania wyznaczników

Wykonaj na stronie:
1)Na stronie zapisz ile wynoszą elementy tablicy dla układu równań.

$$\begin{cases} nr_z_dziennika*x+miesiąi\ _urodzenia*y=dzien_urodzenia \\ (nr_z_dziennika-40)*x+liczba_liter_nazwska*y=liczba_liter_imienia \end{cases}$$

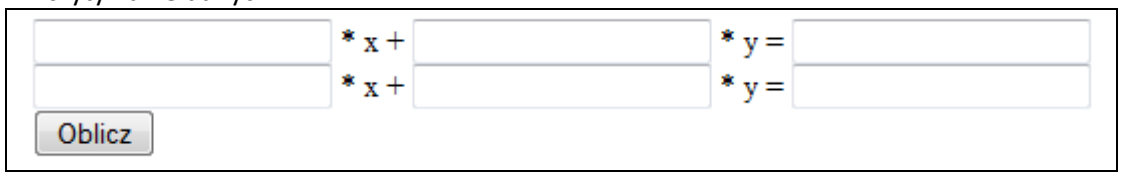
czyli A[0][0]=…… wypisz wszystkie sześć elementów tak aby tworzyły układ równań, łącznie klamrą (powiększ zwykła klamrę).

2) Do operacji na tablicach użyj metody **.lenght**

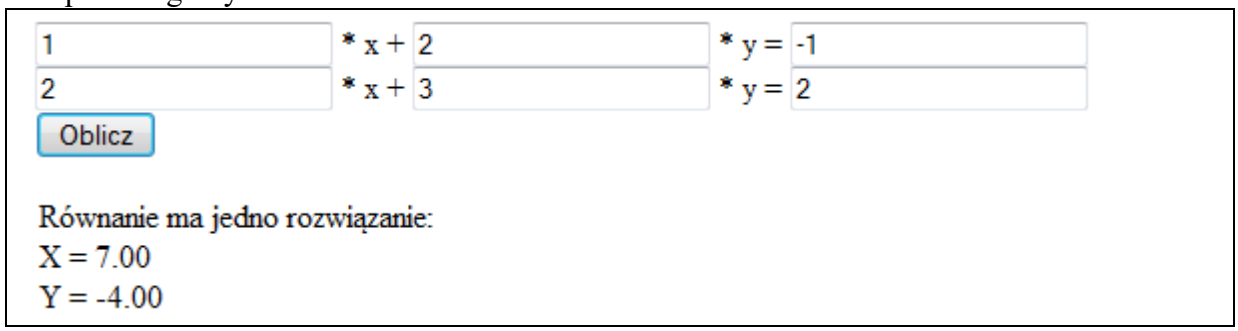
Ilość wierszy uzyskuje się przez **tablica.length**
a liczbę kolumn w danym wierszu uzyskuje się przez **tablica[numerWiersza].length**.

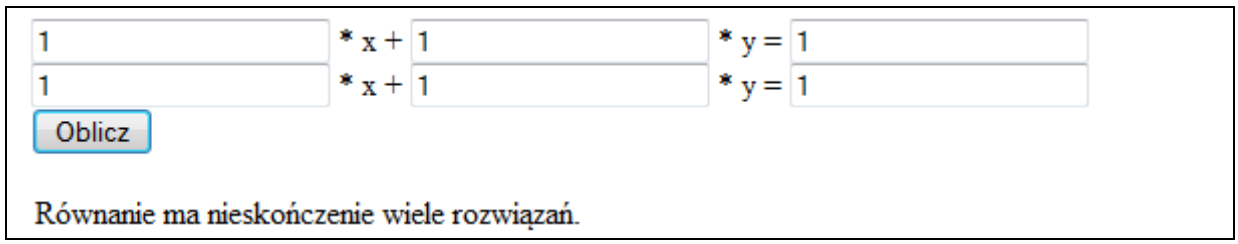
```
for(int i = 0; i < tablica.length ; i++)
 {
  for(int j = 0; j < tablica[i].length ; j++)  //sprawdzić dlaczego nie działa
    {
     // tutaj dostępny jest element tablica[i][j]
    }
 }
```

Wczytywanie danych



Obliczenia→ dwa miejsca po przecinku metoda .toFixed(2) +x.toFixed(2);
Na poszczególnych ekranach ma wartości do testowania.







3)Program pisz etapami :

ETAP 1

Zapewnić wczytywanie danych do tablicy :

Rozwiązanie Etapu 1:

1)Wczytywanie danych czyli współczynników układu równań z formularza

2)utwórz obiekt tablicy o nazwie A_pierwsza_litera_nazwiska
np.

```
float $A[][] = new float [?][?];
```

w miejsce znaków zapytania wpisz odpowiednie liczby zgodne z treścią zadania.

3)Wpisz danych do tablicy, pamiętaj o Twojej nazwie tablicy (innej niż w przykładzie poniżej.
np.

$A[1][1]= dana z formularza

7)Wypisz dane z tablicy czytanej w podpunkcie 6)

ETAP 2

Obliczyć Wx, Wy, W, X, Y oraz wydrukować Wx, Wy, W, X i Y z dwoma miejscami po przecinku.
np. String.format("%.2f",obwod)
Potrzebne zmienne zadeklaruj jako double.

ETAP 3

Sprawdzenie warunków czy układ jest

- oznaczony
- sprzeczny
- ma nieskończenie wiele rozwiązań

wraz z wyprowadzeniem stosownego komunikatu na monitorze

układy do testowania:

- oznaczony

wpisz swoje dane z układu

$$\begin{cases} nr_z_dziennika * x + miesiąi\ _urodzenia * y = dzien_urodzenia \\ (nr_z_dziennika - 40) * x + liczba_liter_nazwska * y = liczba_liter_imienia \end{cases}$$

- sprzeczny

$$\begin{cases} 1 * x + 2 * y = 2 \\ 1 * x + 2 * y = -5 \end{cases}$$

- ma nieskończenie wiele rozwiązań

$$\begin{cases} -3 * x + 4 * y = 2 \\ -3 * x + 4 * y = 2 \end{cases}$$

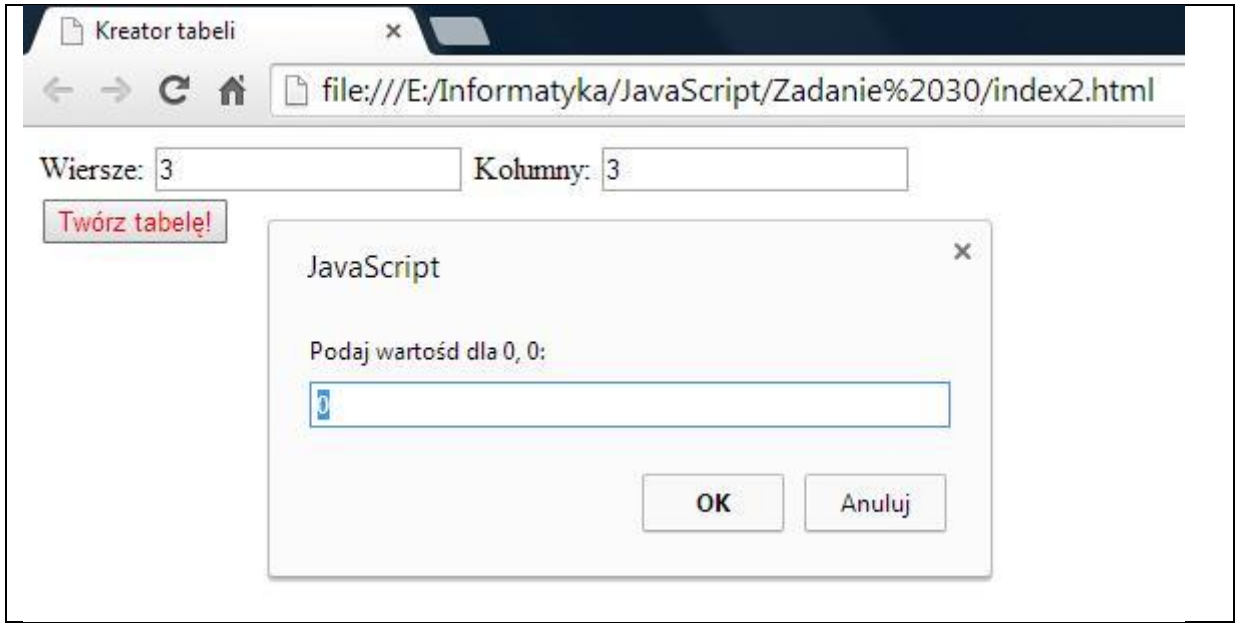
ETAP 4

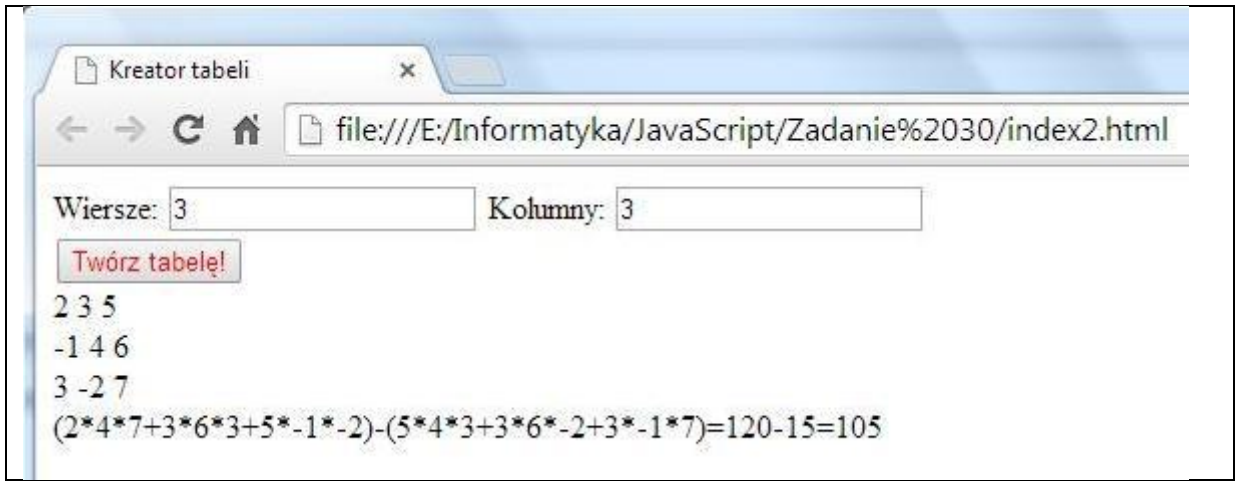
Zapewnienie powtarzalności obliczeń
Wypisanie na monitorze układu równań w postaci np.

```
 2*X + 3*Y = 5
-4*X + 8*Y = 0
```

Wykonaj specyfikację problemu.

Zadanie 70.(ZADANIE 31)
Napisz program obliczający wyznacznik 3x3 metodą Sarussa. Wczytaj macierz oraz wydrukuj ją po wczytaniu wierszami oraz wartość wyznacznika.
Znajdź w Internecie jak obliczamy wyznacznik 3x3 metodą Sarussa.





Dział 8 Programowanie obiektowe-definiowane w przeglądarce

Przykład

Kod	Wytłumaczenie
var napisz="Witaj w szkole";	utworzenie obiektu **napisz** i przypisanie wartości "Witaj w szkole"
document.write(napisz.toUpperCase());	UpperCase() jest metodą, która działa na obiekt **napisz,** →zamienia małe litery na duże
document.write(napisz.length);	napisz.length jego właściwością obiektu **napisz** →podaje długość testu zapisanego w obiekcie **napisz**
Wynikiem będzie wypisanie tekstu: "WITAJ W SZKOLE15"	

Zadanie 71.

Udziel odpowiedzi na następujące pytania teoretyczne:

Wymień oraz opisz obiekty przeglądarki

- window
- navigator

Koniec pytania teoretycznego

==========

Przeglądarka internetowa wykonana jest z użyciem programowania obiektowego.

Obiekty przeglądarki

Dla każdej strony internetowej zdefiniowane są następujące obiekty:

- window
- navigator
- document
- location
- history

Obiekt window

1. Opisuje bieżące okno przeglądarki.
2. Jest najważniejszym obiektem w hierarchii, dlatego odwołanie do jego właściwości lub metod nie wymaga podania nazwy obiektu.
3. Tworzony jest automatycznie podczas otwierania okna przeglądarki.
4. Posiada wiele właściwości przydatnych podczas tworzenia strony.

Obiekt navigator

1. Pozwala na dostęp do informacji dotyczących przeglądarki.
2. Służy do ustalenia wersji przeglądarki używanej przez użytkownika.
3. Właściwości tego obiektu mogą być tylko odczytywane nie mogą być zmieniane.
4. Najczęściej używa się go do sprawdzenia, czy przeglądarka jest odpowiednia do zastosowanej wersji języka JavaScript.
5. Szczególnie przydatnym jest window.navigator.userAgent – to tekst identyfikujący przeglądarkę.

Zadanie 72. (dawniej Zadanie 32a)

Uruchom poniższy przykład:

Przykład1 do zadania 32a

Temat:

Rozpoznawanie typu przeglądarki przy użyciu **navigator** i jej właściwości **userAgent**

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html lang="pl">
 <head>
  <meta http-equiv="Content-Type" content="text/html; charset=utf-8">
  <meta http-equiv="Content-Script-Type" content="text/javascript">
  <title>Moja strona WWW</title>
 </head>
 <body>
  <script type="text/javascript">
  var agent = navigator.userAgent.toLowerCase();
  var nazwa = "nieznany";
  if(agent.indexOf('firefox') != -1){nazwa = "Firefox";}
  if(agent.indexOf('opera') != -1){nazwa = "Opera";}
  else if(agent.indexOf('konqueror') != -1){nazwa = "Konqueror";}
        else if(agent.indexOf('netscape') != -1){nazwa = "Netscape Navigator";}
             else if(agent.indexOf('msie') != -1){nazwa = "Internet Explorer";}
                                document.write("Twoja przeglądarka to: " + nazwa);
  </script>
 </body>
</html>
```

Opis do przykładu powyżej:

Tekst znajdujący się we właściwości **userAgent** jest konwertowany za pomocą metody **toLowerCase**, tak aby zawierał włącznie małe litery (co ułatwia dalsze operacje), i jest zapisywany w pomocniczej zmiennej **agent**. Następnie za pomocą metody **indexOf** jest sprawdzane, czy w ciągu zapisanym w agent znajduje się któreś ze słów charakterystycznych dla jednej z rozpoznawanych przeglądarek. Jeśli tak, zmiennej nazwa jest przypisywana nazwa rozpoznanego produktu. Nazwa ta jest wyświetlana na ekranie za pomocą instrukcji document.write.

koniec przykładu

==========

Zadanie 73.

Uruchom poniższy przykład (dopisując część kodu aby całość działała):
Uruchom dwa poniższe przykłady w jednym skrypcie aby otrzymać wygląd ekranu jak poniżej.

```
pierwszy sposób
mam przeglądarkę= Netscape
drugi sposób
Masz niewłaściwą wersję przeglądarki!!!
```

Przykłady do uruchomienia:

```
document.write("mam przeglądarkę=  "+navigator.appName);
```

inny przykład

```
if((navigator.appName=="Firefox") && (parseInt(navigator.appVersion)>=3))
        { document.write("mam przeglądarkę="+navigator.appName);}
                        else
       {document.write("Masz niewłaściwą wersję przeglądarki!!!");}
```

Opis właściwość użytych w przykładzie
appName → zawiera nazwę przeglądarki,
appVersion → zawiera numer wersji przeglądarki,
funkcja parseInt() → zamienia dowolny łańcuch na liczbę.

koniec przykładu
==========

Zadanie 74.

Uruchom poniższy przykład:
Przykład3 do zadania 32c → do **navigator** i jej właściwości **userAgent**
Uruchom dwa poniższe przykłady w jednym skrypcie aby otrzymać wygląd ekranu jak poniżej.

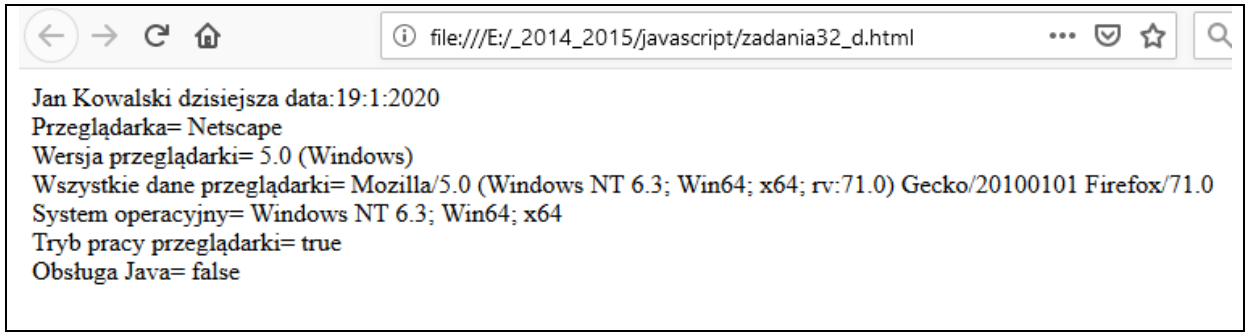
Rozpoznawanie typu systemu operacyjnego

```
<!DOCTYPE html>
<html lang="pl">
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=utf-8">
    <meta http-equiv="Content-Script-Type" content="text/javascript">
    <title>Moja strona WWW</title>
  </head>
  <body>
    <script type="text/javascript">
    var agent = navigator.userAgent.toLowerCase();
    var nazwa = "nieznany";
    if(agent.indexOf('windows') != -1)   {nazwa = "Windows";}
    if(agent.indexOf('linux') != -1)        {nazwa = "Linux";}
    else if(agent.indexOf('mac') != -1)   {nazwa = "MacOS";}
        document.write("Twój system to: " + nazwa);
    </script>
  </body>
</html>
```

koniec przykładu
==========

Zadanie 75.

Treść zadania:
Na podstawie przykładów oraz tabeli wygeneruj następujący tekst w oknie przeglądarki:



W celu rozwiązania zadania:
1)Użyj przykład, który pokazuje jak uzyskać datę:

```
<!DOCTYPE html>
<!-- metatagi -->
<head>
<script type="text/javascript">
function wyswietlDate(){
    var Today = new Date();
    var Month = (Today.getMonth()+1);
    var Day = Today.getDate();
    var Year = Today.getFullYear();
       if(Year <= 99)     Year += 1900
    return  Day + "-" + Month + "-" + Year;
}
</script>
</head>
<body>
 <script type="text/javascript">
       document.write("Dzisiaj jest " + wyswietlDate());
  </script>
</body>
</html>
```

2) Podpowiedzi na temat rozwiązania zadania:

Uczeń [tutaj twoje imię i nazwisko] ustalił w dniu [dzisiejsza data→ ustalona z użyciem odpowiedniego obiektu JS, patrz przykład powyżej] następujące dane:

Wykrywana cecha	Wygląd ekranu
Typ przeglądarki z użyciem **appNane**	Przeglądarka=…….
Wersja przeglądarki z użyciem **navigator.appVersion**	Wersja przeglądarki=…….
Wszystkie dane dotyczące przeglądarki z użyciem **userAgent**	Wszystkie dane Przeglądarki=……
Określenie systemu operacyjnego, na którym jest uruchomiona przeglądarka **osepu**	System operacyjny =……..
W jakim trybie pracuje przegladarka **onLine**	Przegladarka pracuje w trybie=……..
Czy przegladarka obsługuje Java. Użyj metody **javaEnabled()**	Obsługa Java=……..

Tabela: Właściwość obiektu navigator

Nazwa	Znaczenie	Dostępność
appCodeName	Nazwa kodowa przeglądarki.	FF, IE, NN, OP
appName	Oficjalna nazwa przeglądarki.	FF, IE, NN, OP
appVersion	Wersja kodowa przeglądarki.	FF, IE, NN, OP
appMinorWersion	Podwersja przeglądarki	IE, OP
cookieEnabled	Określa, czy w przeglądarce jest włączona obsługa plików cookie (true — tak, false — nie).	FF, IE, NN, OP
cpuClass	Rodzina procesorów urządzenia, na którym jest uruchomiona przeglądarka.	IE
language	Kod języka przeglądarki.	FF, NN, OP
mimetypes	Tablica zawierająca listę typów mime obsługiwanych przez przeglądarkę. W niektórych przypadkach ta właściwość jest wartością pustą.	FF, IE, NN, OP

online	Określa, czy przeglądarka pracuje w trybie online.	FF, IE
Oscpu	Określa system operacyjny, na którym jest uruchomiona przeglądarka.	FF, NN
Platform	Określa platformę systemową, dla której jest przeznaczona przeglądarka.	FF, IE, NN, OP
Plugins	Tablica zawierająca odniesienia do rozszerzeń zainstalowanych w przeglądarce.	FF, IE, NN, OP
Product	Nazwa produktowa przeglądarki (np. Gecko).	FF, NN
productSub	Wersja produktowa przeglądarki.	FF, NN
systemLanguage	Język systemu operacyjnego, w którym jest uruchomiona przeglądarka.	IE
userAgent	Ciąg wysyłany przez przeglądarkę do serwera jako nagłówek HTTP_USER_AGENT. Z reguły pozwala na dokładną identyfikację przeglądarki.	FF, IE, NN, OP
userLanguage	Język użytkownika (z reguły wersja językowa przeglądarki).	IE, OP
Vendor	Dostawca (producent) przeglądarki.	FF, NN
vendorSub	Numer produktu według producenta (np. 5.1, 6.0).	FF, NN

Metody obiektu navigator

Metoda javaEnabled

Wywołanie: javaEnabled()

Metoda javaEnabled zwraca wartość true, jeżeli dana przeglądarka obsługuje Javę, lub false w przeciwnym razie.

==========

Zadanie 76. (dawniej Zadanie 33 -T)

Opisz teorię dotyczącą Obiekt dokument:

Obiekt document

- Zawiera informacje o bieżącym dokumencie HTML.
- Poprzez jego właściwości document mamy wpływ na elementy strony (np. kolor tła, kolor czcionki).
- Metody document write umożliwia wyświetlenie np. tekstu w oknie przeglądarki.

Koniec teorii

==========

Zadanie 77. (dawniej Zadanie 33 P)

Uruchom następujący przykład a następnie dopisz i wyświetlanie dwie własności właściowści z tabeli poniżej.

```
<!DOCTYPE html>
<html lang="pl">
<head>
   <head>
      <meta charset="utf-8">
  <meta http-equiv="Content-Script-Type" content="text/javascript">
  <title>Moja strona WWW</title>
 </head>
 <body>
  <script type="text/javascript">
        document.write("Podstawowe informacje o dokumencie:<br />");
        document.write("Tryb kompatybilności: ");
```

```
        document.write(document.compatMode + "<br />");
        document.write("URL: " + document.URL + "<br />");
        document.write("Liczba apletów: ");
        document.write(document.applets.length + "<br />");
        document.write("Liczba obrazów: ");
        document.write(document.images.length + "<br />");
        document.write("Liczba formularzy: ");
        document.write(document.forms.length + "<br />");
        document.write("Tytuł: " + document.title + "<br />");
        document.write("Data ostatniej modyfikacji: ");
        document.write(document.lastModified + "<br />");
   </script>
  </body>
</html>
```

koniec przykładu

==========

Tabela Wybrane właściwości obiektu document

Nazwa	Znaczenie	Dostępność
All	Obiekt zawierający odniesienia do wszystkich elementów dokumentu, charakterystyczny dla przeglądarki Internet Explorer.	IE, OP
alinkColor	Kolor aktywnego odnośnika.	FF, IE, NN, OP
Anchors	Tablica zawierająca odniesienia do znajdujących się w dokumencie obiektów typu Anchor.	FF, IE, NN, OP
Applets	Tablica zawierająca odniesienia do znajdujących się w dokumencie obiektów typu Applet.	FF, IE, NN, OP
bgColor	Kolor tła dokumentu.	FF, IE, NN
Body	Obiekt zawierający treść dokumentu HTML.	FF, IE, NN, OP
characterSet	Ciąg określający kodowanie znaków w dokumencie.	FF, NN
compatMode	Ciąg określający tryb kompatybilności dokumentu ze standardami HTML.	FF, IE, NN, OP
Cookie	Ciąg znaków zawierający cookies danego dokumentu.	FF, IE, NN, OP
docType	Obiekt określający typ dokumentu (DTD).	FF, NN, OP
Domain	Nazwa domenowa serwera, z którego pochodzi dokument.	FF, IE, NN, OP
Embeds	Tablica zawierająca odniesienia do znajdujących się w dokumencie obiektów zagnieżdżonych.	FF, IE, NN, OP
fgColor	Kolor tekstu dokumentu.	FF, IE, NN, OP
fileCreatedDate	Data utworzenia pliku zawierającego aktualnie otwarty dokument, w formacie mm/dd/rrrr. Właściwość charakterystyczna dla przeglądarek z rodziny Internet Explorer.	IE
fileModifiedDate	Data ostatniej modyfikacji pliku zawierającego aktualnie otwarty dokument, w formacie mm/dd/rrrr. Właściwość charakterystyczna dla przeglądarek z rodziny Internet Explorer. Należy wziąć pod uwagę fakt, że nie każdy serwer WWW dostarcza taką informację.	IE
fileSize	Rozmiar pliku zawierającego aktualnie otwarty dokument. Właściwość charakterystyczna dla przeglądarek z rodziny Internet Explorer. Należy wziąć pod uwagę fakt, że nie każdy serwer WWW dostarcza taką informację.	IE
Forms	Tablica zawierająca odniesienia do znajdujących się w dokumencie obiektów formularzy.	FF, IE, NN, OP
Height	Wysokość dokumentu.	FF, NN
Images	Tablica zawierająca odniesienia do znajdujących się w	FF, IE, NN, OP

	dokumencie obrazów.	
implementation	Obiekt typu DOMImplementation pozwalający stwierdzić, które elementy modelu DOM są implementowane przez bieżące środowisko.	
lastModified	Zwiera datę i czas ostatniej modyfikacji dokumentu.	FF, IE, NN, OP
linkColor	Definiuje kolor odnośnika.	FF, IE, NN, OP
Links	Tablica zawierająca odniesienia do znajdujących się w dokumencie obiektów typu Link (odnośników).	FF, IE, NN, OP
Location	Obiekt przechowujący URL bieżącego dokumentu.	FF, IE, NN, OP
Plugins	Tablica zawierająca odniesienia do znajdujących się w dokumencie obiektów typu Plugin.	FF, IE, NN, OP
Referrer	Zawiera URL dokumentu, z którego nastąpiło odwołanie do bieżącego dokumentu.	FF, IE, NN, OP
styleSheets	Zawiera wszystkie style zdefiniowane w dokumencie.	FF, IE, NN, OP
Title	Zawiera tytuł dokumentu zdefiniowany za pomocą znacznika <title>.	FF, IE, NN, OP
URL	Ciąg zawierający URL bieżącego dokumentu.	FF, IE, NN, OP
vLink	Kolor odwie własnościdzonego odnośnika.	FF, IE, NN, OP
Width	Szerokość dokumentu.	FF, NN

==========

Zadanie 78.

Opisz teorię dotyczącą Obiekt location.

Obiekt location

Obiekt ten zawiera informacje o bieżącym adresie URL. Jego właściwości odpowiadają kolejnym członom adresu.

Ogólna postać lokalizatora URL to:

protocol://host:port/path#fragment?query

Właściwości obiektu location

protocol →to łańcuch znakowy określający protokół, zgodnie z którym należy nawiązać łączność z podanym serwerem (np. http:, ftp:, file:),

host → to łańcuch znakowy określający nazwę serwera z bieżącego adresu, port to łańcuch znakowy odpowiadający portowi, przez który należy połączyć się z serwerem,

pathname →to łańcuch znakowy określający ścieżkę dostępu do dokumentu na serwerze,

hash → to łańcuch znakowy odpowiadający nazwie zakotwiczenia (przypisanie tu jakiejś wartości spowoduje przewinięcie dokumentu do wskazanego punktu),

serach→ to człon lokalizatora URL.

Właściwość obiektu location w formie tabeli.

Nazwa	Znaczenie	Dostępność
Hash	Część adresu URL znajdująca się za znakiem #.	FF, IE3, NN2, OP7
Host	Część adresu URL zawierająca nazwę hosta oraz numer portu.	FF, IE3, NN2, OP7
hostname	Część adresu URL zawierająca nazwę hosta.	FF, IE3, NN2, OP7
Href	Pełny adres URL.	FF, IE3, NN2, OP7
pathname	Część adresu URL zawierająca ścieżkę dostępu i nazwę pliku.	FF, IE3, NN2, OP7
Port	Część adresu URL zawierająca numer portu.	FF, IE3, NN2, OP7
protocol	Część adresu URL zawierająca nazwę protokołu (np. http, ftp).	FF, IE3, NN2, OP7

Search	Część adresu URL znajdująca się za znakiem zapytania.	FF, IE3, NN2, OP7

Metody obiektu location

1)Metoda assign
Wywołanie: location.assign(url)
Metoda assign wczytuje dokument o adresie wskazanym przez argument url.

2)Metoda reload
Wywołanie: location.reload(force)
Metoda reload wymusza ponowne wczytanie bieżącej strony. Jeśli obecny jest argument force i ma on wartość true, wymusza to ponowne wczytanie zawartości witryny z serwera. W pozostałych przypadkach przeglądarka może wczytać ją z pamięci cache.

3)Metoda replace
Wywołanie: location.replace(url)
Metoda replace zastępuje bieżący dokument przez wczytany spod adresu wskazanego przez argument url.
Różnica między assign a repleace
W przeciwieństwie do metody assign, po zastosowaniu replace bieżąca strona nie zostanie zapisana w historii przeglądanych witryn, nie będzie więc można się do niej ponownie odwołać poprzez wciśnięcie przycisku Wstecz bądź też wywołanie history.go(-1).
Tę metodę można wykorzystać np. w sytuacji, kiedy witryna została przeniesiona pod inny adres, a pod starym ma być umieszczona stosowna informacja oraz automatyczne przekierowanie użytkownika (po zadanym czasie).

4)Metoda toString
Wywołanie: location.toString()
Metoda toString przekształca zawartość obiektu location w ciąg znaków reprezentujący przechowywany przezeń adres URL.

1. window.location.href →przechowuje pełny adres
2. window.location.hostname →wyłącznie domenę.
3. window.location.reload() → odświeża stronę,
4. window.location.assign() → przechodzi na podany adres,
5. window.location.replace() → przechodzi na podany adres i nie tworzy nowego wpisu w historii przeglądania.

Zadanie 79. (dawniej Zadanie 34 P)
Na podstawie przykładów zapisanych poniżej oraz zapisów teoretycznych wykonaj stronę z czterema przyciskami, przycisk powinien mieć odpowiedni opis np. Kowalski-strona CKE (nazwisko ucznia):

1. przycisk 1 →strona CKE uruchamiana po 4 sekundach
2. przycisk 2 → strona oficjalna Gdańska uruchamiana po 6 sekundach
3. przycisk 3 → strona zaproponowana przez ucznia uruchamiana po 10 sekundach
4. przycisk 4 → strona odświeżenie aktualnej strony uruchamiana po 8 sekundach

Przykład:
Aby zmienić adres strony wyświetlanej w oknie, wystarczy zmienić lokalizator URL.

```
window.location = https://www.google.pl/search?q=warszawa+lotnisko

window.location = "http://helion.pl/ksiazki/cwjas2.htm";
```

Przykład:
Wyświetlanie strony po naciśnięciu PRZYCISKU (button)

```
<!DOCTYPE html>
<html lang="pl">
<head>
   <head>
       <meta charset="utf-8">
       <title>Przykład nr tutaj wpisz odpowiedni numer oraz nazwisko </title>
      <script language="JavaScript">
        function ZSE_Open()
         {
           window.location = "http://zse.gda.pl";

         }

        function OKE_Open()
         {
           window.location = "http://oke.gda.pl";

         }
      </script>
    </head>
<body>
  <form>
     <input type="button" name="przycisk1" value="Strona ZSE" onclick="ZSE_Open(' ')">
     <input type="button" name="przycisk2" value="Strona OKE" onclick="OKE_Open(' ')">
   </form>
</body>
</html>
```

Przykład:
Przekierowanie użytkownika na nowy adres

setTimeout(fn, time)
Służy ona wywołania z opóźnieniem funkcji przekazanej w pierwszym parametrze. W drugim parametrze podajemy czas w milisekundach po jakim zostanie ta funkcja wywołana.

fn → wywoływana funkcja
time → czas opóźnienia w milisekundach

```
window.setInterval()

funkcję co określoną liczbę milisekund, wiele razy.
```

```
function dot()
{
console.log('.');
}
setTimeout(dot,2000);        //po 2 sekundach na konsoli pojawi się kropka
```

```
function dot()
```

```
{
console.log('.');
}
var funId=setTimeout(dot,2000);
clearTimeout(funId);                    //zanim upłynie 2 s, odwołujemy funkcję,
var id=setInterval(dot, 2000);          //po upływie każdych 2 s, wykona się funkcja
clearInterval(id);                      //od tego momentu funkcja przestanie się wykonywać
```

```
<!DOCTYPE html>
<html lang="pl">
<head>
   <meta charset="utf-8">
   <title>Moja strona WWW</title>
   <script type="text/javascript">
   function js_zmienStrone()
   {
    window.location.replace("http://helion.pl/ksiazki/jscpk.htm");
   }
   setTimeout("js_zmienStrone();", 5000);
   </script>
 </head>
 <body>
   <p>Witryna została przeniesiona pod nowy adres:<br />
     http://helion.pl/ksiazki/jscpk.htm<br />
     Za 5 sekund zostaniesz przeniesiony do nowej lokalizacji.
   </p>
 </body>
</html>
```

==========

Zadanie 80.

Opisz teorię dotycząca Obiekt history.

Obiekt history

1. Zawiera historię stron odwie własnościdzanych w bieżącej sesji.
2. Posiada dwie własności właściwości:
   - current → zawiera bieżący lokalizator URL,
   - length → zawiera długość listy historii.

window.history→wszystkie możliwości

window.history.length → możesz sprawdzić ile stron odwie własnościdził użytkownik zanim wszedł na Twoją, ze względów bezpieczeństwa nie da się jednak pobrać ich adresów.
window.history.forward()→Działa to jak przyciski "w przód" w przeglądarce.
window.history.back()→Działa to jak przyciski "w tył" w przeglądarce.
window.history.go(n) →pozwala przejść o kilka stron do przodu lub do tyłu naraz.
window.history.go(-2); →przejdzie dwie własności strony wstecz
window.history.go(0); →odświeży aktualną stronę

==========

Zadanie 81.

Zadanie na history

Zadanie 82.

Zadanie teoretyczne na przyszły rok screen, close, windowo itp

Obiekt screen

window.screen
window.screen.colorDepth→ odpowiada za głębię kolorów.
window.screen.width → szerokość ekranu,
window.screen.height → wysokość ekranu,
window.screen.availWidth → tylko dostępna szerokość ekranu, to znaczy odejmuje wszelkie systemowe paski menu,
window.screen.availHeight → analogicznie jak wyżej,

==========

Metoda .open()

window.open()→ to metoda, która pozwala na otworzenie nowego okna.
Jej parametrami są:
URL – adres strony, która ma zostać załadowana w nowym oknie,
name – określa sposób otwarcia nowego okna,
specs – lista wartości rozdzielonych przecinkami, które dotyczą takich właściwości okna jak możliwość zmiany rozmiaru, wysokość, szerokość, …,
replace – parametr, który mówi czy nowe okno ma dodać nowy wpis do historii przeglądarki czy zastąpić aktualny.

Metoda .close()

window.close() → służy do zamknięcia aktualnego okna.

Metoda → window.moveTo(), window.moveBy(), window.resizeTo()

window.moveTo()→metody służą do zmiany położenia okna przeglądarki na pulpicie. Przyjmują parametry określające pozycję w poziomie i pionie
window.moveBy() → metody służą do zmiany położenia okna przeglądarki na pulpicie. Przyjmują parametry określające pozycję w poziomie i pionie
window.resizeTo() → Trzecia metoda zmienia rozmiar okna przeglądarki. Przyjmuje nową szerokość i wysokość.

Zadanie 83.

- **Zadanie praktyczne na przyszły rok screen, close, windowo itp**

==========

Ciekawostka

```
/*jeśli nie jesteśmy pewni czy dana funkcjonalność jest w danej przeglądarce, robimy tak:*/

if( typeof window.addEventListener==='function' ){
  //kod jeśli funkcjonalność JEST wspierana
}else{
  //kod jeśli funkcjonalność NIE jest wspierana
}
```

console.log(answer); → gdzieś to umieścić

Ten obiekt odnosi się do aktualnie załadowanego dokumentu czyli strony WWW. Wszystkie jego pola i metody są związane z modelem DOM. Zostaną omówione w kolejnej lekcji.

Zadanie 84.

Wykonaj:

1. Zapisz w „szkielecie", co to jest model DOM.
2. Na podstawie przykładu schematu/drzewa (umieszczonego poniżej) narysuj „szkielecie"schemat DOM dla kodu (musisz wybrać odpowiedni numer schematu/kodu). Metoda rysownia schematu/drzewa dowolna.

Wykonujesz układ jak układach zademonstrowanych poniżej.

Obliczenie	**Który kod**
(Numer_w_dzienniku) mod 4=1	kod1
(Numer_w_dzienniku) mod 4=2	kod2
(Numer_w_dzienniku) mod 4=0	kod3
(Numer_w_dzienniku) mod 4=3	kod4

Model DOM

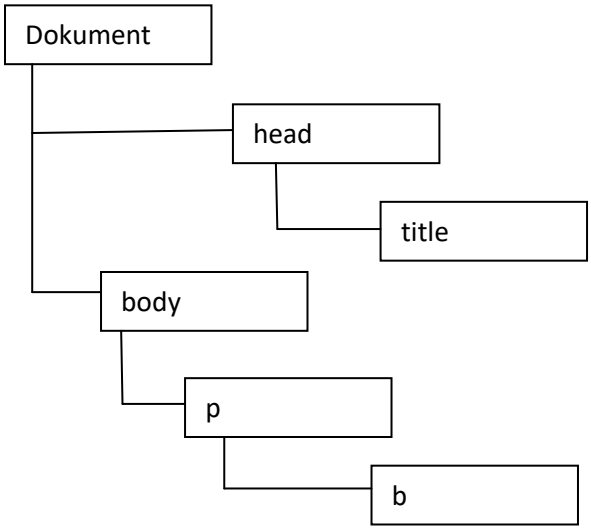
DOM (Document Object Model) to sposób opisu złożonych dokumentów XML i HTML w postaci hierarchii modeli (niektóre obiekty są nadrzędne nad innymi, cała strona tworzy drzewo obiektów) modelu obiektowego.

Mechanizm działania DOM →Kod HTML zamieniany jest przez przeglądarkę na stronę WWW. Przeglądarka przechowuje informację o stronie WWW w postaci modelu obiektowego strony. DOM ma hierarchię obiektów, udostępniane są metody oraz właściwości obiektów.

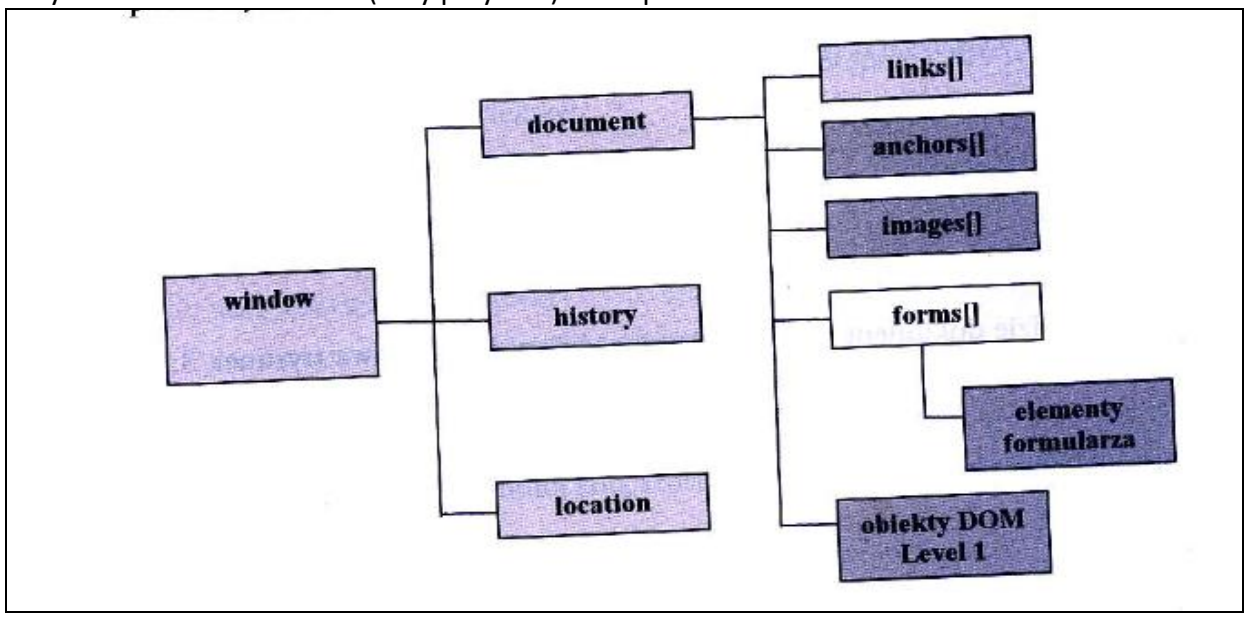
Przykład kodu + schemat/drzewo DOM

```
<!DOCTYPE HTML>
<html>
  <head>
        <title>Obiekty</title>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
  </head>
  <body>
        <p>Strona DOM<b>pokaz</b></p>
  </body>
</html>
```

Drzewo/Hierarchia obiektów z **kodu** poprzedniego (przykład)



Wycinek hierarchii DOM→(inny przykład) oraz sposób odwolania do obieku links1.



W celu odwolania się do obiektu należy podać nazwy obiektów w hierarchi oddzielonych kropkami.

np. window.document.link1

==========

Zadanie 85.

Będzie strona i trzeba narysować schemat hierarchii Dom

==========

Zadanie 86.

Teoria

Dodatkowe elementy DOM:

- form — formularz,
- anchor — zakotwiczenie,
- link— odsyłacz,
- image— obrazek,
- embed— dodatek,
- applet— aplet Javy,
- frame— ramka,
- area— mapa graficzna.

==========

Predefiniowane obiekty w JavaScript

- String — łańcuch tekstowy. Posiada własność ***length*** i metody: ***slice(), split().***
- Array— tablica. Posiada własność ***length*** i metody: ***concat(), pop(), push().***
- Date— data. Posiada metody: ***getMonth(), getDay()***.
- Match— obiekt matematyczny.

==========

Zadanie 87.

Zadanie teoretyczne

Dodatkowe elementy DOM:

- form — formularz,
- anchor — zakotwiczenie,
- link— odsyłacz,
- image— obrazek,
- embed— dodatek,
- applet— aplet Javy,
- frame— ramka,
- area— mapa graficzna.

==========

Dział 9 Obiekt String (teksty)

Zadanie 88.

Zadanie na praktyczne na

String — łańcuch tekstowy. Posiada własność ***length*** i metody: ***slice(), split().***

Obiekt String

Obiektem zawsze występującym w języku JavaScript jest obiekt String. Posiada on jedną właściwość,
length, określającą długość łańcucha.

Przykład

```
tekst = "Obiekty języka JavaScript"
dl = tekst.length
```

Zmiennej **dl** przypisana zostanie wartość 25, określająca długość tekstu.

Obiekt **String** posiada dwa typy metod.

Typ1
metoda substring(), która zwraca podzbiór tego łańcucha. Jej parametry określają położenie początku i końca podzbioru.

Przykład

```
x = tekst.substring (15,19)
```

Zostanie zwrócony łańcuch Java.

Typ2
Metoda **toUpperCase()** zamienia wszystkie litery ciągu **na duże**,
Metoda **toLowerCase()** zamienia wszystkie litery ciągu **na małe**.

Przykład

```
tekst = "Obiekty języka JavaScript"
x = tekst.toLowerCase()
/*Zostanie zwrócony łańcuch obiekty języka javascript */
```

Przykład

```
tekst = "Obiekty języka JavaScript"
x = tekst.toUpperCase()
/*Zostanie zwrócony łańcuch OBIEKTY JĘZYKA JAVASCRIPT*/
```

Przykład
Temat: Jak wykonać aby pierwsza litera ciągu znaków była pisana dużą literą.

Wykonanie:
Do istniejących obiektów można dodawać nowe właściwości i można definiować dla nich nowe metody. Do obiektu String dodamy dodatkową funkcjonalność, dzięki której pierwsza litera łańcucha będzie pisana dużą literą.

```
String.prototype.duzaLitera = function() { return this.charAt(0).toUpperCase() + this.substr(1);}
```

Opis działania zdefiniowanej powyżej metody

Metoda **charAt()** zwraca znak z pierwszej pozycji łańcucha znaków. Metoda **substr()** zwraca podzbiór łańcucha znaków. Jako parametr tej metody został podany indeks pierwszego znaku podzbioru. Zadeklarowana metoda została dołączona do obiektu **String** i od tej pory każdy nowy tekst będzie posiadał metodę, która zamieni jego pierwszą literę na dużą.

Przykład
Temat: Zamiana pierwszej litery na dużą.

```
var tx1 = 'komputer';
document.write(tx1.duzaLitera())
```

W wyniku wykonania skryptu wyświetli się tekst 'Komputer'.

Obiekt Date

Obiekt specjalny Date, który służy do przechowywania wartości daty i czasu. Przy jego pomocy można odczytać wartość daty i czasu, można też rozłożyć te wartości na części, odczytując oddzielnie dzień, miesiąc, rok itp. Można również te części niezależnie modyfikować.

Przykład
Temat: Odczytanie bieżącej datę i czas do zmiennej **dat_cz**.

```
var dat_cz = new Date();
```

Przykład
Temat:Utworzenie obiektu, który będzie zawierał ściśle określoną wartość daty i godziny.

```
var data = new Date(2018,2,27);
```

Opis:
Można utworzyć obiekt z określoną liczbą parametrów. Tych parametrów może być od dwóch do siedmiu **(rok, miesiąc, dzień, godzina, minuty, sekundy, milisekundy).**
W języku JavaScript wartości daty i czasu są przechowywane w formacie **timestamp**, czyli jako liczba milisekund, które upłynęły od północy 1 stycznia 1970 roku.

Do konwersji obiektu Date na tekst służy kilka funkcji:
toString() — zwraca datę, czas oraz informacje o strefie czasowej w języku angielskim,
toLocaleString() — zwraca datę i czas dla bieżących ustawień regionalnych,
toUTCString()— zwraca datę, czas oraz informacje o strefie czasowej dla formatu UTC (Universal Coordinated Time),
toGMTString()— działa jak funkcja toUTCString(),
toDateString()— zwraca tylko datę w języku angielskim,
toLocaleDateString() — zwraca tylko datę dla bieżących ustawień regionalnych,
toTimeString()— zwraca tylko czas w języku angielskim,
toLocaleTimeString()— zwraca tylko czas dla bieżących ustawień regionalnych.

UWAGA
Dla metod **toString() i toLocaleString()** różne przeglądarki zwracają wyniki w różny sposób.

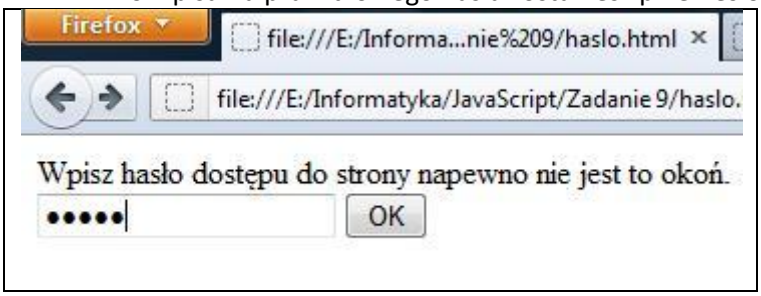
porównywanie tekstów ułożyć zadanie

Hasła

Zadanie 89.
Temat: Hasło na stronę wersja 1.

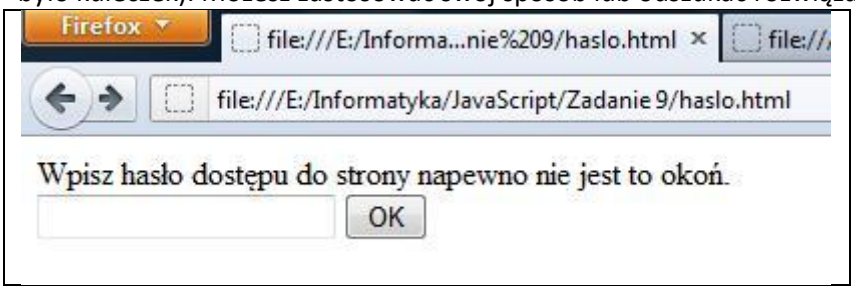
Wykonaj:

- Posługując się opisem dotyczącym wykonanie hasła do strony zabezpiecz hasłem stronę z zadania powyżej (kliknij link zawarty w tytule).
- Wykonaj stronę na, której będzie tylko pytanie o hasło w celu uzyskania dostępu do strony.
- Po wpisaniu prawidłowego hasła zostaniesz przeniesiony do strony



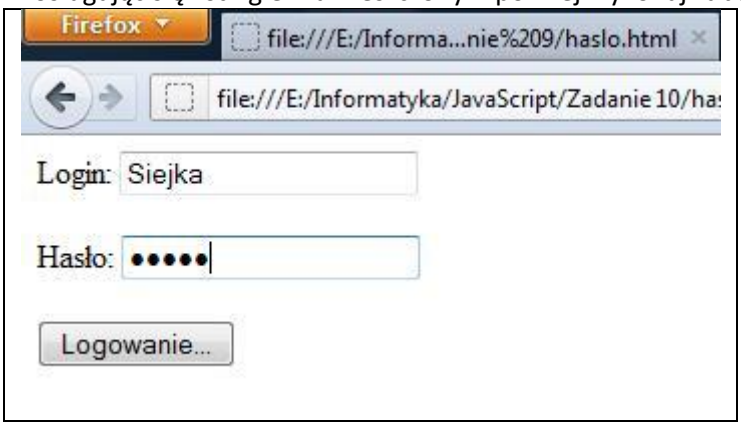
Zadanie 90.
Temat: Hasło na stronę wersja 1→ modyfikacja.

Zmień zadanie poprzednie tak aby przy ponownym logowaniu nie było wpisanego hasła (czyli nie było kuleczek). Możesz zastosować swój sposób lub odszukać rozwiązanie w instrukcji.



Zadanie 91.
Temat: Hasło na stronę wersja 2

Wykonaj:
Posługując się listingiem umieszczonym poniżej wykonaj zabezpieczenie strony z zadania8.

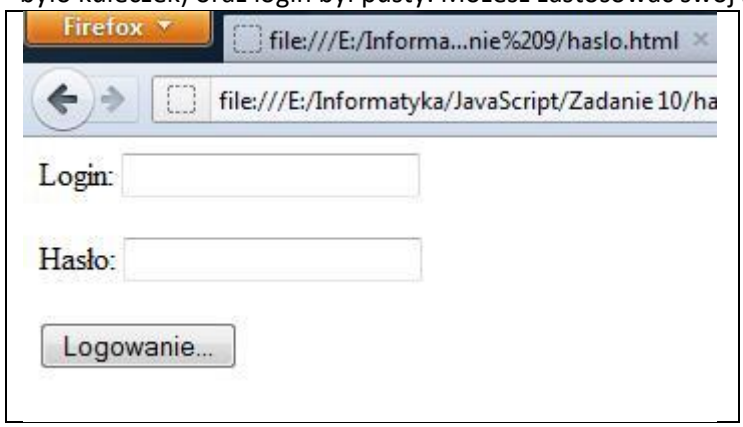


Zmień:

login→ na Twoje nazwisko
hasło→na Twoje imie
parametr→ funkcji Twoje inicjały (bez polskich liter)

```
<html>
<head>
<script type="text/javascript">
function logowanie(dane)
{
 if(dane.login.value=="k"&dane.haslo.value=="k")
    {
     window.location.replace("obliczenia.html");
     }
                    else
    {
      window.location.replace("");
    }
}
</script>
</head>
 <body>
  <form name="formularz">
          Login:  <input type="text" name="login"/><br><br>
          Hasło:  <input type="password" name="haslo"/><br><br>
          <input type="button" value="Logowanie..." onClick="logowanie(this.form)"/><br><br>
   </form>
 </body>
</html>
```

Zadanie 92.
Temat: Hasło na stronę wersja 2→modyfikacja.
Zmień zadanie poprzednie tak aby przy ponownym logowaniu nie było wpisanego hasła (czyli nie było kuleczek) oraz login był pusty. Możesz zastosować swój sposób lub wzorować się na zadaniu 9a.



Zadanie 93.
Temat: Hasło na stronę wersja 3

Zmień:
Zmień tak zadanie11 aby nieistotne było login i hasło pisane małymi czy dużymi literami (całość lub część) np. KOwaLski było prawidłowe.

Skomplikowana funkcja sprawdzająca hasło

```
<script language='JavaScript' type='text/JavaScript'>

function weryfikacja_hasla()
{
var prob = 1;
var haslo = prompt('Podaj hasło','');
if (haslo != null)
{
while (prob < 3)
{
if (!haslo) history.go(-1);
//Metoda toLowerCase zwraca wartość łańcucha znaków skonwertowanego na
małe litery
if (haslo.toLowerCase() == 'asdf')
{
alert('Hasło OK!');
window.open('skrypt1.html','_top');
break;
}
prob+=1;
var haslo = prompt('Złe hasło. Spróbuj ponownie','Hasło');
}
}
if (haslo !=null) {
if (haslo.toLowerCase()!='Hasło' + prob ==3) history.go(-1);return ' ';}}
</script>
```

==========

Zadanie 94.

Zadanie na praktyczne na (może z obliczeń przenieść na tablice)

- Array— tablica. Posiada własność ***length*** i metody: ***concat(), pop(), push().***

==========

Zadanie 95.

Zadanie na praktyczne na (może z obliczeń przenieść na tablice)

- Date— data. Posiada metody: ***getMonth(), getDay()***.

==========

Tworzenie obiektów

Aby utworzyć nowy obiekt w języku JavaScript, należy skorzystać z konstrukcj (funkcji)i, która zdefiniuje:

- nazwę obiektu
- utworzy właściwości
- utworzy metody.

Obiekty wbudowane JavaScript

Przykład
Temat: Wykorzystania JavaScript na stronie WWW do wyświetlania daty i czasu jako elementu strony internetowej.

```
<html>
<head>
<title>JavaScript - Data i czas</title>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<body>
<h2>Wyświetlam bieżącą datę i czas</h2>
<p>
<script type="text/javascript">
data_n = new Date();
data_l = data_n.toString();
data_u = data_n.toGMTString();
data_r = data_n.toLocaleString();
document.write("<b>Czas lokalny:</b> " + data_l + "<br>");
document.write("<b>Czas uniwersalny:</b> " + data_u + "<br>");
document.write("<b>Czas regionalny:</b> " + data_r + "<br>");
</script>
</p>
</body>
</html>
```

Opis działania przykładu powyżej:
Utworzonej zmiennej o nazwie data_n przypisany został obiekt Date. Zmienne data_l, data_u i data_r zawierają tekstową postać czasu lokalnego, uniwersalnego i regionalnego, odpowiadającego wartości przechowywanej w zmiennej data_n. Do wyświetlenia wartości otrzymanych zmiennych została użyta funkcja document.write.

Zadanie 96.

Zdefiniuj funkcję, która będzie wyświetlała datę i czas po polsku

Zadanie 97.

Utwórz stronę internetową, na której data i czas będą wyświetlane w postaci kartki z kalendarza.

Dział 10 Data i czas

Zadanie 98.

Temat: Strona wyświetlająca:

- ostatnią datę modyfikacji,
- komunikat Cześć na wciśniecie klawisz,
- aktualną datę oraz czas.

Wykonaj:
1)Dwa przyciski jeden Zadanie 82 drugi Zadanie 82 kod,
2)Dokonaj kopiowania i uruchom zadanie 82,
3)Dopisz do przykładu linie kodu tak, aby otrzymać wygląd jak na zrzucie ekranu, który jest poniżej , (wytłumaczenie dzis = new Date()--> wpisz co oznacza ten zapis),
4)Przestaw wyświetlanie tak, aby otrzymać format daty DD:MM:YY, wielkość wyświetlania 1 cm,
5)
DD→ na czerwono
MM→na zielono
YY→niebiesko

Uwaga:
Do miesiąca należy dodać 1 ponieważ funkcja odczytuje wartości od 0 do 11.

```
<html>
<head>
<title>  Zadanie 82</title>
  <script language="JavaScript">
    function pushbutton() {
      alert("Czesc !");}
  </script>
</head>
<body>
 Ostatnio zmodyfikowany
 <script language="LiveScript">
   document.write(document.lastModified,"<br>")
 </script>
 <form>
  <input type="button" name="Button1" value="Naciśnij mnie" onclick="pushbutton()">
 </form>
 <script language="JavaScript">
  dzis = new Date()
  document.write("<br>Aktualny czas : ",dzis.getHours(),":",dzis.getMinutes())
  document.write("<br>Aktualna data : ",dzis.getMonth()+1,",",dzis.getDate(),",",dzis.getFullYear());
 </script>
</body>
</html>
```

Strona po wyświetleniu powinna wyglądać mniej więcej tak:

Tutaj Twoje nazwisko np. Nowak

dzis = new Date()--> wpisz co oznacza ten zapis
dzis = dzis.getHours()--> wpisz co oznacza ten zapis
dzis = dzis.getMinutes()--> wpisz co oznacza ten zapis
dzis = dzis.getMonth()+1--> wpisz co oznacza ten zapis
dzis = dzis.getDate()--> wpisz co oznacza ten zapis
dzis = dzis.getFullYear()--> wpisz co oznacza ten zapis

Ostatnio zmodyfikowany 04/18/2022 22:31:02

Nacisnij mnie

Aktualny czas : 22:31
Aktualna data : 4,18,2022

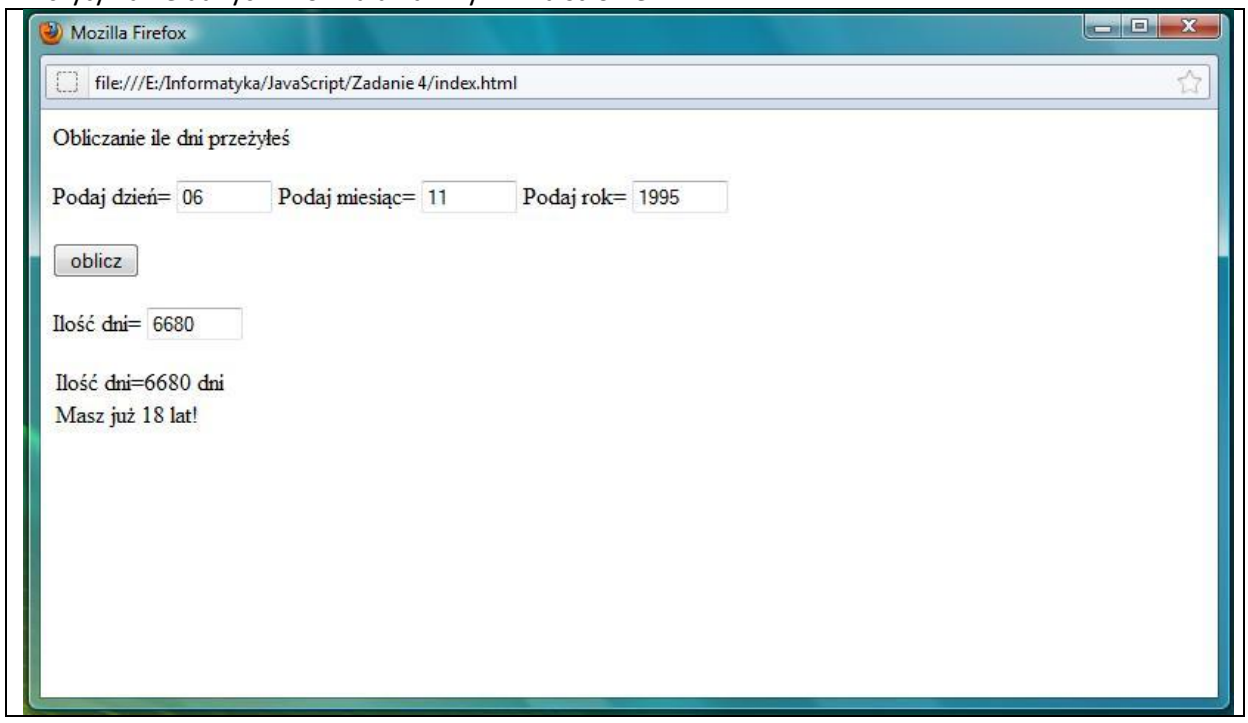
Zadanie 99.

Temat: Utwórz teraz dokument, w którym po podaniu dnia, miesiąca i roku twoich urodzin program stwierdzi:

- ile dni już przeżyłeś
- czy jesteś pełnoletni czy też nie a może właśnie masz osiemnaste urodziny.
- Wykonaj:

1)Dwa przyciski jeden Zadanie 83 drugi Zadanie 83 kod,
2)Wykonaj zadanie 83 etapami,

Wczytywanie danych z formularza. Wyniki na stronie.



Etapy wykonania zadania:

Etap1→wykonanie formularza:

Do szkieletu HTML (poniżej) wstaw kod formularza oraz tabeli (poniżej),

aby zrealizować:

a) wczytywanie daty czyli **dzień miesiąc rok**
b) wykonanie okienka do **wypisywania** obliczeń (obliczona ilość dni)
c) wykonanie tabeli do **wypisywania** obliczeń(obliczona ilość dni oraz czy masz więcej czy mniej niż 18 lat)

SZKIELET:

```
<!DOCTYPE html>
        <html lang="pl-PL">
        <html>
                 <head>
                          <meta charset="utf-8">
                         <title>  Zadanie 83-nazwisko ucznia     </title>
…….  Tutaj wstaw etap2……

                 </head>
                  <body>
…….  Tutaj wstaw etap1……
                 </body>
        </html>
```

FORMULARZ i TABELA

```
<form name="formularz">
                         Podaj dzień=
                <input size="6" name="dzien">
                         Podaj miesiąc=
                Tutaj miesiąc i rok
        <br>
        <br>
<                input TYPE="submit" Value="oblicz" >
        <br>
        <br>
                         Ilość dni=
                <input size="6" name="ilosc">
</form>
        <table>
                <tr id="pokaz1"></tr>
                Dodaj jeszcze jeden wiersz tabeli o nazwie  [czyli id] "pokaz2"
        </table>
```

Etap2→wykonanie funkcji:

Pamiętaj aby funkcje osadzić w <head>

```
<script language="JavaScript">
function dni_nazwisko(dzien,miesiac,rok)  // function dni_kowalski(dzien,miesiac,rok)
 {
        d_liczba=parseFloat(dzien)   // zamiana tekstu na liczbę, tekst który był parametrem funkcji
        m_liczba=parseFloat(miesiac)
        r_liczba=parseFloat(rok)
```

```
        var dzis=new Date();     // utworzenie obiektu z dzisiejszą datą
        var urodziny=new Date(r_liczba,m_liczba-1,d_liczba);
        // utworzenie obiektu z datą urodzin użytkownika
        //  m_liczba-1  musi być -1  bo komputer styczeń=0
        var czyRok;       // zmienne pomocnicze
        var czyMiesiac;
        var czyDzien;

ilosc_wynik=Math.floor((dzis-urodziny)/(??????????????????));
// Math.floor- metoda matematyczna  zaokrąglająca
// różnica (dzis- urodziny) podaje wynik w milisekundach
// musisz przeliczyć na dni, zapisz w miejsce znaków zapytania.

document.getElementsByName('ilosc')[0].value=ilosc_wynik;
// wyprowadzanie do okienka formularza

alert(??????????????????);
// wyprowadzanie do okienka alert

pokaz1.innerHTML="Ilość dni="+ilosc_wynik+" dni"+"<br>";
// wyprowadzenie do tabeli

czyRok = dzis.getFullYear();
czyMiesiac = dzis.getMonth();
czyDzien = dzis.getDate();
// przypisanie dzisiejszej daty do zmiennych

// teraz sprawdzanie warunków czy masz więcej czy mniej niż 18 lat

if ((czyRok==(r_liczba+18))&&(czyMiesiac==(m_liczba-1))&&(czyDzien==d_liczba))
        {
                pokaz2.innerHTML="Masz dziś 18 urodziny!";
        }
if (czyRok<(r_liczba+18))
        {
                pokaz2.innerHTML="Nie masz jeszcze 18 lat!";
        }
if ((czyRok==(r_liczba+18))&&(czyMiesiac<(m_liczba-1)))
        {
                pokaz2.innerHTML="Masz już 18 lat!";
        }
if ((czyRok==(r_liczba+18))&&(czyMiesiac==(m_liczba-1))&&(czyDzien<d_liczba))
        {
                pokaz2.innerHTML="Masz już 18 lat!";
        }
if (czyRok>(r_liczba+18))
        {
                pokaz2.innerHTML="Masz już 18 lat!";
        }
if ((czyRok==(r_liczba+18))&&(czyMiesiac>(m_liczba-1)))
        {
                pokaz2.innerHTML="Nie masz jeszcze 18 lat!";
```

```
        }
if ((czyRok==(r_liczba+18))&&(czyMiesiac==(m_liczba-1))&&(czyDzien>d_liczba))
        {
                pokaz2.innerHTML="Nie masz jeszcze 18 lat!";
        }
}
</script>
```

Etap3→wykonanie akcji dla funkcji:

Jest:
<form name="formularz">
Ma być:
<form name="formularz" action="..."
onsubmit="dni(this.dzien.value,this.miesiac.value,this.rok.value);return false">

Zadanie 100. (Zadanie 5)

Wykonaj program stoper o następującym wyglądzie:



Zasada działania:
przyciskamy START stoper → następuje początek pomiaru czasu
przyciskamy STOP starter→uzyskujemy czas w sekundach i jej tysięcznych.

Algorytm rozwiązania zadania
1)wstaw formularz z dwoma przyciskami:
Pierwszy przycisk
<input type="button" name="Button1" value="START stoper" onclick="kowalski_start()">
Oraz podobnie drugi przycisk

nazwa funkcji→ nazwisko_ucznia_start
nazwa funkcji→ nazwisko_ucznia_stop

2)wyświetl Aktualny czas:
poprzez skrypt w <body>

```
dzis_kow = new Date();
   document.write("<br>Aktualny czas : ",dzis_kow.getHours(),":",dzis_kow.getMinutes());
```

uwaga: obiekt Data() z trzema literami nazwiska ucznia

3)Wykonaj dwie własności funkcje w head

nazwa funkcji→ nazwisko_ucznia_start
nazwa funkcji→ nazwisko_ucznia_stop

poniżej wnętrze funkcji nazwisko_ucznia_start

```
  {
     start_kow = new Date();
     poczatek_kow=start_kow.getTime()
    }
```

poniżej wnętrze funkcji nazwisko_ucznia_stop

```
{
    stop_kow = new Date();
    koniec_kow=stop_kow.getTime()

     roznica_kow=koniec_kow-poczatek_kow;
     pokaz1.innerHTML=roznica_kow/1000+" s";
}
```

4)wstaw tabelę na koniec body

```
<table tutaj obramowanie>
        <tr id="pokaz1"></tr>
</table>
```

Zadanie 101. (Zadanie 6)

Wykonaj program wypisujący dzień tygodnia na podstawie aktualnej daty.
Użyj instrukcji swich case oraz getDay().

Poniżej wytłumaczenie koniecznych instrukcji.

```
switch (wyrazenie) {
   case przypadek1:
      instrukcjaGdyPrzypadek1
           break;
   case przypadek2:
      instrukcjaGdyPrzypadek2
           break;
   default:
      instrukcjaGdyBrakDopasowanegoPrzypadku
}
```

```
<script type="text/javascript">
var liczba = 10;
switch(liczba){
  case 10 :
    document.write("Zmienna liczba = 10.");
    break;
  case 20 :
    document.write("Zmienna liczba = 20.");
    break;
  default :
    document.write("Zmienna liczba nie jest równa ani 10, ani 20.");
}
</script>
```

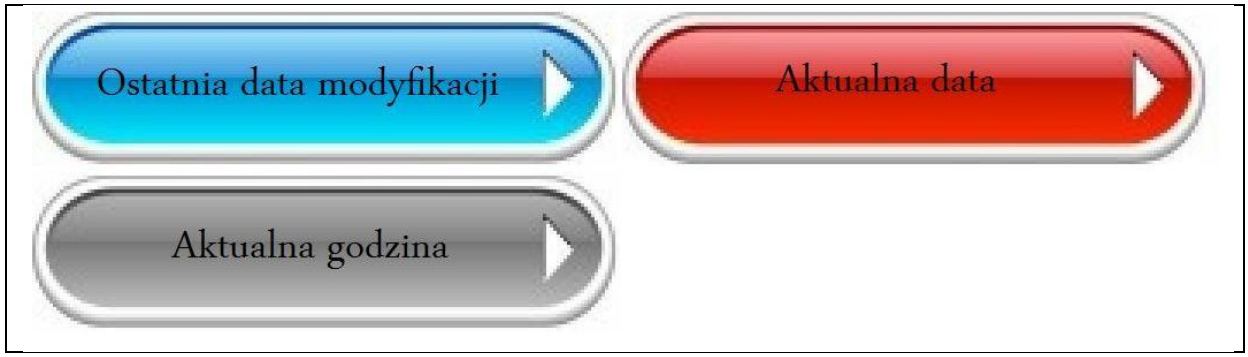
getDay()	dzień tygodnia (0 dla niedzieli, 1 dla poniedziałku, 2 dla wtorku itd.)

Zadanie 102.Zadanie 6a

Wykonaj program który będzie miał trzy aktywne przyciski wykonane z grafik.

Wykonaj;
grafiki ściągnij z Internetu. Będą to trzy komplety przycisków. Komplet składa się z dwóch grafik różniących się od siebie np. kolorem liter lub cieniowaniem.
do przycisków dodaj akcje:
akcja 1→ostania data modyfikacji
akcja 2→aktualna data
akcja 3→aktualna godzina

Widok1



Widok2



Funkcje:
Nazwy
function Przycisk1_nazwisko_ucznia()
function Przycisk2_nazwisko_ucznia()
function Przycisk3_nazwisko_ucznia()

np.
function Przycisk1_nowak()
{
pokaz1.innerHTML="Ostatnio zmodyfikowany : "+document.lastModified;
}

Kod

Wyświetlanie wyników z użyciem właściwości tabeli

```
<form>
<a onclick="Przycisk1_nowak(' ')"><img src="p1.jpg" alt="Przycisk1" border="0"
onmouseover="this.src = 'p2.jpg'" onmouseout="this.src ='p1.jpg'" /></a>
……..
</form>
<table>
……..
<tr id="pokaz2"></tr>
……
</table>
```

Dział 4 Formularze

Zadanie 103. (Przykład 5)

Temat: Pokaz działania formularza.
W formularzu sprawdzana jest zawartość pól:

- czy pole nie jest puste
- czy w adresie email jest znak @

W formularzu ustawiane jest aktywne pole:

- tekstowe→wpisywania nazwiska
- radio→środkowa opcja

Na co zwrócić uwagę;

- document.jeden.txt1.focus();→uruchomienie metody focus() czyli aktywny element formularza,
- <body onLoad="setfocus()">→ uruchomie funkcji w momencie załadowania dokumentu do przeglądarki,
- <input name="choice" type="radio" value="2" checked>Może być
→zaznaczenie aktywnego radio,
- form.txt2.value.indexOf('@', 0) == -1)→wartość wpisaną w polu tekstowym traktujemy jako tablicę znaków, sprawdzamy czy w tablicy znaków występuje znak '@', jeśli nie to metoda indexOf zwraca -1, zero (0) w parametrach oznacza, że przeszukiwanie jest od zerowego elementu tablicy.

Wykonaj:

- zmień domyślne pole tekstowe(czyli miejsce gdzie jest kursor wpisywania danych po uruchomieniu programu),
- zmień domyślne pole radio (czyli zaznaczone pole radio po uruchomieniu programu),
- dopisz takie sprawdzanie aby nie można było wpisać nazwiska krótszego niż 4 znaki→użyj własności tekstu length.

```
<html>
<head>
<title>
Przykład ->pokaz formularza
</title>
<script language="JavaScript">

    function setfocus()
     {
      document.jeden.txt1.focus(); return;
     }

    function test1(form)
     {
      if (form.txt1.value == "")
        {
          alert("Podaj ciąg znaków!")
        }
       else
        {
          alert("Cześć "+form.txt1.value+" ! Informacja poprawna!");
        }
       }
```

```
    function test2(form)
      {
       if (form.txt2.value == "" || form.txt2.value.indexOf('@', 0) == -1)
        {
             alert("Niepoprawny adres poczty elektronicznej!");
        }
           else
        {
           alert("OK!");
        }
      }

</script>
</head>
<body onLoad="setfocus()">

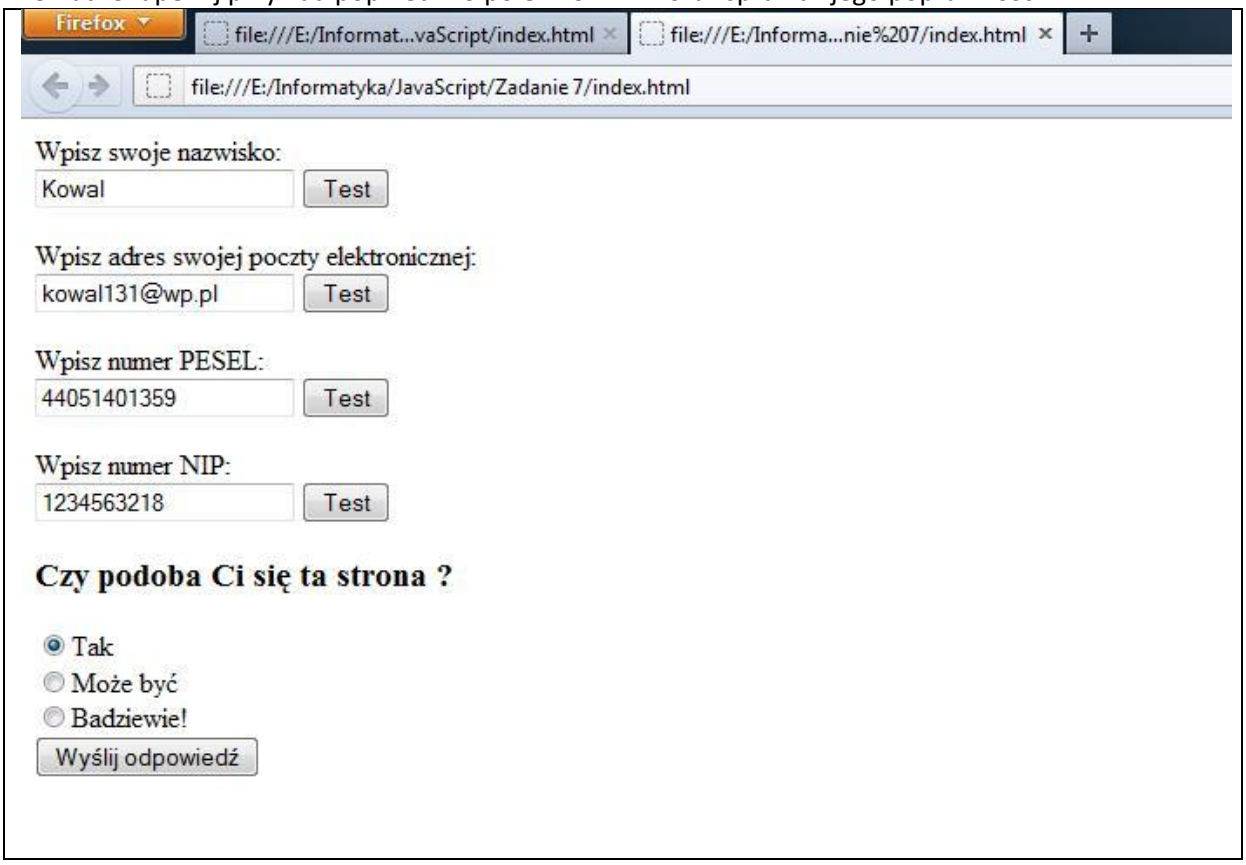
<form name="jeden">
       Wpisz swoje nazwisko:<br>
       <input type="text" name="txt1">
      <input type="button" name="button1" value="Test" onClick="test1(this.form)">
</form>

<form name="dwa">
          Wpisz adres swojej poczty elektronicznej:<br>
          <input type="text" name="txt2">
          <input type="button" name="button2" value="Test" onClick="test2(this.form)">
</form>


<form method=post action="mailto:dowolny@mail.pl">
          <h3>Czy podoba Ci się ta strona ?</h3>
          <input name="choice" type="radio" value="1">Tak<BR>
          <input name="choice" type="radio" value="2" checked>Może być<BR>
          <input name="choice" type="radio" value="3">Badziewie!<BR>
          <input name="submit" type="submit" value="Wyślij odpowiedź">
</form>
</body>
</html>
```

Zadanie 104.(Zadanie 7)

Temat: Uzupełnij przykład poprzedni o pole PESEL NIP oraz sprawdź jego poprawność.



Rozwiazanie:

1)Zapewnij wpisywanie PESEL w formularzu.

```
<form name="pesel">
        Wpisz numer PESEL:<br>
                <input type="text" name="pesel_kowalski">
        <input type="button" name="button3" value="Test" onClick="test_pesel(this.form)">
</form>
```

2)Zapewnij wpisywanie NIP w formularzu.

```
……..
                <input type="text" name="nip_kowalski">
……..
```

3)wykonaj funkcję sprawdzania (przeczytaj algorytm poniżej)

- sprawdzenie czy Pesel ma 11 znaków
- podział oraz zamiana na wartość Int 11 znaków PESEl i zapisanie do osobnych zmiennych l1..l11

Konwersja → parselInt

Wycinanie znaków z tekstu → substring(?,?)

4) obliczenie lk sumy kontrolnej w zmiennej lk

5)obliczenie reszty z dzielenie przez 10 w zmiennej ck (cyfra kontrolna)

6)odejmowanie od 10 ck i zapis w zmiennej ckk (cyfra kontrolna końcowa)

7)Sprawdzenie warunku czy cyfra kontrolna końcowa jest równa ostatniemu znakowi Pesel lub czy cyfra końcowa jest równa zero i czy ostatni znak jest równy zero.
8)Wykonaj podobnie dla NIP

```
function test_pesel(form)
 {
   if (form.txt3.value.length == 11)
    {
        var l1= parseInt (form. pesel_kowalski.value.substring(0,1));
        var l2=parseInt(form. pesel_kowalski.value.substring(1,2));
        ……..

        var lk=(1*l1+3*l2+7*l3+……);
        var ck=……;
        var ckk=……..;
                if (??????????????????????????)
                        {
                                alert("PESEL prawidłowy!");
                        }
                                else
                        {
                                alert("PESEL nieprawidłowy!");
                        }
    }
                        else
    {
                alert("PESEL nieprawidłowy!");
    }
 }
```

Numer PESEL→algorytm sprawdzania poprawności.
Numer PESEL jest to 11-cyfrowy, stały symbol numeryczny, jednoznacznie identyfikujący określoną osobę fizyczną.

Zbudowany jest z następujących elementów:

- zakodowanej daty urodzenia
- liczby porządkowej
- zakodowanej płci
- cyfry kontrolnej

Przykładowa postać:

4	4	0	5	1	4	0	1	4	5	8

cyfry [1-6] – data urodzenia z określeniem stulecia urodzenia
cyfry [7-10] – numer serii z oznaczeniem płci
cyfra [10] – płeć
cyfra [11] – cyfra kontrolna

Data urodzenia w formacie
RRMMDD
RR -rok

MM -miesiąc
DD -dzień

dla osób urodzonych w latach 1900 do 1999 – miesiąc zapisywany jest w sposób naturalny
dla osób urodzonych w innych latach niż 1900 – 1999 dodawane są do numeru miesiąca następujące wielkości:

dla lat 1800-1899 - 80
dla lat 2000-2099 - 20
dla lat 2100-2199 - 40
dla lat 2200-2299 - 60

Przykładowo osoba urodzona 14 lipca 2002 roku ma następujący zapis w numerze ewidencyjnym:

0	2	2	7	1	4

Płeć
Informacja o płci osoby, zawarta jest na 10. (przedostatniej) pozycji numeru PESEL.

cyfry 0, 2, 4, 6, 8 – oznaczają płeć żeńską
cyfry 1, 3, 5, 7, 9 – oznaczają płeć męską

Cyfra kontrolna i sprawdzanie poprawności numeru
Jedenasta cyfra jest cyfrą kontrolną, służącą do wychwytywania przekłamań numeru. Jest ona generowana na podstawie pierwszych dziesięciu cyfr. Aby sprawdzić czy dany numer PESEL jest prawidłowy należy, zakładając, że litery **a-j** to kolejne cyfry numeru od lewej, obliczyć wyrażenie

1*a + 3*b + 7*c + 9*d + 1*e + 3*f + 7*g + 9*h + 1*i + 3*j

Następnie należy odjąć ostatnią cyfrę otrzymanego wyniku od 10. Jeśli otrzymany wynik nie jest równy cyfrze kontrolnej, to znaczy, że numer zawiera błąd.

Uwaga implementacyjna - jeśli ostatnią cyfrą otrzymanego wyniku jest 0, w wyniku odejmowania otrzymamy liczbę 10, podczas gdy suma kontrolna jest cyfrą. Oznacza to tyle, że cyfra kontrolna winna być równa 0 (stąd dobrze jest wykonać na wyniku odejmowania operację modulo 10). W wyniku niezbyt precyzyjnego opisu na stronie MSW ten aspekt jest często pomijany i prowadzi do błędów w implementacji sprawdzania poprawności numeru PESEL.

Przykład dla numeru PESEL 44051401358:

1*4 + 3*4 + 7*0 + 9*5 + 1*1 + 3*4 + 7*0 + 9*1 + 1*3 + 3*5 = 101

Wyznaczamy resztę z dzielenia sumy przez 10:

101:10 = 10 reszta = 1

Jeżeli reszta = 0, to cyfra kontrolna wynosi 0. Jeżeli reszta ≠ 0, to cyfra kontrolna będzie uzupełnieniem reszty do 10, czyli w podanym przykładzie jest to cyfra 9.

10 - 1 = 9

Wynik 9 nie jest równy ostatniej cyfrze numeru PESEL, czyli 8, więc numer jest błędny.

Metoda równoważna→algorytm drugi poprawności zapisu.

Powyższa metoda sprowadza się do obliczenia wyrażenia:

a+3b+7c+9d+e+3f+7g+9h+i+3j+k

gdzie litery oznaczają kolejne cyfry numeru, i sprawdzeniu czy reszta z dzielenia przez 10 jest zerem (ostatnia cyfra powstałej liczby jest zerem) i jeśli nie jest, to numer jest błędny.
Cechy specyficzne algorytmu sprawdzania

Wada algorytmu
Algorytm ma pewną wadę w przydziale wag do poszczególnych elementów, która powoduje, że gdy zamienimy rok z dniem (zamieniając zapis z rr-mm-dd na dd-mm-rr) otrzymamy identyczną sumę kontrolną jak w numerze z poprawnym zapisem.

W praktyce zdarzają się (a przynajmniej zdarzały i wciąż istnieją) numery PESEL z błędami. Błędy w dacie zwykle były zauważane i poprawiane od razu, lecz zdarzały się też powtórzenia numeru porządkowego, błędy w określeniu płci i błędne cyfry kontrolne, które zostały wychwycone po latach przy okazji wprowadzania numeru PESEL do komputerowych baz danych. W związku z tym nie można zakładać, że wynik sprawdzania jednoznacznie określa istnienie bądź nieistnienie podanego numeru PESEL.

Sprawdzanie poprawności numeru NIP

NIP czyli Numer Identyfikacyjny Płatnika to numer przyznawany każdemu podatnikowi przez właściwy Urząd Skarbowy, służący do identyfikowania osoby lub podmiotu gospodarczego przy prowadzeniu rozliczeń podatkowych. NIP składa się z dziesięciu cyfr. Pierwsze dziewięć cyfr to część identyfikacyjna. Dziesiąta cyfra jest cyfrą kontrolną. Sprawdzanie poprawności NIP-u można przeprowadzić za pomocą następujących czynności:

A. Wymnożyć kolejne cyfry (od pierwszej do dziewiątej) przez odpowiednie wagi. Jako wagi stosujemy kolejno liczby
6 5 7 2 3 4 5 6 7

B. Zsumować wyniki uzyskane w punkcie A

C. Wynik otrzymany w punkcie B należy podzielić modulo przez 11

Jeśli NIP jest poprawny to liczba uzyskana w punkcie C będzie równa dziesiątej cyfrze.

Zadanie 105. (Przykład 6)

Temat: Demonstracja obliczeń z użyciem formularza.

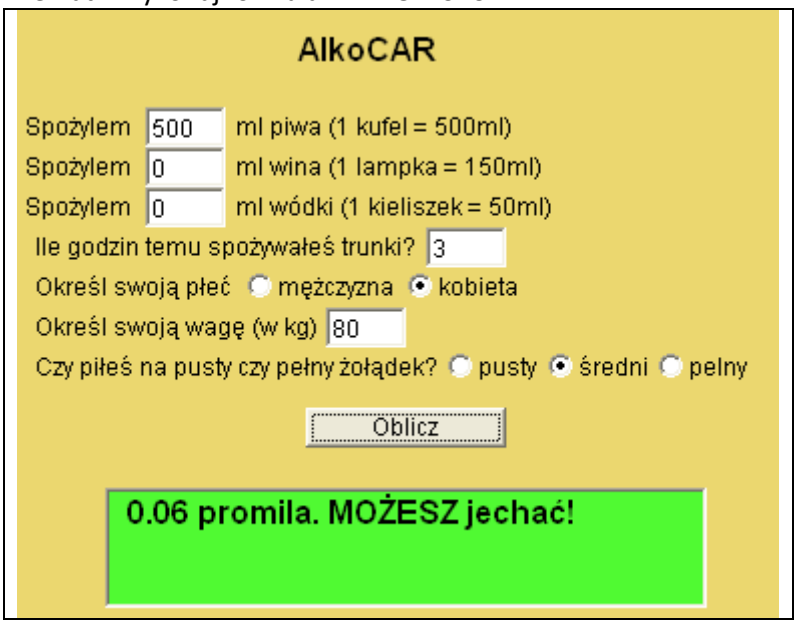
```
<html>
<head>
<script type="text/javascript">
function obliczanie(dane)
{
    wynik.innerHTML="                                                    ";
    cena_koncowa=0;
 if ( dane.wybor[0].checked)
    {
          cena_koncowa=parseFloat(dane.cena.value);
```

```
    }
 if ( dane.wybor[1].checked)
   {
      cena_koncowa=0.9*parseFloat(dane.cena.value);
    }
 if ( dane.wybor[2].checked)
   {
        cena_koncowa=0.8*parseFloat(dane.cena.value);
    }
if (dane.vat.selectedIndex==0)
   {
       cena_koncowa=cena_koncowa;
    }
  if (dane.vat.selectedIndex==1)
   {
       cena_koncowa=1.07*cena_koncowa;
    }
  if (dane.vat.selectedIndex==2)
   {
       cena_koncowa=1.23*cena_koncowa;
    }
  if(dane.ubezpieczenie.checked == true)
 {
    cena_koncowa=1.15*cena_koncowa;
 }
if(dane.gwarancja.checked == true)
 {
 cena_koncowa=1.28*cena_koncowa;
 }
      wynik.innerHTML="do zapłaty    "+cena_koncowa.toFixed(2)+"   zł";
}
</script>
</head>
<body>
<form name="formularz">
<b>Podaj cenę:</b><br>
cena  netto<input type="text" name="cena"/><br><br>
<b>Podaj rabat:</b><br>
rabat 0% <input type="radio" name="wybor" value=0 /><br><br>
rabat 10% <input type="radio" name="wybor" value=10 /><br><br>
rabat 20%  <input type="radio" name="wybor" value=20 /><br><br>
<b>Podaj VAT:</b><br>
wybierz VAT<select  name="vat"/>
                    <option value="0"> 0%
                    <option value="7"> 7%
                    <option value="23"> 23%
          </select><br><br>
<b>Podaj opcje dodatkowe:</b><br>
ubezpieczenie zakup   &nbsp15%<input type="checkbox" name="ubezpieczenie" value=0
/><br>
przedłużona gwarancja 28%<input type="checkbox" name="gwarancja" value=0 /><br><br>
<!-- <input type="button" value="Oblicz....." onClick="obliczanie(this.form)"/><br><br> -->
```

```
<input type="button" value="Oblicz....." onClick="obliczanie(this.form)"/><br><br>
</form>
<table>
        <tr id="wynik"></tr>
</table>
</body>
</html>
```

Zadanie 106. (Zadanie 8a)

Temat: Wykonaj formularz ALKOTESTU



Zadanie 107. (pobrany z Internetu)

Sprawdzanie czy pola pozostały puste – sposób pierwszy.

```
<script type="text/javascript">
   function f(w,i){
   var pola=['imie','nazwisko','adres','kod_pocztowy'];
   dane=w.elements;
   for(i=0;i<pola.length;i++){
   if(dane[pola[i]].value==""){
   dane[pola[i]].focus(); alert("Musisz wypelnic wymagane pola"); return false;
   }
   }
   return true;
   }
   </script>
<form onsubmit="[b]return [/b]f(this)">
<input type="text" name="imie">
<input type="text" name="nazwisko">
<input type="text" name="kod_pocztowy">
</form>
```

Zadanie 108. (pobrany z Internetu)

Temat: Sprawdzanie czy wszystkie pola formularza są wypełnione-przykład2.

```
<script>
function sprawdz(formularz)
{
	for (i = 0; i < formularz.length; i++)
	{
		var pole = formularz.elements[i];
if (!pole.disabled && !pole.readonly && (pole.type == "text" || pole.type == "password" ||
pole.type == "textarea") && pole.value == "")
		{
			alert("Proszę wypełnić wszystkie pola!");
			return false;
		}
	}
	return true;
}
</script>

<form action="mailto:adres e-mail?subject=temat" method="post" enctype="text/plain"
onsubmit="if (sprawdz(this)) return true; return false">
<div>
	<input type="text" name="a"><br>
	<input type="password" name="b"><br>
	<textarea name="c" cols="20" rows="10"></textarea><br>
	<input type="submit" value="OK">
</div>
</form>
```

Metaznak	Znaczenie	Przykład wyrażenia	Zgodne ciągi z wyrażeniem	Niezgodne ciągi z wyrażeniem
^	początek wzorca	^za	zapałka zadra zapłon zarazek	kazanie poza bazar
$	koniec wzorca	az$	uraz pokaz	azymut pokazy
		^.arka$	barka warka	parkan
.	dowolny pojedynczy znak	.an.a	panda Wanda panna kania	rana konia
[...]	dowolny z wymienionych znaków; możemy podawać kolejne znaki lub wpisywać zakres - na przykład [a-z] oznacza wszystkie małe litery. Wymieniając escapowane znaki (następna table) nie musimy ich poprzedzać ukośnikiem	[a-z]an[nd]a	pana panda wanna	Wanda kania
		[a-z][a-zA-Z0-9.-]	pas mAs p2p	Bas bal balu mp3
[^...]	dowolny z niewymienionych znaków	kre[^st]	krew krem	kres kret
|	dowolny z rozdzielonych znakiem ciągów	[nz]a|pod|przed	na za pod przed	
		trzynasty|13-ty|13	trzynasty 13-ty 13	
(...)	zawężenie zasięgu	g(ż|rz)eg(ż|rz)(u|ó)łka	gżegżółka gżegrzółka gżegrzułka grzegrzułka	
		(ósmy|8-my|8)(maj|maja)	ósmy maja 8-my maj 8 maja	
?	zero lub jeden poprzedzający znak lub element; elementem może być na przykład wyrażenie umieszczone wewnątrz nawiasów (...)	ro?uter	router ruter	
		(ósmy|8(-my)?)maja?	ósmy maja ósmy maj 8-mymaja	

			8-my maj 8 maja 8 maj	
+	jeden lub więcej poprzedzających znaków lub elementów; elementem może być na przykład wyrażenie umieszczone wewnątrz nawiasów (...)	[0-9]+[abc]	10a 1b 003c 42334b	a b c z 14 03 12d 1231z
		pan+a	pana panna pannnna	
		(tam)+	tam tamtam tamtamtam	paa panda ta tamta mat
*	zero lub więcej poprzedzających znaków lub elementów; elementem może być na przykład wyrażenie umieszczone wewnątrz nawiasów (...)	[0-9]*[abc]	10a 1b 003c 42334b a b c	k 2335
		pora*n*a*	por poa poranna poraannnaa pornnna	porada panna
{4}	dokładnie 4 poprzedzające znaki lub elementy	[0-9]{4}	8765 8273 2635	12345 234 2123456
{4,}	4 lub więcej poprzedzających znaków lub elementów	[ah]{4,}	haha haaaaahaha ahaaa	haa ha hehe aha
{2,4}	od 2 do 4 poprzedzających znaków lub elementów	p.{2,4}a	piana pola polana	psa poranna

Jeżeli chcemy w wyrażeniu zawrzeć znaki, które normalnie mają specjalne znaczenie, musimy poprzecić taki znak ukośnikiem.

\.	znak kropki	[0-9]{,3}\.[0-9]{,3}\.[0-9]{,3}	128.0.0.2	128-0-0-2
*	znak *	*.+	*nic	nic* nic

\/	znak /	^\/\/$	//	
\?	znak ?	^.+\?$	Czy to jest kot?	Czy to jest kot
\:	znak :	^.+\:$	Oto one:	:nic
\.	znak .	\.+	.....	
\^	znak ^	.*\^	To jest ^	To jest &
\+	znak +	[0-9]+\+[0-9]+	928384+29832	23873-32787 238738278
\\	znak \	c\:\\	c:\	
\=	znak =	[0-9]+\+[0-9]+\=[0-9]+	11+12=23	11-12=23
|	znak |	x || y	x || y	x -- y

Flagi

Poza wymienionymi meta znakami istnieją specjalne parametry (flagi), które oddziałują na wyszukiwanie wzorców.:

```
const reg = /[a-z]*/mg
const reg = new RegExp("[a-z]*","g")
```

znak Flagi	znaczenie
i	powoduje niebranie pod uwagę wielkości liter
g	powoduje zwracanie wszystkich psujących fragmentów, a nie tylko pierwszego
m	powoduje wyszukiwanie w tekście kilku liniowym. W trybie tym znak początku i końca wzorca (^$) jest wstawiany przed i po znaku nowej linii (\n).

Klasy znaków

Dodatkowo Javascript udostępnia specjalne klasy znaków. Zamiast wyszukiwać wszystkie litery za pomocą [a-zA-Z_] możemy skorzystać z klasy znaków \w.

Klasa znaków	znaczenie
\s	znak spacji, tabulacji lub nowego wiersza
\S	znak nie będący spacją, tabulacją lub znakiem nowego wiersza
\w	każdy znak będący literą, cyfrą i znakiem _
\W	każdy znak nie będący literą, cyfrą i znakiem _
\d	każdy znak będący cyfrą
\D	każdy znak nie będący cyfrą

Interfejs programistyczny aplikacji (ang. Application Programming Interface, **API**) – sposób, rozumiany jako ściśle określony zestaw reguł i ich opisów, w jaki programy komputerowe komunikują się między sobą. API definiuje się na poziomie kodu źródłowego dla takich składników oprogramowania jak np. aplikacje, biblioteki czy system operacyjny. Zadaniem API jest dostarczenie odpowiednich specyfikacji podprogramów, struktur danych, klas obiektów i wymaganych protokołów komunikacyjnych. Przykładem API jest POSIX, czy też Windows API.
Windows API, lub krócej: WinAPI – interfejs programistyczny systemu Microsoft Windows – jest to zbiór niezbędnych funkcji, stałych i zmiennych umożliwiających działanie programu w systemie operacyjnym Microsoft Windows.

Dział 5 Przyciski

Zadanie 8

Temat: Różne funkcje przycisków (button).

Wykonaj:
stronę o następującej budowie(możesz wykorzystać stronę, którą już robiłeś)

<table>
<tr><td>MENU

Opcja1
Opcja2
Opcja3</td><td>Otwierane strony zaproponowanych przez ucznia</td></tr>
<tr><td colspan="2">Przyciski
[strona do przodu] [drukuj stronę] [wstecz strona] [odśwież stronę]</td></tr>
</table>

Działanie MENU pionowego, (patrz schemat strony powyżej);
Opcje menu wybierają (otwierają) strony zaproponowane przez ucznia . Wybrana opcja wgra stronę w dużym oknie.
Działanie MENU poziomego, (zbudowane z przycisków, patrz schemat strony powyżej);
Wybór jednego z Przycisków powoduje akcja na stronie w dużym okna (strona do przodu, wstecz, odśwież).

Pomocna teoria:

a) do przodu strona

```
function przod()
{
  history.forward()
}
```

b) wstecz strona
history.back()

c)odświeżanie strony
location.refresh()

d)drukowanie strony
window.print()

Dział 7 Galerie

Zadanie 12

Temat: Klasyczna galeria zdjęć

Wykonanie miniatur.

Wykonaj klasyczna galerię zdjęć:

sześć zdjęć i miniatur związanych z:

Numer z dziennika	Temat
1,17	Parki narodowe
2,18	Gdańsk
3,19	Warszawa
4,20	Samochody wyścigowe
5,21	Tatry
6,22	Kulturyści
7,23	Morze
8,24	Wyspy
9,25	Piłka nożna
10,26	Antyczna Grecja
11,27	Słynne obrazy
12,28	Słynne mosty
13,29	Zamki polskie
14,28	Arbuzy
15,29	Sady
16,30	Motocykle

Dla numerów nieparzystych ułożenie miniatur

Dla numerów parzystych ułożenie miniatur

Po uzyskaniu całego zdjęcia powinien być przycisk wróć do miniatur.

Dział 8 Skrypty z internetu

Zadanie 12a

Temat: Dołączenie 5 grafik uzyskanych z Internetu.

Wykonaj:

1)Do strony na, której będzie Twoje nazwisko (zielone, 3cm→ formatowanie napisu zewnętrznym arkuszem styli CSS o nazwie twoje_nazwisko.css) dokomponuj pięć skryptów JS z Internetu.

2)Wykonaj przyciski (buttony) po, których naciśnięciu wyświetli się kod skryptu JS. Przycisk ma napis z nazwą skryptu.



Po naciśnieciu przycisku Data uzyskujemy kod w okienku np.

```
<SCRIPT LANGUAGE="JavaScript">
<!-- // skrypt pobrano z http://szablony.freeware.info.pl
miesiac = new Array(12)
miesiac[0] = "stycznia "
miesiac[1] = "lutego "
miesiac[2] = "marca "
miesiac[3] = "kwietnia "
miesiac[4] = "maja "
miesiac[5] = "czerwca "
miesiac[6] = "lipca "
miesiac[7] = "sierpnia "
miesiac[8] = "wrzenia "
miesiac[9] = "padziernika "
miesiac[10] = "listopada "
miesiac[11] = "grudnia "
dzien = new Array(7)
dzien[0] = "niedziela "
dzien[1] = "poniedziałek "
dzien[2] = "wtorek "
dzien[3] = "sroda "
dzien[4] = "czwartek "
dzien[5] = "pištek "
dzien[6] = "sobota "
function podaj_date(){
var Dzisiaj = new Date()
var Tygodnia = Dzisiaj.getDay()
var Miesiac = Dzisiaj.getMonth()
var Dnia = Dzisiaj.getDate()
var Rok = Dzisiaj.getFullYear()
if(Rok <= 99) Rok += 1900
return dzien[Tygodnia] + "," + " " + Dnia + " " +
miesiac[Miesiac] + ", " + Rok +"r." }
//-->
</SCRIPT>
<SCRIPT>document.write("Dzis jest " + podaj_date())</SCRIPT>
```

Dział 9 HTML5 canvans

Element **canvas** umożliwia rysowanie **grafiki rastrowej** na stronach internetowych.
Canvas jest prostokątnym obszarem, który po umieszczeniu na stronie **HTML5** umożliwia kontrolowanie każdego punktu za pomocą **JavaScript.**
Technologia ta pozwala on na:

- rysowanie grafiki, (proste figury geometryczne)
- różnego rodzaju krzywe,
- gradienty,
- wykresy,
- tworzenia galerii zdjęć,
- tworzenia animacji 2D (poprzez cykliczne przerysowywanie całego obszaru),
- gry.

http://msdn.microsoft.com/pl-pl/library/kurs-html5--canvas--grafika.aspx

Zadanie 14
Temat: Narysuj żółty kwadrat z mniejszym czerwonym kwadratem pośrodku.

Etap1
Szkielet strony:

```
<!DOCTYPE html>
<html lang="pl">
	<head>
		<meta charset="utf-8">
		<title>Canvas</title>
	</head>
	<body>
	</body>
</html>
```

Etap2
Utworzenie elementu <canvas>→nowy znacznik HTML5

W celu określenia obszaru rysowania, pomiędzy znacznikami <body></body>, dodaj kod wg schematu:

```
<canvas id="nazwa" width="" height="">
</canvas>
```

gdzie:

id – unikalny identyfikator,
width – szerokość elementu canvas (domyślnie 300px),
height – wysokość elementu canvas(domyślnie 150px).

```
<!DOCTYPE html>
<html lang="pl">
	<head>
		<meta charset="utf-8">
		<title>Canvas</title>
	</head>
	<body>
		<canvas id="obrazek" width="300" height="300">
		</canvas>
	</body>
</html>
```

Definiowanie koloru tła obszaru do rysowania definiowanego w canvans.

```
<canvas
	id="plaszczyzna" width="800" height="600" style='background-color: lightgreen;'>
</canvas>
```

Etap3

Po utworzeniu nowego elementu, nie jest on jeszcze widoczny w oknie przeglądarki. Należy skorzystać z metody getContext, umożliwiającej wykorzystanie funkcji rysujących. Metodę getContext umieść w funkcji, którą wykorzystasz do narysowania prostokąta.

Wykonaj:

Do kodu z etapu 2 dodaj:

Pod </title> wpisz skrypt Javascript:

```
<script type="text/javascript">
function rysuj()
{
	var rysowanie = document.getElementById('obrazek');
	var rodzaj = rysowanie.getContext('2d');
}
</script>
```

Dodaj atrybut w znaczniku <body>, który uruchomi funkcję rysuj() po załadowaniu strony:

```
<body onload="rysuj();">
```

Metoda getContext obiektu canvas powoduje pobranie kontekstu płaszczyzny, który jest nam potrzebny, abyśmy mogli wywoływać funkcje rysujące.

Pierwszy parametr metody getContext to identyfikator kontekstu. Jeśli podamy **„2d",** pobierzemy kontekst implementujący interfejs CanvasRenderingContext2D.

Możliwe są też inne wartości. Na przykład, jeśli chcemy rysować grafikę 3D, odpowiednio użylibyśmy wartości **"3d"**. Z kolei, aby zastosować **webgl**, używać będziemy identyfikatora "webgl".

Powstanie kod strony

```
<!DOCTYPE html>
<html lang="pl">
	<head>
		<meta charset="utf-8">
		<title>Canvas</title>
<script type="text/javascript">
function rysuj()
{
	var rysowanie = document.getElementById('obrazek');
	var rodzaj = canvas.getContext('2d');
}
</script>

	</head>
	<body>
		<canvas id="obrazek" width="300" height="300">
		</canvas>
	</body>
</html>
```

Etap 4

Rysowanie w zdefiniowanym obiekcie do rysowania.

Uwaga1:
Punkt (0,0) nie znajduje się jak na układzie kartezjańskim pośrodku obrazka, ale w jego **lewym górnym rogu**.

Metody do rysowania:

fillRect(x,y,s,w) – wypełnia obszar z lewym górnym rogiem, znajdujący się w punkcie (x,y) o szerokości (s) i wysokości (w), podawanych w pikselach,

fillStyle ='kolor' – zmienia kolor wypełnienia.

Możliwe formaty koloru:

- 'red',
- postać hexadecymalna, np. '#aabbcc'
- format rgb, np. rgb(123,123,123) – podobnie jak w CSS,

strokeRect(x,y,s,w) – rysuje kontur prostokąta z lewym górnym rogiem, znajdującym się w punkcie (x,y), o szerokości (s) i wysokości (w), podawanych w pikselach,

strokeStyle = 'kolor' – zmienia kolor konturu prostokąta; używa się go tak samo jak przy fillStyle(),

clearRect(x,y,s,w) – czyści obszar (rysuje przeźroczysty kwadrat) na obszarze z lewym górnym rogiem, znajdującym się w punkcie (x,y), o szerokości (s) i wysokości (w), podawanych w pikselach.

Utwórz nowe linie w funkcji rysuj().
po linii var rodzaj = canvas.getContext('2d'); wpisz:

```
rodzaj.fillStyle = 'yellow';
rodzaj.fillRect(0,0,100,100);
rodzaj.strokeRect(0,0,100,100);
rodzaj.fillStyle = 'red';
rodzaj.fillRect(30,30,30,30);
```

Obsługa przeglądarek, które nie wspierają elementu canvas:
Wykonać to można poprzez umieszczenie tekstu komunikatu pomiędzy znacznikami <canvas></canvas>. Ukaże się on w przypadku, gdy przeglądarka nie będzie wspierała elementu canvas, np. :

```
<canvas id="image" width="300" height="300">
        Twoja przeglądarka nie wspiera Canvas! Zainstaluj Internet Explorer 9.
</canvas>
```

Strona końcowa

```
<!DOCTYPE html>
<html lang="pl">
<head>
   <meta charset="utf-8">
   <title>Canvas</title>
   <script type="text/javascript">
      function rysuj() {
         var rysowanie = document.getElementById('obrazek');
         var rodzaj = rysowanie.getContext('2d');
         rodzaj.fillStyle = 'yellow';
         rodzaj.fillRect(0, 0, 100, 100);
         rodzaj.strokeRect(0, 0, 100, 100);
         rodzaj.fillStyle = 'red';
         rodzaj.fillRect(30, 30, 30, 30);
      }
   </script>
</head>
<body onload="rysuj();">
   <canvas id="obrazek" width="300" height="300">
                  Twoja przeglądarka nie wspiera Canvas!
                  Zainstaluj Internet Explorer 9.x
   </canvas>
</body>
</html>
```

Zadanie 15
Temat: Narysuj zielony kwadrat rogami do góry.

Wykonaj:
1)Dodaj kod do poprzedniego programu:

```
rodzaj.beginPath();
rodzaj.moveTo(50,50);
rodzaj.lineTo(100,0);
rodzaj.lineTo(150,50);
rodzaj.lineTo(100,100);
rodzaj.lineTo(50,50);
rodzaj.fillStyle = 'green';
rodzaj.fill();
```

2)zmień położenie zielonego kwadratu tak aby nie nakładał się elementami z zadania poprzedniego. Współrzędne górnego rogu bedą następujące:

x=200+(numer_miesiaca_rodzenia)*10
y=300+(numer_miesiaca_rodzenia)*11

Podczas tworzenia ścieżek i linii w Canvas, wykorzystaj następujące metody:
beginPath() – rozpoczyna tworzenie ścieżki,
closePath() – kończy tworzenie ścieżki,
moveTo(x,y) – ustawia punkt, od którego rozpoczniesz rysować (x,y) (domyślnie jest on ustawiony w miejscu zakończenia poprzedniego rysunku),
lineTo(x,y) – rysuje linie do punktu (x,y) z miejsca oznaczonego metodą moveTo() lub punktu, w którym zakończone zostało poprzednie rysowanie,
fill() – wypełnia ścieżkę kolorem podanym w parametrze fillStyle(). Warto zwrócić uwagę, że metoda ta sama domyka ścieżki, dlatego nie wymaga użycia metody closePath(),
fillStyle='kolor' – zmienia kolor wypełnienia metody, gdzie kolor może być nazwą angielską, formatem szesnastkowym lub RGB,
stroke() – rysuje kontur ścieżki,
strokeStyle='kolor' – zmienia kolor konturu ścieżki, gdzie kolor może być nazwą angielską, formatem szesnastkowym lub rgb.
lineWidth = size – grubość linii w pixelach.

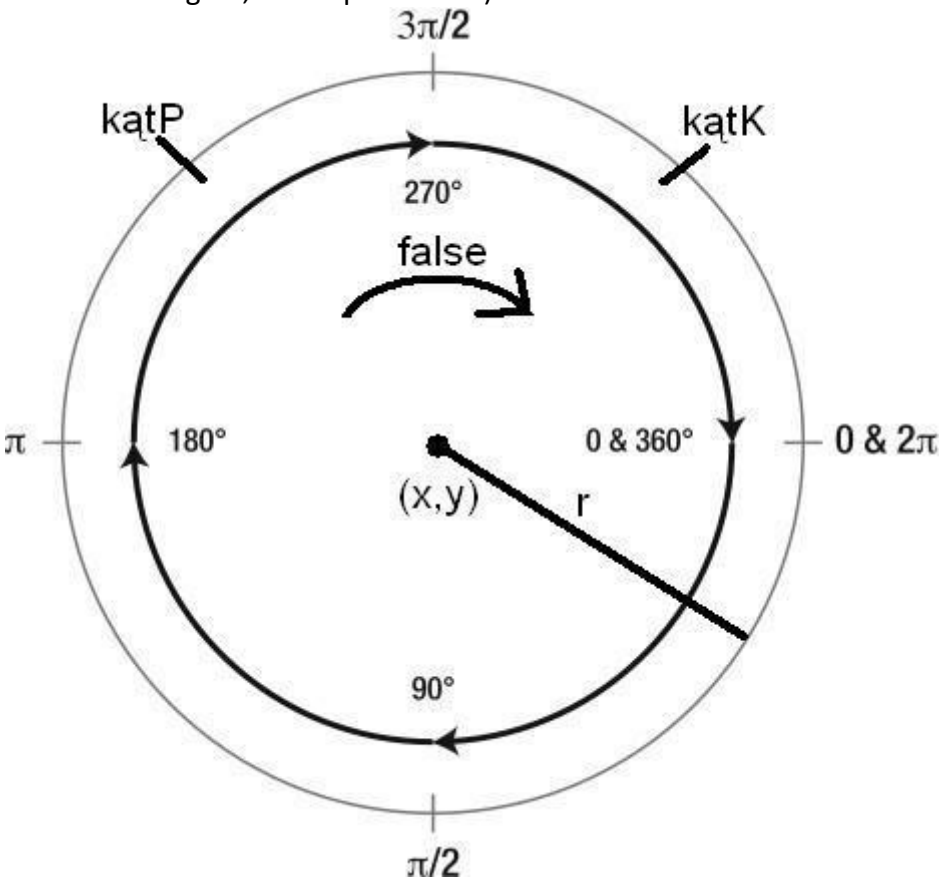
Zadanie 16
Temat: Narysuj kółka olimpijskie.

Wykonaj:
1. Skasuj rysowanie z poprzedniego zadania.
2. Dodaj kod do poprzedniego programu.
3. Wykonaj rysowanie kółek olimpijskich (kolory mają być identyczne i wielkości kółek odpowiedniej wielkości).

```
rodzaj.beginPath();
rodzaj.arc(200,50,30,Math.PI,0,false);
rodzaj.closePath();
rodzaj.lineWidth = 10;
rodzaj.strokeStyle = "black";
rodzaj.stroke();
```

Rysowanie łuku→przy rysowaniu łuków wykorzystujesz metody:

arc(x, y, r, kątP, kątK, kierunek)
x, y to współrzędne środka okręgu, na którym rysowany jest łuk;
r to promień okręgu,
kątP, kątK określają odpowiednio kąt początkowy i końcowy łuku (wartość w radianach);
kierunek to wartość boolowska, określająca kierunek rysowania (false – zgodny z ruchem wskazówek zegara, true – przeciwnie).



Zamiana stopni na radiany jest możliwa na pomocą następującego równania:

miara_w_radianach = miara_w_stopniach * (Math.PI / 180)

Math.PI to wartość liczby PI, która określona jest w bibliotece Math języka JavaScript:

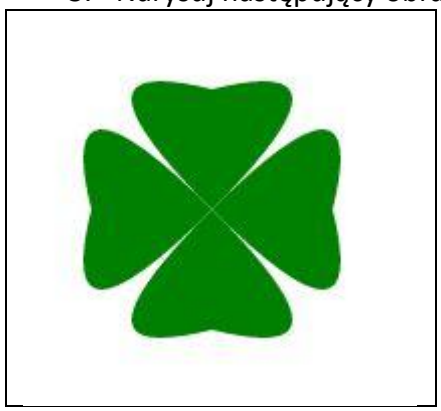
Zadanie 17
Temat: Rysowanie koniczynki z użyciem krzywych kwadratowej.

Wykonaj:

1. Skasuj rysowanie z poprzedniego zadania.
2. Dodaj kod do poprzedniego programu.

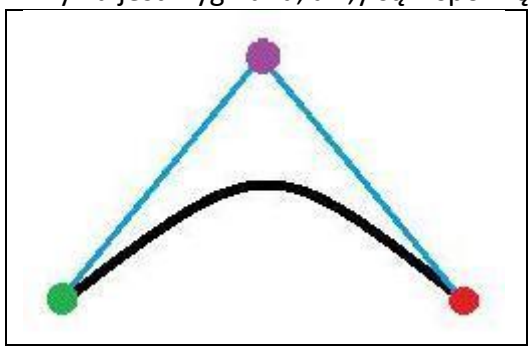
```
rodzaj.beginPath();
rodzaj.moveTo(50,250);
rodzaj.quadraticCurveTo(100,200,180,280);
rodzaj.closePath();
rodzaj.strokeStyle = "red";
rodzaj.stroke();
```

3. Narysuj następujący obrazek:



Rysowanie krzywej kwadratowej

quadraticCurveTo(PktX, PktY, x, y) – PktX, PktY są współrzędnymi punktu, wg którego krzywa jest wyginana, a x,y są współrzędnymi punktu końcowego.



fioletowe kółko to punkt o współrzędnych (PktX, PktY),
czerwone kółko to punkt o współrzędnych (x, y),
zielone kółko to punkt o współrzędnych ustawianych w metodzie moveTo().

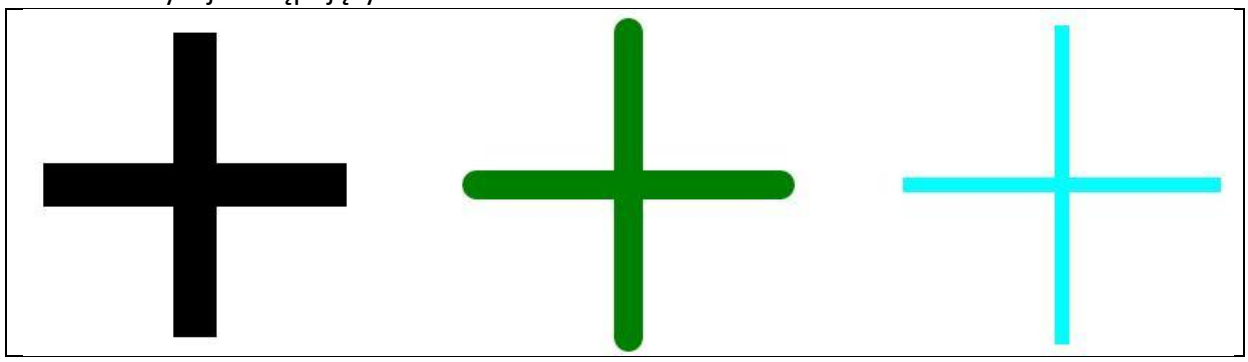
Stylowanie linii
Zadanie 18
Temat: Rysowanie stylizowanych linii

Wykonaj:

1. Skasuj rysowanie z poprzedniego zadania.
2. Dodaj kod do poprzedniego programu.

```
rodzaj.strokeStyle = 'black';
rodzaj.lineWidth = 20;
rodzaj.lineCap = 'butt';
rodzaj.beginPath();
rodzaj.moveTo(30, 250);
rodzaj.lineTo(30, 400);
rodzaj.stroke();
rodzaj.lineCap = 'round';
rodzaj.beginPath();
rodzaj.moveTo(70, 250);
rodzaj.lineTo(70, 400);
rodzaj.stroke();
rodzaj.lineCap = 'square';
rodzaj.beginPath();
rodzaj.moveTo(110, 250);
rodzaj.lineTo(110, 400);
rodzaj.stroke();
```

3. Narysuj następujący obrazek:



Zadanie 19

Temat: Rysowanie stylizowanych linii

Wykonaj:

1. Skasuj rysowanie z poprzedniego zadania.
2. Dodaj kod do poprzedniego programu.

```
rodzaj.beginPath();
rodzaj.moveTo(200, 400);
rodzaj.lineTo(250, 300);
rodzaj.lineTo(300, 400);
rodzaj.lineWidth = 20;
rodzaj.lineJoin = "miter";
rodzaj.stroke();

rodzaj.beginPath();
rodzaj.moveTo(350, 400);
rodzaj.lineTo(400, 300);
rodzaj.lineTo(450, 400);
rodzaj.lineWidth = 20;
rodzaj.lineJoin = "round";
rodzaj.stroke();

rodzaj.beginPath();
rodzaj.moveTo(500, 400);
rodzaj.lineTo(550, 300);
rodzaj.lineTo(600, 400);
rodzaj.lineWidth = 20;
rodzaj.lineJoin = "bevel";
rodzaj.stroke();
```

Zamień w ostatniej linijce parametr round na bevel, a następnie na miter i sprawdź, jak wyglądają połączenia wielokąta.

3. Narysuj następujący obrazek:
   Pagon(jeden) sierżanta.



Czyli



Zadanie 20

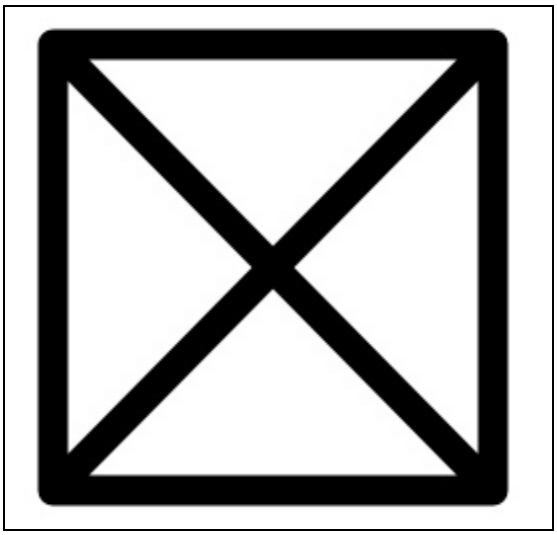
Temat: Rysowanie stylizowanych linii

Wykonaj:

1. Skasuj rysowanie z poprzedniego zadania.
2. Dodaj kod do poprzedniego programu.

```
rodzaj.beginPath();
rodzaj.moveTo(200, 300);
rodzaj.lineTo(300, 350);
rodzaj.lineTo(300, 450);
rodzaj.lineTo(200, 500);
rodzaj.lineTo(100, 450);
rodzaj.lineTo(100, 350);
rodzaj.closePath();
rodzaj.lineWidth = 20;
rodzaj.lineJoin = "round";
rodzaj.stroke();
```

3. Narysuj następujący obrazek:



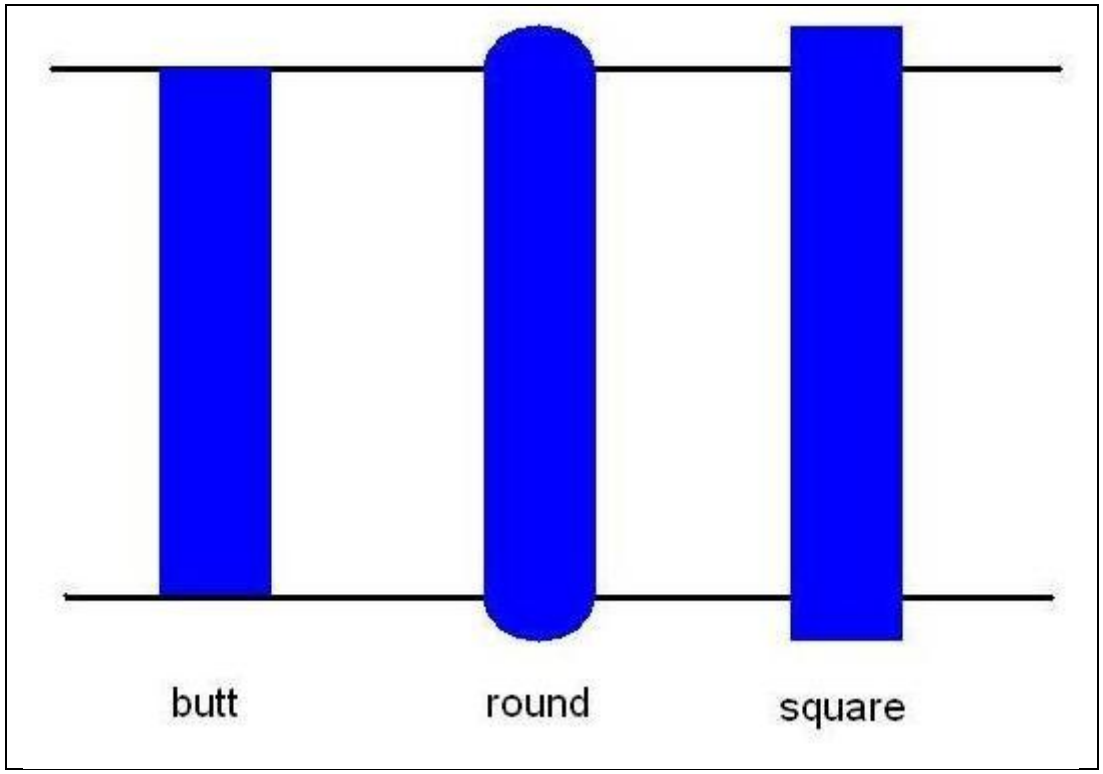
Stylowanie linni opis teoretyczny

Canvas umożliwia określenie właściwości linii za pomocą 4 metod:

lineWidth = x – wartość parametru x podawana w pikselach określa grubość linii,

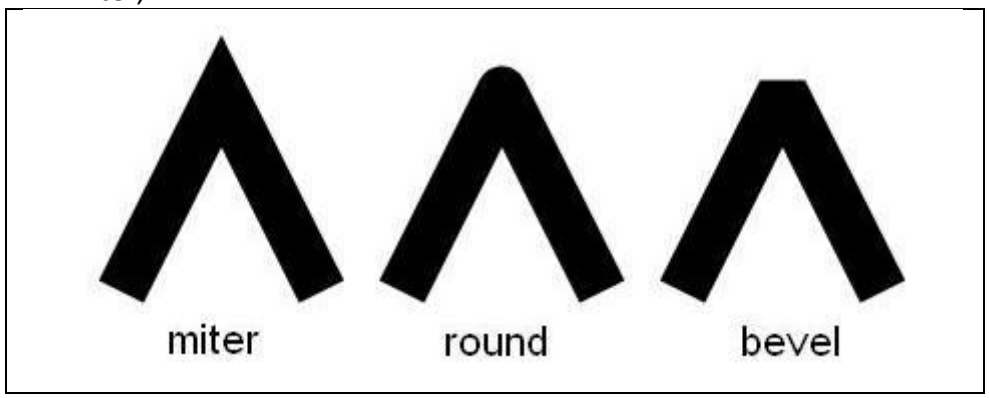
lineCap = typ – określa typ zakończenia linii; parametr typ przyjmuje jedną z trzech wartości:

- butt,
- round,
- square,



lineJoin = typ – określa styl połączenia 2 linii; parametr typ przyjmuje jedną z trzech wartości:

- round,
- bevel,
- miter,



Import i wyrysowanie grafik

Zaczynając importowanie obrazków warto wiedzieć, że źródłem grafiki do importu może być jedynie inny element canvas lub obiekt obrazu. Jeśli masz już odnośnik do obrazka, narysuj go, używając metodydrawImage(), którą możesz wywołać z różnymi parametrami:

drawImage(nazwa_obiektu, x, y),
gdzie:
— nazwa_obiektu – nazwa obiektu przechowującego obrazek,
— x,y – współrzędne lewego, górnego wierzchołka obrazka,

drawImage(nazwa_obiektu, x, y, w, h),
gdzie:
— nazwa_obiektu – nazwa obiektu przechowującego obrazek,
— x,y – współrzędne lewego, górnego wierzchołka obrazka,
— w,h – szerokość i wysokość obrazka.

drawImage(nazwa_obiektu, sx, sy, sw, sh, dx, dy, dw, dh),
gdzie:
— nazwa_obiektu – nazwa obiektu przechowującego obrazek,
— sx, sy – współrzędne lewego, górnego wierzchołka obrazka źródłowego,
— sw, sh – szerokość i wysokość źródłowego obrazka,
— dx, dy - współrzędne lewego, górnego wierzchołka nowego obrazka,
— dw, dh – szerokość i wysokość nowego obrazka.

Obiekt obrazka możesz utworzyć za pomocą następującego kodu:

```
var nazwa_obiektu = new Image();
 nazwa_obiektu.src = "adres_obrazka.jpg";
 image.onload = function () { rodzaj.drawImage(nazwa_obiektu, 0, 0); };
```

gdzie:
pierwsza linia odpowiedzialna jest za utworzenie nowego obiektu (metoda Image()) i przypisaniu mu nazwy "nazwa_obiektu",
druga linia odpowiedzialna jest za ustawienie adresu do obrazka (adres_obrazka.jpg), dzięki wykorzystaniu właściwości src wcześniej utworzonego obiektu,
trzecia linijka umożliwia wyświetlenie obrazka po jego pobraniu, brak image.onload spowoduje, że większość przeglądarek nie wyświetli poprawnie obrazka.

```
<!DOCTYPE html>
<html lang="pl">
<head>
   <meta charset="utf-8">
   <title>Canvas</title>
   <script type="text/javascript">
      function rysuj() {
         var rysowanie = document.getElementById('obrazek');
         var rodzaj = rysowanie.getContext('2d');
```

```
        var obraz_arbuz = new Image();

       obraz_arbuz.src = "arbuz.jpg";
       obraz_arbuz.onload = function () { rodzaj.drawImage(obraz_arbuz, 100,100); }
       rodzaj.font = "30pt Times New Roman";
       rodzaj.fillStyle = "blue";
       rodzaj.fillText("Am-am  Arbuz", 10, 50);
          }
   </script>
</head>
<body onload="rysuj();">
   <canvas id="obrazek" width="600" height="800">
   Twoja przeglądarka nie wspiera Canvas! Zainstaluj Internet Explorer 9.x
</canvas>
</body>
</html>
```

Zadanie 21

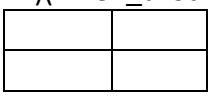
Wykonaj stronę wyświetlająca, co najmniej cztery pliki graficzne z opisami odpowiadającymi grafice. Grafiki możesz ściągnąć o różnych wymiarach (szerokość, wysokość), ale poprzez skalowanie uzyskaj stosunek długości do szerokości 16/9 oraz grafiki powinny być tej samej wielkości po skalowaniu, dostosuj większe grafiki do najmniejszej(wymiary). Opis umieść pod grafiką. Opisy różnymi czcionkami, różną wielkością czcionek. Całość ma tworzyć śmieszną przyzwoita historyjkę. Pliki pobierz z Internetu. Opisy wymyśl sam. Obok grafik wyświetl miniatury grafik wykonane z użyciem instrukcji (drawImage(nazwa_obiektu, x, y, w, h→skalowanie obrazka).

o wymiarze 50+miesiąc_urodzenia*50+miesiąc_urodzenia.

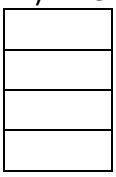
Do miniatur dodaj cienie a do dwóch grafik przeźroczystość.

Ułożenie obrazków i miniatur:

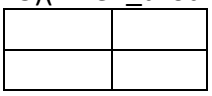
1)(Dzień_urodzenia mod 4=0)→ Mintury po prawej zdjęć nie styka się ze zdjęciem



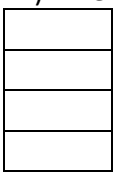
2) Dzień_urodzenia mod 4=1)→ Mintury po prawej zdjęć nie styka się ze zdjęciem



3)(Dzień_urodzenia mod 4=2)→ Mintury po lewej zdjęć nie styka się ze zdjęciem



4) Dzień_urodzenia mod 4=3)→ Mintury po lewej zdjęć nie styka się ze zdjęciem



Skalowanie obrazka

Do zmiany rozmiaru obrazka, wykorzystaj drugą z metod drawImage, podanych powyżej:

drawImage(nazwa_obiektu, x, y, w, h).

Skalowanie obrazka rozpocznij od wstawienia go na stronę za pomocą znacznika <img>. Następnie pobierz jego id i wyświetl go na elemencie canvas, zmniejszonego do rozmiaru 100x100 pikseli.

Ustaw kursor za znacznikiem body i dodaj nową linię.
Wpisz następujący kod:

```
<img id="wielblad" src="wielblad.jpg" alt="Wielbłąd" />
```

Następnie ustaw kursor za znacznikiem z metodą getContext() i dodaj nową linię.
Wpisz następujący kod:

```
var iWielblad = document.getElementById("wielblad");
rodzaj.drawImage(iWielblad, 350, 0, 100, 100);
```

Krojenie grafiki

Do przycięcia grafiki korzystaj z trzeciej z zaprezentowanych metod drawImage():

drawImage(nazwa_obiektu, sx, sy, sw, sh, dx, dy, dw, dh).

Ustaw kursor za znacznikiem z metodą getContext() i dodaj nową linię.
Wpisz następujący kod:

```
rodzaj.drawImage(iWielblad, 215, 10, 50, 45, 500, 0, 250, 225);
```

Przezroczystość
Do określenia przezroczystości możesz wykorzystać następujące metody:

globalAlpha = x – gdzie x to liczba z zakresu <0,1> gdzie 0 – całkowita przezroczystość, 1 – nieprzezroczysty,
rgba(r,g,b,a) – gdzie "r","g","b" to wartości kolorów w formacie rgb, natomiast "a" oznacza przezroczystość i przyjmuje wartości jak w globalAlpha.

Ustaw kursor za znacznikiem z metodą getContext() i dodaj nową linię.
Wpisz następujący kod:

```
var image = new Image();
image.src = "wielblad.jpg";
image.onload = function () {
        rodzaj.drawImage(image, 0, 0);
        rodzaj.fillRect(0,0,100,100);
        rodzaj.globalAlpha = 0.5;
        rodzaj.fillRect(110,0,100,100);
//przezroczystość
};
//linie
```

Teraz, gdy potrafisz już użyć metody globalAlpha(), spróbuj wykonać trudniejszą grafikę, która wykorzystuje przezroczystość.

```
rodzaj.drawImage(image, 350, 0);
rodzaj.fillStyle = "white";
for (var i = 0; i < 1; i += 0.04) {
        rodzaj.globalAlpha = i;
        rodzaj.fillRect(350, i * 225, 350, 10); }
```

Cień w Cavas
Do określenia cienia możesz wykorzystać następujące właściwości:

shadowColor – określa kolor cienia, taką samą wartość jak dla kolorów w CSS.
shadowBlur – określa rozmycie cienia w pikselach, im niższa wartość rozmycia, tym ostrzejsze są cienie. Standardowa wartość wynosi 0.
shadowOffsetX i shadowOffsetY – określa przesunięcie cienia w pikselach, wartości dodatnie przesuwają cień na dół i prawo, ujemne do góry i w lewo. Standardowa wartość wynosi 0.

W funkcji draw zastąp pierwsze wystąpienie polecenia rodzaj.drawImage(image, 0, 0); za pomocą następującego kodu:

```
rodzaj.shadowOffsetX = 10;
rodzaj.shadowOffsetY = 10;
```

```
rodzaj.shadowBlur = 4;
rodzaj.shadowColor = 'rgba(0, 0, 0, 0.5)';
rodzaj.drawImage(image, 0, 0);
rodzaj.shadowOffsetX = 0;
rodzaj.shadowOffsetY = 0;
rodzaj.shadowBlur = 0;
```

Tworząc bardziej skomplikowane rysunki użyteczne są następujące dwie własności metody:
Save() – zapisuje stan canvas,
Restore() – przywraca stan canvas z momentu zapisu za pomocą metody save().
Metody te zostaną użyte w poniższych przykładach w celu ukazania różnic przed i po transformacjach.

Zmiana początku układu współrzędnych

Zmiana początku układu współrzędnych, względem którego odbywa się rysowanie (domyślnie lewy-górny wierzchołek elementu canvas) jest bardzo prosta, wystarczy tylko wykorzystać metodę:
Translate(x,y) – zmienia początek układu współrzędnych na punkt (x,y).

1. Ustaw kursor za znacznikiem z metodą getContext() i dodaj nową linię.
2. Wpisz następujący kod:

```
rodzaj.save();
rodzaj.translate(150,150);
rodzaj.fillRect(0,0,100,100);
rodzaj.restore();
rodzaj.fillStyle = "red";
rodzaj.fillRect(0,0,100,100);
```

Metoda save() zapisuje na stosie stan początkowy canvas, następnie musisz zmienić punkt, względem którego nastąpi rysowanie o wektor [150,150]. Najpierw musisz narysować kwadrat, przywrócić ustawienia domyślne i ponownie narysować kwadrat, który znajduje się już w innym miejscu, gdyż metoda restore() przywraca punkt początku układu współrzędnych z punktu (150,150) na punkt (0,0).

Zmiana jednostki

Canvas umożliwia zmianę wielkości domyślnej jednostki z jednego piksela na większą lub mniejszą. W tym celu użyteczna jest kolejna z metod transformacji:
scale(x,y) – gdzie x i y zwiększają/zmniejszają skalę odpowiednio w poziomie i pionie, np. scale(2,2) zwiększy domyślny rozmiar jednostki w pionie i poziomie dwukrotnie.

1. Ustaw kursor za ostatnio dodanym poleceniem i dodaj nową linię.
2. Wpisz następujący kod:

```
rodzaj.save();
rodzaj.fillStyle = "green";
rodzaj.fillRect(150,0,100,100);
rodzaj.scale(0.5,0.5);
rodzaj.fillStyle = "yellow";
rodzaj.fillRect(300,0,100,100);
rodzaj.restore();
```

3. Zapisz plik index.html. Otwórz go w przeglądarce i sprawdź, czy wygląda jak na Rys. 2.

Zmiana kąta rysowania

W celu narysowania obiektu obróconego o podany kąt, wykorzystaj metodę:
Rotate(kąt) – gdzie argument kąt podawany jest w radianach. W celu zrozumienia, w jaki sposób podawać kąt w radianach lub jak zamienić radiany na stopnie, zajrzyj do samouczka pt. Kształty w Canvas.

1. Ustaw kursor za ostatnio dodanym poleceniem i dodaj nową linię.
2. Wpisz następujący kod:

```
rodzaj.save();
rodzaj.rotate(Math.PI/4);
rodzaj.fillStyle = "green";
rodzaj.fillRect(130, -40, 100, 100);
rodzaj.restore();
rodzaj.fillStyle = "yellow";
rodzaj.fillRect(0,150,100,100);
```

3. Zapisz plik index.html. Otwórz go w przeglądarce i sprawdź, czy wygląda tak, jak na

Podstawowe kompozycje

Tworzenie kompozycji wiąże się z jedną metodą:

globalCompositeOperation=typ – gdzie typ może przybierać jedną z wartości zaprezentowanych na Rys. 2.

Ustaw kursor za znacznikiem z metodą getContext() i dodaj nową linię.
Wpisz:

```
rodzaj.globalCompositeOperation = "source-over";
rodzaj.fillStyle = "blue";
rodzaj.fillRect(0, 0, 100, 100);
rodzaj.fillStyle = "red";
rodzaj.fillRect(50, 50, 100, 100);
rodzaj.fillStyle = "blue";
rodzaj.globalCompositeOperation = "destination-over";
rodzaj.fillRect(200, 0, 100, 100);
rodzaj.fillStyle = "red";
rodzaj.fillRect(250, 50, 100, 100);
```

Łączenie kompozycji

Wiesz już jak stosować kompozycje, teraz spróbuj dodać inne kompozycje.

Ustaw kursor za ostatnio dodanym kodem i dodaj nową linię.
Wpisz:

```
rodzaj.globalCompositeOperation = "source-in";
rodzaj.fillStyle = "blue";
rodzaj.fillRect(0, 200, 100, 100);
```

```
rodzaj.fillStyle = "red";
rodzaj.fillRect(50, 250, 100, 100);
```

Informacja
Kompozycja typu "source-in" pokaże element przykrywający tylko w obszarze elementu przykrywanego. Zmieniając typ kompozycji w canvas, musisz pamiętać, że część z nich usuwa fragmenty obrazu.

Zmień "source-in" na "lighter".

Ustaw kursor za ostatnio dodanym kodem i dodaj nową linię.
Wpisz:

```
rodzaj.globalCompositeOperation = "xor";
rodzaj.fillStyle = "blue";
rodzaj.fillRect(200, 200, 100, 100);
rodzaj.fillStyle = "red";
rodzaj.fillRect(250, 250, 100, 100);
```

Umieszczanie tekstu w Canvas

Zadanie 22
Temat: Teksty.
Wykonaj:

1. Wykonaj następujące teksty.



Do tworzenia podstawowych napisów wykorzystaj następujące metody:

fillText("napis", x, y) – umieszcza napis o treści określonej jako pierwszy argument, rozpoczynając pisanie od punktu (x,y),
fillStyle = "kolor" – określa kolor tekstu,
strokeText("napis", x, y) – umieszcza obramowanie napisu o treści określonej w pierwszym argumencie w punkcie (x,y), wykorzystując strokeStyle,
strokeStyle = "kolor" – określa kolor obramowania czcionki,
font = "Xpt FontFamily" – określa wielkość i font. X jest wartością liczbową, określającą wielkość fontu, aFontFamily nazwą fontu np. "40pt Consolas".

Ustaw kursor za znacznikiem z metodą getContext() i dodaj nową linię.
Wpisz:

```
rodzaj.font = "30pt Times New Roman";
rodzaj.fillStyle = "blue";
rodzaj.fillText("Witaj", 10, 50);
rodzaj.font = "30pt Arial";
rodzaj.strokeStyle  = "red";
rodzaj.strokeText("CANVAS", 110, 50);
```

Pomiar narysowanego tekstu

Na pewno zauważyłeś w poprzednim przykładzie, że aby połączyć style – zastosować różne fonty, kolory, wielkości, należy ręcznie określać położenie kolejnych fragmentów tekstu. Jeśli chcesz zrobić to automatycznie, musisz wykorzystać metodę measureText. Umożliwia ona pomiar tekstu przy aktualnych ustawieniach:

measureText("napis").width – zwracadługość napisu w pikselach przy aktualnych ustawieniach.

Ustaw kursor za ostatnio dodanym kodem i dodaj nową linię.
Wpisz:

```
rodzaj.font = "30pt Times New Roman";
rodzaj.fillStyle = "blue";
rodzaj.fillText("Yo", 10, 100);
var len = rodzaj.measureText("Yo").width;
rodzaj.font = "30pt Arial";
rodzaj.strokeStyle = "red";
rodzaj.strokeText("CANVAS", len + 20, 100);
```

Wyrównanie tekstu

Tekst możesz umiejscowić dowolnie, względem punktu odniesienia. Do tego celu służą dwie własności metody:

textAlign=[X] – określa wyrównanie tekstu w poziomie względem punktu (x,y). Wartość elementu X można ustawić na: "start", "end", "left", "right", "center" (domyślnie: "start"),

textBaseline=[X]; – określa wyrównanie tekstu w pionie. Wartość elementu X można ustawić na: "top", "hanging", "middle", "alphabetic", "ideographic", "bottom" (domyślnie: "alphabetic").

Ustaw kursor za ostatnio dodanym kodem i dodaj nową linię.
Wpisz:

```
rodzaj.arc(100, 150, 2, 2 * Math.PI, 0, true);
rodzaj.textAlign = "left";
rodzaj.fillStyle = "blue";
rodzaj.fillText("left", 100, 150);
rodzaj.arc(100, 200, 2, 2 * Math.PI, 0, true);
rodzaj.textAlign = "right";
rodzaj.fillText("right", 100, 200);
rodzaj.arc(100, 250, 2, 2 * Math.PI, 0, true);
rodzaj.textAlign = "center";
rodzaj.fillText("center", 100, 250);
rodzaj.arc(100, 300, 2, 2 * Math.PI, 0, true);
rodzaj.textBaseline = "middle";
rodzaj.fillText("middle", 100, 300);
rodzaj.arc(100, 400, 2, 2 * Math.PI, 0, true);
rodzaj.textBaseline = "bottom";
rodzaj.fillText("bottom", 100, 400);
rodzaj.textBaseline = "top";
rodzaj.fillText("top", 100, 400);
rodzaj.fillStyle = "black";
rodzaj.fill();
```

Strona końcowa

Poniżej możesz zobaczyć końcowy kod strony HTML:

```
<!DOCTYPE html>
<html lang="pl">
	<head>
		<meta charset="utf-8">
		<title>Tekst w Canvas</title>
			<script type="text/javascript">
			  function draw() {
			    var canvas = document.getElementById('image1');
			    var rodzaj = canvas.getContext('2d');
			    rodzaj.font = "30pt Times New Roman";
			    rodzaj.fillStyle = "blue";
			    rodzaj.fillText("Witaj", 10, 50);
			    rodzaj.font = "30pt Arial";
			    rodzaj.strokeStyle = "red";
			    rodzaj.strokeText("CANVAS", 110, 50);
			    rodzaj.font = "30pt Times New Roman";
			    rodzaj.fillStyle = "blue";
			    rodzaj.fillText("Yo", 10, 100);
			    var len = rodzaj.measureText("Yo").width;
			    rodzaj.font = "30pt Arial";
			    rodzaj.strokeStyle = "red";
			    rodzaj.strokeText("CANVAS", len + 20, 100);
			    rodzaj.arc(100, 150, 2, 2 * Math.PI, 0, true);
			    rodzaj.textAlign = "left";
			    rodzaj.fillStyle = "blue";
			    rodzaj.fillText("left", 100, 150);
			    rodzaj.arc(100, 200, 2, 2 * Math.PI, 0, true);
			    rodzaj.textAlign = "right";
			    rodzaj.fillText("right", 100, 200);
			    rodzaj.arc(100, 250, 2, 2 * Math.PI, 0, true);
			    rodzaj.textAlign = "center";
			    rodzaj.fillText("center", 100, 250);
			    rodzaj.arc(100, 300, 2, 2 * Math.PI, 0, true);
			    rodzaj.textBaseline = "middle";
			    rodzaj.fillText("middle", 100, 300);
			    rodzaj.arc(100, 400, 2, 2 * Math.PI, 0, true);
			    rodzaj.textBaseline = "bottom";
			    rodzaj.fillText("bottom", 100, 400);
			    rodzaj.textBaseline = "top";
			    rodzaj.fillText("top", 100, 400);
			    rodzaj.fillStyle = "black";
			    rodzaj.fill();
			  }
```

```
                </script>
        </head>
        <body onload="draw();">
                <canvas id="image1" width="300" height="450">
                        Twoja przeglądarka nie wspiera elementu canvas!
                        Pobierz Internet Explorer 9 lub nowszą wersję!
                </canvas>
        </body>
</html>
```

Zadanie 23
Temat: Animowana piłka (tor lotu góra dół) zmieniającą kolor.

Teoria

Uwaga1

Animacja w elemencie canvas polega na cyklicznym przerysowywaniu całości obrazka z uaktualnionymi obiektami.

Animacja składa się z następujących kroków:

1. usunięcie poprzedniego rysunku,
2. zapisanie stanu elementu canvas (jest to używane w przypadku, kiedy w kolejnych krokach zmieniamy style lub transformujemy element canvas),
3. wykonanie operacji przeliczenia nowych współrzędnych elementu i ponowne narysowanie wszystkich elementów na stronie,
4. przywrócenie stanu elementu canvas (jak w pkt. 2.).

Uwaga2

Do cyklicznego przerysowywania obrazka możesz wykorzystać dwie własności funkcje języka JavaScript:

setInterval(funkcja, czas) - funkcja jest nazwą funkcji, która ma być wywołana co czas podany w milisekundach (1s=1000ms); możesz zatrzymać jej cykliczne wywoływanie za pomocą funkcji clearInterval(),

zastosowanie: Funkcja setInterval jest używana w sytuacji, kiedy zachodzi potrzeba wykonania operacji w dokładnych odstępach czasowych, ponieważ nie czeka ona na zakończenie operacji, a jedynie wykonuje cyklicznie zadanie. Zapewne bardzo często spotkasz się z potrzebą zapewnienia cykliczności wykonywanych operacji. Wówczas użyjesz funkcji setInterval.

setTimeout(funkcja, czas) - funkcja jest nazwą funkcji, która ma być wywołana po czasie określonym w parametrze czas, podanym w milisekundach (1s=1000ms).

zastosowanie: Funkcja setTimeout jest wywoływana na końcu operacji i na czas pomiędzy kolejnym wywołaniem funkcji wpływa czas jej realizacji.

Przygotowanie strony

Uwaga1:

Przygotuj szablon i obramuj element canvas kolorem niebieskim, tak, aby zaznaczyć obszar elementu. W edytorze HTML utwórz nowy plik pod nazwą index.html i wklej do niego szkielet strony:

```
<!DOCTYPE html>
<html lang="pl">
	<head>
		<meta charset="utf-8">
		<title>Animacja w Canvas</title>
		<script type="text/javascript">
			var canvas
			var rodzaj;
			function draw(){
				canvas = document.getElementById('image1');
				rodzaj = canvas.getContext('2d');
			}
		</script>
		<style type="text/css">
			canvas { border: 1px solid blue; }
		</style>
	</head>
	<body onload="draw();">
		<canvas id="image1" width="400" height="400">
			Twoja przeglądarka nie wspiera elementu canvas!
			Pobierz Internet Explorer 9 lub nowszą wersję!
		</canvas>
	</body>
</html>
```

Uwaga2

Przygotuj:

- zmienne przechowujące aktualne współrzędne piłki (x,y),
- przesunięcie zmiennej y (dy),
- paletę kolorów (kolory)
- aktualny kolor (i).

Za linią var rodzaj; dodaj:

```
var x = 200;
var y = 20;
var dy = 10;
var kolory = new
Array("blue","red","yellow","green");
var i = 0;
```

Zapisz zmiany w pliku index.html i otwórz go w przeglądarce wspierającej element canvas,

Przygotowanie animacji

1)Utwórz nową funkcję, umieszczoną w funkcji draw, oraz cykliczne jej wywołanie za pomocą setInterval. Czyli ustaw kursor za znacznikiem z metodą getContext() i dodaj nową linię. Wpisz:

```
setInterval(anim, 100);
```

2)Ustaw kursor przed znacznikiem </script>i dodaj nową linię. Wpisz:

```
function anim() {
        rodzaj.clearRect(0, 0, canvas.width, canvas.height);
}
```

3)Narysuj piłkę – koło o promieniu 20 pikseli w punkcie określonym przez zmienne x i y. Koło ma zostać wypełnione kolorem z listy (kolory) na pozycji określonej przez wartość zmiennej i. Poniżej metody clearRect wpisz:

```
rodzaj.beginPath();
rodzaj.fillStyle = kolory[i];
rodzaj.arc(x, y, 20, 0, Math.PI * 2, true);
rodzaj.closePath;
rodzaj.fill();
```

Animacja piłki

Teraz, gdy wiesz już jak narysować piłkę, nadszedł czas na jej animację. Animacja będzie polegała na ruchu piłki w dół, następnie odbiciu od krawędzi elementu canvas i ruchu do góry, aż do krawędzi, gdzie piłka ponownie się odbije. Przy każdym odbiciu piłka będzie cyklicznie zmieniała kolor.

Dodaj zmianę pozycji piłki.

Poniżej metody clearRect wpisz:

```
y += dy;
```

Zapisz stronę index.html i odśwież ją w przeglądarce. Sprawdź, jak zachowuje się piłka.

Informacja

Piłka przesuwa się z góry na dół, ale nie odbija się od krawędzi. Aby mogła się odbić, musisz ograniczyć jej ruch do pola elementu canvas. Pole to ma wysokość określoną przez wartość canvas.height. Należy również uwzględnić promień piłki. Aby aby nie „wychodziła" za pole, musisz ograniczyć jej ruch od 20 do (canvas.height-20) pikseli.

Ogranicz ruch piłki do od 20 do (canvas.height-20) pikseli.

Za poprzednio dodaną linijką wpisz:

```
if (y > (canvas.height-20)) {
        dy = -10;
}
if (y < 20) {
        dy = 10;
}
```

Zapisz stronę index.html i odśwież ją w przeglądarce. Sprawdź, jak zachowuje się piłka.

Informacja
Piłka przesuwa się z góry na dół i odbija się od krawędzi. Aby możliwa była zamiana koloru, musisz po każdym odbiciu od górnej lub dolnej krawędzi zmienić kolor wypełnienia – wartość zmiennej i od 0 do ilości elementów w tablicy kolorów.

Zmień cyklicznie kolor po odbiciu od krawędzi.

Za linijką dy = -10; dodaj :

```
i = ++i % kolory.length;
```

Za linijką dy = 10; dodaj :

```
i = ++i % kolory.length;
```

Zapisz stronę index.html i odśwież ją w przeglądarce. Sprawdź, jak zachowuje się piłka.

Strona końcowa

Poniżej możesz zobaczyć końcowy kod strony HTML:

```
<!DOCTYPE html>
<html lang="pl">
        <head>
                <meta charset="utf-8">
                <title>Animacja w Canvas</title>
                        <script type="text/javascript">
                           var canvas
                           var rodzaj;
                           var x = 200;
                           var y = 20;
                           var dy = 10;
                           var kolory = new Array("blue", "red", "yellow", "green");
                           var i = 0;

                           function draw() {
                              canvas = document.getElementById('image1');
                              rodzaj = canvas.getContext('2d');
                              setInterval(anim, 100);
                           }

                           function anim() {
                              rodzaj.clearRect(0, 0, canvas.width, canvas.height);
                              y += dy;
                              if (y > (canvas.height - 20)) {
                                 dy = -10;
                                 i = ++i % kolory.length;
```

```
                        }
                        if (y < 20) {
                           dy = 10;
                           i = ++i % kolory.length;
                        }
                        rodzaj.beginPath();
                        rodzaj.fillStyle = kolory[i];
                        rodzaj.arc(x, y, 20, 0, Math.PI * 2, true);
                        rodzaj.closePath;
                        rodzaj.fill();
                     }
                  </script>
      <style type="text/css">
            canvas { border: 1px solid blue; }
      </style>
       </head>
       <body onload="draw();">
            <canvas id="image1" width="400" height="400">
                  Twoja przeglądarka nie wspiera elementu canvas!
                  Pobierz Internet Explorer 9 lub nowszą wersję!
            </canvas>
       </body>
</html>
```

Zadanie 24
Temat: Gra zaproponowana przez jednego z uczniów o nazwie Karol_game.

Zasady gry:
Odbijamy piłkę do góry, każde odbicie przyspiesz ruch piłki. Licznik zlicza ilość odbić.

Zdarzenia

http://webmaster.helion.pl/index.php/kursjs-obsluga-zdarzen-i-elementow-strony/kursjs-zdarzenia

Z JavaScriptem i DHTML-em nieodłącznie związane jest pojęcie zdarzeń. Zdarzenie to przesunięcie kursora myszy, kliknięcie, wczytanie strony, opuszczenie strony itp. Każdy element strony ma przypisany zestaw obsługiwanych przez niego zdarzeń, a każdemu z nich można przypisywać własne procedury obsługi, napisane w JavaScripcie.

Procedurę obsługi można przypisać do danego zdarzenia na dwa sposoby.

Jeśli jest ona bardzo krótka, np. składa się tylko z jednej, dwóch instrukcji, i może być zapisana w jednej linii kodu, można ją przypisać bezpośrednio do znacznika obsługującego dane zdarzenie:

Sposób 1

<*znacznik* **zdarzenie**="instrukcja;">

Sposób 2

W przypadku dłuższych procedur obsługi zdarzeniu należy przypisać jedynie wywołanie funkcji JavaScript:

<*znacznik* **zdarzenie**="nazwa_funkcji();">

natomiast w kodzie strony (z reguły w sekcji head) umieścić samą funkcję. Całość miałaby zatem schematyczną postać:

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN""http://www.w3.org/TR/html4/strict.dtd">
    <html>
      <head>
        <meta http-equiv="Content-Type" content="text/html; charset=utf-8">
        <meta http-equiv="Content-Script-Type" content="text/javascript">
       <title>Moja strona WWW</title>
           <script type="text/javascript">
                function nazwa_funkcji()
                    {
                       //tutaj kod funkcji
                    }
            </script>
      </head>
  <body>
      <znacznik zdarzenie="nazwa_funkcji();">
  </body>
 </html>
```

Przykładowy znacznik obsługujący zdarzenie miałby postać:

<p onclick="pClicked();">

Lista typowych zdarzeń.

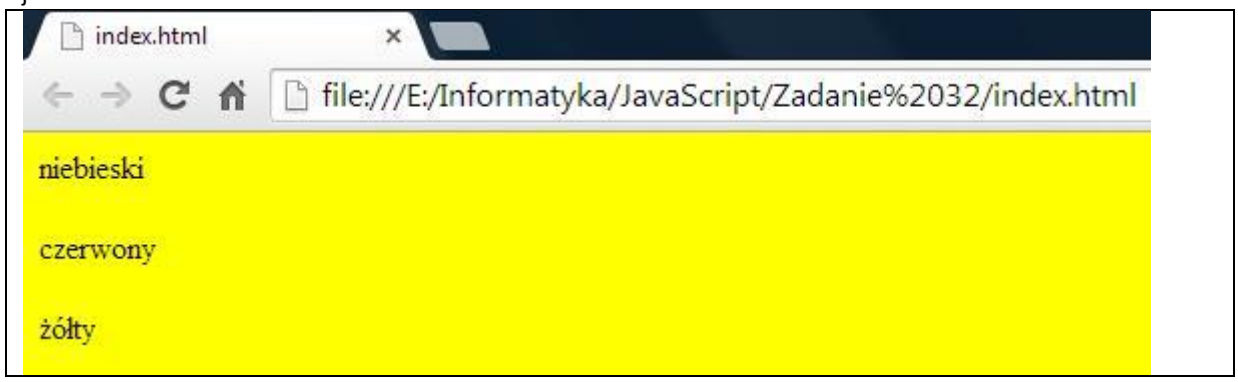
Nazwa zdarzenia	opis	Elementy HTML obsługujące zdarzenie
onAbort	Zdarzenie, które powstaje, kiedy zostanie przerwane ładowanie obrazu.	img
onBlur	Zdarzenie, które powstaje, kiedy element traci fokus.	większość elementów strony

onChange	Zdarzenie, które powstaje, kiedy element traci fokus i jednocześnie zmienia się wartość zawarta w tym elemencie (np. polu tekstowym).	input, select, textarea
onClick	Zdarzenie, które powstaje, kiedy element został kliknięty	większość elementów strony
onDblclick	Zdarzenie, które powstaje, kiedy element został dwukrotnie kliknięty.	większość elementów strony
onError	Zdarzenie, które powstaje, kiedy w trakcie ładowanie obrazu wystąpi błąd.	Img
Onfocus	Zdarzenie, które powstaje, kiedy element otrzymuje fokus.	większość elementów strony
onkeydown	Zdarzenie, które powstaje, kiedy naciśnięty zostanie klawisz klawiatury.	większość elementów strony
onkeypress	Zdarzenie, które powstaje, kiedy klawisz klawiatury zostanie naciśnięty i puszczony.	większość elementów strony
Onkeyup	Zdarzenie, które powstaje, kiedy klawisz klawiatury zostanie puszczony.	większość elementów strony
onload	Zdarzenie, które powstaje, kiedy przeglądarka zakończy ładowanie strony lub ramki.	body, frameset
onmousedown	Zdarzenie, które powstaje, kiedy klawisz myszy zostanie naciśnięty nad elementem.	większość elementów strony
onMouseMove	Zdarzenie, które powstaje, kiedy kursor myszy jest przesuwany nad elementem.	większość elementów strony
onMouseOut	Zdarzenie, które powstaje, kiedy kursor myszy opuści obszar elementu.	większość elementów strony
onMouseOver	Zdarzenie, które powstaje, kiedy kursor myszy wejdzie w obszar elementu.	większość elementów strony
onmouseup	Zdarzenie, które powstaje, kiedy klawisz myszy zostanie zwolniony nad elementem.	większość elementów strony
on reset	Zdarzenie, które powstaje podczas resetowania (wywołanie w kodzie metody reset, kliknięcie przycisku reset) formularza.	Form
Onresize	Zdarzenie, które powstaje, gdy zmieni się rozmiar okna.	body, frameset
Onselect	Zdarzenie, które powstaje podczas zaznaczania fragmentu tekstu.	input (text, textarea)
onsubmit	Zdarzenie, które powstaje podczas wysyłania (wywołanie w kodzie metody submit, kliknięcie przycisku submit) formularza.	Form
onUnload	Zdarzenie, które powstaje, kiedy przeglądarka usuwa bieżący dokument.	body, frameset

Zadanie 32

Napisać stronę używającą zdarzenia (onMouseOver) oraz zdefiniowanych funkcji, która będzie zmieniać kolor tła. Wykorzystaj metodę .bgColor=kolor; dla obiektu dokument.
Nazwa funkcji:

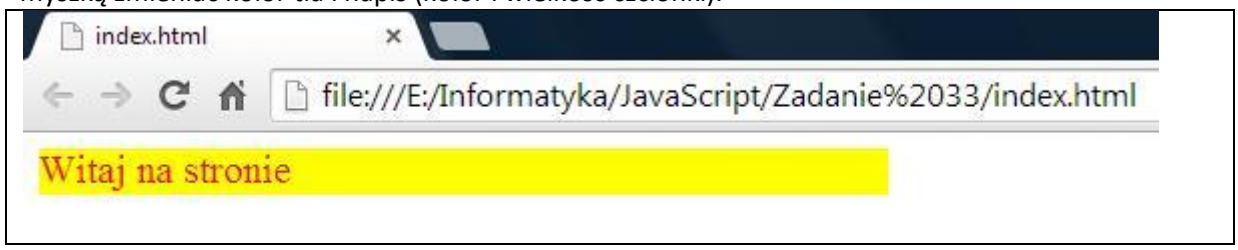
funtion kolor_tla_nazwisko_ucznia(kolor)
{
treść funkcji;
}



Dopisac dokładne rozwiązanie

Zadanie 33

Napisać stronę używającą zdarzenia onMouseMove oraz onMouseOut, która będzie po najechaniu myszką zmieniać kolor tła i napis (kolor i wielkość czcionki).

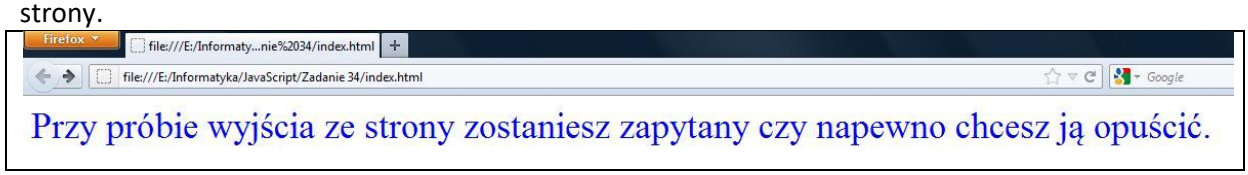


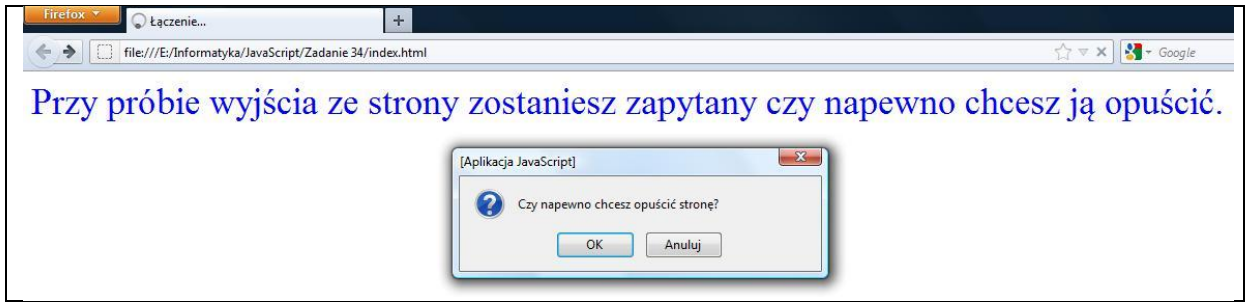


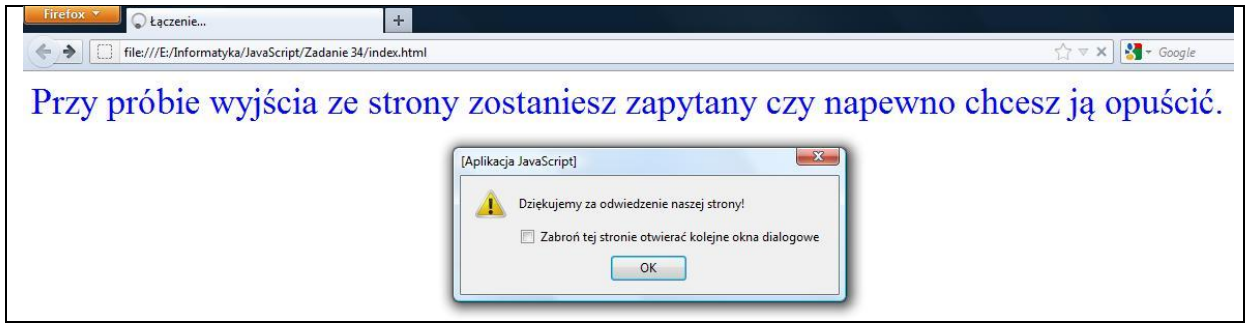
Dopisac dokładne rozwiązanie

Zadanie 34

Napisać stronę używającą zdarzenia onUnload która przy próbie wyjścia ze strony pytało się o potwierdzenie. Gdy nie będziesz chciał opuścić strony do użyj funkcji history.back();
Po opuszczeniu uruchom jeszcze okno Alert z napisem Dziękujemy za odwie własnościdzenie naszej strony.







Dopisac dokładne rozwiązanie

Uruchamianie program Windows ze strony

Nasza strona tematycznie nie powinna tak bardzo odbiegać jednak wątek uruchamiania programów środowiska Windows za pomocą Apletów Java stanowił dość spore zainteresowanie. Alternatywnym rozwiązaniem dla rozszerzenia zasięgu przeglądarki internetowej (realizacji możliwości uruchamiania w/w procesów) może być wykorzystanie użycie JavaScript.

Nasz przykład zawiera funkcję odczytu typu przeglądarki oraz skrypty dla uruchomienia konkretnych programów systemu operacyjnego wprowadzone za pomocą parametru:

```
<html>
  <head>
    <title></title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
  </head>
  <body>
  </p>
  Test uruchomienia aplikacji za pomocą JavaScript.</p>
  Skrypty testowane na IE oraz FireFox.</p>
  Polecenie : <input id = "polecenie" type="text" value="C:windowsnotepad.exe"></input></br>
  <input type="button" width="15" value="Uruchom" onclick="RunExe();"/></input></p>
  <script type="JavaScript">
  </script>
  <script type="text/javascript">
  function RunExe()
  {
    var bname = navigator.appName;
    var pol = document.getElementById('polecenie');
    if (bname =="Netscape")
    {
      netscape.security.PrivilegeManager.enablePrivilege("UniversalXPConnect");
      var exe =
window.Components.classes['@mozilla.org/file/local;1'].createInstance(Components.interfaces.nsIL
ocalFile);
      exe.initWithPath(pol.value);
      var run =
window.Components.classes['@mozilla.org/process/util;1'].createInstance(Components.interfaces.n
sIProcess);
      run.init(exe);
      var parameters = [""];

      run.run(false, parameters,parameters.length);

    }
    else
    {
      var oShell = new ActiveXObject("WScript.Shell");
      var prog = pol.value;
      oShell.Run('""'+prog+'"',1);
    }
  }
```

```
  </script>
  <a href="/ http://www.gmmobile.pl">Strona główna</a>
  </body>
</html>
```

http://gajdaw.pl/ajax/jquery-tutorial/p1.html
http://jquery.com
https://mansfeld.pl/programowanie/jquery-co-to-jest-czy-warto-uzywac/

Biblioteka jQuery

Dołączanie biblioteki.
Biblioteka jQuery jest dołączana do stron WWW jako zewnętrzny plik JavaScript, a zatem nie wymaga instalacji żadnego oprogramowania. Po dołączeniu do dokumentu HTML pliku jquery-1.2.3.js uzyskujemy dostęp do obiektów i metod biblioteki jQuery.

Znaczenie metoda ready() klasy jQuery.
Zasadniczą rolę odgrywa metoda ready() klasy jQuery. Jest ona wywoływana po kompletnym załadowaniu strony, a zatem w jej treści możemy operować pełnym modelem DOM dokumentu. Wewnątrz funkcji ready() definiujemy anonimową funkcję, która zostanie wywołana po zakończeniu ładowania dokumentu. W treści anonimowej funkcji możemy umieścić dowolny kod, np. wywołanie okna informacyjnego alert():

```
<html>
 <head>
  <title>Przykład 1-1</title>
  <script type="text/javascript" src="jquery-1.2.3.js"></script>
  <script type="text/javascript">

  $(document).ready(function(){

    alert('Witaj');

  });

  </script>
 </head>
<body>

<p>Zespół Szkół Energetycznych</p>

</body>
</html>
```

Obsługa zdarzenia onclick elementu p na dwa spsoby:

Separacja zachowania (tj. JavaScript) od struktury (tj. HTML) polega na tym, że wewnątrz elementu body nie występują żadne zdarzenia JavaScript.
Tradycyjne podejście:
Zdarzenie onclick akapitu p, które w tradycyjnym dokumencie HTML/JavaScript opisalibyśmy jako:

```
<p onclick="alert('Witaj')">Lorem</p>
```

Podejście z użyciem jQuery

przy użyciu jQuery definiujemy stosując selektor 'p' oraz metodę click(), w której umieszczamy wywołanie funkcji alert():

```
<html>
 <head>
  <title>Przykład 1-2</title>
  <script type="text/javascript" src="jquery-1.2.3.js"></script>
  <script type="text/javascript">

  $(document).ready(function(){

    $('p').click(function(){
      alert('Witaj');
    });

  });

  </script>
 </head>
<body>

<p> Zespół Szkół Energetycznych </p>

</body>
</html>
```

Składnia jQuery

$ w kodzie powyższych przykładów, **jest aliasem do funkcji o nazwie jQuery()**.
W odróżnieniu od Pascal-a czy C++, w języku JavaScript znak **$** jest dozwolony w identyfikatorach funkcji.

Zadanie 35

Powyższy przykład możemy zapisać jako:

```
<html>
 <head>
  <title>Przykład 1-3</title>
  <script src="http://ajax.googleapis.com/ajax/libs/jquery/1.11.1/jquery.min.js"></script>
<script type="text/javascript">
  jQuery(document).ready(function(){
    jQuery('p').click(function(){
      alert('Witaj');
    });
   });
     </script>
```

```
 </head>
<body>
<p> Zespół Szkół Energetycznych </p>
</body>
</html>
```

Analizując składnię powyższego przykładu, powiemy, że w skrypcie występuje funkcja jQuery() wywołana z parametrem document:

```
jQuery(document)
```

Funkcja ta zwraca obiekt, na rzecz którego wywołujemy metodę ready:

```
jQuery(document).ready(
   ...
);
```

Parametrem metody ready() jest funkcja anonimowa, tj. taka, która nie otrzymała nazwy. Jej definicja występuje w miejscu wywołania:

```
function(){

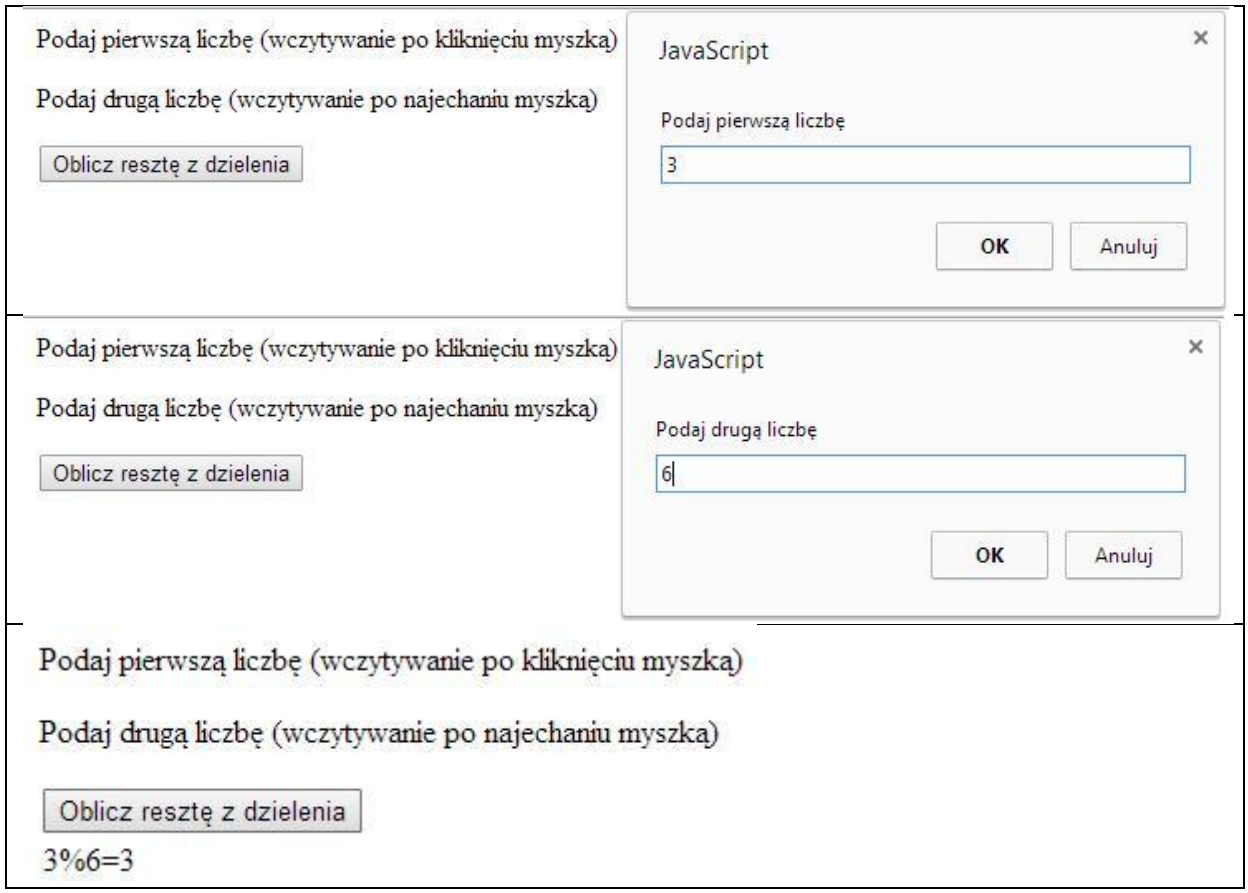
}
```

W treści funkcji może występować dowolny kod. W powyższym przykładzie było to wywołanie funkcji jQuery() z parametrem 'p'. Zwróconemu obiektowi wywołaliśmy metodę click(), której parametrem jest funkcja anonimowa wywołująca okno dialogowe alert():

```
jQuery('p').click(function(){
   alert('Witaj');
});
```

Zadanie 36

Wykonaj program w jQuery który, po kliknięciu myszką na napis „Podaj pierwszą liczbę” uruchomi okienko **prompt** → nastąpi wczytanie liczby do zmiennej a_trzy_litery_nazwiska_ucznia oraz
po najechaniu myszką na napis „Podaj drugą liczbę” uruchomi okienko **prompt** → nastąpi wczytanie liczby do zmiennej b_trzy_litery_nazwiska_ucznia. Komputer obliczy resztę z dzielenia liczby a i b. Użyj znacznika <p> oraz .
I wypisze wynik:
Np.
a=7
b=4
7 % 4 = 3


Podaj pierwszą liczbę (wczytywanie po kliknięciu myszką)
Podaj drugą liczbę (wczytywanie po najechaniu myszką)
Oblicz resztę z dzielenia
JavaScript
Podaj pierwszą liczbę
3
OK
Anuluj
Podaj pierwszą liczbę (wczytywanie po kliknięciu myszką)
Podaj drugą liczbę (wczytywanie po najechaniu myszką)
Oblicz resztę z dzielenia
JavaScript
Podaj drugą liczbę
6
OK
Anuluj
Podaj pierwszą liczbę (wczytywanie po kliknięciu myszką)
Podaj drugą liczbę (wczytywanie po najechaniu myszką)
Oblicz resztę z dzielenia
3%6=3

Selektory CSS

Element

Do wyboru wszystkich elementów zadanego typu służy selektor będący nazwą elementu. Akapity wybierzemy selektorem p, sekcje div — selektorem div, tabele — selektorem table.

Skrypt:

```
$(document).ready(function(){
  $('p').click(function(){
    alert('kliknięto p');
  });
});
----------------------------
<p> Zespół Szkół Energetycznych </p>
```

przypisze obsługę zdarzenia onclick wszystkim akapitom **<p>** występującym w dokumencie.
Uwaga:
W powyższym przykładzie fragment znajdujący się przed linią jest kodem JavaScript, zaś dolny fragment — kodem HTML. Wszystkie kolejne przykłady zostały przedstawione w podobnej, skróconej formie.

Uwaga:

Technologia	oddzielenie	Od
CSS	Kodu HTML	prezentacji-wygladu
jQuery	Kodu HTML	działania-akcji

Praktyczne odzielenie kodu HTML od **prezentacji-wygladu** dla CSS

```
p {
  color: blue;
}
```

to wszystkie akapity staną się niebieskie.
Praktyczne odzielenie kodu HTML od **działania-akcji** dla CSS
Podobnie, selektor **p** w jQuery:

```
$('p').click(function(){
  alert('kliknięto p');
});
```

spowoduje, że wszystkie elementy p otrzymają obsługę zdarzenia onclick.

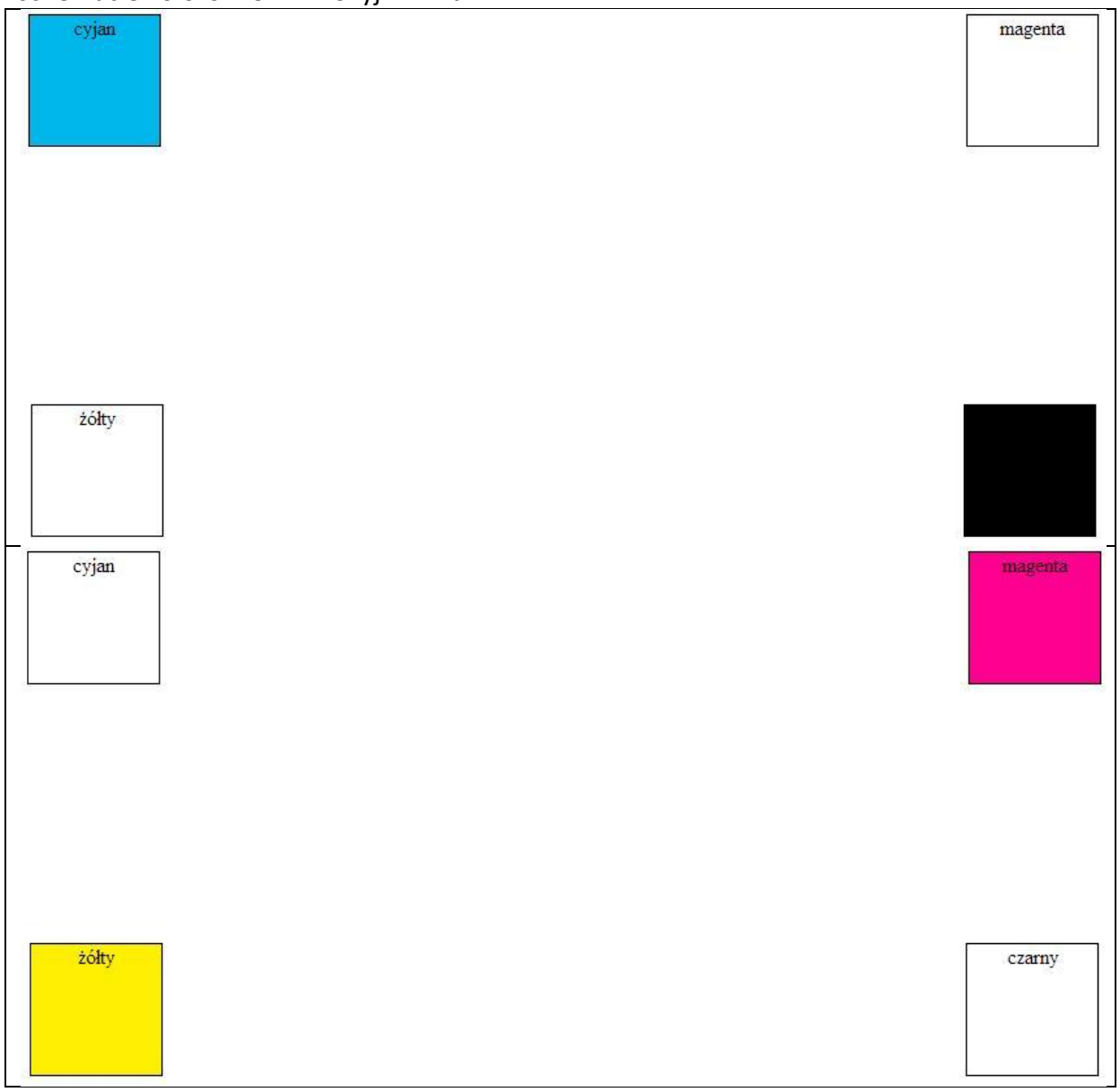
Identyfikator

Jeśli element HTML posiada identyfikator id, wówczas stosujemy selektor składający się ze znaku # oraz identyfikatora:

```
$(document).ready(function(){
  $('#tresc').click(function(){
    alert('kliknięto #tresc');
  });
});
----------------------------
<p id="tresc"> Zespół Szkół Energetycznych </p>
```

Zadanie 38

Wykonaj program w jQuery który, po kliknięciu myszką na kwadraty będzie gasił i zapalał w schemacie kolorów CMYK. Użyj Div i id.



Klasa

Selektor elementów wzbogaconych o klasę składa się z kropki i nazwy klasy:

```
$(document).ready(function(){
  $('.inny').click(function(){
    alert('kliknięto .inny');
  });
});
----------------------------
<p class="inny"> Zespół Szkół Energetycznych </p>
```

Wartość atrybutu

Selektor:

a[href="ipsum.html"]
odnosi się do elementu a, którego atrybut href przyjmuje wartość ipsum.html:

```
<a href="ipsum.html">ipsum</a>
Zatem kod:
$(document).ready(function(){
  $('a[href="ipsum.html"]').click(function(){
    alert('kliknięto ipsum.html');
  });
});
----------------------------
<ol>
<li><a href="lorem.html">lorem</a></li>
<li><a href="ipsum.html">ipsum</a></li>
<li><a href="dolor.html">dolor</a></li>
</ol>
```

będzie ustalał obsługę zdarzenia onclick wyłącznie środkowego hiperłącza.

Zmiana treści i wyglądu

Dostęp do treści elementu

Dostęp do treści elementu zapewnia metoda html(). Wywołana bez parametru zwraca bieżącą treść elementu. Wywołana z parametrem — ustala nową treść.
Jeśli metodę html() wywołamy na rzecz kliknięty akapitu, wówczas kliknięcie akapitu będzie powodowało zmianę treści elementu:

```
$(document).ready(function(){
  $('p').click(function(){
    $(this).html('one, two, ...');
  });
});
----------------------------
<p> Zespół Szkół Energetycznych </p>
```

Dostęp do kliknięty akapitu wewnątrz metody click umożliwia this — referencja do obiektu, który wygenerował zdarzenie.

Zmiana stylu

Styl elementu zmienimy metodą css(), która pobiera dwa parametry: nazwę właściwości oraz wartość:

```
$(document).ready(function(){
  $('p').click(function(){
    $(this).css('color', 'yellow');
  });
});
----------------------------
<p> Zespół Szkół Energetycznych </p>
```

Wadą powyższego rozwiązania jest połączenie dwóch języków: CSS oraz JavaScript. Wygodniej będzie do zmiany formatu wykorzystać klasy CSS oraz metody addClass() i removeClass().

Dodanie i usunięcie klasy

Każdy element wybrany selektorem możemy dynamicznie wzbogacić o klasę. Umożliwia to metoda **addClass()**. Poniższy przykład jest rozbity na trzy języki: CSS, JavaScript oraz HTML:

```
.inny {
  color: yellow;
}

$(document).ready(function(){
  $('p').click(function(){
    $(this).addClass('inny');
  });
});
----------------------------
<p> Zespół Szkół Energetycznych </p>
```

W celu usunięcia klasy wywołujemy metodę **removeClass():**

```
.inny {
  color: yellow;
}
$(document).ready(function(){
  $('p.inny').click(function(){
    $(this).removeClass('inny');
  });
});
----------------------------
<p> Zespół Szkół Energetycznych </p>
```

Zdarzenia

Biblioteka jQuery zapewnia dostęp do pełnego zestawu zdarzeń HTML. Na przykład obsługę zdarzeń onmouseover oraz onmouseout zdefiniujemy metodami mouseover() oraz mouseout():

```
.inny {
  color: yellow;
}

$(document).ready(function(){
  $('p').mouseover(function(){
    $(this).addClass('inny');
  });
  $('p').mouseout(function(){
    $(this).removeClass('inny');
  });
});
----------------------------
<p> Zespół Szkół Energetycznych </p>
```

zaś zdarzenie ondblclick metodą dblclick():

```
$(document).ready(function(){
```

```
  $('p').dblclick(function(){
    $(this).css('text-decoration', 'underline');
  });
});
----------------------------
<p> Zespół Szkół Energetycznych </p>
```

Duża część funkcji odpowiedzialnych za zdarzenia może być wywoływana na dwa sposoby.

Pierwszym sposobem jest wywołanie z parametrem, którym jest funkcja:

```
$('p').click(function(){
  ...
});
```

Takie wywołanie definiuje funkcję obsługi zdarzenia.

Drugi sposób wywołania to wywołanie bezparametrowe:

```
$('p').click();
```

Powoduje ono uruchomienie zdarzenia (ang. trigger).

Zestawienie części zdarzeń HTML i odpowiadających im metod klasy jQuery.

Zdarzenie HTML	Metoda jQuery
onchange	change()
onclick	click()
ondblclick	dblclick()
onkeydown	keydown()
onkeypress	keypress()
onkeyup	keyup()
onload	load()
onmousedown	mousedown()
onmousemove	mousemove()
onmouseout	mouseout()
onmouseover	mouseover()
onmouseup	mouseup()
onsubmit	submit()

Zagadnienia dalsze

Automatyczna iteracja

Jak już zostało powiedziane, selektor jQuery obejmuje swoim działaniem wszystkie wybrane elementy. Jeśli użyjemy selektora li:

```
$(document).ready(function(){
  $('li').css('color', 'blue');
});
----------------------------
<ul>
  <li>one</li>
  <li>two</li>
  <li>three</li>
  <li>four</li>
</ul>
```

to przetwarzaniu będą podlegały wszystkie elementy listy. Każdy z nich stanie się niebieski. Nie wpisujemy ręcznie żadnej iteracji (np. pętli for). Jest ona wykonywana podobnie jak w CSS automatycznie dla wszystkich elementów pasujących do selektora.

Kaskada

Większość metod jQuery zwraca w wyniku obiekt jQuery. Pozwala to na tworzenie kaskadowych wywołań metod:

```
$(document).ready(function(){
  $('li').css('color', 'yellow').append(' - read more').css('background', 'black');
});
----------------------------
<ul>
  <li>one</li>
  <li>two</li>
  <li>three</li>
  <li>four</li>
</ul>
```

Powyższy kod jQuery możemy równoważnie zapisać jako trzy osobne wywołania:

```
$('li').css('color', 'yellow');
$('li').append(' - read more');
$('li').css('background', 'black');
```

Odczyt atrybutów

Dostęp do atrybutów zapewnia metoda attr(), której parametrem jest nazwa atrybutu HTML:

```
$(document).ready(function(){
  $('input').focus();
  $('input').keyup(function(){
    $('div').html($('input').attr('value'));
  });
});

----------------------------
<form action="#" method="post">
  <input type="text" name="imie" value="" />
</form>
<div></div>
```

Wywołanie w jQury:
$('input').attr('value')
powoduje odczytanie atrybutu value elementu input:
<input name="" value="" />

wytłumaczenie
Pierwsza instrukcja:
$('input').focus();
uruchamia zdarzenie onfocus dla elementu input. Dzięki niej po bezpośrednio po otworzeniu dokumentu w przeglądarce kursor będzie znajdował się w polu edycyjnym formularza.

Ajax, czyli asynchroniczne ładowanie pliku

Do załadowania zewnętrznego pliku służy metoda load():

```
$(document).ready(function(){
  $('#content').load('lorem.html');
});
----------------------------
<div id="content"></div>
```

Działa ona w oparciu o obiekt XMLHttpRequest. Dokument, którego adres URL jest podany jako parametr jest pobierany asynchronicznie w tle (Ajax).

Menu

Ładowanie treści po kliknięciu

Wykonanie menu, które nie będzie przeładowywało całej strony rozpoczynamy od załadowania zewnętrznego dokumentu (tj. pliku fragment-abba.html) do elementu o identyfikatorze #content po wystąpieniu zdarzenia onclick. Zadanie to zrealizuje następujący kod jQuery:

```
$('#a1').click(function(){
  $('#content').load('fragment-abba.html');
});

<ol>
  <li id="a1">ABBA</li>
</ol>
<div id="content"></div>
```

Najpierw wybieramy element o identyfikatorze #a1, któremu definiujemy zdarzenie onclick:

```
$('#a1').click(function(){
  ...
});
```

Wewnątrz obsługi zdarzenia występuje ładowanie zewnętrznego dokumentu fragment-abba.html do elementu o identyfikatorze #content:

```
$('#content').load('fragment-abba.html');
```

7.2 Kilka opcji menu

Jeśli menu zawiera kilka opcji:

```
<ol id="menu">
  <li id="a1">ABBA</li>
  <li id="a2">AC DC</li>
  ...
</ol>
<div id="content"></div>
```

wówczas definiujemy zdarzenie onclick każdej opcji z osobna:

```
$(document).ready(function(){

  $('#a1').click(function(){
    $('#content').load('fragment-abba.html');
  });
```

```
 $('#a2').click(function(){
   $('#content').load('fragment-ac_dc.html');
 });

 ...

});
```

7.3 Menu ol/li/a

Opcje menu należy przekształcić w hiperłącza a:

```
<ol>
 <li><a href="fragment-abba.html">ABBA</a></li>
 <li><a href="fragment-ac_dc.html">AC DC</a></li>
 ...
</ol>
<div id="content"></div>
```

W tym przypadku stosujemy selektor wybierający element o zadanej wartości atrybutu. W ten sposób kod jQuery przyjmie postać:

```
$(document).ready(function(){

 $('a[href="fragment-abba.html"]').click(function(){
   $('#content').load('fragment-abba.html');
   return false;
 });

 $('a[href="fragment-ac_dc.html"]').click(function(){
   $('#content').load('fragment-ac_dc.html');
   return false;
 });

 ...

});
```

Instrukcje return zapewniają, że kliknięcie hiperłącza nie będzie powodowało przeładowania strony.

7.4 Ajaksowe menu działające po wyłączeniu JavaScript

Witryna, z menu, które nie przeładowuje strony, wymaga przygotowania każdej podstrony w dwóch wersjach: pełnej i okrojonej. Strona w wersji okrojonej zawiera przedrostek fragment-. Jeśli na przykład witryna zawiera dwie własności opcje menu, które mają ładować pliki abba.html oraz ac_dc.html, to należy przygotować cztery pliki:

abba.html
fragment-abba.html

ac_dc.html
fragment-ac_dc.html

W menu witryny, w atrybutach href, stosujemy nazwy pełnych stron (tj. bez przedrostka fragment-): Dzięki temu witryna będzie działała po wyłączeniu JavaScript:

```
<ol>
  <li><a href="abba.html">ABBA</a></li>
  <li><a href="ac_dc.html">AC DC</a></li>
  ...
</ol>
<div id="content">
```

W powyższym kodzie ujawnia się cały urok rozwiązania stosującego jQuery. Jest to bowiem zwykłe menu, jakie byśmy przygotowali wykonując statyczne pliki HTML.

Modyfikację hiperłączy menu, w takie, które nie przeładowują strony zapewnia kod zawarty w pliku menu.js (plik ten należy oczywiście dołączyć do każdej pełnej podstrony):

```
$(document).ready(function(){

  $('a').click(function(){
    $('#content').load(
      'fragment-' + $(this).attr('href')
    );
    return false;
  });

});
```

Kolejno:

- wybieramy wszystkie hiperłącza $('a'),
- ustalamy obsługę zdarzenia onclick,
- w obsłudze zdarzenia onclick do elementu o identyfikatorze #content ładujemy zewnętrzny plik,
- adres url pobieranego pliku zewnętrznego powstaje z adresu odczytanego z atrybutu href kliknięty hiperłącza this przez dodanie prefiksu fragment-,
- kliknięcie hiperłącza nie może powodować przeładowania strony (return false).

---

8. Wyszukiwarka

Ajaksowa wyszukiwarka, której użycie nie przeładowuje strony wykorzystuje formularz oraz element #wyniki:

```
<form action="index.php" method="get">
  <input id="sz" name="szukaj" />
  <input type="submit" value="szukaj" />
</form>
<div id="wyniki"></div>
```

Działanie wyszukiwarki definiuje następujący kod JavaScript:

```
$(document).ready(function(){

  $('#wyniki').hide();

  $('form').submit(function(){
    $('#wyniki').load('server.php?co=' + $('#sz').attr('value'));
    $('#wyniki').show();
    return false;
  });

});
```

Kolejno:

- zaraz po załadowaniu dokumentu ukrywamy element #wyniki (metoda hide()),
- ustalamy obsługę zdarzenia onsubmit elementu form,
- obsługa zdarzenia polega na załadowaniu zewnętrznego dokumentu do elementu #wyniki (metoda load()) i pokazaniu elementu #wyniki (metoda show()),
- adres url pobieranego elementu powstaje przez dodanie na końcu adresu sever.php?co= tekstu wprowadzonego w formularzu do elementu o identyfikatorze #sz.
- instrukcja return false zakazuje przeładowania strony.

Wykorzystując odwołania kaskadowe, obsługę zdarzenia onsubmit można zakodować następująco:

```
$('#wyniki').load('server.php?co=' + $('#sz').attr('value')).show();
```

Podana wyszukiwarka, podobnie jak opisane wcześniej menu, działa także po wyłączeniu obsługi JavaScript.

Co to jest JavaScript?

Ten wszechobecny dziś język skryptowy został stworzony – początkowo jako LiveScript – na potrzeby firmy Netscape i przeglądarki Navigator. Zamierzeniem jego projektantów było rozszerzenie możliwości języka HTML w taki sposób, aby strony stały się bardziej atrakcyjne pod względem sposobu prezentacji danych oraz możliwości interakcji z użytkownikiem. W 1995 roku, na mocy porozumienia z firmą Sun Microsystems, w zamian za włączenie do przeglądarki Netscape obsługi technologii Java, język otrzymał oficjalną nazwę JavaScript. W grudniu tego samego roku został zaimplementowany w najnowszej wówczas wersji Navigatora i rozpoczął triumfalny pochód przez Sieć.

JavaScript jest przede wszystkim **językiem skryptowym – to znaczy interpretowanym**. Nie musi zostać skompilowany do kodu maszynowego, aby można było zobaczyć efekty jego działania. Wystarczy nam do tego przeglądarka internetowa, która ten język obsługuje – czyli w zasadzie każda z liczących się na rynku aplikacji. Ze względów bezpieczeństwa JavaScript ma znacznie ograniczone uprawnienia dostępu do zasobów komputera, przy użyciu którego przeglądana jest dana strona, a wszelkie odwołania do funkcji i obiektów wykonywane są w trakcie wykonywania programu.

JavaScript a Java

Zbieżność nazw obu języków może być dla początkującego użytkownika myląca. Przyjęcie takiej, a nie innej nazwy dla produktu Netscape'a najprawdopodobniej podyktowane było względami marketingowymi, ale zasadność łączenia obu pakietów do dziś jest przedmiotem kontrowersji. Istnieją tutaj dwie własności szkoły: jedni uważają, że JavaScript sięga swoimi korzeniami środowiska Java, wskazując na przykład na podobieństwo składni, inni zaś dowodzą, że poza nazwą oba języki niewiele mają ze sobą wspólnego. Jakiekolwiek miałoby być rozstrzygnięcie tego sporu, debiutujący programiści powinni pamiętać o tym, że mają do czynienia z zupełnie odrębnymi mechanizmami.

Zalety JavaScriptu

Największą zaletą JavaScriptu jest niewątpliwie jego prostota. Język ten uważa się więc za relatywnie łatwy do opanowania i rzeczywiście – nic nie stoi na przeszkodzie, aby zacząć używać go już po kilku minutach od rozpoczęcia nauki. Ponieważ jest to obecnie najbardziej rozpowszechniony język skryptowy, w Internecie można znaleźć miliony praktycznych przykładów – witryn wykorzystujących różnorodne funkcje JavaScriptu.

O olbrzymiej popularności języka zadecydowały z pewnością jego uniwersalność oraz fakt, iż pozwala on na odciążenie serwerów i ograniczenie ilości zbędnych danych przesyłanych pomiędzy nimi a przeglądarką klienta. Najprostszym przykładem tych zalet mogą być choćby sklepy internetowe, których nieodzownym elementem są formularze wypełniane przez użytkownika. Dzięki JavaScriptowi można sprawdzić poprawność wprowadzonych danych na komputerze klienta, unikając – przy ewentualnym błędzie – urzeczywistnienia scenariusza, w którym dane są wysyłane, a serwer zużywa moc obliczeniową na ich sprawdzenie, wygenerowanie strony z opisem błędu i odesłaniem jej z powrotem.
JavaScript jest nieustannie rozwijany. Podstawowe funkcje z biegiem czasu uzupełniane są o nowe narzędzia, jak chociażby obiekt Image i tablica document.images, które pojawiły się w JavaScripcie 1.1 i wzbogaciły język o opcje manipulacji plikami graficznymi. Wraz z wersją 1.2, koncepcją dynamicznego HTML-a i zastosowaniem warstw możliwości projektowania interaktywnych stron zwiększyły się wręcz zdumiewająco. Obecnie przy użyciu odpowiednich bibliotek – jak na przykład script.aculo.us czy mootools – tworzyć można aplikacje sieciowe o dużym stopniu zaawansowania i atrakcyjnym interfejsie.

Umieszczanie skryptu na stronie

Skrypt JavaScript można umieścić w czterech różnych miejscach:

W treści strony
JavaScript

```
<body>
   <script type="text/javascript">
      <!--
         alert("Hello World!");
      //-->
   </script>
</body>
```

```
<body>
   <script type="text/javascript">
      <!--
         alert("Hello World!");
      //-->
   </script>
</body>
```

W nagłówku strony wewnątrz znaczników <head>
JavaScript

```
<head>
   <script type="text/javascript">
      <!--
         alert("Hello World!");
      //-->
   </script>
</head>
```

```
<head>
   <script type="text/javascript">
      <!--
         alert("Hello World!");
      //-->
   </script>
</head>
```

Skrypty w nagłówku HTML nie wpływają od razu na dokument HTML, lecz mogą się do nich odwoływać inne skrypty. Nagłówek często wykorzystywany jest do umieszczania funkcji – grup instrukcji JavaScriptu, które mogą być użyte jako jedna całość.

W znaczniku HTML
JavaScript

```
<body onLoad="window.alert('Okno komunikatu')">
1
```

```
<body onLoad="window.alert('Okno komunikatu')">
```

Nosi to nazwę funkcji obsługi zdarzenia (ang. event handler) i pozwala na pracę skryptu z elementami HTML. Używając JavaScriptu w funkcjach obsługi zdarzeń, nie musimy używać znacznika <script>.

W osobnym pliku
JavaScript

```
<head>
   <script type="text/javascript" src="plik.js"></script>
</head>

<head>
   <script type="text/javascript" src="plik.js"></script>
</head>
```

JavaScript pozwala korzystać z plików o rozszerzeniu .js zawierających skrypty; można je dołączać do dokumentu HTML, wskazując nazwę pliku w atrybucie src.
Kod źródłowy zagnieżdżony w HTML-u
Kod JavaScript musi być zawarty pomiędzy znacznikami HTML <script> i </script>:

Przykład
Temat: Zagnieżdżanie skryptu

```
<script>
        kod naszego skryptu
</script>
```

Znaczniki takie można umieszczać w dowolnym miejscu dokumentu, ale dobrą praktyką jest osadzanie kodu na początku strony, w sekcji HEAD, która przez przeglądarkę jest wczytywana jako pierwsza. Pozwala to na uniknięcie sytuacji, kiedy użytkownik aktywuje element strony powiązany z kodem, który jeszcze nie został załadowany. Oczywiście możemy w obrębie strony używać znaczników <script> wielokrotnie – na przykład w nagłówku i sekcji BODY.

Znacznik <script> ma **atrybut type** – nadając mu odpowiednią właściwość, definiujemy język, w którym napisany będzie nasz kod. W wypadku JavaScriptu wartością atrybutu będzie oczywiście: „text/javascript”.

Przykład
Temat: Kod HTML strony używającej JavaScript.

```
<html>
        <head>
                <script type="text/javascript">
                        kod skryptu
                </script>
                <script type="text/javascript">
                        przykładowy kod
                </script>
        </head>
<body>
        <script type="text/javascript">
                przykładowy kod
        </script>
</body>
```

</html>

Przykład
Temat: Kod HTML strony używającej JavaScript.

Kod źródłowy zamieszczony w oddzielnym pliku

Bardzo dobrą i ułatwiającą pracę programisty praktyką jest wielokrotne wykorzystywanie napisanego wcześniej kodu. Aby uniknąć każdorazowego przeszukiwania dokumentów, otwierania, kopiowania i wklejania, kod źródłowy skryptu możemy umieścić w osobnym pliku. Będzie to plik tekstowy o rozszerzeniu .JS, zawierający kod pisany już bezpośrednio, bez użycia znaczników <script>. O tym, że kod źródłowy jest w pliku zewnętrznym, informujemy przeglądarkę, wykorzystując atrybut src:

Skladnia:
<script type="text/javascript" src="nazwa_pliku.js"></script>

Przykład
Temat: Zewnętrzny pliku zawierający kod JavaScript. Wyświetlanie datę ostatniej modyfikacji pliku html.

Plik *.html

```
<HTML>
        <HEAD>
                <TITLE>PIERWSZY SKRYPT</TITLE>
        </HEAD>
<BODY>
    <SCRIPT type="text/javascript" src="przyklad0_zewnetrzny.js"></SCRIPT>
</BODY>
</HTML>
```

Plik *.js o nazwie przyklad0_zewnetrzny.js

```
document.write(document.lastModified);
```

Dzięki takiej metodzie postępowania możemy wkrótce stworzyć własną kolekcję najczęściej używanych skryptów, co zaowocuje oszczędnością czasu potrzebnego na tworzenie nowych rozwiązań od podstaw.

Jak zadbać o przeglądarki nieobsługujące JavaScriptu?

Mimo że w zasadzie wszystkie używane dziś przeglądarki bez problemu interpretują kod JavaScript, nie zaszkodzi, jeżeli zadbamy o użytkowników, którzy na przykład wyłączyli obsługę języka w ustawieniach przeglądarki. Zaoszczędzimy im trudnych do przewidzenia zachowań aplikacji lub komunikatów o błędach, umieszczając kod w HTML-owych znacznikach komentarza. Znacznik zamykający komentarz HTML powinniśmy przy tym wzbogacić o sekwencję komentarza JavaScript – //. Unikniemy w ten sposób konfliktów wynikających z faktu, że symbole tego znacznika są w obrębie składni JavaScriptu nieprawidłowe.

Przykład
Temat: Gdy przeglądarka nie obsługuje JavaScript. Kod JavaScript umieszczony pomiędzy znacznikami komentarza

```
<script type="text/javascript">
	<!--
		kod skryptu
	//-->
</script>
```

Dobrą praktyką jest także poinformowanie użytkowników, że strona zawiera skrypty, które nie zostały wykonane przez ich przeglądarkę. W tym celu stosuje się **znaczniki <noscript>**. Pomiędzy nimi powinniśmy umieścić element (np. odpowiedni komunikat), który zostanie wyświetlony zamiast interpretacji kodu. Uwzględniając wszystkie powyższe wskazówki, szablon naszej strony HTML będzie wyglądał następująco:

Przykład
Temat: Kod strony zawierającej JavaScript i użycie znacznika <noscript>

```
<html>
	<head>
		<script type="text/javascript">
			<!--
				kod skryptu
			// -->
		</script>
	</head>
<body>
	<noscript>
		Twoja przeglądarka nie obsługuje JavaScriptu lub wyłączyłeś jego obsługę.
	</noscript>
kod HTML strony
</body>
</html>
```

Komentarze do kodu

Do wyboru mamy dwa typy komentarzy:

- **Komentarz liniowy** zaczyna się od dwóch ukośników, a kończy przy przejściu do następnej linii. Oznacza to, że przeglądarka zignoruje wszystko za znacznikiem // aż do końca linii, w której znacznik ten występuje.
  np. // to jest komentarz liniowy
- **Komentarz blokowy rozpoczyna** się z kolei od sekwencji: /*, a kończy sekwencją: */. Może on więc ciągnąć się przez wiele linii – pamiętajmy jednak, że niemożliwe jest jego zagnieżdżanie, czyli stosowanie jednego komentarza zawartego w innym.
  np. /* komentarz blokowy */

Dzięki komentarzom możemy poinformować użytkownika, że jego przeglądarka nie obsługuje skryptów, nie używając przy tym znacznika <noscript>. Wymieniony w listingu nr 5 przykład zmieni więc swoją postać na:

Przykład
Temat: Kod strony zawierającej w JavaScript z komentarzami.

```
<html>
	<head>
		<script type="text/javascript">
			// Twoja przeglądarka nie obsługuje JavaScriptu.
			/* Aby zobaczyć stronę w pełnej funkcjonalności, zainstaluj inną przeglądarkę
			Internet Explorer, Netscape Navigator, Mozilla, Opera... */
		<!--
			kod skryptu
		// -->
		</script>
	</head>
<body>
	kod HTML strony
</body>
</html>
```

Wyświetlanie informacji na stronie i działania na zmiennych

Kilka podstawowych zasad programowania

- Po pierwsze, interpreter kodu rozróżnia tekst **pisany wielkimi i małymi literami**. Musimy być więc niezwykle ostrożni, aby nie popełnić błędu – na przykład wywołując wcześniej zdefiniowaną funkcję „program()" za pomocą instrukcji „Program()". W takim wypadku przeglądarka zwróci błąd, a początkujący programista może mieć duże problemy ze zidentyfikowaniem jego przyczyny.
- Czynnością należącą już raczej do katalogu dobrych praktyk jest ponadto umieszczanie na **końcu każdej instrukcji średnika** – głównie w sytuacji, gdy w jednej linii tekstu chcemy zawrzeć kilka poleceń. Ma to często miejsce na przykład podczas konstruowania pętli. Przejrzystości kodu służy także umiejętne stosowanie spacji. Odstępy nie są „widoczne" dla interpretera, ale zdecydowanie ułatwiają naszą pracę ze skryptem lub też zapoznanie się innych użytkowników z jego budową.

Instrukcja document.write

Document to obiekt JavaScript, który reprezentuje akurat wyświetlaną stronę, **write** to natomiast jego metoda, czyli funkcja wykonująca określone działania na obiekcie – w tym wypadku wpisująca tekst. Nasz tekst umieszczamy w nawiasach jako argumenty wywołania metody. Postępujemy więc zgodnie z ogólną regułą składni, która wygląda następująco:
obiekt.metoda(argumenty metody);

Przykład
Temat: Zastosowanie metody write

```
document.write("Witaj, świecie!");
```

Przykład
Temat: Metoda write – użycie znaczników HTML

```
document.write("<h1>Strona tytułowa</h1>");
document.write("<b>Spis treści</b>");
```

Jako składników tekstu możemy także używać znaczników HTML, dbając oczywiście o to, żeby w odpowiednich miejscach otwierać je i zamykać:

Uwaga:
Metoda write – użycie cudzysłowów w łańcuchu znaków.
W tym miejscu powinniśmy zwrócić naszą uwagę na pewien istotny fakt: otóż łańcuch znaków przekazywany metodzie write ograniczony jest z obu stron podwójnym cudzysłowem. Co zrobić zatem w wypadku, gdy w tym łańcuchu będziemy chcieli umieścić znaczniki HTML, których atrybuty również używają cudzysłowu? Jeżeli użyjemy symbolu ", popełnimy błąd, gdyż interpreter JavaScriptu przyjmie, że w tym miejscu zakończyliśmy wprowadzanie łańcucha znakowego do metody write. Aby uniknąć wynikających z tego problemów, musimy stosować zamiennie znaki " oraz '. Jeżeli będziemy pamiętali o tym, że powinny się one parami otwierać i zamykać, możemy wielokrotnie zagnieżdżać jedne w drugich, stosując je przemiennie: "1 '2 "3 – 3" 2' 1".

Nieco łatwiej przyjdzie nam rozstrzygnąć problem ewentualnego użycia cudzysłowu w samym tekście, który chcemy umieścić na stronie. Zarówno cudzysłów, jak i inny znak specjalny powinniśmy w takim wypadku poprzedzić backslashem – czyli znakiem \ lub specjalnymi sekwencjami podstawienia HTML.

```
document.write("Witamy na naszej stronie filmowej.");
document.write("Polecamy film \"Władca Pierścieni\"");
document.write("<b><a href='spis.htm'>Spis treści</a></b>");
```

Zmienne

Typy zmiennych:

- **string** →czyli łańcuch znakowy – dlatego jest on umieszczony w cudzysłowie.
- **integer**→ liczbę całkowitą
- **float**→zmiennoprzecinkową

W JavaScripcie nie musimy deklarować, jakiego typu zmiennej będziemy używali.
Ponadto typ zmiennej elastycznie dopasowuje się do naszych potrzeb, dlatego nic nie stoi na przeszkodzie, aby przypisywać na zmianę typ łańcuchowy i liczb całkowitych.

Jedyne, co musimy zrobić, to poinformować interpreter o korzystaniu ze zmiennej.
var nazwa zmiennej = wartość zmiennej;
Przykład
Temat: Zastosowanie instrukcji var

```
var imie="Janek"                                    // zmienna typu string
var wiek=20                                         // zmienna typu integer
document.write("Nasz gość ma na imie "+imie+".");
document.write(imie+" ma "+wiek+" lat");
```

Nazwy zmiennych zaczynamy od litery i konstruujemy je w całości z liter, cyfr lub znaku podkreślenia (_). Powinniśmy też zadbać, aby nazwa była zrozumiała dla osoby czytającej kod – na przykład wiązała się określonym elementem czy funkcją. Wszystko to oczywiście po to, byśmy w przyszłości nie musieli się zastanawiać, z jakich przyczyn daną zmienną w kodzie umieściliśmy.

Operatory

Operatory służą dokonywaniu działań na wartościach zmiennych. Już w powyższym przykładzie zastosowania funkcji var użyliśmy operatora łączenia łańcuchów tekstu – +. Stosując go, możemy wpisywać do ciągu złożone zdania, zmieniające się w zależności od wartości wprowadzanych zmiennych. Należy przy tym pamiętać, że jeśli przy łączeniu różnych typów zmiennych występuje łańcuch znakowy (string) i operator łączenia +, pozostałe zmienne są również przekształcane na typ string.

Ważniejszym jeszcze od manipulacji tekstem zastosowaniem operatorów będą oczywiście działania matematyczne. Możemy przy tym wykorzystać szeroką ich gamę – JavaScript oddaje nam do dyspozycji nie tylko operatory arytmetyczne, ale także logiczne oraz operatory przypisania i porównania. Poniższe tabele w sposób wyczerpujący prezentują wszystkie dostępne nam możliwości.

Operatory arytmetyczne

Operator	Opis	Przykład	Wynik		
+	Dodawanie	x=3; x=x+4	7		
-	Odejmowanie	x=4;x=6-x	2		
*	Mnożenie	x=3;x=x*5	15		
/	Dzielenie	10/5 9/2	2	4.5	
%	Modulo (reszta z dzielenia)	4%3 12%8 8%2	1	4	0
++	Zwiększanie o 1	x=2;x++	x=3		
--	Zmniejszanie o 1	x=4;x--	x=3		

Operatory przypisania

Operator	Przykład	Równoważne z
=	x=y	
+=	x+=7	x=x+7
-=	x-=3	x=x-3
*=	x*=y	x=x*y
/=	x/=y	x=x/y
%=	x%=y	x=x%y

Operatory porównania

Operator	Opis	Przykład	
==	jest równe	2==3	wynik:fałsz
!=	nie jest równe	2!=3	wynik:prawda
>	jest większe	25>3	wynik:prawda
<	jest mniejsze	2<3	wynik:prawda
>=	większe lub równe	25>=3	wynik:prawda
<=	mniejsze lub równe	2<=3	wynik:prawda

Operatory logiczne

Operator	Opis	Przykład

&&	i	x=3 y=4	(x < 9 && y > 2)	wynik:prawda
||	lub	x=3 y=4	(x==8 || y==6)	wynik:fałsz
!	zaprzeczenie	x=3 y=4	!(x==y)	wynik:prawda

KONWERSJA TYPÓW ZMIENNYCH
isFinite(zmienna) - zwraca true jeżeli zmienna ma wartość nieskończoność

isNaN(zmienna) - zwraca true jeżeli zmienna nie jest liczbą ("Not a Number")

Number(zmienna) - sprawdza czy zmienna jest typu liczbowego (zwraca NaN jeżeli nie, zwraca wartość jeżeli tak)

parseFloat(zmienna) - konwersja zmiennej do typu float

parseInt(zmienna) - konwersja zmiennej do typu int (liczbowego)

toString(zmienna) - konwersja zmiennej do typu łańcuchowego

```
var txt = "Hello World!";

document.write("The original string: " + txt);
document.write("<p>Big: " + txt.big() + "</p>");
document.write("<p>Small: " + txt.small() + "</p>");
document.write("<p>Bold: " + txt.bold() + "</p>");
document.write("<p>Italic: " + txt.italics() + "</p>");
document.write("<p>Fixed: " + txt.fixed() + "</p>");
document.write("<p>Strike: " + txt.strike() + "</p>");
document.write("<p>Fontcolor: " + txt.fontcolor("green") + "</p>");
document.write("<p>Fontsize: " + txt.fontsize(6) + "</p>");
document.write("<p>Subscript: " + txt.sub() + "</p>");
document.write("<p>Superscript: " + txt.sup() + "</p>");
document.write("<p>Link: " + txt.link("http://www.w3schools.com") + "</p>");
document.write("<p>Blink: " + txt.blink() + " (works only in Opera)</p>");
```

Funkcje i obiekty
Co to jest funkcja?
Funkcja jest oddzielnym blokiem kodu, który może być wielokrotnie wykonywany w danym programie przez jego wywoływanie. W większości wypadków do funkcji przekazujemy odpowiednie argumenty, aby otrzymać wartość wyjściową wynikającą z zaprogramowanych w funkcji działań. Dobrze jest tworzyć funkcję tak, by wykonywała ona jedno określone zadanie – większe operacje zaplanowane w ramach algorytmu powinniśmy więc rozdzielać pomiędzy kilka, wywoływanych kolejno, funkcji. Dzięki temu stworzymy swego rodzaju „cegiełki", z których zbudujemy następnie cały skrypt, a które (gdy zapiszemy je w oddzielnych plikach – o czym była mowa już wcześniej) będziemy także mogli później wykorzystać w zupełnie innych konfiguracjach czy programach.

Funkcję powinniśmy zdefiniować na początku kodu strony – czyli w sekcji HEAD. Wywołać ją możemy w dowolnym miejscu poniżej, gdy tylko zajdzie taka potrzeba. Dzięki takiemu rozplanowaniu struktury kodu będziemy pewni, że funkcja zostanie załadowana, zanim nastąpi jej wywołanie.

Jak zdefiniować funkcję→skladnia?
Po słowie function wpiszemy zatem nazwę funkcji, a w nawiasach argumenty, jakie chcemy przekazać. Kod wykonywany przez funkcję umieścimy pomiędzy klamrą otwierającą i zamykającą – równoznaczną z końcem funkcji.

Przykład

Temat: Przykładowa funkcja bezargumentowa będzie więc wyglądała tak:

definiowanie

```
function pomoc()
        {
        document.write("<a href=\"pomoc.html\"><img src=\"help.gif\" width=\"15\" height=\"10\"
        alt=\"help\" /></a>");
        }
```

wywołanie

```
document.write("Jeżeli nie wiesz, co zrobić dalej, zobacz pomoc: ");
pomoc()                                                    // to jest wywołanie funkcji w programie
```

Funkcja zwracająca wartość

Przykład
Temat: Funkcję dodającą dwie własności liczby, których wartość będzie przekazywana w argumentach, w trakcie wywołania.

definiowanie funkcji

```
function suma(liczba1, liczba2)
        {
                sumaliczb=liczba1+liczba2        // dodaje liczby i przypisuje nowej zmiennej
                wynik=sumaliczb;
return wynik                                     // zwraca zmienną wynik
}
```

wywołanie funkcji

Aby wywołać funkcję w dalszej części programu, wystarczy jedynie wcześniej przypisać zmiennym odpowiednie wartości:

```
a=1;
b=2;
c=suma(a,b)                    // funkcja zwraca wartość, która jest podstawiana do zmiennej c
document.write("c="+c+" to samo co: "+ suma(a,b));
```

Zasięg zmiennych

Zasięg zmiennej nazywany jest inaczej, obrazowo, czasem jej życia. Określa on, w jakich miejscach kodu interpreter JavaScript zmienną „widzi" i może z niej korzystać. Pod tym względem zmienne dzielimy więc na **lokalne i globalne**.
Zmienne lokalne to takie, które są deklarowane wewnątrz funkcji i które „giną" wraz z zakończeniem jej działania. Zatem zmiennej „sumaliczb", utworzonej w przykładzie, nie będziemy mogli użyć nigdzie poza funkcją „suma".
Inny charakter mają zmienne globalne – deklarowane poza funkcją, są dostępne od momentu ich pierwszego użycia aż do końca kodu HTML.

Funkcje predefiniowane JavaScriptu

Okienka dialogowe

Javascript udostępnia nam trzy typy okien dialogowych:

Z jednej strony okienka takie są łatwe we wdrożeniu i skuteczne (bo gdy się pojawią, nie pozwalają kliknąć niczego innego na stronie), z drugiej strony bardzo ładnie odciągają uwagę odwie własnościdzającego od zawartości strony. Nie odradzam ich używania. Po prostu czasami trzeba się zastanowić, czy nie ma lepszej metody, by poinformować użytkownika o danym zdarzeniu. Przykładowo zamiast pokazywać alerta z informacją o źle wpisanym mailu, lepiej jest wyświetlić obok takiego pola czerwony znaczek czy coś symbolizującego, że ów pole jest źle wypełnione. Użytkowanie strony będzie wtedy o wiele przyjemniejsze.
Formatowanie tekstu w okienkach dialogowych

W oknach dialogowych można formatować tekst za pomocą znaków:

\n - służący do łamania linii - jak użycie ENTERA w edytorze tekstu.
\t - służący do wstawiania znaku tabulacji.

Jedynym wymogiem jest to, że znaki te muszą znajdować się między podwójnymi cudzysłowami ("...") a nie między apostrofami ('...') - na przykład "To jest łamany\ntekst.".
okienko typu alert()

Metoda alert ma postać

alert('treść komunikatu');

Po wywołaniu metody alert() zostaje wyświetlone okienko z komunikatem i przyciskiem OK. Po kliknięciu przycisku nie zostaje zwrócona żadna wartość, tak więc okno to nadaje się tylko do informowania użytkownika o pewnych zdarzeniach. Zaznaczyć trzeba, że nie ma możliwości zmiany tytułu tego okna (jest zależny od przeglądarki), a tylko treści jakie to okienko wyświetla.

```
<input type="button" id="alert" value="Okienko Alert">

<script type="text/javscript">
   function oknoAlert() {
      alert('To jest okienko alert');
   }

   document.getElementById('alert').onclick = function() {
      oknoAlert()
   }
</script>
```

Za pomocą metody getElementById pobraliśmy input i przypisaliśmy do niego zdarzenie onclick, które odpala funkcję pokazującą okienko alert

okienko typu confirm()

Metoda confirm ma postać:

confirm('treść komunikatu');

Można powiedzieć, że okienko confirm służy do tego, aby użytkownik coś potwierdził (stąd nazwa confirm). Do metody confirm() przekazywany jest tylko jeden parametr zawierający treść jaka będzie

wyświetlana w okienku. Okienko to wyświetla dwa guziki: [OK] i [ANULUJ] (CANCEL). W zależności od tego który guzik został kliknięty przez użytkownika, metoda ta zwróci true lub false.

```
<input type="button" id="confirm" value="Okienko Confirm">

<script type="text/javascript">
   function oknoConfirm() {
      if (confirm('Czy jesteś pewien, że chcesz kontynuować?')) {
         alert('No to kontynuuj...');
      } else {
         alert('Przykro mi, że nie chcesz kontynuować...');
      }
   }

   document.getElementById('confirm').onclick = function() {
      oknoConfirm()
   }
</script>
```

okienko typu prompt()

```
prompt('treść komunikatu', 'domyślna wartość');
```

Do metody prompt przekazujemy dwa parametry - jeden jest treścią, która będzie wyświetlana w okienku, a drugi jest domyślną wartością, która będzie wyświetlana w polu, w które wpisujemy tekst. Okienko to wyświetla dwa guziki: [OK] i [ANULUJ] (CANCEL). Jeżeli użytkownik kliknie [OK], to zostanie zwrócona wartość z pola tekstowego znajdującego się w tym okienku (lub też 'domyślna wartość', jeżeli użytkownik nie zmieniał zawartości tego pola). Jeżeli użytkownik kliknie [ANULUJ] (CANCEL), to zostanie zwrócona wartość null.

```
<input type="button" id="prompt" value="Okienko Prompt" />

<script type="text/javascript">
   function oknoPrompt() {
   var imie = prompt('Podaj swoje imię:', 'Kartofel');
      if (imie != null) {
         alert('Witaj ' + imie);
      } else {
         alert('Anulowałeś akcję');
      }
   }

   document.getElementById('prompt').onclick = function() {
      oknoPrompt()
   }
</script>
```

Okienko **alert** – wyświetlająca okienko dialogowe z informacją i przyciskiem OK,
Okienko **prompt** – generująca okno, w którym użytkownik może wpisać wartość, zwracaną następnie do programu.
Okienko **confirm** służy do tego, aby użytkownik coś potwierdził (stąd nazwa confirm).

Kod przedstawiony na poniższym przykładzie wyświetli więc komunikat powitalny, a po podaniu przez użytkownika imienia umieści w zawartości strony odpowiednią treść:

```
alert("Witam na mojej stronie");
document.write("Miło mi Cię gościć "+ prompt("Podaj imię:") +" na mojej stronie");
```

Instrukcje warunkowe i pętle

Instrukcja warunkowa if
Instrukcje warunkowe są jednym z podstawowych elementów tworzących skomplikowane systemy interakcji z użytkownikiem na potrzeby współczesnych aplikacji online. Zasadą ich działania jest to, że polecenia w nich zawarte będą wykonywane, jeżeli umieszczony na wstępie warunek okaże się prawdziwy. Składnia jest więc w tym wypadku następująca: if (warunek) {kod wykonywany, gdy warunek zostanie spełniony}.

Przykład
Temat: Wykorzystanie instrukcji warunkowej if
W oparciu o ten schemat możemy na przykład stworzyć funkcję zamykającą dostęp do strony osobom niepełnoletnim:

```
wiek=prompt("W jakim jesteś wieku?");
if (wiek>18) {
document.write("Jesteś pełnoletni, więc możesz wejść dalej.");
document.write("<br><a href=\"adult.html\">Wejście dla dorosłych</a>");
}
```

Instrukcja warunkowa if ... else
Składnia zyskuje więc postać:

```
if (warunek)
        {kod wykonywany, gdy warunek zostanie spełniony}
                else
        {kod wykonywany, gdy warunek nie zostanie spełniony}.
```

Przykład
Temat: Wykorzystanie instrukcji warunkowej if ... else

Aby sprawdzić działanie tej instrukcji w praktyce, możemy na przykład stworzyć kod, który skontroluje stan naszej pamięci:

```
odpowiedz=prompt("W którym roku narodził się JavaScript?");

if (odpowiedz=="1995")
        {
        document.write("Brawo! Masz dobrą pamięć!");
        }
                else
        {
        alert("Źle!!!");
        document.write("Nie zapamiętałeś dokładnie pierwszej części kursu?");
        }
```

Konstrukcja switch

Jeśli chcemy mieć jeszcze więcej możliwości określenia zachowania programu przy zróżnicowanych wartościach wejściowych, użyjemy konstrukcji switch. Pozwala ona wykonać jeden z wielu bloków kodu, w zależności od różnych wartości zmiennej. Jej składnia ma postać:

```
switch (zmienna)
{
case wartosc1:
kod wykonywany, gdy zmienna ma wartość wartosc1
break
case wartosc2:
kod wykonywany, gdy zmienna ma wartość wartosc2
break
// ... itd
case wartoscX:
kod wykonywany, gdy zmienna ma wartość wartoscX
break
default:
kod wykonywany, gdy zmienna nie przyjmuje żadnej z powyższych wartości
}
```

Przykład:
Temat:Wykorzystując obiekt Date zwracający datę, możemy uzależnić wygląd tekstu wyświetlanego na stronie od dnia tygodnia:

```
var dzis=new Date()                    // tworzony jest obiekt z datą
dzien=dzis.getDay()                    // wiemy, jaki jest dzień, na podstawie daty
switch (dzien);
{
        case 5:
        document.write("Wreszcie piątek!");
        break;
        case 6:
        document.write("Imprezowa sobota");
        break;
        case 0:
        document.write("Śpiąca niedziela");
        break;
        default:
        document.write("Kiedy wreszcie będzie weekend?!");
}
```

Operator warunkowy
Operator ten pozwala wielokrotnie zaoszczędzić czas programisty i kilka linijek skryptu, gdyż doskonale zastępuje prostą konstrukcję if ... else. W wypadku operatora warunkowego w zależności od zrealizowania warunku uzyskamy pojedynczą reakcję.
Jeśli zostanie on spełniony, zmiennej przypisana będzie jedna wartość, jeżeli nie – druga:

```
zmienna=(warunek)?wartoscTRUE:wartoscFALSE.
```

Przykład
Temat: Sprawdzanie, czy podana przez użytkownika liczba jest parzysta:

```
liczba=prompt("podaj jakąś liczbę:");
jaka=(liczba%2==0)?"parzysta":"nieparzysta";
document.write("podana liczba jest "+ jaka);
```

Pętla while

Zastosowanie pętli pozwala wykonywać kod przez określoną liczbę powtórzeń, aż do spełnienia zdefiniowanego warunku.

W wypadku pętli while polecenia są wykonywane przez cały czas, gdy warunek jest spełniony:

```
while (warunek)
        {
        kod pętli
        }
```

Przykład

Temat: W poniższym przykładzie zostaną wyświetlone po kolei wszystkie litery od „a" do „y". Litera „z" nie ukaże się, gdyż warunek straci status spełnionego i wykonywanie kodu z wnętrza pętli while zostanie zatrzymane:

```
litera="a";
while(litera!="z")
        {
        document.write(litera);
        kodLitery=litera.charCodeAt()                   // pobierany jest kod ASCII litery
        kodLitery++                                     // zwiększamy kod o 1
        litera=String.fromCharCode(kodLitery)                   // tworzymy literę z kodu ASCII
        }
```

Pętla do ... while

Konstrukcja tej pętli jest bardzo podobna do poprzednio opisanej while, z tym że kod w niej umieszczony zawsze **musi być wykonany co najmniej raz** – nawet gdy warunek nie zostanie spełniony. Jest to związane z tym, że warunek interpreter sprawdza dopiero po wykonaniu poleceń – składnia ma bowiem postać

```
do
        {
                kod wykonywany w pętli
        }
while (warunek).
```

Przykład

Sprawdzanie czy konwersja wczytanej liczby nastąpiła poprawnie.

```
do
         {
           var liczba_konc = prompt('Podaj liczbę całkowitą końcową=','');
               k=parseInt(liczba_konc);
         }
        while (isNaN(p)==true);
```

Pętla for

Pętla for może być wykorzystana do wykonania pewnego kodu przez określoną liczbę powtórzeń.

Jej składnia ma postać:

```
for (inicjalizacja_zmiennej; warunek; zmiana_zmiennej)
{
kod wykonywany w pętli
}
```

Określana jest więc początkowa postać zmiennej, wartość, do czasu osiągnięcia której pętla będzie wykonywana, oraz „stopień" zmiany zmiennej w każdej iteracji.

Przykład

Temat: W poniższym przykładzie posłużymy się pętlą do ... while, aby „zmusić" użytkownika do podania liczby z przedziału od 1 do 100, a następnie wyświetlimy graficzny, różnokolorowy słupek, obrazujący stosunek pomiędzy częścią przez niego wybraną a pozostałą:

```
do
        {
        procent=parseInt(prompt("Podaj procent zawartości (0-100):",""));
        }
 while (procent<0 | procent>100 | isNaN(procent));

/* parseInt zamienia wartość, którą podał użytkownik, na liczbę całkowitą; pętla wykonuje się dotąd,
aż użytkownik wpisze liczbę z przedziału 0-100; jeżeli użytkownik wpisze inny znak lub ciąg znaków,
wówczas operacja parseInt zamienia taki ciąg na wartość NaN (Not a Number) - co jest również
sprawdzane w jednym z warunków działania pętli */

slupek="<font color=\"#00FF00\">";

for (i=0;i<=procent;i++)
        {
        slupek+="|";
        }
slupek+="</font><font color=\"#0000FF\">";
for (i=procent+1;i<=100;i++)
        {
        slupek+="|";
        }

slupek+="</font>";
document.write("<br>"+ slupek +" = "+ procent +"%");
```

Rdzenne Obiekty JavaScriptu

Obiekt Array

Array to po prostu tablica – wykorzystując ją, możemy przechowywać zbiory wartości, kolejno ułożonych, do których odwołujemy się przez nazwę tablicy i indeks.

Tablicę tworzymy, korzystając z operatora new.
Do dyspozycji mamy kilka sposobów:

```
var owoce=new Array("Gruszka","Banan","Ananas");

var owoce=new Array(3);
owoce[0]="Gruszka";
owoce[1]="Banan";
owoce[2]="Ananas";

sok=new Array();
sok.smak="Pomidorowy";
sok.pojemnosc="1 litr";
sok.producent="Pomidorek";
sok.length                                    // zwrócony wynik=3;
```

Dana jest tablica myArray i pięciu elementach. W trakcie wykonywania programu konieczne może się okazać usunięcie niektórych elementów z tablicy – tak jak na poniższym przykładzie. Stosując właściwość length, zobaczymy jednak, że usunięcie elementu z tablicy **nie** powoduje skrócenia jej długości:

```
myArray.length                                // wynik 5
delete.myArray[2]
myArray.length                                // wynik 5
myArray[2] // wynik: undefined
```

Listing 27 – usuwanie elementów z tablicy

Posługiwanie się tablicami w sposób bardziej zaawansowany wymaga użycia określonych właściwości i metod. Umieszczona poniżej lista pozwoli z pewnością zrealizować wszystkie działania dotyczące przechowywania i modyfikowania danych, jakie tylko mogą przyjść na myśl początkującemu programiście JavaScriptu:

Właściwości

nazwa	opis
constructor	Zawiera funkcję tworzącą prototyp obiektu.
length	Zwraca liczbę elementów w tablicy.
prototype	Pozwala na dodanie właściwości do tablicy.

Metody

nazwa	opis
.concat(obiektArray2)	Łączy dwie własności lub więcej tablic i zwraca nową.
.join("separ")	Zwraca łańcuch znaków elementów tablicy, przedzielonych separatorem.
.pop()	Usuwa i zwraca ostatni element tablicy.
.push("el1"[,"el2"])	Dodaje element lub więcej na koniec tablicy i zwraca nową długość.
.reverse()	Zwraca tablice z elementami w odwrotnej kolejności.
.slice(indexPocz [,indexKońc])	Zwraca tablicę utworzoną z części danej.

.sort([funkcjaPorównawcza])	Zwraca tablicę posortowaną.
.splice(index,ile [,el1, el2])	Dodaje i/lub usuwa elementy tablicy.
.toSource()	Zwraca łańcuch znaków, który reprezentuje źródło tablicy.
.toString()	Zwraca łańcuch znaków, który reprezentuje daną tablicę.
.unshift("el1"[,"el2"])	Dodaje jeden lub kilka elementów na początek tablicy i zwraca nową długość.
.valueOf()	Zwraca wartość tablicy.

Obiekt Boolean

Obiekt ten, zgodnie ze swoją nazwą, służy do tworzenia i przechowywania wartości logicznych „prawda” lub „fałsz”. Podobnie jak w wypadku konstruowania tablicy, mamy do wyboru kilka możliwości stworzenia obiektu, którego wartość logiczna wynosi „fałsz” bądź „prawda”:

```
//tworzymy obiekt, którego wartość logiczna to fałsz
var b1=new Boolean();
var b2=new Boolean(0);
var b3=new Boolean(null);
var b4=new Boolean("");
var b5=new Boolean(false);
var b6=new Boolean(NaN);

//tworzymy obiekt, którego wartość logiczna to prawda
var b1=new Boolean(true);
var b2=new Boolean("true");
var b3=new Boolean("false");
var b4=new Boolean("prawda");
var b5=new Boolean("fałsz");
var b6=new Boolean("Janek");
```

Obiekt Date

Posługiwanie się datą i czasem w wypadku wielu skryptów okazuje się niezbędne. Przydatność tego typu danych zdążyliśmy już wykazać w jednym z poprzednich przykładów. Teraz przyjrzyjmy się dokładniej obiektowi Date, który w JavaScripcie odpowiada za całość operacji dotyczących czasu. Konstruując go, możemy użyć określonych parametrów lub też tego zaniechać – wówczas zostanie zwrócona aktualna data:

```
// bez parametrów – zostanie zwrócona aktualna data
var today=new Date();

//liczba milisekund od 1 stycznia 1970 roku
var someDate=new Date(milisekundy);

//data w formacie miesiąc dd, rrrr gg:mm:ss
var someDate=new Date("Jan 13, 1982 21:12:45");

//data w formacie miesiąc dd, rrrr
var someDate=new Date("Jun 23, 1998");

//data w formacie rrrr,mm(od 0 do 11),dd,gg,mm,ss
var someDate=new Date(1981,11,28,14,15,45);

//data w formacie rrrr,mm,dd
```

var someDate=new Date(1985,10,11);

Skuteczne i wydajne operowanie obiektem Date wymaga jeszcze zapoznania się z metodami, za pomocą których będziemy mogli manipulować jego zawartością. Należy także pamiętać o tym, że w wypadku obliczeń dokonywanych na datach najlepiej posługiwać się wartościami milisekund.

Metody

nazwa	opis
.getTime()	Liczba milisekund od 1970-01-01 od godziny 00:00:00 czasu Greenwich.
.getTimezoneOffset()	Zwraca różnicę czasu lokalnego i GMT.
.getYear()	Określony rok minus 1900 – czterocyfrowe oznaczenia dla roku 2000 i kolejnych.
.getFullYear()	Rok w postaci czterocyfrowej.
.getUTCFullYear()	Rok w postaci czterocyfrowej czasu uniwersalnego.
.getMonth()	Miesiąc roku (0–11).
.getUTCMonth()	Miesiąc roku czasu uniwersalnego.
.getDate()	Dzień miesiąca (1–31).
.getUTCDate()	Dzień miesiąca czasu uniwersalnego.
.getDay()	Dzień tygodnia (niedziela=0) (0–6).
.getUTCDay()	Dzień tygodnia czasu uniwersalnego.
.getHours()	Godzina dnia w zapisie 24-godzinnym (0–23).
.getUTCHours()	Godzina dnia czasu uniwersalnego.
.getMinutes()	Minuta godziny (0–59).
.getUTCMinutes()	Minuta godziny czasu uniwersalnego.
.getSeconds()	Sekunda minuty (0–59).
.getUTCSeconds()	Sekunda minuty czasu uniwersalnego.
.getMilliseconds()	Milisekunda sekundy (0–999).
.getUTCMilliseconds()	Milisekunda sekundy czasu uniwersalnego.
.setTime(val)	Ustawia liczbę milisekund od 1970-01-01, od godziny 00:00:00 czasu Greenwich.
.setYear(val)	Ustawia rok minus 1900 – czterocyfrowe oznaczenia dla roku 2000 i kolejnych.
.setFullYear(val)	Ustawia rok w postaci czterocyfrowej.
.setUTCFullYear(val)	Ustawia rok w postaci czterocyfrowej czasu uniwersalnego.
.setMonth(val)	Ustawia miesiąc roku (0–11).
.setUTCMonth(val)	Ustawia miesiąc roku czasu uniwersalnego.
.setDate(val)	Ustawia dzień miesiąca (1–31).
.setUTCDate(val)	Ustawia dzień miesiąca czasu uniwersalnego.
.setDay(val)	Ustawia dzień tygodnia (niedziela=0) (0–6).
.setUTCDay(val)	Ustawia dzień tygodnia czasu uniwersalnego.
.setHours(val)	Ustawia godzinę dnia w zapisie 24-godzinnym (0–23).
.setUTCHours(val)	Ustawia godzinę dnia czasu uniwersalnego.
.setMinutes(val)	Ustawia minutę godziny (0–59).
.setUTCMinutes(val)	Ustawia minutę godziny czasu uniwersalnego.
.setSeconds(val)	Ustawia sekundę minuty (0–59).
.setUTCSeconds(val)	Ustawia sekundę minuty czasu uniwersalnego.
.setMilliseconds(val)	Ustawia milisekundę sekundy (0–999).
.setUTCMilliseconds(val)	Ustawia milisekundę sekundy czasu uniwersalnego.
.parse()	Zwraca łańcuch znaków z datą – jako liczbę milisekund od 1 stycznia 1970 00:00:00.
.toGMTString()	Zwraca łańcuch znaków z datą ustawioną na GMT.
.toLocaleString()	Zwraca łańcuch znaków z datą lokalną.
.toString()	Zwraca łańcuch znaków z datą.

Obiekt Math

przykład dla wylosowania pewnej liczby powinniśmy użyć polecenia:

liczba=Math.random();

Stałe

symbol	opis
E	Podstawa logarytmu naturalnego – liczba e.
LN2	Logarytm naturalny z 2.
LN10	Logarytm naturalny z 10.
LOG2E	Logarytm z e przy podstawie 2.
LOG10E	Logarytm z e przy podstawie 10.
PI	Liczba Pi.
SQRT1_2	Odwrotność pierwiastka kwadratowego z 2.
SQRT2	Pierwiastek kwadratowy z 2.

Metody

nazwa	opis
.abs(x)	Wartość bezwzględna liczby.
.acos(x)	Arcus cosinus liczby.
.asin(x)	Arcus sinus liczby.
.atan(x)	Arcus tangens liczby.
.atan2(y,x)	Zwraca kąt od osi x do punktu y.
.ceil(x)	Zwraca najbliższą liczbę całkowitą większą lub równą x.
.exp(x)	Zwraca e do potęgi x.
.floor(x)	Zwraca najbliższą liczbę całkowitą mniejszą lub równą x.
.log(x)	Logarytm naturalny z x.
.max(x,y)	Zwraca większą z dwóch liczb.
.min(x,y)	Zwraca mniejszą z dwóch liczb.
.pow(x,y)	Zwraca x do potęgi y.
.random()	Zwraca losową liczbę rzeczywistą z przedziału 0–1.
.round(x)	Zaokrągla x do liczby całkowitej.
.sqrt(x)	Pierwiastek liczby.
.sin(x)	Sinus liczby.
.cos(x)	Cosinus liczby.
.tan(x)	Tangens liczby.

Obiekt Number

Obiekt Number pozwala, na wyższym stopniu wtajemniczenia, na bardziej precyzyjne operowanie na liczbach – ich przechowywanie i wyświetlanie. Oprócz przypisywanych wartości obiekt ten zawiera kilka predefiniowanych właściwości, określających na przykład nieskończoność i ujemną nieskończoność. Aby stworzyć obiekt Number, używamy polecenia: new Number(liczba).

Stosując obiekt Number oraz odnoszące się do niego metody, możemy używać zapisu:

- dziesiętnego,
- binarnego,
- ósemkowego
- szesnastkowego.

Musimy jednak pamiętać, że

- wartości dziesiętne nie mogą zaczynać się od zer wiodących,
- szesnastkowe powinny zaczynać się od „0x” lub „0X”,
- ósemkowe natomiast rozpoczynają się zerem wiodącym, a przy ich opisywaniu należy korzystać z cyfr od 0 do 7.
- Liczby możemy także zapisywać z wykładnikiem, na przykład „1e6 = 1*10^6”.

Poniższy przykład obrazuje możliwość konwersji liczby z pierwotnego formatu na ciąg binarny lub szesnastkowy:

```
var x=30;
var y=x.toString(16)                    // wynik="1e"
var z=x.toString(2)                     // wynik="11110"
```

Właściwości

nazwa	opis
MAX_VALUE	Określa największą możliwą do przedstawienia liczbę.
MIN_VALUE	Określa najmniejszą możliwą do przedstawienia liczbę.
NaN	Specjalna wartość określająca, że dane wyrażenie nie jest liczbą.
NEGATIVE_INFINITY	Specjalna wartość określająca nieskończoność.
POSITIVE_INFINITY	Specjalna wartość określająca ujemną nieskończoność.

Metody

nazwa	opis
.toExponential(precyzja)	Zwraca łańcuch znaków określających liczbę w notacji wykładniczej. Opcjonalny argument precyzja oznacza liczbę miejsc po przecinku.
.toFixed(precyzja)	Zwraca łańcuch znaków określających liczbę w notacji dziesiętnej. Opcjonalny argument precyzja oznacza liczbę miejsc po przecinku.
.toPrecision(precyzja)	Zwraca łańcuch znaków określających liczbę w notacji dziesiętnej, z dokładnością wyznaczoną argumentem precyzja.
.toString(system)	Zwraca łańcuch znaków reprezentujący daną liczbę. Argument system określa w tym wypadku rodzaj zapisu – dziesiętny (domyślny), dwójkowy (2), ósemkowy (8) lub szesnastkowy (16).
.valueOf	Zwraca pierwotną wartość określonego obiektu.

Obiekt String

Obiekt String to po prostu ciąg znaków. Przy jego tworzeniu możemy więc wykorzystać poznane wcześniej metody „tradycyjne” lub skorzystać z konstruktora postaci: new String("łańcuch znaków"). Podobnie jak w wypadku obiektu Number, przypisane do String metody pozwalają z dużą precyzją operować na danych – tym razem łańcuchach znaków. Wykorzystując je, możemy nie tylko zmieniać wartości ciągów (patrz: „Metody rozbioru łańcuchów”), ale także dodawać do nich określone znaczniki formatujące (patrz: „Metody formatujące łańcuchy”).

Znaki specjalne

\"	podwójny cudzysłów
\'	pojedynczy cudzysłów (apostrof)
\\	ukośnik lewy
\b	backspace
\t	znacznik tabulacji
\n	znak nowej linii
\r	znak powrotu karetki
\f	znak przewinięcia strony

Właściwości

nazwa	opis	uwagi
length	Zwraca długość łańcucha (int).	tylko odczyt
prototype	Obiekt – prototyp.	odczyt i zapis;

Metody rozbioru łańcuchów

nazwa	opis	uwagi
.charAt(n)	Zwraca znak na pozycji n w łańcuchu	np. ostatni znak: napis.charAt(napis.length-1)

.charCodeAt(n) Zwraca kod znaku znajdującego się na pozycji (w indeksie) n w ciągu.
String.fromCharCode(kod1[,kodn]) Zwraca znaki, których kody zostały przekazane w argumentach.
.concat(napis2) Zwraca połączony łańcuch znaków, np. "abc".concat("def") da wynik "abcdef".
.indexOf(szukane[,indexPocz]) Zwraca numer indeksu łańcucha, w którym występuje ciąg szukane – gdy łańcuch nie zostanie odnaleziony, zwracane jest -1. Opcjonalny parametr indexPocz określa początkowe miejsce w indeksie, od którego zaczyna się szukanie.
.lastIndexOf(szukane[,indexPocz]) Przeszukiwanie w tym wypadku rozpoczyna się od końca, także od podanego indeksu w kierunku początku łańcucha, na przykład polecenie "banany".lastIndexOf("a",2) da wynik równy 1.
.match(wyrReg) Zwraca tablicę łańcuchów spełniających określone w wyrażeniu regularnym kryteria.
.replace(wyrReg, nowy_lancuch) Podmienia nowym łańcuchem ciągi odpowiadające wyrażeniu regularnemu.
.search(wyrReg) Zwraca liczbę całkowitą określającą pozycję wyrażenia regularnego w łańcuchu.
.slice(indexPocz[, indexKońc]) Działa podobnie jak substring() – polecenia łańcuch.substring(4, (łańcuch.length-2)) oraz łańcuch.slice(4, -2) dadzą ten sam wynik. NN4 IE4
.split("znakOddzielający" [,maksymalnaLiczba]) Zwraca tablicę oddzielonych od siebie elementów.
.substr(początek[,długość]) Podobnie jak substring – drugi parametr określa długość łańcucha.

.substring(indexA, indexB) Zwraca łańcuch znaków od indexA do indexB.
.toLowerCase() Zmienia litery na małe.
.toUpperCase() Zmienia litery na wielkie.

Metody formatujące łańcuchy

sposób użycia	wynik
napis.anchor("abc")	<A NAME="abc">napis</A>
napis.blink()	<BLINK>napis</BLINK>
napis.bold()	<B>napis</B>
napis.fixed()	<TT>napis</TT>
napis.fontcolor("red")	<FONT COLOR="red">napis</FONT>
napis.fontsize(2)	<FONT SIZE="2">napis</FONT>
napis.italics()	<I>napis</I>
napis.link("http://www.w3c.org")	<A HREF="http://www.w3c.org">napis</A>
napis.big()	<BIG>napis</BIG>
napis.small()	<SMALL>napis</SMALL>
napis.strike()	<STRIKE>napis</STRIKE>
napis.sub()	_{napis}
napis.sup()	^{napis}
escape("Ala ma\nkota")	Ala%20ma%0Akota
unescape("Ala%20ma%20kota")	Ala ma kota

Zdarzenia elementów HTML-a

Zdarzenia są to akcje podejmowane przez przeglądarkę w odpowiedzi na działania użytkownika bądź ogólnie – na zmianę stanu dokumentu w danym momencie (np. załadowanie się obrazka czy kodu strony). JavaScript daje nam możliwość kontrolowania i odpowiadania na większość zdarzeń, z jakimi możemy się spotkać w trakcie korzystania z naszej witryny. Najczęściej wykorzystywane zdarzenia przedstawione są poniżej. Należy jedynie pamiętać, że dla prawidłowego korzystania z tych dobrodziejstw
JavaScriptu należy dobrze poznać tak zwany obiektowy **model dokumentu HTML – czyli DOM.** Określa on sposoby dostępu oraz zmiany zawartości i parametrów poszczególnych elementów struktury strony. Dzięki niemu do dokumentów HTML mogą mieć dostęp wszystkie typy przeglądarek i języków programowania. Zamieszczanie w tym miejscu pełnego opisu elementów DOM mija się z celem. Wszystkie potrzebne informacje zainteresowani programiści JavaScriptu mogą znaleźć pod adresami:

http://www.w3schools.com/htmldom/default.asp
http://www.w3schools.com/js/js_obj_htmldom.asp

Zdarzenia JavaScript

zdarzenie	opis	z obiektami
onclick	Zachodzi, gdy klikamy obiekt.	button, checkbox, image, layer, link, radio, reset, submit
onmouseover	Zachodzi, gdy przesuwamy wskaźnik myszy nad obiekt.	image, layer, link
onmouseout	Zachodzi, gdy przesuwamy wskaźnik myszy znad obiektu.	image, layer, link
onmousedown	Zachodzi, gdy nie zwalniamy przycisku myszy, a jej wskaźnik znajduje się nad obiektem.	image, layer
onmouseup	Zachodzi, gdy zwalniamy przycisk myszki, a jej wskaźnik znajduje się nad obiektem.	image, layer
onblur	Zachodzi, gdy opuszczamy obiekt (przejście na inny obiekt).	password, select, text, textarea

onfocus	Zachodzi, gdy aktywujemy obiekt.	password, select, text, textarea
onchange	Zachodzi, gdy opuszczamy obiekt, którego zawartość została zmieniona.	password, select, text, textarea
onselect	Zachodzi, gdy zaznaczamy tekst wewnątrz pola tekstowego.	password, text, textarea
onsubmit	Zachodzi, gdy wysyłamy formularz.	form
onreset	Zachodzi, gdy resetujemy formularz.	form
onload	Zachodzi w momencie załadowania dokumentu do przeglądarki.	document, frame
onunload	Zachodzi w momencie opuszczania strony.	document, frame
onabort	Zachodzi, gdy przerwane zostaje ładowanie strony.	document, frame
onerror	Zachodzi, gdy przeglądarka nie może załadować obrazu.	image

Okna i okienka

Uwagi wstępne:

Uwaga 1

W przeszłości bardzo często w linkach stosowało się atrybut target="_new", który nakazywał otwierać strony w nowym oknie. W3C na szczęście stwierdziło, że nie ma podstawy używania tego atrybutu.

Uwaga 2

Dzisiejsze wszystkie [dobre] przeglądarki pozwalają na otwieranie stron:

- w zakładkach ,
- nową kartę,
- okno ,
- widok.

Uwaga 3

Za pomocą Javascriptu możemy otwierać nowe okna, formatować ich wygląd, przesyłać między nimi informacje itp.

Uwaga 4

Pamiętaj, że większość przeglądarek domyślnie blokuje wszelkie próby otwarcia nowego okna, dlatego lepiej nie uzależniać działania naszej strony od okienek. O wiele lepszym pomysłem jest używanie dynamicznie pozycjonowanych warstw (np DIV).

Otwieranie nowego okna

Standardowa konstrukcja otwierająca okno ma postać:

```
window.open('url do strony','nazwa strony','ustawienia nowego okna')
```

1. W **miejscu url** do strony wstawiamy adres do strony która ma być umieszczona w nowo otwartym oknie.
2. W **miejscu nazwa strony** podajemy nazwę strony, którą będziemy wykorzystywać przy korzystaniu z atrybutu target (mało użyteczne). Ma ona znaczenie gdy chcemy korzystać z atrybutu target. W miejscu ustawienia nowego okna umieszczamy opcjonalnie ustawienia nowo powstałego okna.
3. Poniżej zamieszczam wszystkie możliwe **ustawienia okna:**

Directories	- wartość yes lub no (1 lub 0): pokazuje lub ukrywa przyciski katalogów	
Location	- wartość yes lub no (1 lub 0): pokazuje lub ukrywa pasek adresowy	
Menubar	- wartość yes lub no (1 lub 0): pokazuje lub ukrywa menu przeglądarki	
Resizable	- wartość yes lub no (1 lub 0): określa czy okno może zmienić rozmiar	
Scrollbars	- wartość yes lub no (1 lub 0): pokazuje lub ukrywa paski przewijania	
Status	- wartość yes lub no (1 lub 0): pokazuje lub ukrywa pasek statusu	
Toolbar	- wartość yes lub no (1 lub 0): pokazuje lub ukrywa standardowy pasek narzędzi	
Height	- wartość w pixelach	: ustawia wysokość okna
Width	- wartość w pixelach	: ustawia szerokość okna
Top	- wartość w pixelach	: ustawia położenie okna względem góry ekranu
Left	- wartość w pixelach	: ustawia położenie okna względem lewej strony ekranu

Wszystkie powyższe właściwości są typu boolean. Oznacza to, że jeżeli nie podamy ich wartości (0 lub 1), to domyślnie przypisana zostanie im wartość true (1). W niektórych przeglądarkach trzeba

zwracać uwagę, by przy wypisywaniu kolejnych właściwości nie używać spacji. Może to powodować błędy.

Przykłady

```
function okno()
{
var NoweOkienko = window.open('strona.html', 'strona',
'toolbar=0,menubar=0,scrollbars=0,resizable=0,status=0,location=0,directories=0,top=20,left=20,
height=300,width=400');
}

<input type="button" value="Pokaz Okno" id="guzikOkna" />
<script type="text/javascript">
document.getElementById('guzikOkna').onclick = function() { okno('strona.html') }
</script>
```

Modyfikacja okna

Dzięki przypisaniu nowego okna pod zmienną, możemy nim w łatwy sposób manipulować. Zasady są identyczne jak dla skryptów w głównym oknie przeglądarki - jedyna różnica polega na odwołaniu:

```
window.document //dokument w oknie przeglądarki

NoweOkienko.document //dokument w nowym oknie
NoweOkienko.document.getElementsByTagName('body')[0] //pobieramy sekcję body w nowym
okienku
NoweOkienko.document.getElementsByTagName('p').style.fontSize = '20px' //ustawiam wiekość
czcionki dla wszystkich akapitów w nowym oknie
NoweOkienko.width = '400'; //ustawiam szerokość okienka na 400px
NoweOkienko.moveTo(100,100); //przesuwam okienko do pozycji 100x100
NoweOkienko.close() //zamyka okienko
```

Jak widać powyżej otwarte okno możemy dodatkowo formatować za pomocą metod i właściwości.
Okna on-the-fly

Dzięki temu, że możemy odwoływać się do dokumentu w nowym oknie, możemy dynamicznie generować jego treść:

```
function noweOkno() {
   var noweOkno = window.open('', 'okno', 'toolbar=no,location=no,width=200,height=200');
   with (noweOkno) {
      document.writeln('<html>');
      document.writeln('<head>');
      document.writeln('<title>Tytuł okienka</title>');
      document.writeln('<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />');
      document.writeln('</head>');
      document.writeln('<body>');
      document.writeln('<p>To jest okno utworzone dynamicznie <strong>On-The-Fly</strong></p>');
```

```
    document.writeln('</body>');
    document.writeln('</html>');
    document.close() //kończymy zapis do dokumentu
  }
}
```

Właściwość opener

Jedną z ciekawszych właściwości otwartych okien jest właściwość opener. Dzięki niej możemy się odwoływać do okna, z którego utworzyliśmy nowe okno.

```
opener.location.href = '#rozdzial_1';
```

Powyższy kod przeniesie stronę w głównym oknie do kotwicy rozdzial_1.

Metody i właściwości obiektu WINDOW

Poniżej przedstawiam metody i właściwości z których możemy korzystać przy pracy z okienkami. Oczywiście nie ze wszystkich jest sens korzystać, ale korzystanie z większości z nich daje dość ciekawe efekty:

Właściwość: Opis:
closed - zwraca wartość czy okno jest zamknięte czy nie
defaultStatus - ustawia/zwraca domyślny status okna
document - zwraca odwołanie do dokumentu w oknie
event - zwraca obiekt event (1, 2)
frames - zwraca odwołanie do obiektu frames
history - zwraca odwołanie do historii skoków danego okna
innerHeight - zwraca/ustawia wysokość zawartości okna
innerWidth - zwraca/ustawia szerokość zawartości okna
length - zwraca ilość ramek dla danego okna
location - zwraca/ustawia adres strony w aktualnym oknie
locationbar - zwraca obiekt locationbar, którego widoczność można ustawić dla danego okna
menubar - zwraca obiekt menubar, którego widoczność można ustawić dla danego okna
name - zwraca/ustawia nazwę okna
navigator - zwraca obiekt navigator
opener - zwraca odwołanie do okna, z którego utworzono dane okno
outerHeight - zwraca wysokość zewnętrzną okna przeglądarki
outerWidth - zwraca szerokość zewnętrzną okna przeglądarki
pageXOffset - zwraca wielkość kawałka strony, który został ukryty po przesunięciu strony w prawo
pageYOffset - zwraca wielkość kawałka strony, który został ukryty po przesunięciu strony w dół
parent - zwraca dokument ze zdefiniowanymi ramkami
personalbar - zwraca obiekt personalbar, którego widoczność można ustawić dla danego okna
screen - zwraca obiekt screen (np. screen.colorDepth)
screenX- zwraca lewe położenie okna na ekranie
screenY- zwraca górne położenie okna na ekranie
scrollbars - zwraca obiekt scrollbars, którego widoczność można ustawić dla danego okna
self - zwraca odwołanie do aktualnego okna
status - zwraca/ustawia tekst na statusie danego okna
statusbar - zwraca obiekt statusbar, którego widoczność można ustawić dla danego okna
toolbar - zwraca obiekt toolbar, którego widoczność można ustawić dla danego okna
top - zwraca odwołanie do okna położonego najwyżej w hierarchi

Metoda: Opis:

alert() - wywołuje okno alert
back() - wraca do strony poprzedniej w historii skoków
blur() - ustawia aktualne okno pod innymi oknami
close() - zamyka aktualne okno. Okno główne przeglądarki pyta o potwierdzenie zamknięcia
confirm() - wywołuje okno confirm
focus() - ustawia aktualne okno na pierwszym planie (ponad innymi oknami)
forward() - skacze do następnej strony w historii skoków
home() - wraca do strony startowej
moveBy(x,y) - przesuwa aktualne okno o dane wartości x,y
moveTo(x,y) - przesuwa aktualne okno do x,y
open() - otwiera nowe okno
print() - drukuje zawartość strony
prompt() - otwiera okno dialogowe prompt
resizeBy(x,y) - zmienia rozmiar okna o dane wartości x,y
resizeTo(x,y) - zmienia rozmiar okna do x,y
scroll(x,y) - przewija zawartość strony do x,y
scrollBy(x,y) - przesuwa zawartość strony o dane wartości x,y
scrollTo(x,y) - przewija zawartość strony do x,y
stop() - zatrzymuje ładowanie zawartości strony (to samo co po naciśnięciu przycisku STOP w przeglądarce)

Kilka okien

Możliwe jest otwarcie jednocześnie kilku nowych okien. Należy wtedy oddzielić średnikami (";") kolejne polecenia window.open('adres', 'nazwa'):
<body onload="window.open('adres1', 'nazwa1'); window.open('adres2', 'nazwa2')">...</body>
Trzeba wtedy jednak pamiętać, aby nazwy okien w kolejnych poleceniach były inne (albo ich wcale nie podawać). Wpisanie takich samych nazw, spowoduje otwarcie tylko jednego okna i załadowanie do niego strony ostatniej w kolejności podawania.

Strona na hasło

Wykorzystując pole typu "password" można w prosty sposób zabezpieczyć wybraną stronę hasłem. W tym celu wystarczy na stronie głównej, do której wszyscy mają normalny dostęp, wstawić następujący kod:

```
<form action="?" onsubmit="window.location.href = this.password.value + '.html'; return false">
        <input type="password" name="password" />
        <input type="submit" value="OK" />
</form>
```

Hasłem w tym przypadku jest nazwa strony bez rozszerzenia, którą chcemy zabezpieczyć.

Galeria zdjęć

Galeria tworzona jest w ten sposób, że na głównej stronie umieszcza się pomniejszone kopie obrazków oraz odsyłacze, po kliknięciu których następuje wczytanie obrazka w pełnych rozmiarach. Pozwala to uchronić się od wczytywania wszystkich dużych obrazków jednocześnie (użytkownik powiększa tylko te, które mu odpowiadają), a także zachowuje estetykę strony.

Uwaga! Miniaturki grafik muszą być pomniejszone fizycznie, tzn. nie mogą to być oryginalne duże obrazki ze zmniejszonymi rozmiarami wyświetlania za pomocą atrybutów WIDTH oraz HEIGHT. Tylko wstawienie naprawdę pomniejszonych obrazków uchroni od niepotrzebnego wczytywania dużych plików.

Jak wykonać:

- Najczęściej ustala się takie same rozmiary dla wszystkich miniatur, ponieważ ułatwia to utrzymanie estetyki strony.
- Obrazki w galerii zwykle ustawia się w komórkach tabeli (najczęściej bez obramowania), dzięki czemu można je dokładnie ustawić w rzędach i kolumnach.
- Jeżeli zamierzasz umieścić na swojej stronie obszerniejszą galerię grafik, zaleca się podzielenie jej na kilka części i stworzenie kilku podstron z miniaturkami.
- Wczytanie oryginalnego dużego obrazka następuje najczęściej po kliknięciu bezpośrednio jego miniaturki. Aby zastosować taki efekt, należy użyć odsyłacza obrazkowego (podstawowego). Prawie zawsze stosuje się również atrybut target="_blank", który powoduje otwarcie nowego okna przeglądarki.

```
<a target="_blank" href="ścieżka do dużego obrazka">
<img src="ścieżka do pliku miniaturki obrazka" alt="Tekst alternatywny" border="0" /></a>
```

Wykonywanie miniatur w programie gimp

1)Wczytaj pliku którego chcesz wykonać miniatur (opcja Plik → Otwórz).
2)Opcja menu Obraz→Skaluj Obraz…→Okno dialogowe „Skalowanie obrazu" umożliwia wybór jednostki, w jakiej podawane są wymiary obrazu, podanie nowej szerokości obrazu lub wysokości oraz zmianę współczynników X i Y. Domyślnie Szerokość i Wysokość oraz Współczynniki X oraz Y są połączone, co powoduje, że obraz jest skalowany proporcjonalnie (zmiana wysokości z 600 na 300 spowoduje automatycznie zmianę szerokości z 800 na 400). Jeśli chcemy zmieniać niezależnie szerokość i wysokość lub skalowanie obrazu należy rozłączyć klikając na przycisk o wyglądzie spinacza. Na koniec Skaluj

Aktywne przyciski obrazkowe

Aby wprowadzić na stronę przyciski obrazkowe, które zmieniają swój wygląd, po wskazaniu ich myszką, wystarczy dodać do znacznika <img /> dwa dodatkowe atrybuty: onmouseover="..." i onmouseout="...":

```
<a href="adres"><img src="obrazek_podstawowy.gif" alt="Tekst alternatywny" border="0"
onmouseover="this.src = 'obrazek_podswietlony.gif'" onmouseout="this.src =
'obrazek_podstawowy.gif'" /></a>
```

Ochrona strony

Czasami zdarza się, że chcemy opublikować w sieci jakieś ważne informacje. Zależy nam jednak, aby można się było z nimi zaznajomić tylko bezpośrednio na naszej stronie WWW i nie chcemy udostępniać takich materiałów do dalszej publikacji innym osobom. Chodzi tutaj głównie o opracowania typu: praca dyplomowa, ważny referat lub obszerny artykuł, unikalna grafika nad którą długo pracowaliśmy czy wyniki badań doświadczalnych. Niestety zwykle jedyną ochroną przed plagiatami jest podanie wyraźnej informacji: "Wszelkie prawa zastrzeżone". Jak wiadomo taki zapis nie może uchronić przed podkradaniem naszej własności intelektualnej, na co najlepszym dowodem jest konieczność istnienia serwisów i organizacji zajmujących się plagiatami w sieci, jak np. BOWI - Biuro Ochrony Witryn Internetowych.
Istnieją pewne metody utrudniające kradzież materiałów ze strony WWW. Od razu chciałbym wyraźnie podkreślić, że nie są to prawdziwe zabezpieczenia, a tylko pewne przeszkody, mogące zatrzymać raczej osoby początkujące, które jednak stanowią większość wśród użytkowników sieci. Według mnie nie warto w ten sposób zabezpieczać każdej strony internetowej, ponieważ irytuje to tylko internautów, a i tak prawdopodobnie nie zatrzyma osób bardzo zdeterminowanych, znających w jakimś stopniu język HTML i JavaScript. Ponadto większość z zabezpieczeń przedstawionych na tej stronie działa tylko w Internet Explorerze 5.0 lub nowszym. Pewnym pocieszeniem w tej sytuacji może być fakt, że przeglądarka ta zdobyła zdecydowaną większość rynku (nie dyskutując w tej chwili: słusznie czy nie).

Dalej w tym rozdziale znajdziesz wybrane formy blokady niedozwolonych działań na stronie. Oczywiście można je ze sobą łączyć.

UWAGA!
Metody ochrony stron WWW opisane w tym rozdziale w większości przypadków działają tylko w Internet Explorerze 5.0 lub nowszym!

Blokada prawego klawisza myszki

Zabezpiecza przed wybraniem "Pokaż źródło" z menu kontekstowego.

```
<body oncontextmenu="return false">
...
</body>
```

Uwaga! Źródło dokumentu nadal będzie można podejrzeć, wybierając odpowiednią opcję z górnego menu przeglądarki. Poza tym nie wszystkie przeglądarki interpretują to polecenie. Częściowo można zlikwidować ten problem, otwierając stronę w nowym oknie bez paska menu. Można również próbować otwierać stronę zawsze w ramkach - wtedy z menu będzie można zobaczyć jedynie źródło "mało ciekawej" strony startowej.

Blokada zaznaczania i kopiowania tekstu

```
<body onselectstart="return false" onselect="return false" oncopy="return false">
...
</body>
```

Blokada przeciągania elementów strony

Zabezpieczana m.in. przed przeciąganiem obrazków do innego okna lub programu w celu ich zapisania na dysku.

```
<body ondragstart="return false" ondrag="return false">
```

...
</body>

Blokada drukowania

Zaznacz kod Wypróbuj kod

<body onbeforeprint="document.body.style.visibility = 'hidden'; alert('Wydruk jest niedostępny!')" onafterprint="document.body.style.visibility = 'visible'">
...
</body>

Blokada pamięci podręcznej przeglądarki

Przeciwdziała zapisywaniu strony i jej elementów (np. obrazów) na dysku użytkownika w tzw. cache'u przeglądarki - zobacz: Cache.

Blokada zapisu wybranych zdjęć

[Zobacz: Obrazek]
Blokada paska narzędziowego obrazów

Pasek narzędziowy obrazów to zestaw ikon pojawiających się nad dużymi zdjęciami po "najechaniu" myszką (zwykle oba wymiary grafiki - szerokość i wysokość - muszą wynosić co najmniej 200 pikseli), co umożliwia m.in. wydruk lub zapisanie grafiki na dysku (Internet Explorer 6)

Dla wszystkich zdjęć na stronie (wstaw w nagłówku dokumentu - <head>...</head> - poniższy kod):

<meta http-equiv="Imagetoolbar" content="no" />

Tylko dla wybranych zdjęć:

Blokada klawisza Print Screen

Klawisza Print Screen umożliwia wykonanie prostego zrzutu ekranu i późniejszego wklejenia do programu graficznego. Należy wstawić do dokumentu specjalny kod w następujący sposób:

<html>
<head>
<script type="text/javascript">
// <![CDATA[
var browser = navigator.userAgent;

```
var ie = 0;
if (browser.indexOf("MSIE") != -1 && browser.indexOf(") ") == -1) ie =
parseFloat(browser.substring(browser.indexOf("MSIE")+4));

var id_status_blink = 0;
function status_blink(txt)
{
	window.status = txt;
	if (!txt) id_status_blink = setTimeout('status_blink("KLIKNIJ WEWNĄTRZ OKNA
PRZEGLĄDARKI !!!!!")', 250);
	else id_status_blink = setTimeout('status_blink("")', 1500);
	return true;
}

function blur_ie()
{
	document.all["body"].style.visibility = "hidden";
	clipboardData.clearData();
	status_blink("");
}

function focus_ie()
{
	document.all["body"].style.visibility = "visible";
	if (id_status_blink) clearTimeout(id_status_blink);
	window.status = '';
	return true;
}

if (ie >= 5)
{
	window.onblur = blur_ie;
	window.onfocus = focus_ie;
}
// ]]>
</script>
</head>
<body>
<div id="body">

Treść dokumentu

</div>
</body>
</html>
```

Oczywiście w nagłówku dokumentu mogą - a nawet powinny (!) - znaleźć się również inne znaczniki (<meta />, <title>...</title> itp.). Nie można zapomnieć o deklaracji DTD. Nic nie stoi również na przeszkodzie, aby dodać atrybuty do znacznika BODY, określające np. kolor tekstu i tła strony. Ważne jest jedynie, aby skrypt został wstawiony w ramy dokumentu tak jak pokazano.

Podsumowanie

Niestety takie rozwiązania zwykle nie są idealne i zawsze znajdzie się droga, aby je obejść. Mogą one natomiast utrudnić życie początkującym "hakerom". Powtarzam jeszcze raz: w większości przypadków stosowanie tych poleceń nie ma dużego sensu, ponieważ nie taka jest idea Internetu. Nie widzę większego celu w zabezpieczaniu w ten sposób wszystkich stron zwykłego serwisu. Irytuje to tylko użytkowników, a jeśli ktoś naprawdę będzie chciał podejrzeć źródło dokumentu, skopiować tekst lub zdjęcie, prawdopodobnie i tak znajdzie sposób, żeby to zrobić. A poza tym zastanów się, czy na Twojej stronie rzeczywiście są aż tak tajne dane, że naprawdę nikt nie może mieć do nich dostępu? Jeśli tak, to uzmysłów sobie, że powyższe sposoby stanowią tylko utrudnienie, a nie prawdziwe i pewne zabezpieczenie.

Sprawdzanie formularzy w javascript

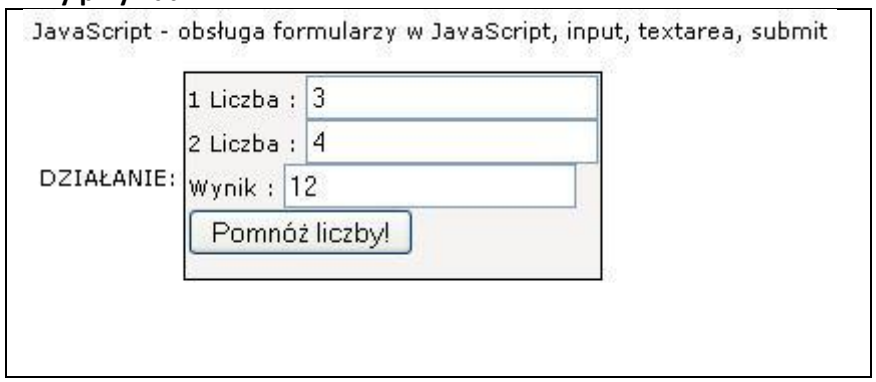
formularz

```
<form id="formularz" action="skrypt.php" method="post"
    onsubmit="return sprawdz_formularz()">
  <div>
      Podaj swoje imię: <input type="text" name="imie"><br>
      Podaj swoje nazwisko: <input type="text" name="nazwisko"><br>
      <input type="submit" value="Wyślij">
   </div>
</form>
```

funkcja sprawdzająca w javascript

```
function sprawdz_formularz()
{
                                    // przypisanie obiektu formularza do zmiennej
  var f = document.forms['formularz'];
                                    // sprawdzenie imienia
  if (f.imie.value == '')
  {
    alert('Musisz wpisać imię!');
    f.imie.focus();
    return false;
  }
                                    // sprawdzenie nazwiska
  if (f.nazwisko.value == '')
  {
    alert('Musisz wpisać nazwisko!');
    f.nazwisko.focus();
    return false;
  }
                                    // formularz jest wypelniony poprawnie
  return true;
}
```

inny przykład




```
script type="text/javascript">
<!--
function obslugaFormularza()
{
        var liczba1 = window.document.kalkulator.licz1.value;
```

```
        var liczba2 = window.document.kalkulator.licz2.value;
        var iloczyn = liczba1*liczba2;
        window.document.kalkulator.wynik.value = iloczyn;
}
//-->
</script>
<form name="kalkulator" method="post">
1 Liczba : <input type="text" name="licz1"/><br/>
2 Liczba : <input type="text" name="licz2"/><br/>
Wynik : <input type="text" name="wynik"/><br/>
<input type="submit" value="Pomnóż liczby!" onclick="obslugaFormularza(); return false;"/>
</form>
```

Popatrzmy na przykład - mamy formularz, dwa pola input gdzie wpisujemy liczby, jedno pole gdzie ma pojawić się wynik iloczynu no i wiadomo przycisk submit, który po kliknięciu wywołuje napisaną przez nas funkcję obslugaFormularza , czyli :
- zmiennej liczba1 jest przypisana wartość z pola formularza o nazwie licz1, czyli uzyskujemy tą wartość poprzez : window.document.nazwa_formularza.nazwa_pola.value
- podobnie odczytujemy drugą liczbę
- obliczamy iloczyn
- do pola input o nazwie wynik wpisujemy wynik iloczynu

Zwróć uwagę, że po wywołaniu funkcji mamy return false, po to aby strona się nam nie przeładowała po kliknięciu, tylko żeby wykonał się JS i nic więcej. Należy pamiętać aby nadawać nazwy polom formularza oraz samemu formularzowi - w przypadku PHP zazwyczaj nazywanie samego formularza nie było konieczne.

Jeśli chodzi o inne elementy formularzy HTMLowych działamy podobnie, na przykład dopisanie wartości do jakiegoś elementu textarea :
- window.document.formularz.nazwa_pola.value = "Tutaj jakiś tekst, który sobie dopisujemy do pola typu textarea";

No i analogicznie można odczytać przez : var odczyt = window.document.formularz.nazwa_pola.value;

dobra strona o formularzach:
http://www.doman.art.pl/kursjs/kurs/formularze/formularze.html

jQuery

jQuery to jedna z najbardziej popularnych bibliotek/frameworków do javascript. Jej popularność oczywiście znikąd się nie bierze. Dzięki tej bibliotece jesteśmy w stanie o wiele szybciej i sprawniej pisać nasze skrypty, nie martwiąc się przy tym o kompatybilność z różnymi przeglądarkami.
Główne zalety tej biblioteki:

Mała objętość skryptów, prosta składnia

Mała objętość samej biblioteki

Bardzo dużo dodatkowych pluginów

Minusy? Pisanie z wykorzystaniem jQuery rozleniwiają ^^.

http://webmaster.helion.pl/starocie/skrypt/skrypt.htm

Zadanie bez numeru

Temat: Obliczanie wyznaczniak trzeciego stopnia metodą Sarussa. Wczytywanie tablicy danych wg pomysłu ucznia Patryka z 3F.

Wykonaj:
a)uruchom program do wczytywania danych do tablicy(pamiętaj aby po kopiowaniu przenieść złamane linie, [są takie dwie własności] tak aby tworzyły jedna linię kodu),

```
<html>
<head>
<title>Kreator tabeli</title>
<meta http-equiv="Content-Type" content="text/html;charset=utf-8" >
<script LANGUAGE="JavaScript">
	function generuj_tabele(kolumny,wiersze)
	{
		var tabela = document.createElement('table');
		for(var x = wiersze;x>0;x--)
		{
			var wiersz = tabela.appendChild(document.createElement('tr'));
			for(var y = kolumny;y>0;y--)
			{
				var komorka = document.createElement('td');
				komorka.appendChild(document.createTextNode(prompt('Podaj
wartość dla '+((wiersze-x)+1)+', '+((kolumny-y)+1)+':',"0")));
				wiersz.appendChild(komorka);
			}
		}
		document.getElementById('wstaw').appendChild(tabela);
	}
</script>
</head>
<body>
	Wiersze:	<input type="text" name="rows" maxlength="10" value='3' id="rows" />
	Kolumny:	<input type="text" name="cols" maxlength="10" value='3' id="cols" />
	<p
onClick="generuj_tabele(document.getElementById('cols').value,document.getElementById('rows').v
alue);" style="color: red;">Twórz tabelę!</p>
	<div id="wstaw"></div>
</body>
</html>
```

```
document.write("<p style=\"color:red; \">jjjj</p>");
```

Konstruktor metoda w klasie, która nazywa się tak samo jak klasa oraz nie zwraca żadnej wartości, nazywa się konstruktorem i jest wywoływana automatycznie w chwili tworzenia obiektu tej klasy.

Zadanie 14a

Wczytać dwa boki trojkata c_kow i b_kow.

c_kow>b_kow (nie sprawdzamy)

obliczyć
cos(alfa)=... dwa miejsca po przecinku
.....

do formularza program i kod

Zadanie 14b
zmienne x_kow

podać stopnie obliczyć radiany

x=(PI*s)/180

podać radiany obliczyć stopnie

s=(180*x)/PI

dwa miejsca po przecinku

Przykład użycia textarea

```
<html>
 <head>
   <script type="text/javascript">
      function okno_zamknij_siejka()
        {
             window.close()
        }
    </script>
 </head>
<body>
   <form>
      <textarea name="siejka" cols="80" rows="20" readonly>
```

```
                przyklad3.html

        <HTML>
         <HEAD>
            <TITLE>SKRYPT trzeci --> zewnętrzny</TITLE>
         </HEAD>
    <BODY>
        <SCRIPT type="text/javascript" src="przyklad3_zewnetrzny.js"></SCRIPT>
  </BODY>
</HTML>
przyklad3_zewnetrzny.js
         document.write("ostatnia modyfikacja strony".fontcolor("red").bold().fontsize(7)+"<br>");
         document.write(document.lastModified);
        </textarea>
</form>
<input type="button" value="zamknij okno" onclick="okno_zamknij_siejka()"/>
<script type="text/javascript">
</script>
</body>
</html>
```

W celu wyświetlenia kodu programu możesz użyć instrukcji **textarea**

```
<textarea name="nazwa" cols="x" rows="y" readonly>
          jakiś tekst
</textarea>

cols- ilość kolumn
rows- ilość wierszy

Wstawienie atrybutu readonly powoduje, że tekstu zapisanego domyślnie w tym polu, nie będzie
można modyfikować.

================================================================================

<textarea name="nazwa" cols="x" rows="y" disabled>
        jakiś tekst
 </textarea>

Wstawienie atrybutu disabled powoduje zablokowanie pola.
```

```
<!DOCTYPE html>
<html lang="pl">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Kowalski Z39</title>
</head
 <html>
  <body>
     <h2>Pobierz wielkość okna przeglądarki</h2>
```

```
    <p>Przykład wyświetla szerokość i wysokość okna przeglądarki: (NIE uwzględniając pasków narzędzi/pasków przewijania): </p>
        <p id="demo"></p>
         <script>
            var w = window.innerWidth;
            var h = window.innerHeight;
            var x = document.getElementById("demo");
                  x.innerHTML = "szerokość przeglądarki: " + w + ", wysokość: " + h + ".";
         </script>
  </body>
 </html>
```